Le PYTHON en bref

Par Alexandre GALODÉ

Date de publication : 5 avril 2013

Dernière mise à jour : 16 juillet 2013

Ce livre a été rédigé sous Libre Office et écrit avec des polices d'écriture libres. Les exemples ont été conçus sous Geany sur une distribution Linux Ubuntu.

« La connaissance appartient à tout le monde »

Film Antitrust

Conçu et rédigé par Alexandre GALODÉ.

Ouvrage placé sous licence Creative Commons BY-NC-SA.

Texte complet de la licence depuis **http://creativecommons.fr/licences/les-6-licences/**

Ce livre est dédié à ma fiancée, mes parents, ma famille, mon « frère » et mes amis, présents ou regrettés.
Commentez : **Commentez**

I - Introduction....11
II - Présentation....11
II-A - Langages compilés et interprétés....11
II-B - Le langage PYTHON en bref....12
III - La Programmation Orientée Objet....12
III-A - Définition....12
III-B - Les objets par l'exemple....13
III-B-1 - Les classes....13
III-B-2 - Les propriétés....13
III-B-3 - Les méthodes....13
III-B-4 - En bref....13
III-C - L'encapsulation....14
III-D - L'héritage....14
IV - Le langage PYTHON....14
IV-A - Les types de données....14
IV-A-1 - Booléens....15
IV-A-2 - Integer....15
IV-A-3 - Long....15
IV-A-4 - Float....15
IV-A-5 - String....15
IV-A-5-a - Méthodes....16
IV-A-6 - Liste....17
IV-A-6-a - Méthodes....18
IV-A-7 - Dictionnaire....18
IV-A-7-a - Méthodes....19
IV-A-8 - Tuple....19
IV-B - Le transtypage....19
IV-B-1 - Transtypage de type....19
IV-B-2 - Transtypage de format....19
IV-B-3 - Détection de la plateforme d'exécution....19
IV-C - La portée des variables....20
IV-D - Quelques fonctions prédéfinies....20
IV-D-1 - PRINT....20
IV-D-2 - LEN....20
IV-D-3 - TYPE....21
IV-D-4 - INPUT....21
IV-D-5 - GETPASS....21
IV-E - Le caractère de césure....21
IV-F - Le caractère de commentaire....22
IV-G - Les opérateurs....22
IV-H - Les tests conditionnels....23
IV-H-1 - IF, ELIF, ELSE....23
IV-I - Les boucles....23
IV-I-1 - FOR....23
IV-I-2 - WHILE....24
IV-I-3 - Break et continue....24
IV-J - PYTHON et les fichiers....24
IV-J-1 - Chemin absolu et chemin relatif....24
IV-J-2 - Ouverture d'un fichier....24
IV-J-3 - Fermeture d'un fichier....25
IV-J-4 - Lecture....25
IV-J-5 - Écriture....26
IV-K - La POO PYTHON....26
IV-K-1 - PYTHON et les principes de la POO....26
IV-K-1-a - L'encapsulation....26
IV-K-1-b - L'héritage....26
IV-K-2 - La modularité avec PYTHON....26
IV-K-2-a - L'instruction import....26

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

IV-K-2-b - L'instruction SELF....27
IV-K-2-c - Les fonctions et les procédures....27
IV-K-2-d - Les classes....28
IV-K-2-d-i - Les attributs....28
IV-K-2-d-ii - Les accesseurs....28
IV-K-2-d-iii - Les mutateurs....29
IV-K-2-d-iv - Mise en situation....29
IV-K-2-e - Les modules....30
IV-K-2-f - Les packages....30
IV-K-2-f-i - Composition d'un package....30
IV-K-2-g - En résumé....31
IV-K-3 - Stockage d'objet dans des fichiers....31
IV-K-3-a - Lecture....31
IV-K-3-b - Enregistrement....32
IV-L - Les expressions régulières....32
IV-L-1 - Le module re....33
IV-L-1-a - Search....33
IV-L-1-b - Sub....33
IV-M - Les exceptions....33
IV-N - Les mots réservés....34
IV-O - Convention de programmation....34
IV-O-1 - La PEP20....34
IV-O-2 - La PEP8....35
IV-O-3 - Règles de codage....36
IV-O-3-a - Les variables....36
IV-O-3-b - Les fonctions/procédures....36
IV-O-3-c - Les modules et packages....36
IV-O-3-d - Les classes....36
IV-O-3-d-i - Le nom des classes....36
IV-O-3-d-ii - Les propriétés et les méthodes....36
IV-O-3-e - Les exceptions....36
IV-O-3-f - Les DocStrings....36
IV-O-3-g - Début de code....37
IV-O-3-h - Sortie de code....37
IV-O-3-i - Autre....37
IV-O-3-j - En plus....38
IV-O-4 - Bonne structure type d'un programme....38
IV-O-4-a - La mise en page....39
IV-O-4-b - Les règles....39
V - Modules complémentaires....40
V-A - Pypi....40
V-B - Le temps....40
V-B-1 - Le timestamp....40
V-B-2 - Date complète....41
V-B-3 - La mise en sommeil....41
V-C - Les mathématiques....42
V-C-1 - Le module de base....42
V-C-2 - Le module NumPY....42
V-C-2-a - Tableaux multidimensionnels....42
V-C-2-b - Manipulation sur les tableaux....43
V-C-2-b-i - Ajout....43
V-C-2-b-ii - Modification....43
V-C-2-b-iii - SUPPRESSION....44
V-C-2-b-iv - COPIE....44
V-C-2-c - Transposition....44
V-C-2-d - Méthodes associées aux tableaux....44
V-C-2-e - Calcul sur tableau....45
V-C-2-f - En plus....45

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

V-D - Imagerie....45
V-D-1 - Ouverture d'une image....46
V-D-2 - Création d'une image....46
V-D-3 - Modification d'une image....46
V-D-4 - Sauvegarde d'une image....46
V-D-5 - Affichage d'une image....47
V-D-6 - Connaître les composantes d'un pixel....47
V-D-7 - Conversion en gris....47
V-D-8 - Exemple....47
V-E - Les graphiques avec Matplotlib....48
V-E-1 - Création d'une courbe....48
V-E-1-a - Le conteneur....48
V-E-1-b - Ajout d'une courbe....49
V-E-2 - Paramétrage complémentaire....50
V-E-2-a - Axes....50
V-E-2-b - Légende de la courbe....50
V-E-2-c - Labels....50
V-E-2-d - Grille....51
V-E-2-e - Utilisation de date en X ou Y....51
V-E-2-f - Image de fond....51
V-E-2-g - Effacement de la courbe....51
V-E-2-h - Transformer un graphique en image....52
V-E-3 - Exemple....52
V-F - Les bases de données....53
V-F-1 - Présentation rapide....53
V-F-1-a - Composants....53
V-F-1-b - Fonctionnement générique....54
V-F-2 - PYSQLITE....54
V-F-2-a - Connexion....55
V-F-2-b - Exécuter une requête....55
V-F-2-b-i - Curseur....55
V-F-2-b-ii - Récupération des résultats....55
V-F-2-c - Sauvegarde....56
V-F-2-d - Déconnexion....56
V-F-3 - PSYCOPG....56
V-F-3-a - Connexion....56
V-F-3-b - Exécuter une requête....57
V-F-3-b-i - Curseur....57
V-F-3-b-ii - Récupération des résultats....57
V-F-3-b-iii - Lecture de la dernière requête exécutée....57
V-F-3-b-iv - Procédures....57
V-F-3-b-v - Gestion d'erreurs....58
V-F-3-c - Sauvegarde....58
V-F-3-d - Déconnexion....58
V-G - Le module OS....58
V-G-1 - Taille d'un fichier....58
V-G-2 - Renommage d'un fichier....58
V-H - Scripting....59
V-H-1 - Les arguments....59
V-H-1-a - Passage d'argument....59
V-H-1-b - Tester le nombre d'arguments....59
V-H-1-b-i - Ouverture et accès aux données....59
V-H-1-b-ii - Décompression complète....60
V-H-2 - L'exécution de commande....60
V-H-3 - Exemple....60
V-I - Les fichiers Zip....61
V-I-1 - Création/modification d'un ZIP....61
V-I-2 - Ouverture d'un ZIP....62

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

V-J - Le XML....62
V-J-1 - Codage....62
V-J-1-a - L'entête....62
V-J-1-b - Le corps....63
V-J-1-c - Exemple....63
V-J-2 - Les principes de base....63
V-J-2-a - Les commentaires....63
V-J-2-b - La racine et les nœuds....64
V-J-2-c - Les balises....64
V-J-2-d - Chevauchement....64
V-J-2-e - Les attributs....64
V-J-2-f - Les instructions de traitement....64
V-J-2-g - Le XPATH....65
V-J-2-g-i - Sélection d'un nœud précis....65
V-J-2-g-ii - Sélection avec attributs....65
V-J-2-g-iii - Récupérer toutes les données d'un nœud....65
V-J-2-g-iv - Combinaison de XPATH....66
V-J-2-g-v - En bref....66
V-J-3 - Le module LXML....66
V-J-3-a - Utilisation de LXML....66
V-J-3-b - Création d'un fichier XML....66
V-J-3-c - Lecture d'un fichier XML....67
V-J-3-d - Modification d'un fichier XML....68
V-J-3-d-i - Serialisation et deserialisation....68
V-J-3-d-ii - Méthode générale....68
V-K - L'Open Document Format....69
V-K-1 - Historique....69
V-K-2 - Les différents types de documents....69
V-K-3 - Les templates....69
V-K-4 - Description du format ODF....69
V-K-5 - Interaction avec PYTHON....69
V-K-6 - Modification de template....70
V-K-6-a - Théorie....70
V-K-7 - Interaction complète....70
V-K-7-a - Création et ouverture d'un document ODF....71
V-K-7-b - ODT....71
V-K-7-c - ODS....72
V-K-8 - Exemple....72
V-L - Serveur FTP....73
V-L-1 - Connexion à un serveur FTP....73
V-L-2 - Dépôt de fichiers/dossiers....74
V-L-3 - Récupération de fichiers/dossiers....74
V-L-4 - Annulation d'un transfert en cours....75
V-L-5 - Liste des éléments du dossier....75
V-L-6 - Renommage d'un fichier/dossier....75
V-L-7 - Création d'un dossier....75
V-L-8 - Effacement d'un fichier/dossier....75
V-L-9 - Connaître la taille d'un fichier....75
V-L-10 - Savoir où l'on se trouve....75
V-L-11 - Utilisation de ligne de commandes....76
V-L-12 - Déconnexion du serveur FTP....76
V-L-13 - Les messages d'erreurs possibles....76
V-L-14 - Exemple....77
V-M - Les mails avec SMTP....78
V-M-1 - Données nécessaires....78
V-M-2 - Exemple....78
V-N - Le port série....79
V-N-1 - Paramétrage....79

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

V-N-2 - Exemple ... 81
V-O - Le port parallèle ... 81
V-O-1 - Paramétrage ... 82
V-O-2 - Exemple ... 82
V-P - Le réseau: les bases ... 82
V-P-1 - Le module socket ... 83
V-P-2 - Ouverture d'un port ... 83
V-P-3 - Envoi de données ... 83
V-P-4 - Réception de données ... 83
V-P-5 - Fermeture d'un port ... 83
V-P-6 - Exemple ... 84
V-Q - La webcam avec OpenCV ... 84
V-Q-1 - Utilisation basique ... 84
V-Q-1-a - Connexion au flux vidéo ... 84
V-Q-1-b - Snapshot du flux ... 85
V-Q-1-c - Affichage des snapshots ... 85
V-Q-1-d - Exemple d'affichage d'un flux webcam ... 85
V-Q-2 - Utilisation avancée ... 86
V-Q-2-a - Les images ... 86
V-Q-2-b - Les enregistreurs vidéo ... 87
V-Q-2-c - Les polices d'écriture ... 88
V-Q-2-d - Utilisation du clavier ... 89
V-Q-3 - Paramétrage de la webcam ... 89
V-Q-3-a - SetCaptureProperty ... 90
V-Q-3-b - GetCaptureProperty ... 90
V-Q-4 - Traitements sur les frames ... 90
V-Q-4-a - Les types de données OpenCV ... 90
V-Q-4-b - AbsDiff ... 91
V-Q-4-c - CvtColor ... 92
V-Q-4-d - Dilate ... 93
V-Q-4-e - Erode ... 94
V-Q-4-f - Combinaison : Ouverture ... 95
V-Q-4-g - Combinaison : Fermeture ... 95
V-Q-4-h - Combinaison : Gradient ... 96
V-Q-4-i - Combinaison : Top Hat ... 96
V-Q-4-j - Combinaison : Black Hat ... 97
V-Q-4-k - ContourArea ... 97
V-Q-4-l - DrawContours ... 98
V-Q-4-m - EqualizeHist ... 98
V-Q-4-n - FindContours ... 99
V-Q-4-o - HaarDetectObject ... 100
V-Q-4-p - InRangeS ... 101
V-Q-4-q - Load ... 102
V-Q-4-r - MatchTemplate ... 102
V-Q-4-s - MemoryStorage ... 103
V-Q-4-t - MinMaxLoc ... 103
V-Q-4-u - RunningAvg ... 103
V-Q-4-v - Smooth ... 104
V-Q-4-w - Threshold ... 105
V-Q-5 - Ajout d'élément sur une image/frame ... 106
V-Q-5-a - Rectangle ... 106
V-Q-5-b - Cercle ... 107
V-Q-5-c - Ligne ... 107
V-Q-6 - Détection de mouvement ... 108
V-Q-6-a - Exemple ... 109
V-Q-7 - Reconnaissance des couleurs ... 109
V-Q-7-a - Exemple ... 111
V-Q-8 - Détection de contours ... 111

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

V-Q-8-a - Exemple basique....111
V-Q-8-b - Exemple avancé....112
V-Q-9 - Détection d'une forme....113
V-Q-9-a - Recherche d'une image dans l'image....113
V-Q-10 - Détection de visage....114
V-Q-10-a - Exemple....114
V-Q-11 - Création d'un algorithme de détection....115
V-Q-11-a - Création des positifs....115
V-Q-11-b - Création des négatifs....116
V-Q-11-c - Marquage des positifs....116
V-Q-11-d - Création du fichier vecteur....116
V-Q-11-e - Création du classificateur....117
V-R - Les Threads....117
V-R-1 - Création....118
V-R-2 - Lancement....118
V-R-3 - Arrêt....118
V-R-4 - Pause....119
V-R-5 - Appel toutes les x secondes....119
V-R-6 - Exemple....119
V-S - Les PDF : Reportlab....120
V-S-1 - Principes de fonctionnement....120
V-S-2 - Les bases....120
V-S-2-a - Canvas, format de page et unités....120
V-S-2-b - Texte....121
V-S-2-b-i - Polices d'écriture....121
V-S-2-b-ii - Simple ligne texte....122
V-S-2-c - Paragraphe....122
V-S-2-c-i - Les styles....122
V-S-2-c-ii - Création d'un paragraphe....123
V-S-2-c-iii - Ajout d'un paragraphe à un canvas....123
V-S-2-d - Couleur....123
V-S-2-d-i - Principe....123
V-S-2-d-ii - Couleur de ligne....124
V-S-2-d-iii - Couleur de remplissage....124
V-S-2-d-iv - Transparence....124
V-S-2-d-v - Couleur de texte....124
V-S-2-d-vi - Définir une couleur....124
V-S-2-e - Rotation....124
V-S-2-f - Tableau....125
V-S-2-f-i - Tableau de base....125
V-S-2-f-ii - Les styles de tableaux....125
V-S-2-f-iii - Taille de cellules....126
V-S-2-f-iv - Les bordures....126
V-S-2-f-v - Les images dans les tableaux....127
V-S-2-f-vi - Insertion d'un tableau dans un canvas....127
V-S-2-g - Dessin....127
V-S-2-g-i - Ligne....128
V-S-2-g-ii - Cercle....128
V-S-2-g-iii - Rectangle....128
V-S-2-g-iv - Épaisseur de lignes....128
V-S-2-g-v - Type de ligne....128
V-S-2-h - Image....129
V-S-2-i - Graphique....129
V-S-2-i-i - Support de graphique....129
V-S-2-i-ii - Histogramme....129
V-S-2-i-iii - Linéaire....131
V-S-2-i-iv - Camembert....132
V-S-2-i-v - Intégration du support dans un canvas....133

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

V-S-2-j - Changer de page....133
V-S-2-k - Métadonnées PDF....134
V-S-2-k-i - Auteur....134
V-S-2-k-ii - Titre....134
V-S-2-k-iii - Sujet....134
V-S-2-l - Sauvegarde....134
V-S-3 - Exemple....134
VI - Les interfaces graphiques....136
VI-A - PYGTK....136
VI-A-1 - Les fenêtres....137
VI-A-1-a - Main....137
VI-A-1-b - About....138
VI-A-1-c - Info / erreur / question / Attention....139
VI-A-1-d - Print....141
VI-A-1-e - File....142
VI-A-2 - Les conteneurs....143
VI-A-2-a - Hbox / Vbox....143
VI-A-2-b - Boîte à boutons....144
VI-A-2-c - Tableau....144
VI-A-2-d - Fixed....145
VI-A-2-e - ScrolledWindow....146
VI-A-2-e-i - Création....146
VI-A-2-e-ii - Paramétrage....146
VI-A-2-e-iii - Ajout de l'enfant....146
VI-A-3 - Les widgets....146
VI-A-3-a - Boutons à cliquer....146
VI-A-3-b - Boutons à commuter....147
VI-A-3-c - Boutons à cocher....148
VI-A-3-d - Boutons radio....148
VI-A-3-e - Combobox....149
VI-A-3-f - Les onglets....149
VI-A-3-g - Les tableaux de données....151
VI-A-3-g-i - Les TreeStores....152
VI-A-3-g-ii - Les TreeViews....154
VI-A-3-g-iii - Les TreeViewColumn....155
VI-A-3-h - Label....157
VI-A-3-i - Textbox....157
VI-A-3-i-i - Monoligne....157
VI-A-3-i-ii - Multilignes....158
VI-A-3-j - Compteur....159
VI-A-3-k - Progressbar....160
VI-A-3-l - Calendrier....161
VI-A-3-m - Image....161
VI-A-3-m-i - Image en fond de fenêtre....162
VI-A-3-n - Cadre....163
VI-A-3-o - Menu....164
VI-A-3-o-i - Structure d'un menu....164
VI-A-3-o-ii - Menu texte....165
VI-A-3-o-iii - Sous-menu....165
VI-A-3-p - Barre d'outils....166
VI-A-3-q - Barre de statut....166
VI-A-3-r - Les curseurs....167
VI-A-3-s - Bulles d'aide....168
VI-A-3-t - Les stocks (icônes)....168
VI-A-3-u - Exemple....170
VII - La 3D....171
VII-A - Le format STL....172
VII-A-1 - STL ASCII....172

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

VII-A-2 - STL Binaire 173
VII-B - OpenGL et PYTHON: les bases 174
VII-B-1 - Le repère XYZ 174
VII-B-2 - Notions de base 174
VII-B-2-a - Les vertex 174
VII-B-2-b - Les polygones 175
VII-B-2-c - Les formes de base 175
VII-B-2-c-i - Les points 176
VII-B-2-c-ii - Les lignes 176
VII-B-2-c-iii - Les triangles 176
VII-B-2-d - État de surface d'un polygone 177
VII-B-2-d-i - Les couleurs 177
VII-B-2-d-ii - Les textures 177
VII-B-2-e - La caméra 178
VII-B-2-e-i - Les différents types de caméra 178
VII-B-2-e-ii - Positionnement 178
VII-B-2-e-iii - Zoom, translation, et rotation 179
VII-B-3 - Fenêtre 180
VII-B-3-a - Les indispensables 180
VII-B-3-a-i - Le constructeur de la classe 180
VII-B-3-a-ii - Le main 180
VII-B-3-a-iii - L'InitGL 181
VII-B-3-a-iv - Le ReSizeGLScene 181
VII-B-3-a-v - Le DrawnGLScene 182
VII-B-3-b - Insertion dans une frame PYGTK 183
VII-B-3-b-i - Principe 183
VII-B-3-b-ii - Mise en œuvre 184
VII-B-4 - Gestion de la souris 186
VII-B-4-a - Les boutons et la molette 186
VII-B-4-b - Le déplacement de la souris 187
VII-B-5 - Gestion du clavier 187
VII-B-5-a - Touches ALT, SHIFT, CTRL 187
VII-B-5-b - Touches alphanumériques 187
VII-B-5-c - Touches spéciales 188
VII-B-6 - Exemple 189
VII-C - Concepts mathématiques 2D 191
VII-C-1 - Les plans 2D 191
VII-C-2 - La trigonométrie 192
VII-C-2-a - Cercle trigonométrique 192
VII-D - Concepts mathématiques 3D 193
VII-D-1 - Formules 193
VII-D-1-a - Équation d'une droite en 3D 193
VII-D-1-b - Équation d'un plan en 3D 194
VII-D-1-b-i - Méthode 1 194
VII-D-1-b-ii - Méthode 2 196
VII-D-1-b-iii - Cas particuliers 196
VII-D-1-c - Calcul de la normale au plan 3D 196
VII-D-1-d - Calcul des coordonnées d'un point en 3D 197
VII-D-1-e - Rotation d'un point en 3D 197
VII-D-1-f - Un point appartient-il à un triangle ? 198
VII-E - Quelques techniques 3D 199
VII-E-1 - Depthmaps 199
VII-E-2 - Stéréogramme 199
VII-E-3 - Scanner 3D laser 200
VIII - Gestion d'un projet 202
VIII-A - Organisation 203
VIII-A-1 - Description du PJ 203
VIII-A-2 - Structure du projet 203

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

VIII-A-3 - Priorisation....203
VIII-A-4 - Planification....204
VIII-A-4-a - Planner....204
VIII-A-4-b - Calligraplan....205
VIII-B - Viabilité et pérennité....206
VIII-B-1 - Code....206
VIII-B-2 - Tests....206
VIII-B-2-a - Test unitaire....207
VIII-B-2-b - Test d'intégration....207
VIII-B-2-c - Test de validation ou fonctionnel....207
VIII-B-2-d - Construction d'un test....207
VIII-B-2-e - Logiciel dédié : Unittest....208
VIII-B-2-e-i - Fonctionnement de base....208
VIII-B-2-e-ii - Possibilités offertes....209
VIII-B-2-e-iii - Test simple....209
VIII-B-2-e-iv - Tests multiples....210
VIII-B-3 - Documentation....210
VIII-B-3-a - Epydoc....210
VIII-B-4 - Fichier de log....211
VIII-C - Versionning....211
VIII-D - Les licences libres....212
VIII-D-1 - Licences logicielles....212
VIII-D-1-a - GPL....212
VIII-D-2 - Licences Documentaires....214
VIII-D-2-a - Creative Commons....214
IX - Déploiement....216
IX-A - Création d'un .deb manuel....216
IX-B - Création d'un .DEB automatique....218
IX-C - Utilisation du logiciel Alien....219
X - Ressources....220
XI - Annexes....223
XI-A - Spécifications fonctionnelles....223
XI-B - Spécifications techniques....225
XI-C - Priorisation....227
XI-D - Tests unitaires....228
XI-E - Tests intégration....228
XI-F - Tests de validation....229
XI-G - Code ASCII....229
XI-H - Nuancier RGB hexa....230

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

I - Introduction

Que l'informaticien soit débutant ou averti, se pose à un moment la fatidique question du langage à adopter.

En effet, tantôt nous avons besoin d'un langage procédural, tantôt d'un langage objet ; tantôt d'un langage de script, tantôt d'un langage avec IHM…

C'est à ce niveau que **PYTHON** est intéressant. En effet, ce langage sait s'adapter à de nombreuses situations. Capable de créer des clients lourds ou légers, des applications standards ou mobiles, il sait tout faire.

De plus, langage **OPEN SOURCE** par excellence, ce langage bénéficie d'une très large communauté et d'appuis sérieux et solides telle la société GOOGLE qui s'en sert comme langage principal, la NASA ou encore de nombreux logiciels OPEN SOURCE reconnus comme BLENDER.

Prenant de plus en plus d'ampleur dans l'univers de la programmation, **PYTHON** remplace peu à peu ceux qui avaient la préférence dans de nombreux établissements informatiques, car contrairement à d'autres, **PYTHON** ne nécessite nullement de multiples mises à jour par mois. Chaque évolution est mûrement réfléchie avant d'être déployée.

Ce livre est conçu à la fois comme un condensé d'informations pour apprendre **PYTHON**, mais également comme un aide-mémoire toujours utile à avoir sous la main.

Ce livre a été conçu sous Linux, et par conséquent, certains exemples peuvent ne pas fonctionner sous d'autre OS.

Bonne lecture.

II - Présentation



Wikimedia Commons,Start_hand.svg

II-A - Langages compilés et interprétés

Lorsque vient le moment de faire un choix de langage, deux grandes catégories s'affrontent : les langages compilés et les **langages interprétés**.

Les langages compilés comme l'indique leur nom nécessitent un compilateur afin de transformer le code source en langage machine. Cela rend le fichier résultant non modifiable, et lui confère une certaine vélocité d'exécution, la machine se contentant de lire des instructions et de les exécuter. C'est le cas du très connu langage C.

Les langages interprétés, eux, ne passent pas par cette phase de compilation. On utilise non pas un compilateur, mais un interpréteur. Cet interpréteur va lire l'ensemble du code et le traduire en temps réel à la machine pour exécution.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Si cela peut paraître à certains moins rapide à l'exécution qu'un langage compilé, il n'y a en fait plus forcément beaucoup de différences entre ces deux types de langages avec nos machines récentes.

Le code du langage interprété reste lisible. Il y a aussi bien entendu la possibilité de créer un **Byte Code** qui est une préinterprétation du code pour la machine (extension .pyc).

Deux avantages : une amélioration des performances, ce qui peut parfois s'avérer utile voir indispensable, et la possibilité de ne pas divulguer le code, ce que certains verront comme un point positif.

De fait, un langage interprété est très pratique en débogage, puissant et réactif en exécution.

PYTHON est un de ces langages interprétés, avec toute la puissance que cela implique.

II-B - Le langage PYTHON en bref

Le langage **PYTHON** tire son nom du fait que le créateur de ce langage est un fan des comiques anglais les « Monthy PYTHON ».

Créé en 1990 par **Guido VAN ROSSUM**, c'est un langage open source, multiplateforme et multifonction. Les programmeurs s'en servent aussi bien pour remplacer du shell en effectuant du scripting que pour réaliser de la modélisation 3D avec interface graphique.

Ce langage est désormais géré par la **PYTHON** Software Foundation. Principalement utilisée en branche 2.x, la version 3.x est une version « nettoyée » dans le sens où de nombreux doublons et de nombreuses redondances ont été supprimés afin d'épurer le code du langage.

De fait un code développé en 2.x peut se révéler partiellement incompatible avec la 3.x. Il est souvent recommandé de commencer à développer directement avec la 3.x.

Dans les faits, cette branche est actuellement encore peu utilisée, mais se tenir à ce conseil est garantir une certaine viabilité du code, ce qui est non négligeable.

III - La Programmation Orientée Objet



Wikimedia Commons,1328102023 Copy.png

III-A - Définition

Il existe de nombreuses définitions de la **POO**. Pour faire simple, disons qu'il s'agit d'utiliser des copies de briques logicielles afin d'aboutir à notre fin.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Ces copies de briques de base, de références, sont ce qu'on appelle des **OBJETS**. Chaque objet va posséder des caractéristiques (appelées **PROPRIÉTÉS**) et des possibilités (appelées **MÉTHODES**).

Cette façon de travailler, va permettre une plus grande rigueur dans la façon de coder, mais également une viabilité du travail effectué.

Chaque brique est ainsi réutilisable à l'infini dans n'importe quel projet, possède sa propre documentation et est maintenable facilement.

III-B - Les objets par l'exemple

L'**objet** est une brique de référence pour programmer en POO.

Afin de rentrer plus amplement dans le détail, nous allons prendre un exemple concret, beaucoup utilisé : la voiture.

Qu'est-ce qu'une voiture ? Quelle est la définition d'une voiture ?

Selon Wikipedia, c'est un véhicule terrestre à roues équipé avec un moteur embarqué. En simplifié, une voiture possède des roues, un moteur et sert à se déplacer.

Retenons cette définition simpliste. Cette voiture peut avoir des formes, couleurs ou encore marques différentes. De même, elle peut réaliser différentes actions.

En POO, la voiture sera notre objet, sa forme ou sa couleur ses propriétés et les différentes actions qu'elle peut réaliser ses méthodes.

III-B-1 - Les classes

Cet ensemble, ce qui constitue en quelque sorte la quintessence d'une voiture dans notre exemple est ce qu'on appelle une **classe**. On peut ainsi comparer une classe à un moule servant à créer des objets

III-B-2 - Les propriétés

Les **propriétés** sont ce qui définit notre objet. Par exemple, sa couleur, ou encore sa marque pour notre exemple.

Ces propriétés peuvent être accessibles uniquement en lecture, uniquement en écriture ou bien les deux. Elles vous permettront d'interagir avec votre objet afin de le paramétrer au mieux à vos besoins.

III-B-3 - Les méthodes

Les **méthodes** sont les possibilités qu'offre notre objet. Dans notre exemple, une voiture peut avancer, reculer, allumer ses phares…

Chacune de ces actions constitue une méthode différente de l'objet voiture.

III-B-4 - En bref

Pour résumer, nous avons une classe voiture à partir de laquelle nous pouvons créer des objets :

```
ma_voiture = Voiture()
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Notre objet possède des propriétés sur lesquelles nous pouvons agir pour le configurer :

ma_voiture.couleur = vert

Il possède également des méthodes pour nous permettre de lui dire quoi faire :

ma_voiture.phare(ON)

III-C - L'encapsulation

Le principe d'**encapsulation** est un des principes forts de la programmation objet. Pour simplifier, cela signifie que pour des raisons de sécurité ou de gestion, nous allons rendre certaines variables accessibles ou non depuis l'extérieur du code.

Si cela peut parfois sembler abstrait, dans les faits cela permet souvent d'éviter qu'un attribut soit changé en cours de calcul et ne provoque un crash du code.

Cela peut également être un moyen d'encadrer précisément l'utilisation d'un module.

III-D - L'héritage

Le principe de l'**héritage**, autre principe fort de la programmation objet, est qu'une classe peut elle-même hériter d'une autre classe.

Ainsi, pour reprendre l'exemple des voitures, une classe Twingo héritera d'une classe mère voiture. Cette classe Twingo possédera les mêmes caractéristiques que celles définies dans la classe voiture (un moteur, un châssis, un volant…), mais en possédera des complémentaires (options spécifiques à la voiture).

IV - Le langage PYTHON



IV-A - Les types de données

PYTHON dispose d'un certain nombre de types de donnée. Nous allons ici voir les principales.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

IV-A-1 - Booléens

Les **booléens** comme dans tout langage peuvent prendre deux valeurs uniquement. En **PYTHON**, ces valeurs sont **True** et **False**.

```
>>> mon_booleen = True
>>> mon_booleen
True
>>> type(mon_booleen)
<type 'bool'>
```

La dernière commande type est expliquée plus loin.

IV-A-2 - Integer

Le premier du trio des classiques. L'entier/**Integer** est un nombre sans virgule. Il est codé sur quatre octets et sa valeur peut s'étendre **de -2 147 483 648 à +2 147 483 647**.

```
>>> mon_integer = 56321
>>> mon_integer
56321
>>> type(mon_integer)
<type 'int'>
```

IV-A-3 - Long

Lorsque la précision d'un integer est insuffisante, ou peut potentiellement l'être, il faut utiliser un **long**. Ce type de valeur entière **n'a comme limite que la capacité mémoire de l'ordinateur.**

```
>>> mon_long = 2147483648
>>> mon_long
2147483648L
>>> type(mon_long)
<type 'long'>
```

IV-A-4 - Float

Le second du trio. On utilise le point pour indiquer qu'il s'agit d'un **float** (par exemple: a = 3. ou a = 3.0). Encodées sur huit octets, leurs valeurs peut aller **de 10^{-3}## à 10^{3}##, avec une précision de 12 chiffres significatifs après le caractère de séparation.**

Attention : le caractère de séparation est le point et non la virgule.

```
>>> mon_float = 3.5632
>>> mon_float
3.5632
>>> type(mon_float)
<type 'float'>
```

IV-A-5 - String

Le troisième du trio de tête. Une chaîne de caractères est écrite entre simples ou doubles-quotes en **PYTHON**, au choix du programmeur.

```
>>> ma_string = "hello"
>>> ma_string
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
'hello'
>>> type(ma_string)
<type 'str'>
```

Le caractère d'échappement est l'antislash \ . Pour écrire un antislash, on saisit simplement \\ .

```
>>> ma_string2 = "texte avec antislash: \\"
>>> print ma_string2
texte avec antislash: \
```

Un **string** en **PYTHON** est comparable à un tableau de caractères. Ainsi, si ma_string = " Test ", alors ma_string[1] vaut " e ".

```
>>> ma_string3 = "Test"
>>> ma_string3[1]
'e'
```

Il est également possible de ne sélectionner qu'une partie de la chaîne avec le caractère ": "

```
>>> ma_string3 = "Test"
>>> ma_string3[0:2]
'Te'
>>> ma_string3[2:]
'st'
>>> ma_string3[:3]
'Tes'
```

Remarque : les index commencent à 0 en PYTHON.

Enfin, pour assembler deux chaînes de caractères, il suffit d'utiliser le " + "

```
>>> ma_string = "hello"
>>> ma_string2 = "world"
>>> ma_string3 = ma_string + ma_string2
>>> ma_string3
'helloworld'
```

IV-A-5-a - Méthodes

On peut manipuler une chaîne de caractères grâce à certaines de ces méthodes. Les plus usitées sont les suivantes :

>Changement de casse

```
>>> ma_chaine='Hello World'
>>> ma_chaine
'Hello World'
>>> ma_chaine.lower()
'hello world'
>>> ma_chaine.upper()
'HELLO WORLD'
```

>Mettre la première lettre en majuscule

```
>>> ma_chaine='hello world'
>>> ma_chaine.capitalize()
'Hello world'
```

>Séparation de caractères, avec un caractère prédéfini

```
>>> ma_chaine.split('l')
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
['He', '', 'o Wor', 'd']
```

>Concaténation de chaînes, avec un caractère prédéfini

```
>>> ma_chaine.join('l')
'l'
>>> ma_chaine
'Hello World'
```

>Trouver la position d'une lettre

```
>>> ma_chaine = 'Hello world'
>>> ma_chaine.find('w')
6
```

>Compter le nombre d'occurrences d'un caractère

```
>>> ma_chaine = 'Hello world'
>>> ma_chaine.count('l')
3
```

>Supprimer les espaces en début et fin de chaîne

```
>>> ma_chaine = '    Ceci est un test    '
>>> ma_chaine
'    Ceci est un test    '
>>> ma_chaine.strip()
'Ceci est un test'
```

>Enfin, tester le type de donnée contenu dans la chaîne (**True** si vrai, **False** sinon)

```
>>>ma_chaine.isalpha()    #Teste s'il n'y a exclusivement que des lettres
>>>ma_chaine.isdigit()    #Teste s'il n'y a que des chiffres
>>>ma_chaine.isalnum()    #Teste s'il y a des caracteres alphanumeriques
>>>ma_chaine.isspace()    #Teste s'il n'y a que des espaces
```

IV-A-6 - Liste

Comme son nom l'indique, une **liste** est un ensemble d'éléments divers : nombre, texte…

```
>>>#creation d'une liste pleine
>>>jour_ouvre =
 [" lundi ", " mardi ", " credi ", " jeudi ", " vendredi ",1,2,
>>>jour_semaine = [] #creation d'une liste vide
```

Ici, deux listes sont créées de manière différente.

Une liste une fois remplie, telle jour_ouvre, se comporte comme un tableau :

```
>>>jour_ouvre [1]
mardi
```

On peut également modifier un élément d'une liste :

```
>>>jour_ouvre[9] = jour_ouvre[9]-2
>>>print jour_ouvre[9]
5
>>>jour_ouvre[2] = " mercredi "
>>>print jour_ouvre[2]
mercredi
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

nous pouvons également utiliser la fonction prédéfinie **del** pour effacer un élément par son index :

```
>>>del(jour_ouvre[9])
>>>print jour_ouvre
[" lundi ", " mardi ", " mercredi ", " jeudi ", " vendredi ",1,
```

IV-A-6-a - Méthodes

Il existe également une méthode pour effacer un élément d'une liste, en passant comme paramètre sa valeur

```
>>>jour_ouvre.remove(4)
>>>print jour_ouvre
[" lundi ", " mardi ", " credi ", " jeudi ", " vendredi ",1,2,3
```

Pour réaliser des ajouts, on utilise la méthode **append** des listes :

```
>>>jour_ouvre.append(4)
>>>jour_ouvre.append(5)
>>>print jour_ouvre
[" lundi ", " mardi ", " credi ", " jeudi ", " vendredi ",1,2,3
```

Attention : on peut insérer une liste dans une liste. Bien que cela ne soit pas la meilleure manière de procéder, on peut ainsi créer des matrices sommaires.

Il est aussi possible d'insérer un élément dans une liste à une position donnée :

```
>>>jour_ouvre.insert(5, " samedi ")
>>>print jour_ouvre
[" lundi ", " mardi ", " credi ", " jeudi ", " vendredi ", 
```

Nous pouvons concaténer deux listes avec la méthode **extend :**

```
>>>jour_semaine.extend(jour_ouvre)
>>>print jour_semaine
[" lundi ", " mardi ", " credi ", " jeudi ", " vendredi ", 
```

IV-A-7 - Dictionnaire

Un **dictionnaire** peut, par certains points, être comparé à une liste. Cependant, dans un dictionnaire, les différents éléments ne possèdent pas de valeur, mais une clé.

Pour vous représenter cela, imaginez que chaque valeur soit une définition (un string) et que chaque clé associée soit un mot. Pour obtenir la définition d'un mot, vous faites appel au dictionnaire en précisant le mot-clé.

La création et l'ajout d'élément sont très simples.

```
1. >>>mon_dico = {} #creation d'un dictionnaire vide
2. >>>mon_dico [" PYTHON "] = " langage informatique "
3. >>>mon_dico [" voiture "] = " vehicule automobile "
4. >>>mon_dico [" PYTHON "]
5. " langage informatique "
6. >>>mon_dico
7. {" voiture ": " vehicule automobile ", " PYTHON ": " langage informatique&#
```

Comme on peut le voir sur cet exemple pour créer une entrée il suffit de la déclarer dans le dictionnaire, tel ligne 2 ou 3. Pour lire une valeur, il suffit de préciser la clé au dictionnaire, sinon, il affichera la totalité de ses entrées.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

IV-A-7-a - Méthodes

Pour supprimer une entrée, on utilise la méthode **pop** qui renvoie la valeur liée à la clé transmise :

```
>>>mon_dico.pop(" voiture ")
vehicule automobile
```

IV-A-8 - Tuple

Un **tuple** est comparable à une liste, à une exception : on utilise des parenthèses et non des crochets lors de la définition

```
>>>mon_tuple1 = (1,)         #tuple à un parametre (virgule obligatoire)
>>>mon_tuple2 = (2, 3)     #tuple à deux parametres
>>>mon_tuple3 = (4, 5, 6)     #tuple à trois parametres
```

IV-B - Le transtypage

Le **transtypage** correspond à un changement de type de variable, comme transformer une variable de int en string; ou encore de format (binaire, hexa…)

Le transtypage est très pratique, surtout quand l'on doit afficher un nombre dans une chaîne de caractères par exemple ou encore effectuer un travail sur un type donné de variables.

En langage anglophone, il s'agit d'une opération dite **cast**.

IV-B-1 - Transtypage de type

Pour effectuer un transtypage de type en **PYTHON**, il suffit de taper le type désiré puis la variable à transtyper entre parenthèses :

```
>>> a
2
>>> str(a)
'2'
>>> float(a)
2.0
>>> int(a)
2
```

IV-B-2 - Transtypage de format

Pour effectuer un transtypage de format en **PYTHON**, il suffit de taper le format désiré puis la variable à transtyper entre parenthèses :

```
>>> a
2
>>> bin(a)
'0b10'
>>> hex(a)
'0x2'
```

IV-B-3 - Détection de la plateforme d'exécution

Il peut parfois être pratique de connaître l'OS sur lequel tourne le script.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Pour cela il faut passer par une commande du module **sys :**

```
>>sys.platform
'linux2'
```

IV-C - La portée des variables

La **portée des variables** est une notion importante dans la programmation.

En **PYTHON**, tout comme dans beaucoup d'autres langages, une variable peut être locale ou globale.

Dans le premier cas, la variable n'existe qu'à l'intérieur de la fonction(/procédure/…) où elle a été définie. Même si ailleurs dans le code une variable porte le même nom, il s'agira néanmoins de deux variables distinctes.

Cependant, il peut parfois être utile, même si cela n'est pas recommandé, d'avoir une variable globale, autrement dit, accessible depuis n'importe où dans le code.

Pour qu'une variable soit globale en **PYTHON**, il faut la définir au début du code en utilisant le mot-clé **global**.

De même au début de chaque fonction(/procédure /…), il faudra redéfinir cette variable en globale pour que **PYTHON** comprenne que l'on veut faire référence à la variable globale et non à une variable locale portant le même nom.

```
global ma_variable
&#8230;
def ma_procedure():
    global ma_variable #j'appelle ici la variable globale
    &#8230;
&#8230;
def ma_procedure2():
    ma_variable = 3 #ici, c'est une variable locale
```

IV-D - Quelques fonctions prédéfinies

IV-D-1 - PRINT

La fonction **print** n'est utile qu'en mode ligne de commande. Cette fonction permet d'afficher du texte, ou des variables, sur l'écran

```
>>>mon_texte = " hello world !!! "
>>>print mon_texte
hello world !!!
>>>print " la variable mon_texte vaut: ", mon_texte
la variable mon_texte vaut: hello world !!!
```

Noter dans le dernier exemple la présence de la virgule. Elle permet de préciser à **PYTHON** qu'il faut afficher les données à la suite et non à la ligne.

IV-D-2 - LEN

La commande **len** permet de connaître le nombre de caractères dans un string ou encore le nombre d'éléments dans une liste

```
>>>ma_string = " bonjour "
>>>len(ma_string)
7
>>>ma_liste = [4,5," hello "]
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
>>>len(ma_liste)
3
```

IV-D-3 - TYPE

La commande **type** permet de connaître le type d'une variable. Cette fonction est souvent utilisée afin de déterminer quel traitement est le plus adapté à une variable donnée.

```
>>> mon_integer = 56321
>>> type(mon_integer)
<type 'int'>
```

IV-D-4 - INPUT

La fonction **input** permet, en ligne de commande, de demander à l'utilisateur de renseigner des paramètres ou informations.

Il existe deux façons d'utiliser cette fonction :

```
>>>#Methode 1
>>>print " Merci de renseigner votre nom: "
>>>nom = input()
>>>print " votre nom est: ", nom
>>>
>>>#Methode 2
>>>nom = input(" Merci de renseigner votre nom: ")
>>>print " votre nom est: ", nom
```

Remarque : la fonction input renvoie ce que saisit l'utilisateur. C'est-à-dire que si l'utilisateur saisit un entier à la place de son nom, cela posera problème. Pour cette raison, il peut sembler utile d'utiliser la fonction raw_input qui renvoie systématiquement la saisie de l'utilisateur, convertie en string.

IV-D-5 - GETPASS

Le rôle de la fonction **getpass** est identique à celui de la fonction input à un détail près : la confidentialité.

En effet, cette fonction ne réalise pas d'écho de votre saisie.

```
>>>mot_passe = getpass (" Merci de saisir un mot de passe:")
Merci de saisir un mot de passe:
>>>print mot_passe
MotDePasse
```

IV-E - Le caractère de césure

Le caractère de césure permet d'écrire sur plusieurs lignes une instruction qui serait trop longue à écrire sur une seule ligne. Il s'agit du caractère ****

Lorsque l'on utilise le caractère de césure, on effectue la coupure après un opérateur, jamais avant, pour des questions de lisibilité.

De plus, l'utilisation de ce caractère de césure fait que les tabulations ne sont plus prises en compte. Aussi faut-il faire bien attention à faire en sorte que le code reste propre et lisible.

```
>>> ma_string = "texte \
ecrit sur \
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
trois lignes"
>>> ma_string
'texte ecrit sur trois lignes'
```

IV-F - Le caractère de commentaire

En **PYTHON**, il n'existe qu'un seul caractère pour écrire un commentaire. Il s'agit du symbole **#**.

Aussi pour écrire vos commentaires sur plusieurs lignes, il faut faire commencer chaque ligne par ce symbole.

Cependant, les éditeurs de code modernes, tel **GEANY**, vous permettent d'écrire l'intégralité de votre commentaire comme un texte normal, avant de le sélectionner et de demander à commenter la sélection.

Remarque : pour éviter tout problème lors de l'exécution du code, je vous conseille de prendre l'habitude de ne jamais utiliser de caractères spéciaux (accents par exemple) dans vos commentaires.

IV-G - Les opérateurs

Il existe un certain nombre d'astuces pour le codage.

Ainsi au lieu d'écrire ma_variable = ma_variable + 1, nous pouvons écrire ma_variable += 1. De même existe -=, *=, /=.

Il est également possible d'écrire a = b = 1 au lieu de a = 1 et ensuite b = 1.

Plus subtil, au lieu de a = 2 puis b = 3, nous pouvons saisir a, b = 2, 3. Cela ouvre la porte aux permutations rapides comme a, b = b, a.

Pour le reste, les opérateurs sont classiques :

Paramètre	Description
+	Addition
-	Soustraction
*	Multiplication
/	Division (avec chiffre après le symbole des décimales)
//	Partie entière d'une division
%	Reste d'une division (modulo)
**	Puissance
==	Test d'égalité
<>	Test de différence
>=	Test supérieur ou égal
<=	Test inférieur ou égal
>	Test supérieur à
<	Test inférieur à

Il existe en plus de cela les instructions **and**, **or** et **not** qui peuvent servir dans des tests.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

IV-H - Les tests conditionnels

IV-H-1 - IF, ELIF, ELSE

La forme complète d'une boucle conditionnelle **if** est la suivante :

```
if a > 0:
    #code 1
elif a < 0:
    #code 2
else: #a vaut 0
    #code 3
```

Remarque : les conditions de test placées entre if et « : » sont appelées prédicats.

IV-I - Les boucles

IV-I-1 - FOR

La boucle **for** en **PYTHON** n'a pas le même fonctionnement que dans d'autres langages tels le C.

En effet, l'instruction for permet ici de parcourir une variable.

```
>>>ma_chaine = " hello "
>>>for letter in ma_chaine:
       print letter
```

Ce code aura pour effet d'afficher sur l'écran les lettres de ma_chaîne les unes après les autres.

Concrètement, cela revient au fait que letter parcourt ma_chaîne depuis son index 0 jusqu'à la fin.

Cela peut être très utile comme avec des listes :

```
>>>for jour in jour_ouvre:
       print jour
```

Cette façon de faire n'est pas cependant la plus adéquate. On lui préfère alors ceci :

```
>>>for index, jour in enumerate(jour_ouvre):
          print index, jour
```

La fonction **enumerate** prend comme paramètre une liste et nous renvoie un tuple contenant son index et la valeur de l'index. Pour le constater, il suffit de n'indiquer qu'une variable à la place de deux après le for et de l'imprimer.

Pour les dictionnaires, c'est approximativement le même principe :

```
>>>for cle, valeur in mon_dico.items():
       print cle, valeur
```

Cependant, à la place de **.items()**, vous pouvez utiliser **.keys()** pour les clés ou **.values()** pour les valeurs.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

IV-I-2 - WHILE

La boucle **while**, comme dans tout autre langage permet d'effectuer des opérations tant que la condition de la boucle est remplie

```
>>>while a <> 0:
       #code
```

IV-I-3 - Break et continue

Dans les **for** et **while**, il existe deux mots-clés qui peuvent servir occasionnellement.

Le mot **break** permet d'interrompre une boucle quelle que soit sa condition.

Le mot-clé **continue** revient à faire un saut directement de l'endroit où il est codé au **for** ou **while**, et ce sans exécuter le code qui aurait pu rester en dessous.

IV-J - PYTHON et les fichiers

La gestion d'un fichier avec **PYTHON** est extrêmement simple

IV-J-1 - Chemin absolu et chemin relatif

La distinction entre ces deux types est très importante. En effet, lors de la programmation **PYTHON**, vous serez amené à utiliser les deux.

Le **chemin absolu** est le chemin complet (par exemple : c:\windows\solitaire.exe). Ce type de chemin est à utiliser par exemple pour accéder en ouverture/écriture à un fichier utilisateur.

Le **chemin relatif**, lui, part de la position du code exécuté. « ./ » correspond alors au dossier en cours, et « ../ » au dossier parent.

Ce type de chemin est pratique pour accéder à des fichiers de configuration du logiciel qui seraient stockés dans le même dossier ou dans un sous-dossier.

Attention : sous Windows on utilise des antislash dans les chemins, mais sous Linux/UNIX, on utilise des slashs. La notation des chemins relatifs utilisée ici par exemple est la notation Linux.

IV-J-2 - Ouverture d'un fichier

Avant d'écrire ou de lire un fichier, il faut l'ouvrir. Pour cela, nous avons besoin d'un chemin d'accès contenant le nom du fichier, ainsi que de son mode d'accès :

Paramètre	Description
r	Lecture seule
w	Écriture seule
a	Mode ajout (append). On complète le fichier

On peut également écrire **rb**, **wb** ou **ab** pour signifier que nous n'accédons pas au fichier en mode ASCII, mais en mode binaire.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Pour ouvrir un fichier, on exécute la commande suivante :

```
mon_fichier = open("CHEMIN ", " MODE ")
```

Ceci est la méthode la plus simple, mais pas la plus sécurisée. En effet, si jamais votre logiciel crash pour une raison ou pour une autre, alors le fichier que vous aviez ouvert risquerait de devenir inutilisable. Pour éviter cela, nous utilisons le mot-clé **« with » :**

```
with open(" ./config.txt ", w) as fichier_config:
    #Code
```

Cette dernière façon de faire est la meilleure d'un point de vue sécurité. Avec cette méthode, même en cas de crash, le fichier sera fermé proprement.

IV-J-3 - Fermeture d'un fichier

La fermeture d'un fichier est on ne peut plus simple. Il suffit d'écrire :

```
mon_fichier.close()
```

Pour vérifier si le fichier est bien fermé, vous pouvez tester **mon_fichier.closed**. Il s'agit d'un **booléen.**

IV-J-4 - Lecture

Il existe différentes façons de lire un fichier. Nous verrons ici les deux principales.

Tout d'abord lire l'intégralité d'un fichier :

```
contenu = mon_fichier.read()
```

La variable contenue sera ainsi de type **string**, et contiendra une chaîne de caractères qui sera en réalité l'intégralité du contenu de mon_fichier.

Cela peut être utile pour analyser des données par exemple ou pour rechercher une information précise. Cette fonction peut également prendre en paramètre un nombre qui correspond au nombre de caractères que l'on désire lire.

À noter l'instruction **readlines** produit sensiblement le même effet, à savoir la lecture intégrale du fichier. Cependant, readlines fait la distinction entre les différentes lignes. Ainsi, il est possible d'effectuer le code suivant :

```
f = open('myfile.txt','r')
for line in f.readlines():
    print line
f.close()
```

L'autre façon de faire est de lire ligne par ligne avec l'instruction **readline** (remarquer l'absence de « s » final). Cette instruction est à utiliser dans une boucle **while**. Quand un **readline** a atteint la fin de fichier, line vaut alors "". C'est la condition de sortie de la boucle**.**

```
f = open('myfile','r')
f_line = f.readline()
while f_line <> "":
    &#8230;        #code
f_line = f.readline()
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Attention : quand vous lisez un fichier, assurez-vous de connaître sa taille précise afin de ne pas avoir un dépassement de mémoire. Ce dépassement de mémoire peut être évité de manière sure avec la méthode readline, mais cette dernière est beaucoup plus lente que la méthode readlines. Pour connaître la taille d'un fichier, il faut utiliser la méthode os.path.getsize(" chemin_fichier ") du module os.

IV-J-5 - Écriture

Pour écrire dans un fichier nous utilisons la fonction **write**.

```
mon_fichier.write(" HelloWorld \n ")
```

Attention toutefois car la fonction write n'accepte que des chaînes de caractères. N'oubliez donc pas de faire des transtypages si besoin.

IV-K - La POO PYTHON

IV-K-1 - PYTHON et les principes de la POO

IV-K-1-a - L'encapsulation

L'encapsulation côté **PYTHON** est un peu particulière. La notion de privé ou public de certains langages est inconnue.

En effet, **PYTHON** privilégie le bon sens à la répression. En clair, nous partons du principe que si l'auteur du code indique qu'une variable ne doit pas être accédée depuis l'extérieur du module, alors l'utilisateur ne devra pas chercher à y accéder.

Il est cependant possible de ruser pour faire en sorte que l'utilisateur soit automatiquement redirigé en cas de tentative d'accès non correcte. On utilisera pour cela les **accesseurs** et les **mutateurs.**

IV-K-1-b - L'héritage

En **PYTHON**, voici comment se réalise un **héritage** de classe.

```
>>>class MaClasse1:
    #code1
>>>class MaClasse2(Maclasse1):
    #code2
```

PYTHON respecte l'ensemble des principes de l'héritage. MaClasse2 héritera donc de l'ensemble des caractéristiques de MaClasse1.

IV-K-2 - La modularité avec PYTHON

IV-K-2-a - L'instruction import

La fonction **import** permet de préciser à **PYTHON** que vous allez utiliser du code externe et où le trouver. Cas échéant, vous pouvez également préciser quelles fonctions vous désirez importer (**from…import…**)

```
import math #importe le module de mathematique
from math import sqrt #importe uniquement la fonction sqrt
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

La question qui peut se poser ici est l'utilité d'un code tel celui ligne 2. En réalité, il trouve toute son utilité dans des systèmes possédant peu de ressources. Cela permet de contrôler au mieux notre consommation des ressources hôte.

IV-K-2-b - L'instruction SELF

L'instruction **self** est très importante en **PYTHON**. En effet, il indique au langage que vous faites référence à l'objet que vous utilisez et non à la classe mère.

Par exemple, prenons deux objets : une Twingo et une A6. Ce sont tous les deux des objets créés à partir de la classe mère voiture. Sans l'utilisation du self, le fait de changer une propriété sur l'objet Twingo n'affecterait pas cet objet, mais directement la classe mère voiture.

Ainsi dans une classe, le premier et/ou seul paramètre des fonctions/procédures sera toujours self. De même, les variables dans une classe devront être précédées de self. On parle alors d'attributs de l'objet.

La notion de self sera précisée par des exemples lors de l'explication des classes ci-après.

IV-K-2-c - Les fonctions et les procédures

Les **fonctions** et les **procédures** fonctionnent sur le même principe. La différence réside dans le fait qu'une fonction renvoie un résultat, une procédure ne renvoie rien.

Pour définir une fonction ou une procédure, nous utilisons simplement le mot-clé **def :**

```
>>>def ma_fonction():
    print " Hello World !!! "
    return True
```

Remarque : les parenthèses sont obligatoires, même si aucun argument n'est passé.

Nous pouvons voir ici que nous retournons un booléen. On utilise pour cela le mot-clé **return**. Si nous désirons renvoyer plusieurs variables, il suffit de les écrire à la suite et de les récupérer dans des variables adaptées :

```
>>> def ma_fonction(fnom, fprenom):
    return "nom: " + fnom, "prenom: "+fprenom
>>> nom, prenom = ma_fonction("DUPONT", "Jean")
>>> print nom
nom: DUPONT
>>> print prenom
prenom: Jean
```

Il est également possible de définir une ou plusieurs valeurs par défaut :

```
>>> def ma_fonction(fnom, fprenom= 'Jean'):
    return "nom: " + fnom, "prenom: "+fprenom
>>> nom, prenom = ma_fonction("DUPONT")
>>> print nom
nom: DUPONT
>>> print prenom
prenom: Jean
```

Remarque : quand certains paramètres n'ont pas de valeur par défaut, ces paramètres doivent être placés avant ceux possédant une valeur par défaut.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

IV-K-2-d - Les classes

Lorsque nous allons créer un objet, nous nous référerons en fait à une **classe**. Pour reprendre la définition donnée précédemment, une classe est une brique logicielle de référence.

Pour déclarer une classe en **PYTHON**, on utilise le mot-clé **class**. La définition d'une classe doit s'accompagner de sa méthode constructeur :

```
class MaClasse:
    def __init__(self, param1, param2, &#8230;, paramn):
        #code
```

Le rôle de ce constructeur est de créer un objet copie de la classe lorsque nous définissons un nouvel objet.

Pour créer un objet depuis une classe, il suffit de l'appeler :

```
>>>mon objet = Maclasse()
```

IV-K-2-d-i - Les attributs

Il est possible de déclarer des **attributs** propres non pas à l'objet (avec l'utilisation du self), mais à la classe.

```
class MaClasse:
    mon_compteur = 0
    def __init__(self, param1, param2, &#8230;, paramn):
        MaClasse.mon_compteur += 1
        #code
```

Les attributs ne sont ni plus ni moins que les variables de notre objet. Selon le principe d'encapsulation, nous ne devons pas avoir un accès direct à ces variables.

Bien que **PYTHON** autorise cet accès direct de par sa philosophie (rappel : en **PYTHON**, tout est public), il est recommandé, lors de la création d'une classe de créer des méthodes d'accès en lecture et en écriture sur chaque attribut de la classe.

Elles permettent entre autres, outre le fait de rajouter de la sécurité et ainsi de respecter au mieux le principe d'encapsulation, de pouvoir exécuter, de façon totalement transparente pour l'utilisateur, du code complémentaire.

Ces méthodes dédiées et spécifiques sont ce qu'on appelle des **accesseurs** (lecture) et des **mutateurs** (écriture).

IV-K-2-d-ii - Les accesseurs

Les **accesseurs** se présentent toujours sous la même forme

```
_get_ATTRIBUT()
```

En **PYTHON**, déclarer un accesseur se fait de la façon suivante :

```
def _get_ATTRIBUT(self):
    """Accesseur de l'attribut"""
    #Code, finissant souvent par " return self._ATTRIBUT "
```

Remarque : notez le underscore en sus dans le _get_ATTRIBUT_ et le self._ATTRIBUT. Il a son importance que nous verrons dans l'exemple final

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

IV-K-2-d-iii - Les mutateurs

Tout comme les accesseurs, les **mutateurs** présentent également toujours la même forme :

```
_set_ATTRIBUT()
```

En **PYTHON**, nous déclarons un mutateur de la façon suivante :

```
def _set_ATTRIBUT(self, paramètre):
    """Mutateur de l'attribut"""
    #code, finissant souvent par " self._ATTRIBUT = new_value "
```

Remarque : notez bien ici aussi la présence de l'underscore.

IV-K-2-d-iv - Mise en situation

Dans cet exemple, nous allons mettre en situation une classe simplifiée « Twingo » et son attribut « phares ». Cet exemple va nous permettre de présenter les derniers éléments nécessaires à l'implémentation des accesseurs et des mutateurs.

```
class Twingo:
    """Classe Twingo simplifiée possédant un seul attribut: phares"""

    def __init__(self):
        """Constructeur de la classe Twingo"""
        self._phares = False #Phares éteints par défaut


    def _get_phares(self):
        """Accesseur de l'attribut phares, permet de récupérer l'état des phares'"""
        if self._phares:
            return 'ON'
        else:
            return 'OFF'


    def _set_phares(self, on_off):
        """Mutateur de phares, permet de changer l'etat"""
        if on_off = 'ON':
            self._phares = True
        else:
            self._phares = False
        phares = property(_get_phares, _set_phares)
```

Analysons cet exemple de plus près. Vous aurez reconnu maintenant les accesseurs et les mutateurs. Aussi allons nous nous attarder sur les underscores en sus, et sur la dernière ligne.

Les underscores en sus n'influent en rien sur le fonctionnement du programme. Ils servent juste à distinguer visuellement les éléments auxquels on peut, ou non, accéder depuis l'extérieur. Il s'agit uniquement d'une convention de codage.

Bien entendu, comme rappelé plus tôt, **PYTHON** n'effectuant aucune restriction ou contrôle à ce niveau, c'est à vous de ne pas chercher à passer outre les recommandations du programmeur.

Dans notre exemple, les attributs _phares et phares sont donc deux variables différentes.

_phares ne doit pas être accédé depuis l'extérieur. À la place nous passerons par l'attribut phares, ce qui nous amène à la dernière ligne.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Nous y utilisons le mot-clé **property**. Ce mot-clé permet d'indiquer à **PYTHON** pour l'attribut concerné l'accesseur (1er paramètre) et le mutateur (2d paramètre) qui lui sont liés : ***property(accesseur, mutateur).***

Grâce à cette dernière ligne, et au principe d'accesseur et mutateur, alors que l'utilisateur pense accéder directement à l'attribut, il passe en réalité par un ensemble de procédures/fonctions de manière totalement transparente.

IV-K-2-e - Les modules

En **PYTHON**, un **module** est un fichier contenant différents codes (fonctions, procédures, classes…) ayant un lien entre eux.

Pour utiliser le code contenu dans un module, on doit l'importer avec le mot-clé **import**. Cependant, lorsque nous développons un module, il est recommandé de créer la fonction de test du module.

Cette fonction de test a pour but de réaliser un ensemble d'instructions utilisant le code défini dans le module afin de le tester sans avoir besoin d'utiliser un logiciel dédié.

Cette fonction se déclare ainsi :

```
if __name__ == " __main__ ":
```

La variable **__name__** est une variable système initialisée au lancement de l'interpréteur **PYTHON**. Si cette dernière vaut **__main__**, cela signifie que le fichier appelé est le fichier exécuté.

IV-K-2-f - Les packages

Les **packages** sont le niveau supérieur des modules. Ils permettent donc de regrouper plusieurs modules.

Dans la pratique, un package n'est rien d'autre qu'un dossier. Ces dossiers peuvent contenir d'autres dossiers (package) ou d'autres fichiers (module).

Tout comme pour les modules, pour les utiliser dans un code, il faut utiliser le mot-clé import. Pour utiliser ensuite un sous-package ou un module du package, on utilise le point « . » afin de modéliser le chemin menant à la fonction ou la procédure que nous désirons utiliser.

IV-K-2-f-i - Composition d'un package

Un package **PYTHON** n'est jamais vide. En effet, chaque package (et sous-package, quel que soit le niveau) doit contenir outre les modules qui s'y trouvent, un fichier **__init__.py**.

Ce fichier **__init__.py** est la plupart du temps totalement vide. Il est surtout là pour indiquer à **PYTHON** qu'il ne s'agit pas d'un dossier classique, au sens répertoire d'un OS, mais d'un package **PYTHON**.

Si vous en avez l'utilité, vous pouvez par exemple écrire à l'intérieur de ce fichier **__init__.py**, du code qui sera exécuté à l'import du package.

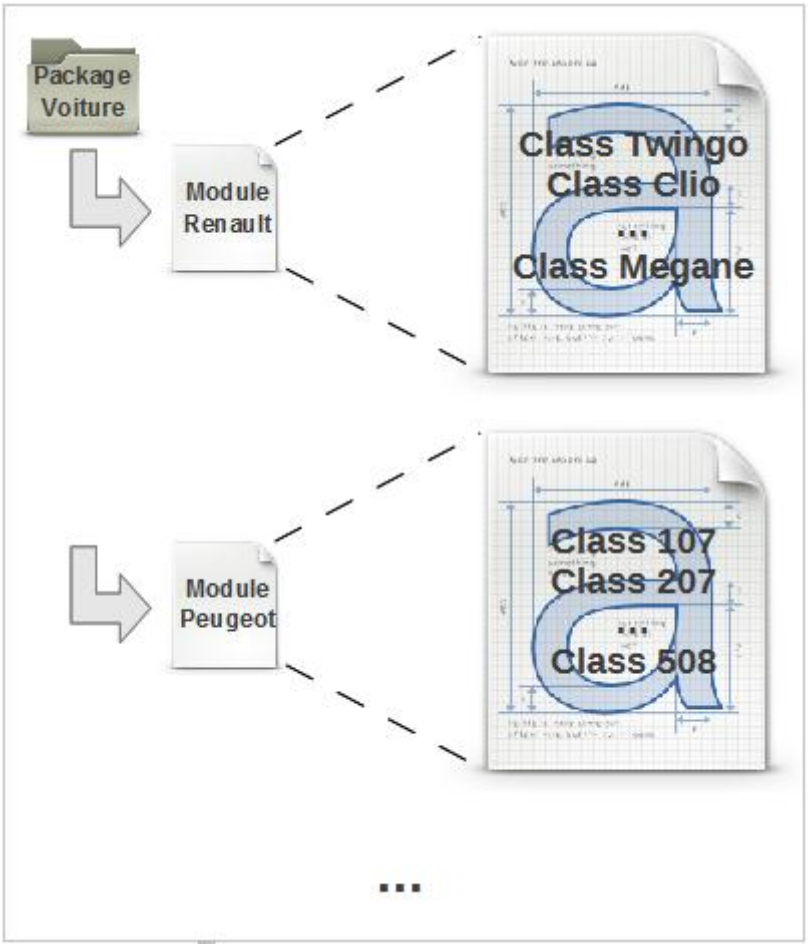
Enfin, vous pouvez y placer une simple **docstring** afin de documenter le package (sous-package…). Cette documentation permettra une future maintenance que ce soit par vous-même ou par un tiers (voir).

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

IV-K-2-g - En résumé

Pour reprendre l'exemple des voitures, nous pourrions avoir un package voiture, puis un module par marque (Renaud, Jugeote…), puis une classe par modèle de voiture de la marque (Clio, 504, DS5…).

Chacune de ces classes, représentant une voiture, aurait alors des propriétés et des méthodes qui lui sont propres.



Principe de Package/Module/Classe

IV-K-3 - Stockage d'objet dans des fichiers

Comme dans beaucoup de langages objet, en **PYTHON** nous pouvons enregistrer nos **objets** dans des fichiers pour nous en resservir plus tard.

Pour cela, nous utiliserons le module **pickle**. À l'intérieur de ce module, nous utiliserons deux classes : **Pickler** et **Unpickler**.

IV-K-3-a - Lecture

Après avoir ouvert un fichier en écriture, les commandes types à utiliser sont :

```
mon_pickler = pickle.Pickler(fichier) #Cree le pickler dans le fichier
mon_pickler.dump(objet) #stocke l'objet dans le pickler
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

IV-K-3-b - Enregistrement

Après avoir ouvert un fichier en lecture, nous utiliserons une commande de type :

```
mon_unpickler = pickle.Unpickler(fichier) #recupere le pickler dans fichier
mon_objet = mon_unpickler.load()
```

IV-L - Les expressions régulières

Les **expressions régulières**, aussi appelées **REGEX** pour REGular EXpressions, sont très utilisées dans le monde de l'informatique. Elles permettent notamment de s'assurer qu'une saisie est conforme aux attentes et besoins du programme informatique.

Elles permettent de définir la structure d'une donnée attendue ou recherchée.

Le fonctionnement d'une **REGEX** est plus ou moins normalisé. Ci-dessous un résumé des éléments pouvant composer une **REGEX**.

Paramètre	Description
^	Début de chaîne
$	Fin de chaîne
.	Tout caractère sauf retour à la ligne
*****	Nombre d'occurrences indifférent (0 compris)
+	Nombre d'occurrences indifférent (0 exclu)
?	Nombre d'occurrences égal à 0 ou 1
|	OU logique
( )	Groupe avec fonction équivalente aux parenthèses en équation mathématique
[ ]	Intervalle
{ }	Répétition

Paramètre	Description
A{n}	A apparaît exactement n fois
A{n,}	A apparaît au moins n fois
A{,n}	A apparaît n fois maximum

Côté fonctionnement, on commence par indiquer ce que l'on attend, puis on indique le nombre de fois désiré. Par exemple, si on attend quatre caractères entre 0 et F, alors on saisira **[0-9A-F]{4}**.

Remarque : nous avons indiqué ici deux intervalles de caractères potentiels. Le OU logique est ici implicite.

Ainsi, si l'on attend la saisie d'un numéro de téléphone tout attaché (soit 10 chiffres), la **REGEX** sera **^[0-9]{4}$**. Si l'on désire une forme avec des tirets, alors ce sera **^([-][0-9]){4}$**.

De même, si on attend la saisie d'une adresse mail de notre société, comme identifiant par exemple, et que l'extension est en @societe.fr, alors nous pourrons saisir comme **REGEX** **^.{1,}(@societe\.fr){1}$**

Remarque : on notera ici la présence de " \ ", caractère d'échappement pour les caractères spécifiques REGEX.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

IV-L-1 - Le module re

En **PYTHON**, nous avons la possibilité d'utiliser ces **REGEX**. Et pour nous faciliter le travail, il existe le **module re**.

Ce module met à notre disposition les méthodes **search** et **sub**.

IV-L-1-a - Search

La première de ces méthodes, **search**, permet de rechercher une REGEX dans une chaîne de caractères.

```
>>> import re
>>> test = "ma_chaine"
>>> re.search("_cha", test)
<_sre.SRE_Match object at 0xa670870>
>>> re.search("toto", test)
>>>
```

Comme on peut le constater, rien de très compliqué. Il faut deux paramètres : le premier la **REGEX**, et le second la chaîne où l'on désire chercher. Si une occurrence est trouvée, re.search nous renvoie une expression. Si aucune occurrence n'est trouvée, alors **re.search** nous renvoie None.

IV-L-1-b - Sub

La méthode **sub** permet, elle, d'effectuer un remplacement dans une chaîne de caractères.

Il s'agit ni plus ni moins d'une fonction rechercher/remplacer, mais en plus brut.

```
>>> mail = "toto@masociete.fr"
>>> re.search("^.{1,}(@masociete\.fr){1}$", mail)
<_sre.SRE_Match object at 0xa602860>
>>> re.search("^.{1}(@masociete\.fr){1}$", mail)
>>> re.sub("(@masociete\.fr){1}", "@societe.com", mail)
'toto@societe.com'
```

Le premier argument de **re.sub** est la **REGEX** recherchée, le second la chaîne de remplacement, et le troisième la chaîne où cherche la **REGEX** et effectue le remplacement.

Nous pouvons ici constater dans cet exemple que la méthode **re.sub** renvoie le résultat de la substitution.

IV-M - Les exceptions

Les **exceptions** permettent d'intercepter une erreur dans un code (telle une division par zéro) et de réaliser une action donnée. Pour cela, nous utilisons **try:… except:… finally: …**

```
try:
    #code1 a tester
except:
    #code2  execute en cas d'erreur
finally:
    #code3 execute quoiqu'il arrive à la fin du code1
```

Dans cet exemple, le mot-clé **finally** est utilisé. Dans la réalité, ce mot-clé sert assez peu.

Après **except**, nous pouvons trouver un type d'erreur donné prédéfini tel **ZeroDivisionError**, ou encore un type d'erreur prédéfini personnalisé par nos soins et déclenché via l'instruction **raise:**

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
1. >>> def ma_div(num, denom):
2.     try:
3.         div = num / denom
4.         if num % 2 is 0:
5.             raise ValueError("numerateur impair obligatoire")
6.     except ZeroDivisionError:
7.         print "Division par zero"
8.     except ValueError:
9.         print "numerateur pair"
10.     except:
11.         print "une erreur a eu lieu"
12.
13. >>> ma_div(2,0)
14. Division par zero
15. >>> ma_div(2,1)
16. numerateur pair
17. >>> ma_div("r",2)
18. une erreur a eu lieu
```

Regardons cet exemple de plus près. Si une division par 0 a lieu, alors elle est interceptée par le type prédéfini en ligne 6. Si le numérateur est pair, alors cela déclenche une erreur interceptée par un type que nous avons défini en ligne 8, sur le type d'erreur prédéfini. Enfin, si une erreur a lieu et ne correspond à aucune de celle précisée, alors elle est capturée ligne 10.

Les exceptions peuvent être utilisées par exemple pour le débogage ou pour des sections de code à risque. Tout code **PYTHON** doit comporter un minimum de ces exceptions afin de s'assurer que les erreurs ne passent pas inaperçues/ silencieusement.

IV-N - Les mots réservés

Il existe une série de mots réservés par **PYTHON** que nous ne pouvons jamais utiliser en tant que noms de variables, de classes ou autres.

and	as	assert	break	class	continue	def	del	elif	else	except
False	finally	for	from	global	import	if	in	is	lambda	none
nonloca	not	or	pass	raise	return	True	try	while	with	yield

IV-O - Convention de programmation

IV-O-1 - La PEP20

Le rôle de la **PEP 20** (PEP pour **PYTHON** Enhancement Proposal : proposition d'amélioration de **PYTHON**) est de donner des directives pour coder de la meilleure façon possible.

Énoncée sous forme d'aphorismes plus ou moins compréhensibles, cette PEP est une des briques de base pour tout programmeur **PYTHON**. Elle concerne cependant surtout l'aspect du code.

> Le beau est préférable au laid.

> L'explicite est préférable à l'implicite.

> Le simple est préférable au complexe.

> Le complexe est préférable au compliqué.

> Du code trop imbriqué est plus difficile à lire.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

> L'aéré est préférable au compact.

> La lisibilité compte.

> Les cas particuliers ne sont pas suffisamment particuliers pour casser la règle.

> Il est difficile de faire un code à la fois fonctionnel et « pur ».

> Les erreurs ne devraient jamais passer silencieusement, à moins qu'elles n'aient été explicitement réduites au silence.

> En cas d'ambiguïté, résistez à la tentation de deviner.

> Il devrait exister une et une seule manière évidente de procéder, même si cette manière n'est pas forcément évidente au premier abord, à moins que vous ne soyez Néerlandais (humour : l 'inventeur du PYTHON est Néerlandais).

> Maintenant est préférable à jamais, mais jamais est parfois préférable à immédiatement.

> Si la mise en œuvre est difficile à expliquer, c'est une mauvaise idée. S i la mise en œuvre est facile à expliquer, ce peut être une bonne idée. Les espaces de noms sont une très bonne idée.

IV-O-2 - La PEP8

Le rôle de la **PEP 8** est de donner des directives claires quant à la manière même de rédiger le code. Une fois de plus, ce ne sont cependant que des conseils que vous êtes libre ou non de suivre.

Ci-dessous, une traduction française de ces conseils.

> Une indentation doit équivaloir à quatre espaces (c onfigurez la touche tab).

> Il ne faut jamais mélanger espaces et indentations dans le même code.

> Une ligne ne doit excéder 79 caractères.

> La définition d'une fonction, classe ou autre doit être suivie de deux sauts de lignes.

> Il faut utiliser un import par package.

> Ces imports doivent toujours être en début de code.

> Ils doivent être répartis en trois groupes dans l'ordre suivant, séparés par un saut de ligne

les bibliothèques standards ;

les bibliothèques tierces ;

les bibliothèques « maison ».

> Toujours utiliser des chemins absolus pour l'import de modules.

> Toujours utiliser un espace avant et après un opérateur.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

> Une seule instruction par ligne.

IV-O-3 - Règles de codage

L'ensemble des règles énoncées ci-après ne constitue en rien une obligation, mais uniquement de fortes recommandations, qui sont respectées par de nombreux programmeurs.

Le respect de ces **règles de codage** facilite autant le codage que la future maintenance potentielle par des tiers.

IV-O-3-a - Les variables

Le nom des variables ne peut pas commencer par un chiffre. Il ne doit être constitué que de lettres minuscules et les différents mots séparés par des underscores « _ ».

IV-O-3-b - Les fonctions/procédures

Les règles de nommage des fonctions et des procédures sont identiques à celles des variables.

IV-O-3-c - Les modules et packages

Les noms des modules et des packages doivent être courts et constitués uniquement de lettres minuscules.

De préférence, il faut éviter d'utiliser des underscores et n'avoir un nom ne tenant qu'en un mot, surtout pour les packages.

IV-O-3-d - Les classes

IV-O-3-d-i - Le nom des classes

Le nom d'une classe se compose d'un ensemble de mots, collés les uns aux autres, avec la première lettre de chaque mot en majuscule. Exemple : ClassName.

IV-O-3-d-ii - Les propriétés et les méthodes

On utilise les mêmes règles que pour les variables.

IV-O-3-e - Les exceptions

Les règles de nommage des exceptions suivent les mêmes règles de nommage que les classes.

IV-O-3-f - Les DocStrings

En **PYTHON**, juste après la définition d'une fonction, procédure, classe, ou module, il faut insérer ce qu'on appelle une **docstring**.

Entourée de triples doubles-quotes, elle donne une description du code que nous écrivons, et peut s'écrire sur plusieurs lignes sans utiliser le caractère de césure. Si vous tapez **help**(fonction) c'est cette **docstring** que vous verrez apparaître.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
>>> def ma_fonction(a):
    """Multiplie a par 2"""
    print a*a

>>> help(ma_fonction)
Help on function ma_fonction in module __main__:

>>> ma_fonction(a)
Multiplie a par 2

>>> ma_fonction(2)
4
```

Remarque : mettre une docstring bilingue est souvent apprécié dans les grandes entreprises.

IV-O-3-g - Début de code

Au début de chaque code, il est important de préciser certaines informations, surtout sur Linux.

Ces informations occupant systématiquement les deux premières sont le chemin de l'interpréteur, ainsi que l'encodage utilisé.

```
#! /usr/bin/env PYTHON
# -*- coding: utf-8 -*-
```

La première ligne indique à l'OS où trouver l'interpréteur **PYTHON** afin de traduire le code. Cela n'est à faire que sous LINUX.

La seconde ligne, non obligatoire, quoique vivement conseillée, permet de préciser à l'interpréteur le type d'**encodage** que nous utilisons pour le code. En effet, par défaut, les accents ne sont pas autorisés.

Remarque : ici nous avons précisé un encodage ISO. Ce n'est pas le seul puisqu'il existe également l'encodage LATIN1, UTF-8…

IV-O-3-h - Sortie de code

Vous l'avez peut-être parfois remarqué lors de l'exécution d'un code en ligne de commande, un petit « **exit code: 0 ».**

En fait, il s'agit pour l'OS de vous indiquer si votre traitement s'est bien déroulé (0) ou non (1).

En **PYTHON**, on utilise l'instruction **sys.exit(1)** pour quitter le programme si l'on sait qu'il va crasher. On s'assure ainsi de bien maîtriser le code, respectant ainsi le principe de la PEP indiquant qu'aucune erreur ne doit passer silencieusement.

De même, nous pouvons connecter le bouton de fermeture à un **sys.exit(0)** pour indiquer que tout s'est bien déroulé.

Il n'est cependant pas du tout obligatoire d'employer ce genre d'instructions, mais elles existent et il faut le savoir, car cela peut parfois s'avérer utile de les implémenter.

IV-O-3-i - Autre

Certaines recommandations complémentaires existent. Par exemple, plutôt utiliser **is** et **is not** à la place de **==** et **<>**.

De même, il est déconseillé d'utiliser le L minuscule, le O majuscule ou le i majuscule. En effet, selon la police d'écriture ces caractères peuvent parfois mal s'interpréter.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Enfin, les constantes doivent systématiquement être écrites en majuscules : MA_CONSTANTE

IV-O-3-j - En plus

En général, outre ces règles, on essaie de toujours faire commencer un nom par un préfixe (souvent une lettre) minuscule, suivie d'un underscore. Cette lettre permet d'identifier en un coup d'œil le type auquel nous avons à faire. En fond orange ceux qui s'avèrent réellement indispensables :

Préfixe	Description
vg_	**Indique une variable globale**
vl_	**Indique une variable locale**
f_	**Indique une fonction**
p_	**Indique une procédure**
pkg_	Indique un package
m_	Indique un module
c_	Indique une classe
c_p_	Indique une propriété d'une classe (par exemple)
c_m_	Indique une méthode d'une classe (par exemple)
e_	Indique une exception

IV-O-4 - Bonne structure type d'un programme

```
#!/usr/bin/PYTHON
# -*-coding:utf-8 -*

#==========================================================================#
#--------------------------------------------------------------------------#
#                                 NOM                                      #
#--------------------------------------------------------------------------#
#**************************************************************************#
#                  Société - Auteur - Date initiale                        #
#--------------------------------------------------------------------------#
#                         Notes/Commentaires                               #
#                                                                          #
#--------------------------------------------------------------------------#
#                            HISTORIQUE                                    #
#   V0.1.0    Société - Auteur - Date                                      #
#             Motif de la modification et/ou commentaire bref              #
#==========================================================================#


#------------------------------------------#
#          Importation des packages        #
#------------------------------------------#
import time


#------------------------------------------#
#          Declaration des variables       #
#------------------------------------------#
a = "Hello World"


#------------------------------------------#
#                    Code                  #
#------------------------------------------#
def ma_fonction(a):
""" ma_fonction(a)
     permet d'afficher a à l'écran
"""
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
    print a

if __name__ == " __main__ ":
""" Main
     Permet d'afficher " hello world " toutes les 5 secondes
"""
    ma_fonction("hello world") #on peut aussi simplement passer 'a'
    time.sleep(5) #mise en veille pendant 5s
```

Comme on peut le voir sur cet exemple simple, une structure de code respecte une certaine mise en page et différentes règles, outre celles déjà énoncées précédemment.

IV-O-4-a - La mise en page

On veillera notamment à bien s'assurer de la présence d'un cartouche complet.

De plus, chaque section sera précédée d'un mini **cartouche** résumant l'utilité/l'action du code de cette section.

Les sections seront séparées par quatre sauts de lignes et les fonctions/procédures internes à ces sections par deux sauts de lignes.

Les tabulations seront équivalentes à quatre espaces (les éditeurs sont paramétrables à ce niveau en général).

IV-O-4-b - Les règles

Sous Linux, UNIX, on n'oubliera pas les deux premières lignes, qui doivent OBLIGATOIREMENT se trouver en ligne 1 et 2.

La première permet de stipuler où se trouve l'**interpréteur** **PYTHON**. La seconde permet d'indiquer le type d'encodage à utiliser.

En plus de cela, on veillera à bien mettre en place des docstrings, les plus explicites possible.

Enfin, il est important de tenir à jour toutes les docstrings, cartouches et commentaires afin qu'ils correspondent au code en place.

Le module graphique **TKInter** est considéré comme le module graphique de base de **PYTHON**.

D'apparence un peu austère, elle ne possède que quelques widgets basiques, suffisants la plupart du temps. Certains autres modules graphiques permettent d'améliorer l'aspect et/ou de rajouter des widgets complémentaires à TKInter, mais cela ne sera pas vu ici.

L'import se fait de la façon suivante :

```
>>>from tTkinter import *
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

V - Modules complémentaires



Wikimedia Commons,Gnome-mime-text-x-sh.png

V-A - Pypi

Pypi, pour « **PYTHON** Package Index » est un dépôt mettant à disposition des développeurs tout un ensemble de packages/modules.

Le but est triple : certifier de façon officielle un certain nombre de packages/modules, standardiser l'exécution de certaines actions, et doter notre langage préféré d'un minimum de fonctionnalités de base.

Le site, unique et en anglais **https://pypi.python.org/pypi/**, met à disposition de tout programmeur, quel qu'il soit, des tutoriels, des exemples, des documentations…

La communauté active permet de remonter l'ensemble des bogues identifiés et d'être sûr de toujours disposer d'une version la plus fonctionnelle possible d'un package/module.

De plus, le système **Pypi** simplifie l'installation/désinstallation, ainsi que la maintenance de l'ensemble des bibliothèques utilisées au sein d'un projet.

Au moment de la rédaction de ce livre, 26 704 packages étaient disponibles via le dépôt **Pypi**.

Un flux RSS vous permettra également de vous tenir facilement au courant des évolutions de vos packages préférés et des nouveautés.

Certains des modules présentés ici seront des modules dépendant de **Pypi**.

V-B - Le temps

On utilise ici le module **time**. Ce module permet de manipuler simplement les variables en rapport avec le temps.

V-B-1 - Le timestamp

La variable de base pour la gestion du temps est ce qu'on appelle le **TIMESTAMP**. Ce **timestamp** correspond au nombre de secondes écoulées depuis la date de référence **UNIX** : le 1er janvier 1970 à 00 h 00 m 00 s.

Pour obtenir ce timestamp rien de plus simple, il suffit d'utiliser la méthode **time** :

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
>>> import time
>>> time.time()
1341920301.923005
>>> type (time.time())
<type 'float'>
```

On peut remarquer que le **timestamp** est un float d'une grande précision. Cette précision peut éventuellement vous servir à calculer le temps d'exécution d'un code.

V-B-2 - Date complète

Le module **time** possède une méthode **localtime** permettant de récupérer une date au grand complet avec les éléments suivants :

Paramètre	Description
tm_year	L'année
tm_mon	Le numéro du mois
tm_mday	Le numéro du jour du mois
tm_hour	L'heure
tm_min	Les minutes
tm_sec	Les secondes
tm_wday	Le jour de la semaine (0 (lundi) à 6)
tm_yday	Le jour de l'année
tm_isdst	Indique un éventuel changement d'heure locale

Il est vivement recommandé d'utiliser le **timestamp** pour tout ce qui est calcul et le **localtime** pour tout ce qui est affichage.

Vous pouvez utiliser le **timestamp** comme référence pour le **localtime**, et la méthode **mktime** pour récupérer un **timestamp** depuis un **localtime**.

```
1. >>> mon_timestamp = time.time()
2. >>> print mon_timestamp
3. 1341921025.92
4. >>> mon_localtime = time.localtime(mon_timestamp)
5. >>> print mon_localtime
6. time.struct_time(tm_year=2012, tm_mon=7, tm_mday=10, tm_hour=13,
7. tm_min=50, tm_sec=25, tm_wday=1, tm_yday=192, tm_isdst=1)
8. >>> print time.mktime(mon_localtime)
9. 1341921025.0
```

Remarque : comme on peut le voir en comparant la ligne 3 et la ligne 9, au cours de la conversion, nous perdons un peu de précision puisqu'il y a un arrondi à la seconde près.

V-B-3 - La mise en sommeil

Le module **time** permet également de mettre le code en pause pendant un temps déterminé par un float. Ce float représente un nombre de secondes

```
>>>time.sleep(5.5) #mise en pause pendant 5 secondes 1/2
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

V-C - Les mathématiques

V-C-1 - Le module de base

Le module s'appelle **math**. Il possède différentes fonctions pouvant être utiles

```
>>>math.pow(4,2)  # 4 puissance 2
16
>>>math.sqrt(16) #racine carree de 16
4
>>> 2 * math.exp(5) #exponentiel
296.8263182051532
>>>math.fabs(-5) #valeur absolue
5
>>>ang_deg = 57.29
>>>math.radians(ang_deg) #conversion de degres en radians
0.9998991284675514
>>> math.degrees(0.9998991284675514) #conversion de radians en degres
57.29
>>>math.ceil(5.3) #arrondi par exces
6
>>>math.floor(2.4) #arrondi par defaut
2
>>>math.trunc (4.6) #ne renvoie que la parti entiere
4
>>>import random #import du module random complementaire a math
>>>random.random() #renvoie une valeur float aleatoire entre 0 et 1
0.535093545175585
```

V-C-2 - Le module NumPY

NumPy est le module **PYTHON** spécialisé dans le calcul scientifique.

```
import numpy
```

Sa grande force tient dans le fait que l'on peut créer des tableaux ou des matrices **Numpy** et appliquer un calcul à cet ensemble, en une fois, permettant ainsi de réaliser un gain de temps important.

Bien que possédant de nombreuses spécificités, c'est surtout cette capacité à appliquer un calcul à l'ensemble des éléments d'un tableau que nous verrons ici.

V-C-2-a - Tableaux multidimensionnels

Créer un **tableau multidimensionnel** NumPy est simple :

```
>>>import numpy
>>>tab1 = numpy.array([1,2,3]) #tableau a une dimension
>>>tab2 = numpy.array([[1,2,3],[4,5,6]]) #matrice 3*3
>>>tab2[0][0]
1
>>>tab2[1][2]
6
```

Il y a aussi possibilité de définir le type des éléments d'un tableau à sa création :

```
tab3 = numpy.array([1,2,3], dtype='i') #i pour integer
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Remarque : un tableau de deux dimensions (matrice) est comparable à un tableau vertical contenant des tableaux horizontaux. Ainsi lors de l'appel d'un élément (ex. : tab2[0][1]), le premier indice correspond à la ligne (emplacement dans le tableau vertical), et le second indice à la place de l'élément dans le tableau horizontal.

V-C-2-b - Manipulation sur les tableaux

Un tableau **NumPy** se rapprochant des listes, on peut utiliser les mêmes méthodes de manipulation

V-C-2-b-i - Ajout

Pour effectuer un ajout à un tableau, on utilise deux fonctions de **NumPy** : **hstack** et **vstack**. La première sert à ajouter des éléments en horizontal et la seconde en vertical du tableau.

Elles prennent deux paramètres sous la forme d'un tuple : le tableau **NumPy** concerné, puis l'élément à rajouter.

Attention : ces deux méthodes n'enregistrent pas les modifications effectuées. N'oubliez donc pas de récupérer le résultat.

```
>>>import numpy
>>>tab1=numpy.array([1,2,3])
>>>tab1
array([1,2,3])
>>>numpy.hstack((tab1,4))
array([1,2,3,4])
>>>tab1
array([1,2,3])
>>>tab1=numpy.hstack((tab1,4))
>>>tab1
array([1,2,3,4])
```

On peut également utiliser la méthode **append**. Elle permet un ajout simplifié directement à la fin du tableau.

```
>>>tab2
array([[1,2,3],
           [4,5,6]])
>>>tab2 = numpy.append(tab2, [[7,8,9]], axis = 0)
>>>tab2
array([[1,2,3],
           [4,5,6],
           [7,8,9]])
```

Remarque : axis correspond au comportement désiré par NumPy. S'il vaut 0, alors le tableau garde des dimensions. S'il faut None, alors le tableau est mis à plat et ne possède plus qu'une dimension (tableau classique).

V-C-2-b-ii - Modification

Pour modifier un élément rien de plus simple, il suffit de préciser l'élément à modifier puis de préciser sa nouvelle valeur

```
>>>tab2
array([[1,2,3],
           [4,5,6]])
>>>tab2[0][2] = 4
>>>tab2
array([[1,2,4],
           [4,5,6]])
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

V-C-2-b-iii - SUPPRESSION

La suppression d'un élément d'un tableau **NumPy** passe par la méthode **NumPy.delete**. Cette méthode prend en paramètre le tableau ainsi que l'indice concerné. Dans un tableau à une dimension, l'indice correspond à l'emplacement de l'élément à effacer. Dans un tableau à deux dimensions, on précisera uniquement le premier indice (suppression d'un tableau horizontal entier, cf. remarque en .

```
>>>tab1
array([1,2,3,4])
>>>tab1 = numpy.delete(tab1,3)
>>>tab1
array([1,2,3])
>>>tab2
array([[1,2,4],
           [4,5,6]])
>>>tab2=numpy.delete(tab2,1,axis = 0)
>>>tab2
array([[1,2,4]])
```

V-C-2-b-iv - COPIE

Une chose importante à savoir concernant **NumPy** est qu'il faut utiliser une fonction bien précise de **NumPy** pour réaliser une copie de tableau sous peine de ne posséder qu'une simple référence. On utilise la méthode **copy**.

```
>>>tab3 = tab2.copy()
```

V-C-2-c - Transposition

NumPy offre la possibilité de réaliser simplement des transpositions. Pour rappel, il s'agit d'inverser lignes et colonnes d'une matrice ainsi que ses dimensions de fait

```
>>> tab2.transpose()
array([[1, 4],
           [2, 5],
           [3, 6]])
```

V-C-2-d - Méthodes associées aux tableaux

Il existe également certaines méthodes associées aux tableaux.

Paramètre	Description
max()	Récupère la ou les valeur(s) maximum(s)
min()	Récupère la ou les valeur(s) minimum(s)
sum()	Effectue la somme des éléments
sort()	Trie les éléments
std()	Calcul l'écart type

Le paramètre axis=1 permet de travailler sur les lignes et axis=0 sur les colonnes et non pas l'ensemble du tableau.

```
>>> tab2.max()
6
>>> tab2.sum()
21
>>> tab2.sum(axis=1)
array([ 6, 15])
>>> tab2.max(axis=1)
array([3, 6])
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
>>> tab4=numpy.array([4,2,8,7])
>>> tab4.sort()
>>> tab4
array([2, 4, 7, 8])
```

V-C-2-e - Calcul sur tableau

Comme indiqué au début de cette partie consacré à **NumPy**, ce qui va nous intéresser surtout est la capacité à réaliser des opérations de manière simplifiée :

```
>>> tab2
array([[1, 2, 3],
       [4, 5, 6]])
>>> tab2 = 2 * tab2
>>> tab2
array([[ 2,  4,  6],
       [ 8, 10, 12]])
```

Dans cet exemple, on peut voir que l'opération effectuée sur le tableau affecte la totalité du tableau. Nous économisons ainsi l'utilisation d'une boucle.

V-C-2-f - En plus…

Certaines fonctions fournies par **NumPy** permettent d'effectuer des opérations complexes sur des matrices

Paramètre	Description
dot(mat1, mat2)	Permet d'effectuer un produit entre matrices
inner(mat1, mat2)	Permet de réaliser un produit scalaire
solve(mat1, mat2)	Permet d'obtenir la solution d'un système linéaire avec : mat1 $\begin{matrix} x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \\ x_3 & y_3 & z_3 \end{matrix}$ mat2 $\begin{matrix} reponse1 \\ reponse2 \\ reponse3 \end{matrix}$
numpy.sin(x)	Permet d'obtenir le sinus de l'angle x (existe aussi numpy.arcsin)
numpy.cos(x)	Permet d'obtenir le cosinus de l'angle x
numpyr.tan(x)	Permet d'obtenir la tangente de l'angle x
numpy.pi	Renvoie la valeur de la variable pi
numpy.abs(x)	Renvoie la valeur absolue de x
numpy.ceil(x)	Arrondit à l'entier supérieur
numpy.floor(x)	Arrondit à l'entier inférieur
numpy.round(x)	Arrondit à l'entier le plus proche

V-D - Imagerie

Pour travailler sur des images, nous utilisons le module **PIL** (**PYTHON** Imaging Library).

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Ce module permet de manipuler les images comme pour effectuer des traitements sur ces dernières.

Nous verrons ici l'ouverture et la fermeture d'un fichier image, ainsi que leur création (pixel par pixel), leur enregistrement et enfin l'affichage de ces images. Il existe de nombreuses autres possibilités avec ce module. Pour les connaître toutes, rendez-vous sur **http://effbot.org/imagingbook/image.htm**.

V-D-1 - Ouverture d'une image

Pour ouvrir une image existante, il faut utiliser la méthode **open**

```
>>> from PIL import Image
>>> im = Image.open("/media/PERSO/test.png")
```

V-D-2 - Création d'une image

Pour créer une image, c'est très simple. Il suffit de faire un appel au constructeur en précisant le mode (généralement « RGB ») et la taille désirée (un tuple(x,y)) :

```
>>> from PIL import Image
>>> im=Image.new("RGB",(500,250))
```

Remarque : par défaut, l'image créée est entièrement noire.

Il ne reste ensuite plus qu'à renseigner chaque pixel de votre image.

```
>>> im.putpixel((0,0),(255,255,255))
```

Ici, le premier tuple correspond aux coordonnées (X,Y) du pixel dans l'image en partant du coin supérieur gauche.

Le second tuple, lui, correspond aux couleurs RGB, où 0 représente le noir et 255 le blanc.

Remarque : pour obtenir uniquement une image en dégradé de gris tel que du Depthmaps, il faut que R=G=B en toute occasion.

V-D-3 - Modification d'une image

Pour modifier une image, c'est assez simple. Une fois l'image ouverte, il suffit d'utiliser la même méthode que pour créer une image. Le pixel sera alors remplacé.

Il est également possible de réaliser un traitement sur l'ensemble de l'image. Pour ce faire, il est alors conseillé de créer une matrice avec **NumPy** qui permettra alors de réaliser des traitements facilement (modification de teinte, détection de contours…)

V-D-4 - Sauvegarde d'une image

Pour sauvegarder une image, il suffit d'utiliser la méthode **save**

```
>>> im.save("/media/PERSO/test", "PNG")
```

Comme on peut le constater, on passe ici deux paramètres. Le premier est le chemin avec le nom désiré pour le fichier, mais sans extension. Le second est le type de fichier désiré, ce qui donnera l'extension du fichier.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

V-D-5 - Affichage d'une image

Pour afficher l'image via le visualiseur d'images par défaut du système, il suffit d'utiliser la méthode **show**.

```
>>> im.show()
```

V-D-6 - Connaître les composantes d'un pixel

Pour des raisons diverses, il peut parfois être intéressant de connaître les composantes précises d'une image (RGB).

Pour cela, il faut utiliser la méthode **getpixel**. Elle prend en paramètre les coordonnées du point et renvoie un tuple contenant les composantes

```
r, g, b = im.getpixel(0,0)
```

V-D-7 - Conversion en gris

Comme dit précédemment, une couleur de gris a une composante RGB (Red, Green, Blue) équilibrée. Entendez par là que R = G = B.

Cependant, il existe une formule donnée par la commission internationale de l'éclairage qui qualifie le gris dit luminance, avec la formule suivante :

$$gris = 0{,}2125 * R + 0{,}7154 * G + 0{,}0721 * B$$

V-D-8 - Exemple

```
#!/usr/bin/PYTHON
# -*-coding:utf-8 -*

from PIL import Image



def Step1():
    """
    Cree une image noire de 64 * 64 pixels, puis l'enregistre.
    """
    im = Image.new("RGB",(64,64))
    im.save("/home/steph/demo.png")
    del im #permet de supprimer l'objet image cree




def Step2():
    """
    Ouvre l'image, modifie 1 pixel sur 2, l'enregistre, puis l'affiche.
    """
    im = Image.open("/home/steph/demo.png")
    i, j = 0, 0

    while i < 64:
        while j < 64:
            im.putpixel((i,j),(255,255,255))
            j = j + 2
        i = i + 2
        j = 0
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
    im.save("/home/steph/demo.png")
    im.show()
    del im



if __name__ == '__main__':
    Step1()
    Step2()
```

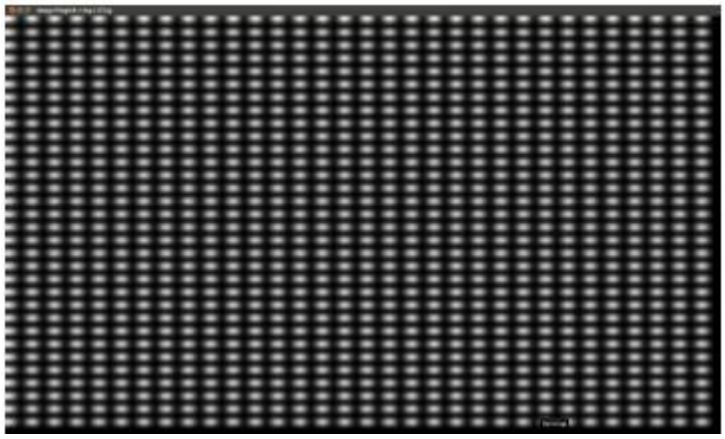


Image générée avec le code exemple

V-E - Les graphiques avec Matplotlib

Le module **Matplotlib** permet de créer des graphiques de type courbe.

Puissant, il n'est cependant pas forcément évident de prime abord. Capable de créer indifféremment des **courbes** 2D ou 3D, en couleur ou non, multiples ou simples, ce module est désormais la référence dans ce domaine.

Nous ne verrons ici que les courbes de base, en 2D, ainsi que la gestion des titres et des légendes associés. Pour des courbes plus complexes, je vous invite à visiter le site et à lire la documentation de Matplotlib.

V-E-1 - Création d'une courbe

V-E-1-a - Le conteneur

Pour créer une courbe, la première chose à faire est de créer un objet pour contenir cette courbe. Cet objet est ici appelé **Figure**.

Il peut prendre différents paramètres, mais nous les laisserons à leur valeur par défaut. Bien entendu, il faudra auparavant importer le module de **matplotlib**

```
from matplotlib.figure import Figure
ma_figure = Figure()
```

Une fois ce conteneur créé, il faut créer un support de courbe et l'ajouter au conteneur via la méthode **add_subplot**.

```
ma_courbe = ma_figure.add_subplot(111)
```

Comme on peut le voir, cette méthode prend un paramètre. Ce paramètre permet de stipuler l'emplacement de la courbe. En effet, un conteneur peut contenir 1,2,4… courbes.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

On définit donc leur emplacement à l'aide de ce paramètre. Ici, 111 signifie que le conteneur aura une définition de 1 courbe en Y, 1 courbe en X, et que ma_courbe occupera la position 1. Si on avait eu 212, on aurait eu 2 courbes en Y, 1 en X, et ma_courbe aurait été la seconde courbe. En résumé donc : YXPosition.

Remarque : si cela vous paraît encore un peu flou, je vous invite à faire quelques tests par vous-même.

V-E-1-b - Ajout d'une courbe

Sur ma_courbe nous pouvons ensuite rajouter une ou plusieurs courbes. Nous utiliserons la méthode **plot(x, y, paramètres optionnels)**.

Cette méthode prend en paramètres optionnels, entre autres, un label, une couleur, un type de marqueur pour chaque point ou encore le type et la largeur de ligne. La couleur, le type de ligne et le type de marqueur peuvent être passés en un seul paramètre.

```
ma_courbe.plot([1,2,3],[1,2,3], 'go--', label='Test1', linewidth=2)
```

Remarque : on peut voir que pour X et Y, on passe une liste de points. Une courbe se trace en une seule fois, c'est pourquoi on doit passer l'ensemble des points constituant la courbe en une fois. De plus, il faut la même quantité de points entre x et y, sinon vous aurez un message d'erreur.

Dans l'exemple ci-dessus, nous réalisons une ligne de trois points. Le « go-- » signifie que nous traçons la ligne en vert, avec des marqueurs circulaires et que la ligne sera pointillée (cf. tableaux ci-après). La ligne aura une largeur double à la normale. Notre ligne s'appellera « Test1 ».

Paramètre	Description
'-'	Solide
'--'	Tiret
'-.'	Alternance tiret pointillé
':'	Pointillé

Paramètre	Description
b	Bleu
g	Vert
r	Rouge
c	Cyan
m	Magenta
y	Jaune
k	Noir
w	Blanc

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Paramètre	Description
'.'	Point
','	Pixel
'o'	Cercle
'v'	Triangle, pointe en bas
'^'	Triangle, pointe en haut
'<'	Triangle, pointe à gauche
'>'	Triangle, pointe à droite
's'	Carré
'p'	Pentagone
'*'	Etoile
'x'	X
'|' (alt gr 6)	Barre verticale
'_' (touche 8)	Barre horizontale

Il est bien entendu possible de mixer ces symboles afin d'obtenir ce qu'on désire. Par exemple « go- » pour une ligne verte avec des marqueurs circulaires et une ligne solide ; ou encore « r:x » pour une ligne rouge en pointillé, avec des marqueurs en x.

Remarque : en cas d'absence d'information sur un type ou un style, le module prendra des valeurs par défaut.

V-E-2 - Paramétrage complémentaire

V-E-2-a - Axes

Il est possible de paramétrer, dans une certaine mesure, les axes du graphique.

Par exemple, on peut définir le **Xmin**, le **Xmax**, le **Ymin** et le **Ymax**, via la commande **axis** :

```
ma_courbe.axis([Xmin,Xmax,Ymin,Ymax])
```

De même, nous pouvons désactiver l'affichage des axes :

```
ma_courbe.axis('off')
```

V-E-2-b - Légende de la courbe

Les légendes permettent de facilement distinguer les différentes courbes présentes sur un graphique.

Pour cela il suffit d'utiliser la méthode **legend(loc='best')**. Cette dernière permet d'afficher la légende de la courbe au meilleur endroit possible.

Autre solution sinon, utiliser un coin du graphique.

```
ma_courbe.legend(loc='upper right')
```

V-E-2-c - Labels

Chaque axe peut posséder son propre label. Pour donner un label à l'axe X ou Y, on utilise les méthodes **set_xlabel** et **set_ylabel**.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
ma_courbe.set_xlabel('Axe X')
ma_courbe.set_ylabel('Axe Y')
```

V-E-2-d - Grille

Il est possible de mettre une grille sur la courbe afin, par exemple, d'en faciliter la lecture. Pour cela, on utilisera la méthode **grid**. On lui passera comme paramètre :

```
ma_courbe.grid(color='r', linestyle='-', linewidth=2)
```

La couleur, le style de ligne sont sur le même principe que pour tracer une courbe.

Pour désactiver la grille, rien de plus simple, il suffit d'appeler **grid** sans paramètre.

```
ma_courbe.grid(False)
```

V-E-2-e - Utilisation de date en X ou Y

Il peut être pratique pour tracer, par exemple, une courbe de données chronologiques de travailler directement avec des dates. Pour cela aussi **matplotlib** peut être utile.

En effet, le module **datetime** est spécialement là pour ça.

```
import datetime
import matplotlib
...
x= [datetime.datetime(2011,02,01), datetime.datetime(2011,03,01)]
y=[100,150]
ma_courbe.plot(X,Y,label = 'Ma courbe avec date en X')
ma_figure.autofmt_xdate(bottom=0.2, rotation=30, ha='right')
```

La dernière ligne mérite une petite explication complémentaire. Elle permet de modifier la position et l'inclinaison des annotations de l'axe X (dans notre cas, les dates).

Les valeurs par défaut placent le texte à 30 °. Mais il est cependant possible de donner un autre angle de rotation.

V-E-2-f - Image de fond

Dans certains cas, il peut être intéressant d'afficher une image en fond de notre courbe. Pour cela, il existe la méthode **imshow**. Cette méthode prend comme paramètre une image PIL (cf 3.14.3).

Attention : la résolution peut poser problème. En effet, cette méthode va traduire l'image avec 1 pixel = 1 unité.

```
imm = Image.open('/home/alex/test.bmp')
ma_courbe.imshow(imm)
```

V-E-2-g - Effacement de la courbe

Effacer une courbe est très utile, lorsque l'on désire afficher une courbe de manière dynamique, ou simplement recharger une donnée sur un graphique.

Pour cela, la méthode **clf** est à utiliser.

```
ma_courbe.clf()
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

V-E-2-h - Transformer un graphique en image

Pour transformer un graphique en image, c'est relativement simple : il suffit d'utiliser la méthode **savefig**.

Cette méthode peut prendre plusieurs paramètres. Nous ne verrons ici que la version simplifiée.

```
ma_figure.savefig(" /home/ag/mon_graphe.PNG ")
```

Remarque : les formats gérés pour l'export sont le png, le pdf, le ps, l'eps et le svg.

Attention : lorsque vous voulez générer des graphiques en masse, n'oubliez jamais de supprimer chaque figure avant de passer à la suivante, car sinon votre mémoire se remplira rapidement. Utilisez pour cela pyplot.close(ma_figure).

V-E-3 - Exemple

```
#!/usr/bin/PYTHON
# -*-coding:utf-8 -*



from matplotlib.figure import Figure
import matplotlib.pyplot as plt #Remplace un conteneur GTK

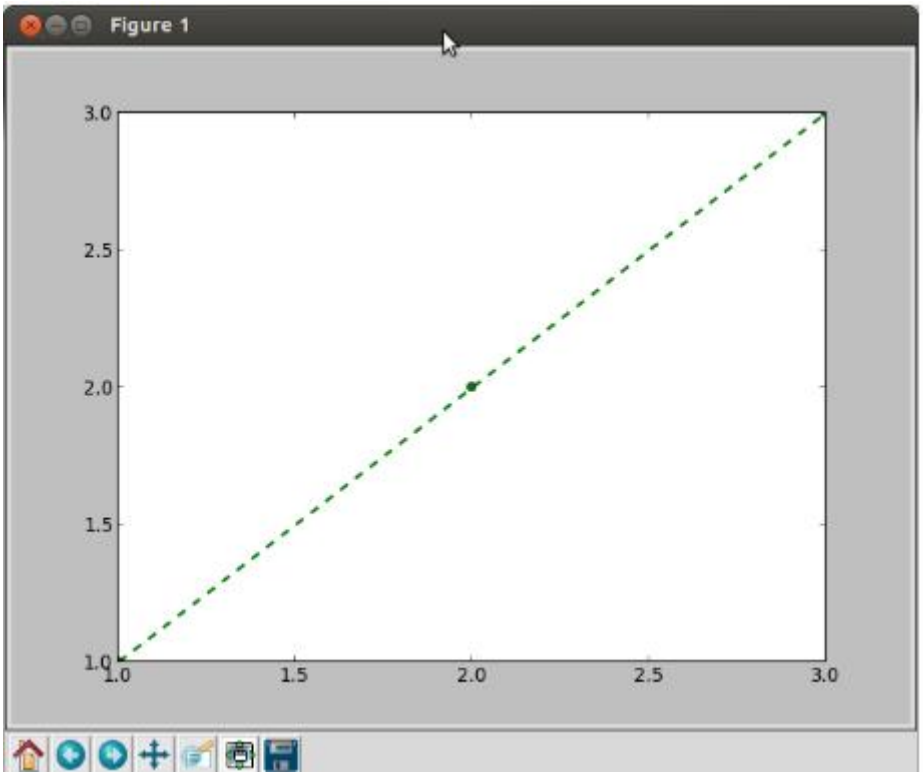


def f_plot():
    """
    Cree et affiche une figure
    """
    ma_figure = plt.figure() #Implemente une interface basique autonome
    ma_courbe = ma_figure.add_subplot(111)
    ma_courbe.plot([1,2,3],[1,2,3], 'go--', label='Test1', linewidth=2)
    plt.show()



if __name__ == '__main__':
    f_plot()
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/



Rsultat du code exemple de matplotlib

V-F - Les bases de données

Lorsque l'on développe des applications, il est très pratique de pouvoir stocker simplement les paramètres de configuration.

Deux choix s'offrent alors à nous : utilisation d'un simple fichier texte ou l'utilisation d'une **base de données**.

C'est cette dernière solution que nous allons aborder ici.

V-F-1 - Présentation rapide

Qu'est-ce qu'une base de données (BDD) ? D'après Wikipédia, c'est « un lot d'informations stockées dans un dispositif informatique ».

En informatique, la BDD est gérée par ce qu'on appelle un SGBD (Système de Gestion de Base de Données). Parmi les plus couramment utilisées de nos jours, nous trouvons **PostGreSQL**, **SQLite**, MySQL ou encore Oracle.

Les SGBD et les BDD sont un sujet très vaste et nous allons tâcher de rester assez simples, et ne verrons donc que les bases.

V-F-1-a - Composants

Une BDD est constituée de différents éléments.

Tout d'abord la base. C'est elle qui accueille l'ensemble des données. Ces données sont réparties dans des tables. Chaque table est organisée en colonnes.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Chacune de ces colonnes, tables, et bases porte un nom. Le nom des colonnes est propre à une table, et le nom des tables est propre à une base. Par exemple, deux tables de même nom ne peuvent coexister dans une même base, mais peuvent exister dans deux bases distinctes.

V-F-1-b - Fonctionnement générique

Une table peut être comparée à une feuille de Libre Office Calc, où chaque ligne possède un numéro unique, appelé ROW ID. Ce ROW ID est unique pour une ligne donnée, quelle que soit la table, à l'intérieur d'une base.

De plus, chaque table possède un index dit unique. Il peut être comparé à l'index d'un livre, à une exception près : l'index unique d'une table (un tuple sur un ou plusieurs éléments) ne peut contenir de doublons. Cela afin de garantir, ce qu'on appelle l'unicité de la table.

Quand nous cherchons une information, nous utilisons un langage en grande partie standardisé appelé SQL pour Structured Query Language.

Remarque : bien que grandement standardisé, chaque éditeur y va de ses touches personnelles, rendant malheureusement difficile l'interopérabilité. C'est pourquoi je vous invite fortement à utiliser des BDD Open Source telles PostGreSQL ou encore SQLite pour les besoins plus faibles.

Pour reprendre l'exemple d'un fichier classeur, imaginez que vous avez un fichier Calc nommé Carnet, contenant une feuille nommée Amis, contenant 50 lignes. Chacune de ces lignes correspond à l'un de vos amis et contient leur nom, leur prénom, leur adresse, leur mail…

Ici, le fichier Calc correspond à la base, la feuille à la table, et le nom des colonnes… au nom des colonnes. Chacun de vos amis y possède un numéro unique (N° de la ligne).

Maintenant, imaginez que vous ayez besoin de l'adresse d'Albert Dupond. Vous vous dites: « je veux l'adresse d'Albert Dupond qui est stockée dans mon fichier carnet.ods, sur la feuille amis ». Eh bien en SQL ce sera la même chose, sauf qu'on précise le nom des colonnes : « Je cherche l'adresse de la personne dont le nom est Dupond et le prénom Albert dans mon fichier carnet.ods, sur la feuille amis ».

Nous nous arrêterons là pour les explications sur les BDD. Il s'agit surtout d'une présentation succincte (pour ceux qui ne connaîtraient pas encore le sujet), mais néanmoins nécessaire et suffisante pour suivre le reste de ce point sur les BDD.

Pour apprendre le langage SQL, je vous renvoie vers Internet qui contient de nombreux cours fort bien conçus sur le sujet.

V-F-2 - PYSQLITE

La base **SQLite** est une base de données extrêmement simple. Intégrée par défaut à **PYTHON**, elle présente cependant l'inconvénient de ne pas être multiuser.

De fait, on la retrouvera plutôt au sein d'un programme autonome pour stocker les données. À titre d'information, c'est notamment souvent une base de données utilisée dans les appareils mobiles type smartphone.

Pour se connecter à une base **SQLite**, il faut importer le module sqlite3, qui s'installe en même temps que l'interpréteur **PYTHON**.

```
import sqlite3
```

Pour créer/manipuler des bases **SQLIte**, je conseille l'excellent **SQLite Manager**, PLUG IN de Firefox.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

V-F-2-a - Connexion

Pour se connecter à une base **SQLite**, on va définir le chemin du fichier **SQLite**.

```
ma_base = sqlite3.connect("./BASE.sqlite")
```

V-F-2-b - Exécuter une requête

V-F-2-b-i - Curseur

La première chose à faire après s'être connecté à une base est de créer un curseur. Il s'agit d'une sorte de zone tampon entre notre programme et la base.

```
mon_curseur = ma_base.cursor()
```

Pour exécuter une requête SQL, rien de compliqué là non plus. On utilise la méthode **execute**, avec la possibilité de passer des paramètres dynamiques.

Imaginons que nos variables mon_nom et mon_prenom varient en fonction de la saisie utilisateur. Pour prendre en compte la saisie dans la requête, on utilisera la syntaxe suivante

```
mon_curseur.execute("SELECT * \
    FROM MA_TABLE \
    WHERE NOM = ? \
    AND PRENOM = ?",\
    (mon_nom, mon_prenom))
```

Comme on peut le voir, nous avions ici deux paramètres dynamiques pour la requête. Nous les avons passés dans l'ordre, grâce au « ? ». Attention toutefois de ne pas oublier de les mettre entre parenthèses.

Le \ en fin de ligne, lui, permet d'indiquer que nous continuons l'instruction à la ligne.

V-F-2-b-ii - Récupération des résultats

Une fois l'interrogation effectuée, le résultat est disponible dans le curseur.

Pour l'exploiter pleinement, il est cependant préférable de transférer le résultat dans une liste temporaire, juste après la requête.

```
ma_liste = []
for r in mon_curseur:
    ma_liste.append(r[0])
```

Cette méthode fonctionne très bien si on n'attend qu'une seule colonne en retour. Mais dans le cas où on en attend plusieurs, on sera vite confronté au problème de savoir quoi correspond à quoi.

Pour pallier cela, nous utiliserons donc le **row_factory**. En association avec sqlite3.Row, nous pouvons non seulement différencier les différentes colonnes, mais en plus connaître le nom des dites colonnes, si ces dernières nous sont inconnues.

Pour connaître le nom des colonnes, nous procédons de la façon suivante :

```
ma_base = sqlite3.connect("./BASE.sqlite")
ma_base.row_factory = sqlite3.Row
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
mon_curseur = ma_base.cursor()
mon_curseur.execute("SELECT * FROM MA_TABLE)
r = mon_curseur.fetchone()
mon_listing = r.keys()
cur.close()
```

Ici, mon_listing est une pseudoliste contenant le nom des colonnes.

Pour récupérer les données d'une colonne dont nous connaissons le nom, tel que dans un logiciel à la recherche d'une donnée, un âge dans notre exemple :

```
1. for r in mon_curseur:
2. if r["Age"] == None:
3.     age = 0
4. else:
5.     age = int(r["Age"])
```

Dans la ligne 5, nous effectuons un transtypage afin de nous assurer que âge sera toujours un int.

V-F-2-c - Sauvegarde

Comme indiqué plus haut, une mise à jour en base n'est effective que lorsque l'exécution **commit** a été exécutée.

```
ma_base.commit()
```

V-F-2-d - Déconnexion

Enfin, une fois nos interrogations terminées, il ne reste qu'à se déconnecter. Pour cela, il suffit de fermer le curseur

```
mon_curseur.close
```

V-F-3 - PSYCOPG

PSYCOPG est un module d'interfaçage entre **PYTHON** et **PostGreSQL**.

Ce SGBD, en plus d'être **Open Source**, est certainement l'un de ceux qui respectent le plus les normes SQL.

PSYCOPG nous permettra de dialoguer au mieux avec ce type de BDD, et le plus simplement possible.

```
Import psycopg2
```

V-F-3-a - Connexion

La connexion à une base est extrêmement simple :

```
ma_connection=psycopg2.connect(database='test',user="name", \
                  password='mdp',host=127.0.0.1, \
                  port=5432) #par defaut
```

Vous voyez ici la forme complète. host et port peuvent être facultatifs selon votre utilisation. Le port 5432 est le port par défaut.

Afin de respecter le principe de **PYTHON** selon lequel « explicite est mieux qu'implicite », je vous invite à toujours utiliser la forme complète.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

V-F-3-b - Exécuter une requête

V-F-3-b-i - Curseur

Une fois connecté à la base, l'exécution d'une requête est semblable à **SQLite**. Il faut d'abord passer par un curseur avant de pouvoir exécuter la requête.

```
1. mon_curseur = ma_connection.cursor()
2. mon_curseur.execute(''SELECT %s FROM DUAL'', ('NULL',))
```

Quelques explications concernant la ligne 2.

Nous voyons que nous avons utilisé un %s. Contrairement au langage C, il n'existe pas de %d, %f… On n'utilisera donc uniquement que le %s, même si on doit passer la valeur 42 par exemple.

Les paramètres seront toujours sous la forme de tuples, ce qui explique que nous ayons « ('NULL',) ».

Nous avons également la possibilité de former la requête avant de l'exécuter grâce à la méthode mogrify :

```
requete = mon_curseur.mogrify(" SELECT %s FROM DUAL ", ('NULL',))
```

Ici, requête vaudra « SELECT NULL FROM DUAL ».

V-F-3-b-ii - Récupération des résultats

```
a = mon_curseur.fetchone()
```

Nous récupérons ici le résultat de la requête.

Cela nous renvoie une seule ligne à la fois. Lorsque nous avons atteint la fin, nous récupérons la valeur None. Les résultats sont renvoyés sous la forme de tuples.

Si l'on désire récupérer tous les résultats d'un coup, il faut utiliser la commande **fetchall()**.

V-F-3-b-iii - Lecture de la dernière requête exécutée

Pour connaître la dernière requête ayant été exécutée, il faut passer par la méthode **QUERY** :

```
last_requete = mon_curseur.query
```

V-F-3-b-iv - Procédures

Afin de faciliter un traitement, il est possible d'écrire des procédures **PostgreSQL** et de simplement les appeler au sein du script **PYTHON**.

Pour les appeler, il faut utiliser la méthode **callproc :**

```
ma_curseur.callproc(ma_procédure(mes_paramètres))
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

V-F-3-b-v - Gestion d'erreurs

Il est possible de gérer simplement les erreurs sur une base **PostgreSQL**.

```
1. try:
2.     mon_curseur.execute(" Select * from test ")
3. except Exception, e:
4.     pass
5. print e.pgcode, ': ', e.pgerror
```

Ligne 5, nous afficherons le code erreur **PostGreSQL**, suivi de la description de l'erreur engendrée.

V-F-3-c - Sauvegarde

Tout comme dans **SQLite**, la sauvegarde s'effectue via un commit :

```
ma_connection.commit()
```

V-F-3-d - Déconnexion

Pour se déconnecter, la procédure est relativement simple. On commence par clore les curseurs ouverts, puis par clore la connexion à la base.

```
mon_curseur.close()
ma_connection.close()
```

Remarque : pour savoir si un curseur est clos ou non, on peut utiliser mon_curseur.closed().

V-G - Le module OS

Nous n'allons pas parler ici d'open source ni de système d'exploitation, même si un lien existe.

En effet, de module sert à s'interfacer avec le **système d'exploitation** afin d'effectuer des opérations précises.

De nombreuses possibilités sont offertes par ce module, mais nous n'en verrons que quelques-unes, celles qui nous servent le plus souvent.

```
import os
```

V-G-1 - Taille d'un fichier

Pour connaître la taille d'un fichier, nous utilisons **os.path.getsize() :**

```
taille_fichier = os.path.getsize(" ./mon_fichier.txt ")
```

V-G-2 - Renommage d'un fichier

Pour renommer un fichier, nous utilisons **os.renames() :**

```
os.renames(" ./ancien_nom.txt ", " ./nouveau_nom.txt ")
```

Remarque : si le dossier stipulé dans le chemin de renommage n'existe pas, alors il sera créé.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

V-H - Scripting

Pour faire du scripting, nous allons utiliser les modules **sys** et **os**. Ces modules sont très utiles en programmation **PYTHON,** car ils permettent de réaliser nombre d'interactions avec la machine et l'OS.

Nous ne détaillerons pas dans ce livre les possibilités offertes par ces modules, car trop vastes, mais vous pourrez trouver beaucoup d'informations sur Internet.

```
import sys
import os
```

V-H-1 - Les arguments

V-H-1-a - Passage d'argument

Pour passer des **arguments**, nous utilisons l'attribut **sys.argv**. Cet attribut est une liste des arguments passés en paramètres.

Imaginons un script nommé script.py. Quand nous l'appelons en lui passant des arguments cela pourrait donner cela :

```
PYTHON script.py arg1 arg2 arg3 " arg4   arg5 "
```

Dans ce cas, sys.argv donnerait le retour suivant :

```
['script.py', 'arg1', 'arg2', 'arg3', 'arg4   arg5']
```

Comme nous pouvons le voir la distinction entre les différents arguments est réalisée grâce aux espaces. Cependant, il est possible de passe un argument contenant des espaces, à condition de le mettre entre doubles-quotes.

Attention : le premier argument (sys.argv[0]) est toujours le nom du programme.

V-H-1-b - Tester le nombre d'arguments

Pour connaître le nombre d'arguments passés en paramètre, il suffit d'exécuter un len sur l'attribut.

```
>>>#sur notre exemple, cela donne :
>>>len(sys.argv)
5
```

V-H-1-b-i - Ouverture et accès aux données

L'accès d'un fichier contenu dans un ZIP peut être utile, notamment quand on ne désire qu'ajouter un peu de texte à un fichier.

Voici comment procéder : création d'un objet zipfile à partir de notre fichier ZIP (accès en lecture/écriture 'mode w)), puis accès en lecture au fichier depuis l'objet **ZIPFILE**, et enfin modification et fermeture du ZIP.

```
import zipfile

mon_zip = zipfile.ZipFile('mon_texte.zip', 'w')
fichier = mon_zip.read('./données.txt') #lecture du fichier
mon_zip.writestr('données.txt', b'ajout de données\n')    #on convertit en binaire
mon_zip.close()
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Remarque : nous voyons ici que nous écrivons en mode binaire (wb). Il faut savoir en effet que les données dans un ZIP sont au format binaire.

V-H-1-b-ii - Décompression complète

La décompression s'effectue de la manière suivante : nous créons un objet **ZIPFILE** avec un accès en lecture. Puis, pour chaque fichier/dossier trouvé, nous l'écrivons sur le disque au chemin désiré.

```
import zipfile

mon_zip = zipfile.ZipFile('mon_arch.zip','r')
for fichier in mon_zip.namelist():
    contenu = mon_zip.read(fichier)
    fichier_sortie = open(fichier, 'wb')
    fichier_sortie.write(contenu)
    fichier_sortie.close()
mon_zip.close()
```

Une autre écriture possible, plus directe, est la suivante :

```
import zipfile

mon_zip = zipfile.ZipFile('mon_arch.zip','r')
mon_zip.extractall('./')    #on passe en parametre le chemin desire
mon_zip.close()
```

V-H-2 - L'exécution de commande

Nous allons ici utiliser le module **os**, et plus précisément la méthode **popen()**. Cette méthode va prendre en paramètre la **commande** à exécuter (au format string), et vous renverra le résultat une fois la commande exécutée.

Le résultat, vous est renvoyé lui aussi sous la forme d'une string. Il ne vous reste plus qu'à le lire grâce la méthode **read()**.

Attention : la méthode read vous renvoie le contenu de la variable puis la réinitialise. Pensez donc à utiliser une variable de type string afin de récupérer le résultat de la commande.

```
>>> ma_commande = os.popen("ls")
>>> ma_variable_string = ma_commande.read()
>>> ma_commande.read()
''
>>> #la variable a été reinitialise
>>> ma_variable_string
'b\nBureau\nDocuments\nImages\nMod\xc3\xa8les\nMusique\nPublic\n'
```

V-H-3 - Exemple

```
#!/usr/bin/PYTHON
# -*-coding:utf-8 -*



import sys
import os



def main():
    """
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
    Prend 2 arguments, les affichent à l'ecran, puis les enregistre.
    """

    nom = sys.argv[1]
    prenom = sys.argv[2]

    ma_commande = os.popen("ls -l /home/steph")
    result = ma_commande.read()

    print "programme ", sys.argv[0]
    print "bonjour ", prenom, nom
    print "contenu de /home/steph: "
    print type(result)

    file("/home/steph/log.txt","w")

    with open("/home/steph/log.txt", "w") as mon_fichier:
        mon_fichier.write(result)
        mon_fichier.close()

    print "Synthese sauvegardee dans log.txt"



if __name__ == '__main__':
    main()
```

V-I - Les fichiers Zip

Nous allons ici parler du module **zipfile**. Ce module permet de gérer les archives ZIP mono fichier jusqu'à 4 GO.

Le module ici retenu est loin d'être le seul et le plus performant existant. Cependant, il est très intéressant, car le format ZIP est extrêmement répandu.

De plus, il possède une capacité de compression minimale non négligeable.

Enfin, comme nous le verrons dans la partie consacrée à l'**Open Document Format** (ODF), il est très utile pour s'interfacer avec des documents ODF.

```
import zipfile
```

V-I-1 - Création/modification d'un ZIP

La création d'une archive Zip est extrêmement simple. La première étape est la création d'un objet zipfile. Ensuite, nous lui ajoutons les différents fichiers que nous souhaitons.

```
1. import zipfile
2.
3. mon_zip = zipfile.ZipFile('mon_arch.zip','w',zipfile.ZIP_STORED)
4. mon_zip.write("fichier1.txt")
5. mon_zip.close()
```

Comme vous pouvez le constater ici, rien de compliqué. Ligne 3, nous passons en paramètre le nom de notre archive (et éventuellement le chemin), le type d'accès (r pour READ, w pour WRITE, a pour APPEND), et le type de compression. Ce dernier paramètre est optionnel et ne peut prendre que deux valeurs: **ZIP_STORED** (par défaut) ou **ZIP_DEFLATED**. Pour utiliser la compression, le module ZLIB peut parfois être demandé.

À noter la possibilité d'une autre écriture

```
import zipfile
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
with zipfile.ZipFile('mon_arch.zip', 'w') as mon_zip:
    mon_zip.write('fichier1.txt')
mon_zip.close()
```

Bien que cette écriture soit plus propre, elle possède néanmoins l'inconvénient de devoir effectuer toutes les opérations d'écriture dans le **WITH**. À vous de voir la méthode la plus pratique dans vos projets.

Attention : l'écriture dans le ZIP ne s'effectue qu'à l'appel du close().

L'ajout de fichier à une archive s'effectue simplement en précisant le mode **append** lors de la création de l'objet **ZIPFILE**.

V-I-2 - Ouverture d'un ZIP

L'ouverture d'un ZIP peut s'interpréter de deux façons différentes : ouverture du ZIP et accès aux données en lecture/écriture, ou bien une décompression complète.

V-J - Le XML

XML n'est pas un langage de programmation, mais un langage de description. Plus précisément, il s'agit d'un langage de balisage, dit générique.

De balisage, car il utilise des balises pour délimiter des zones. Générique, car excepté quelques règles, dont nous verrons les principales, chacun est libre de ses actions.

Ainsi, contrairement au **HTML** (autre langage de balisage), par exemple, nous donnons aux balises le nom que nous désirons.

De plus, de nombreux modules permettent de s'interfacer extrêmement facilement avec ce type de fichiers.

Ces éléments font que le **XML** est extrêmement apprécié dans le milieu informatique. En effet, c'est le langage par excellence qui permet à deux applications de s'interfacer sans difficulté.

La seule chose à connaître est la structure du **XML**. Cette structure est d'ailleurs ce qui permet de vérifier qu'un fichier **XML** est correct.

V-J-1 - Codage

V-J-1-a - L'entête

L'entête d'un **XML** est très simple et en voici un exemple que nous allons analyser. Cet entête s'appelle un **PROLOGUE**.

```
<?xml version="1.0" encoding="UTF-8"?>
```

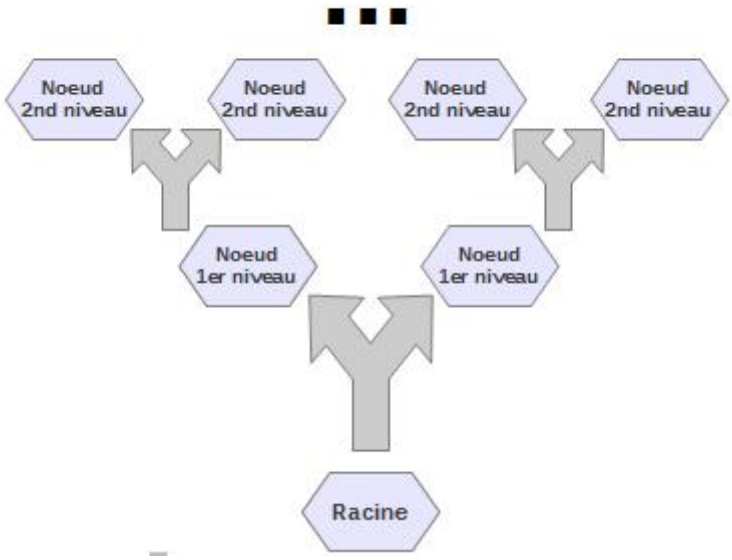
L'élément suivant est donc le langage utilisé (XML). Nous précisons ensuite la version. À l'heure actuelle seule la version 1 est vraiment utilisée, donc pas de questions à se poser. Enfin, le type d'encodage du fichier. Je vous conseille l'UTF-8, standard européen. Vous trouverez plus d'informations sur l'encodage de caractères sur Wikipédia.

Donc, comme vous pouvez le constater, rien de sorcier ici non plus.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

V-J-1-b - Le corps

Le corps du fichier même est organisé en arbre. Par arbre, j'entends ramification. Ainsi, chaque balise peut avoir un certain nombre de sous-balises.



Structure en arbre d'un fichier XML

V-J-1-c - Exemple

Ci-dessous un exemple d'un annuaire codé en **XML**.

```
<?xml version= "1.0 " encoding= "utf-8 "?>
<annuaire>
    <personne dpmt= "sciences ">
        <nom> Dupond </nom>
        <prenom> Jean-Baptiste </prenom>
        <tel> 3637 </tel>
    </personne>
    ...
    <personne dpmt= "langue ">
        <nom> Martin </nom>
        <prenom> Michel </prenom>
        <tel> 5354 </tel>
    </personne>
</annuaire>
```

V-J-2 - Les principes de base

Nous allons voir ici les grands principes de base du **XML**. Certaines spécificités ne seront pas décrites ici. Ce n'est donc pas parce que vous ne trouverez pas d'informations sur ce que vous essayez de faire que ce n'est pas possible. Internet est votre ami.

V-J-2-a - Les commentaires

En **XML**, les commentaires se font très simplement, avec des « <!-- » pour les commencer et un « --> » pour les clore. Cela permet d'avoir des commentaires sur plusieurs lignes.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
<!--  Ceci est un commentaire  -->
```

V-J-2-b - La racine et les nœuds

La racine, aussi appelée nœud document, est la base du document **XML**.

Pour prendre une comparaison, un nœud peut être comparé à un dossier ou à un sous-dossier. Le nœud document, lui, est comparable à la racine système (« / » sous Linux).

V-J-2-c - Les balises

Les balises permettent de délimiter des zones au sein de l'arbre **XML**. La différence entre une balise ouvrante et une balise fermante tient à un caractère.

```
<ma_balise> <!-- balise ouvrante -->
    ...
</ma_balise> <!-- balise fermante -->
```

Dans le cas où il n'y aurait pas de données entre les balises ouvrantes et fermantes, on peut utiliser un raccourci.

```
<ma_balise/>
```

Remarque : dans les faits, je vous recommande d'utiliser exclusivement la première syntaxe, qui sera dans tous les cas plus lisible, et conviendra à toutes les situations.

V-J-2-d - Chevauchement

Les balises n'ont pas le droit de se chevaucher.

```
<balise1><balise2> ... </balise1></balise2> <!-- Ceci est interdit -->
<balise1><balise2> ... </balise2></balise1> <!-- Ceci est la bonne syntaxe -->
```

V-J-2-e - Les attributs

Les attributs, en **XML**, sont des informations complémentaires que l'on insère dans les balises.

Leur intérêt, par rapport à des sous-balises, est de potentiellement permettre une recherche plus rapide, d'un point de vue **XPATH**.

```
<Dupont region= "Bretagne " dpmt= "35 "> <!-- Presence ici de deux attributs -->
...
```

V-J-2-f - Les instructions de traitement

Les instructions de traitement sont spécifiquement dédiées aux divers traitements de l'information contenue dans le fichier **XML**. Ces instructions commencent par un « <? » et finissent par un « ?> ».

L'instruction de traitement la plus courante en **XML** est l'entête.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

V-J-2-g - Le XPATH

Le **XPATH**, correspond à un chemin permettant d'accéder à une donnée bien précise (nœud). Il peut être assimilé à un chemin de fichier complet dans Linux.

Ainsi, si nous reprenons notre exemple de l'annuaire, le **XPATH** nous donnant le nom des personnes dans le fichier sera le suivant :

```
/annuaire/personne/nom
```

On dit qu'on accède aux nœuds « **nom** ». Cela nous ramène cependant l'ensemble des noms, et pas un en particulier. On aurait également pu utiliser « **//nom** », mais ce type de notation est ambigu. En effet, il n'y a aucune description des balises mères, ce qui peut provoquer quelques soucis.

Voyons un peu plus en détail les possibilités offertes par le **XPATH**.

V-J-2-g-i - Sélection d'un nœud précis

Il existe différentes façons de définir un nœud précis. Cependant, toutes sont liées à la position du nœud dans le fichier.

Remarque : ici la numérotation commence à 1 et non à 0.

```
/annuaire/personne[2] <!-- Récupère la seconde personne du fichier -->
/annuaire.personne[position()<10] <!-- Récupère les 9 premières personnes -->
/annuaire/personne[nom=" Dupond "] <!-- Récupère les nœuds ou le nom vaut Dupond -->
```

Outre cela, nous pouvons également faire des recherches avec conditions, notamment au niveau d'un contenu partiel. Ainsi, si nous recherchons toutes les personnes dont le nom, ou le prénom, est ou contient « Jean », nous aurons comme **XPATH :**

```
/annuaire/personne[contains(., " Jean ")]
```

V-J-2-g-ii - Sélection avec attributs

Si l'on désire chercher uniquement les personnes du dpmt science, le **XPATH** sera le suivant :

```
/annuaire/personne[@dpmt= "sciences "]
```

Si l'on désire récupérer les différentes valeurs des attributs dpmt, le **XPATH** sera le suivant :

```
/annuaire/personne/@dpmt
```

Nous pouvons effectuer également des recherches via les attributs. Par exemple, si nous voulons récupérer le nom de toutes les personnes du dpmt sciences :

```
/annuaire/personne[@dpmt=" sciences "]/nom
```

V-J-2-g-iii - Récupérer toutes les données d'un nœud

L'ensemble des données comprises dans un nœud peuvent être récupérées grâce au **XPATH** suivant, par exemple :

```
/annuaire/personne[1]/*
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

V-J-2-g-iv - Combinaison de XPATH

Il est possible de combiner des recherches afin de ne récupérer qu'une partie des informations d'un nœud. Il faut utiliser le caractère pipe, « | » (touches alt-gr + 6)

Ainsi, si nous ne désirons que le nom et le téléphone, nous aurons :

```
/annuaire/personne/nom | /annuaire/personne/tel
```

V-J-2-g-v - En bref…

Comme nous venons de le voir rapidement ensemble, accéder à des informations d'un fichier **XML** est assez simple.

De plus il est possible de combiner les différentes méthodes de recherche afin d'affiner le nœud ciblé.

Cependant, précisons qu'un fichier **XML** n'est pas une BDD. Et cela, même si certaines BDD se basent sur des fichiers **XML**.

V-J-3 - Le module LXML

Maintenant que nous avons vu les grands principes de base du **XML**, nous allons nous attaquer à l'interfaçage d'un fichier **XML** avec **PYTHON**. Comme vous le verrez, il n'y aura rien de bien difficile ici non plus.

Il existe en **PYTHON** deux façons d'aborder un fichier **XML :** de manière événementielle, ou par arborescence. Notre choix retenu ici est le type arborescence.

Notre choix est justifié par la volonté de coller le plus possible à la structure en arbre, définissant un fichier **XML**. Nous allons étudier le module **LXML**.

Ce module est un dérivé de librairie C. Il faut donc installer **lxml**, **libxml2**, et **libxslt**. Vous pouvez passer par Synaptic ou tout autre gestionnaire de paquets pour vous simplifier la vie sur Linux.

V-J-3-a - Utilisation de LXML

Nous n'utiliserons pas la totalité des possibilités offertes par le module. Nous allons ici nous concentrer sur **ETREE**.

ETREE, pour Element Tree, se rapporte comme son nom l'indique au principe d'arborescence en arbre. Grâce à cela, nous allons utiliser le fichier **XML** comme une source de données navigables.

Remarque : notez bien ici que j'ai utilisé l'expression « source de données » et non « base de données ». Un fichier XML, pour rappel, n'est nullement une BDD.

```
from lxml import etree    #import du module

xml_file = etree.parse(" ./annuaire.xml ")    #connexion au fichier XML
```

V-J-3-b - Création d'un fichier XML

La création d'un fichier **XML** diffère un peu de la création d'un fichier classique.

En effet, nous allons ici procéder en deux étapes. Tout d'abord nous allons créer la structure de notre fichier **XML**. Ensuite, nous écrirons cette structure au sein même d'un fichier.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Pour la création de la structure, il faut être particulièrement minutieux et respecter un ordre précis, telle l'arborescence d'un arbre.

Ainsi, la première chose à créer est la racine de l'arbre. Ensuite, nous créons les nœuds permettant de générer chaque branche de l'arbre. Enfin, pour finir, nous nous assurons de la bonne indentation du fichier.

```
1. from lxml import etree
2.
3. racine = etree.Element(" annuaire ")    #Creation de la racine
4. personne1 = etree.SubElement(racine, " personne ")    #Creation d'un nœud
5. personne1.set(" dpmt ", " sciences ")    #Ajout d'un tag
6. nom1 = etree.SubElement(personne1, " nom ")
7. nom1.text = " Dupond "    #Initialisation de la valeur du noeud
8.
9. file_str = etree.tostring(racine, pretty_print = True)    #Convertion en string
10. #ecriture dans un fichier texte
11. ...
```

Analysons un peu plus cet exemple.

Ligne 3, nous créons notre racine, en lui passant le nom de la balise en paramètre.

Ligne 4, nous créons une sous-balise, en passant en paramètre la balise mère, ou nœud père, ainsi que le nom de la balise.

Ligne 5, nous ajoutons un tag sur le même principe. Les paramètres sont le nom du tag et sa valeur.

Ligne 7, nous initialisons la valeur du nœud.

Ligne 9, nous convertissons notre structure, en chaîne de caractères, en précisant que nous voulons que la chaîne soit bien formatée, avec indentations. Pour cela, nous utilisons le paramètre **pretty_print**.

À partir de la ligne 10, il ne nous reste alors plus qu'à écrire la chaîne de caractères dans un fichier texte, à l'extension **.XML**.

V-J-3-c - Lecture d'un fichier XML

Une fois connecté au fichier **XML**, tel que vu précédemment, nous pouvons naviguer au sein des données grâce au **XPATH**. Nous le ferons grâce à un **FOR**.

Une fois dans la boucle, nous utiliserons alors diverses méthodes afin de récupérer les informations qui nous intéressent.

```
for balise in xml_file.xpath(" /annuaire/personne "):
    nom = nalise.xpath(" nom ")
    print nom[0].text
```

Ci-dessous les principales diverses méthodes qui peuvent vous servir.

Méthodes	Utilité
.items()	Récupère la liste des attributs
.text	Récupère le texte du nœud visé
.tag	Récupère le nom de la balise, du nœud
.get(" attribut ")	Récupère la valeur de l'attribut passé en paramètre

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

V-J-3-d - Modification d'un fichier XML

V-J-3-d-i - Serialisation et deserialisation

Qui dit **XML** dit **sérialisation** et **deserialisation**. Derrière ces deux termes se cache un principe fort simple : la conversion d'une chaîne de caractères vers un objet (désérialisation) ou depuis un objet (sérialisation).

Il existe en effet de nombreux modules capables d'effectuer des traitements sur les chaînes.

Convertir un objet en chaîne de caractères permet ainsi de simplifier les traitements.

Remarque : dans notre cas, pour sérialiser un objet de parse, il faut utiliser la méthode .write(<fichier>).

V-J-3-d-ii - Méthode générale

Modifier un fichier **XML** n'est pas tellement plus compliqué qu'une création. Nous allons ici repartir sur notre annuaire **XML** d'exemple. Place au code que nous commenterons.

```
1. #!/usr/bin/env PYTHON
2. from lxml import etree
3.
4. xml_file=etree.parse("./annuaire.xml")
5. racine=etree.Element("annuaire")
6. personne=etree.SubElement(racine,"personne")
7. prenom=etree.SubElement(personne,"prenom")
8. for balise in xml_file.xpath("/annuaire/personne[nom=\"Dupond\"]"):
9.     prenom = balise.xpath("prenom")
10.     prenom[0].text="Charles"
11.
12. body = etree.tostring(xml_file, pretty_print = True)
13. entete = "<?xml version=\"1.0\" encoding=\"utf-8\"?>"
14. contenu = entete + "\n" + body
15.
16. mon_fichier = open("./annuaire.xml", "w")
17. mon_fichier.write(contenu)
```

Ligne 2, nous importons notre module.

Ligne 4, nous nous connectons à notre fichier **XML**, puis nous créons une liaison sur la racine, ligne 5.

Ligne 6 et 7, nous créons d'autres références aux sous-balises personnes et prénom.

Ligne 8 à 10, nous recherchons dans les balises personnes, celles dont le nom est Dupond, afin de changer leur prénom en Charles.

Ligne 12, nous convertissons le fichier **XML** en chaîne de caractères.

Ligne 13, nous créons l'entête d'un fichier **XML.**

Ligne 14, nous assemblons les deux parties précédentes afin de créer le corps de texte du fichier **XML**.

Ligne 16 et 17, enfin, nous générons/mettons à jour le fichier **XML**.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

V-K - L'Open Document Format

Wikipédia dispose d'un très bon article sur l'**Open Document Format**, **ODF** en abrégé. Je vous invite à le lire en entier pour bien comprendre ce qu'il en retourne.

Pour ceux qui ne pourraient pas accéder à Wikipédia, nous allons résumer de manière succincte les grandes lignes de l'article Wikipédia.

V-K-1 - Historique

ODF est un standard ouvert définissant des formats de bureautique. S'étant basé sur les formats d'Open Office, il en a repris les extensions en trois lettres (od pour les documents ou ot pour les templates, suivi d'une lettre indiquant le type de document).

Après de nombreuses années de mise au point, ce standard est certifié ISO en 2006 et de nombreuses sociétés ainsi que plusieurs pays ont déjà pris comme résolution de ne plus utiliser que ces formats bureautiques.

V-K-2 - Les différents types de documents

Il existe plusieurs types possibles : texte (odt), classeur (ods), présentation (odp), graphique (odg). Il ne s'agit là que des principaux.

Ces formats sont des documents à part entière. Mais le standard **ODF** offre également la possibilité pour ces principaux types de réaliser des modèles, aussi appelés templates, aux extensions ott, ots, otp et otg.

V-K-3 - Les templates

Ces **templates** permettent de réaliser une mise en page type, puis de venir remplacer différents champs du template par les valeurs désirées.

Ainsi, une sortie en fichier classeur, par exemple, se déroule en deux étapes :

1-création du fichier template avec des balises type ;

2-copie du template et remplacement des balises par des données.

V-K-4 - Description du format ODF

Le format **ODF** n'est en soi qu'un simple fichier ZIP, contenant des fichiers **XML**, des fichiers de configuration et des dossiers.

En théorie, il suffit donc de dézipper un fichier **ODF** afin d'accéder directement au contenu du fichier sans avoir à l'ouvrir.

V-K-5 - Interaction avec PYTHON

À ce stade, il y a deux façons d'interagir avec des fichiers **ODF :** une simple modification de template ou une interaction totale.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

V-K-6 - Modification de template

V-K-6-a - Théorie

Dans le cas d'une simple modification de template (par exemple, générer une fiche de données avec un format précis), nous allons utiliser la méthode directe :

1-modification de l'extension en .ZIP ;

2-dézippage ;

3-modification des éléments désirés ;

4-rezippage ;

5-modification de l'extension au format **ODF.**

Au niveau de la modification d'un template, ce qui va nous intéresser plus particulièrement ce sera **content.xml**, **meta.xml** et le dossier **Pictures**.

meta.xml contient toutes les informations relatives au document : application, titre, description, sujet, mots-clés, auteur, date de création, modèle utilisé…

content.xml est le fichier qui contient le corps du document, autrement dit son contenu entier.

Le dossier **Pictures**, lui, contient les images du document, auxquelles fait référence **content.xml**.

La méthode directe consiste à fouiller dans les fichiers **XML** à la recherche de nos balises (par exemple MON_NOM, MON_PRENOM…).

Ces balises doivent avoir un nom bien particulier et surtout unique au sein du fichier de manière a être sûr de les repérer simplement.

Une fois repérées, il suffit de les remplacer par les valeurs désirées.

Idem pour les images, qui sont prises entre des balises **XML** <Pictures>. Il suffit de remplacer la référence de l'image entre ces balises **XML**, puis de déposer l'image dans le dossier **Pictures**.

Ainsi, en partant d'un fichier odt ou ods, il est très facile de créer des templates de rapport très simplement en substituant simplement texte et image à notre convenance.

Remarque : il existe également des formats de fichiers TEMPLATE en ODF (ott, ots…). Ces formats sont spécifiques à ODF, et visent plutôt à être modifiés via Libre Office (par exemple), ou via une méthode d'interaction complète.

V-K-7 - Interaction complète

L'interaction complète est très pratique lorsque vous souhaitez créer un document complet de zéro et de manière totalement dynamique (contrairement aux templates qui sont statiques).

Pour cela il faut utiliser des modules spécifiques. Nous verrons ici ensemble les bases de **EZODF**.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Comme d'habitude, nous ne prétendons nullement nous substituer à la documentation officielle. Les strictes bases pour l'odt et l'ods seront vues ici, rien de plus.

Il s'agit avant tout de vous faire une présentation succincte, mais fonctionnelle de l'utilisation de ce package.

Vous serez la plupart du temps confronté à l'utilisation de template, plutôt qu'à la création de fichiers **ODF** de toute pièce. Pour votre information personnelle, même si cela ne sera pas vu ici, sachez que **EZODF** sait très bien gérer les templates **ODF**.

L'utilisation de template fera l'objet de l'exemple général de cette partie sur **l'ODF**.

```
from ezodf import newdoc
```

V-K-7-a - Création et ouverture d'un document ODF

Le constructeur est identique, quel que soit le document :

```
mon_ods = newdoc(doctype='ods', filename='mon_ods.ods')
```

Cela se passe de tout commentaire, au vu de la simplicité. Les paramètres sont le type de document à créer, ainsi que le nom du fichier.

Si nous avons vu comment créer un document de toute pièce, il peut être utile aussi de savoir ouvrir un document existant pour le modifier.

```
mon_odt = ezodf.opendoc('mon_odt.odt')
```

Ici, seul le nom du document à ouvrir est à fournir.

V-K-7-b - ODT

Voici l'exemple officiel pour créer un ODT basique :

```
1. from ezodf import newdoc, Paragraph, Heading
2.
3. odt = newdoc(doctype='odt', filename='text.odt')
4. odt.body += Heading("Chapter 1")
5. odt.body += Paragraph("This is a paragraph.")
6. odt.save()
```

Avant de voir plus en détail les principaux éléments pour un ODT, nous allons commenter ce petit exemple.

Ligne 1, nous importons de quoi créer un nouveau document et de quoi créer un titre et un paragraphe. Il s'agit ici de styles prédéfinis.

Ligne 3, nous créons notre document ODT.

Ligne 4, nous ajoutons au corps de notre ODT, un titre en lui passant en paramètre le texte.

Remarque : ce package permet l'utilisation au choix de « .body += « ou de « .body.append ». À vous d'utiliser celui que vous préférez.

Ligne 5, nous ajoutons à notre corps de document un paragraphe en passant en paramètre le texte.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Ligne 6 nous sauvegardons notre document.

Pour effectuer un saut de page, il suffit d'utiliser :

```
odt.body += SoftPageBreak()
```

V-K-7-c - ODS

Voici l'exemple officiel pour la création d'un ODS :

```
1. from ezodf import newdoc, Sheet
2.
3. ods = newdoc(doctype='ods', filename='spreadsheet.ods')
4. sheet = Sheet('SHEET', size=(10, 10))
5. ods.sheets += sheet
6. sheet['A1'].set_value("cell with text")
7. sheet['B2'].set_value(3.141592)
8. sheet['C3'].set_value(100)
9.
10. pi = sheet[1, 1].value
11. ods.save()
```

Ligne 1, nous importons de quoi générer notre ODS.

Ligne 3, nous créons notre fichier CALC.

Ligne 4, nous créons une feuille CALC, en passant en paramètres le nom de la feuille et la taille désirée (lignes puis colonnes).

Ligne 5, nous ajoutons notre feuille à notre fichier CALC.

Ligne 6 à 8, nous remplissons plusieurs cellules de la feuille.

Ligne 10, nous récupérons une valeur dans une cellule. Notez que nous comptons à partir de 0. A0 est donc (0,0), B2 est (1,1)…

Ligne 11, nous sauvegardons notre ODS.

Parmi les autres actions possibles au niveau d'une feuille ODS, nous retrouvons, entre autres les suivantes :

```
sheets = mon_ods.sheets    #recupere le nom de toutes les feuilles de l'ODS
nom = sheets[0].name        #recupere le nom de la 1ere feuille
count = len(sheets)        #recupere le nombre de feuilles dans l'ODS
rowcount = sheets[0].nrows()    #recupere le nombre de lignes de la feuille
colcount = sheets[0].ncols()    #recupere le nombre de colonnes de la feuille
valeur = sheets[0]['C1']    .value    #recupere la valeur d'une cellule d'une feuille
del sheets[0]            #Supprime la 1ere feuille de l'ODS
```

V-K-8 - Exemple

```
#! /usr/bin/env PYTHON
# -*- coding: utf-8 -*-



import zipfile    #gere les fichiers zip
import shutil    #permet de gerer des actions propres aux dossiers
import os    #interface avec l'os
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
import glob    #permet de gerer les chemins de maniere fine



#fonction d'archivage
def archive(filezip, chemin, leng):
    for i in glob.glob(chemin + '/*'):#os.listdir('./TEST/'):
        print i
        if os.path.isdir(i):
            archive(filezip, i, leng)
        else:
            filezip.write(i, i[leng:]) #Obligation de concevoir un chemin initial



#on dezip le fichier ods
os.rename("./TEST.ods", "TEST.zip")

mon_zip = zipfile.ZipFile('TEST.zip','r')
mon_zip.extractall('./TEST/')
mon_zip.close()


os.remove("./TEST.zip")


#on modifie notre balise $$NOM$$
with open("./TEST/content.xml", 'r') as mon_fichier:
    contenu = mon_fichier.read()
    mon_fichier.close()

contenu = contenu.replace("$$NOM$$", "DUPOND")

#Le fait d'ecrire sans append ecrase le contenu du fichier
with open("./TEST/content.xml", 'w') as fichier_out:
    fichier_out.write(contenu)
    mon_fichier.close()


#on rezip l'ods
with zipfile.ZipFile('TEST.zip', 'w',zipfile.ZIP_DEFLATED) as mon_zip:
    path = './TEST'
    pathsup = len(path)+1
    archive(mon_zip, path, pathsup)
    mon_zip.close()

os.rename("./TEST.zip", "./TEST.ods")


#on efface le dossier apres zippage
shutil.rmtree("./TEST")
```

V-L - Serveur FTP

Nous allons maintenant aborder la question des connexions FTP. En effet, il n'est pas rare qu'un traitement ait besoin d'aller chercher et/ou déposer de(s) fichier(s) sur un **serveur FTP**.

Nous ferons ici appel au module **ftplib**.

```
import ftplib
```

V-L-1 - Connexion à un serveur FTP

Pour se connecter à un serveur **FTP**, il nous faut connaître au minimum :

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

=>l'adresse du serveur (et éventuellement le port) ;

=>vos identifiants d'accès.

```
import ftplibimport ftplib

ftp_server = 'mon_serveur.com
login = 'invite'
passwd = 'invite'

ftpconnection = ftp.ftplib.FTP(ftp_server, login, passwd)
```

Vous voilà désormais connecté à votre serveur **FTP**. Voyons maintenant comment faire pour gérer les fichiers/dossiers à distance.

En général, pour chaque action réalisée, même une simple connexion, le serveur **FTP** vous renvoie des codes pour vous indiquer si l'opération s'est bien déroulée.

L'analyse de ces codes, que nous récupérerons ici dans la variable « ret », vous permettra de détecter de potentielles erreurs.

V-L-2 - Dépôt de fichiers/dossiers

Un dépôt de fichiers se passe en plusieurs étapes :

=>ouverture du fichier en mode binaire ;

=>envoi du fichier au serveur ;

=>fermeture du fichier.

Cette procédure est due au fait que l'envoi d'un fichier en mode binaire au serveur impose que le fichier soit ouvert.

Le choix du transfert en mode binaire se justifie simplement par sa plus grande fiabilité.

```
1. file_name = './mon_fichier.txt'
2. fichier = open(file_name, 'rb')
3.
4. ret = ftpconnection.storbinary('STOR ' + file_name, fichier)
5.
6. fichier.close()
```

Une petite explication concernant la ligne 4. Le premier paramètre est une commande **FTP** classique. On demande au serveur **FTP** d'accepter le fichier dont on passe le nom en paramètre. Le second paramètre est le contenu du fichier que l'on désire transférer.

V-L-3 - Récupération de fichiers/dossiers

En plus du dépôt, la récupération de fichiers/dossiers peut également être utile :

```
ret = ftpconnection.retrbinary('RETR '+ file_name, file_name.wite)
```

La commande **RETR** est également une commande de base **FTP**, qui demande à récupérer une copie du fichier dont le nom est passé en paramètre. Le second paramètre est ici le fichier ou l'on va écrire les données reçues.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

V-L-4 - Annulation d'un transfert en cours

Quels que soient le sens du transfert et le type du transfert (fichier ou dossier), la commande est unique :

```
ftpconnection.abort()
```

V-L-5 - Liste des éléments du dossier

Afin de pouvoir se déplacer dans un serveur, il est important de savoir les éléments disponibles. Pour cela, il faut utiliser la commande dir() :

```
listing = ftpconnection.dir()
```

V-L-6 - Renommage d'un fichier/dossier

Pour renommer un fichier/dossier, il faut utiliser la ligne de code suivante :

```
ret = ftpconnection.rename(nom_actuel, nouveau_nom)
```

V-L-7 - Création d'un dossier

Pour créer un dossier, il faudra utiliser la commande suivante :

```
ret = ftpconnection.mkd(repertory_name)
```

V-L-8 - Effacement d'un fichier/dossier

Pour un fichier il suffit de réaliser l'action suivante :

```
ret = ftpconnection.delete(file_name)
```

Pour un dossier ce sera la commande suivante :

```
ret = ftpconnection.rmd(repertory_name)
```

V-L-9 - Connaître la taille d'un fichier

Il peut parfois être pratique de connaître la taille d'un fichier afin de savoir si un transfert s'est bien effectué par exemple :

```
size = ftpconnection.size(file_name)
```

V-L-10 - Savoir où l'on se trouve

Afin de savoir dans quel dossier/chemin on se trouve, il suffit d'utiliser la commande **pwd :**

```
chemin = ftpconnection.pwd()
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

V-L-11 - Utilisation de ligne de commandes

Il se peut que pour des raisons pratiques, ou pour utiliser une IHM simple, on désire simplement communiquer avec le serveur en ligne de commande.

```
ret = ftpconnection.sendcmd(commande)
```

V-L-12 - Déconnexion du serveur FTP

Le plus simple dans l'accès à un serveur **FTP**. Cela se passe de tout commentaire :

```
ftpconnection.quit()
ftpconnection.close()
```

Une petite explication ici : il n'y a pas d'erreur, il faut bien utiliser **quit()** puis **close()**.

La première commande indique au serveur que l'utilisateur se déconnecte, et la seconde commande que le programme se déconnecte du serveur même.

V-L-13 - Les messages d'erreurs possibles

Comme évoqué plus haut, le serveur **FTP** renvoie des messages pour indiquer comment se déroulent les transferts.

Les principaux codes sont constitués de trois chiffres. Le premier chiffre va de 1 à 5, le second de 0 à 5, et le dernier de 0 à 9.

Les codes vont ainsi de 100 à 559.

Il faut vous préoccuper principalement des codes 4xx et 5xx, indiquant des problèmes.

Voici, en résumé, la signification des principaux codes **FTP** (liste non exhaustive) :

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Code FTP	Signification
120	Service prêt dans xxx minutes
125	Connexion de données déjà ouvertes
150	Fichier OK
202	Commande non prise en compte
221	Le service a fermé la connexion
225	Aucun transfert en cours
226	Action demandée effectuée avec succès
227	Mode passif entrant
230	User authentifié
231	User déconnecté avec succès
232	Demande de connexion prise en compte
331	Mot de passe exigé
332	Autorisation demandée pour la connexion
350	Opération en suspens
421	Service non disponible / connexion fermée
425	Connexion impossible
426	Connexion fermée / transfert impossible
430	User ou mot de passe invalide
450	Fichier indisponible
451	Commande annulée
452	Anomalie espace mémoire
501	Erreur de syntaxe dans les paramètres
502	Commande non prise en compte
503	Mauvais ordre des commandes
504	Commande non prise en compte
530	User non reconnu
532	Autorisation exigée
550	Fichier indisponible
551	Commande interrompue
552	Commande de fichier interrompue
553	Action non prise en compte

V-L-14 - Exemple

L'exemple ici peut sembler superflu, mais néanmoins, il ne peut pas faire de mal.

```
1. #!/usr/bin/PYTHON
2. # -*- coding: utf-8 -*-
3.
4.
5.
6.
7. import ftplib
8.
9.
10.
11.
12. def connect_ftp():
13.     host = 'mon_serveur.linux.net'
14.     username = 'invite'
15.     password = 'invite'
16.
17.     ftpconnection = ftplib.FTP(host, username, password)
18.     ret = ftpconnection.getwelcome()
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
19.     print ret
20.     ftpconnection.quit()
21.     ftpconnection.close()
22.
23.
24.
25.
26. connect_ftp()
```

Une petite précision concernant la ligne 18. La commande utilisée permet de récupérer le message de bienvenue envoyé par le serveur à la connexion.

V-M - Les mails avec SMTP

Nous allons rapidement voir ici comment envoyer des mails.

L'envoi de mails est couramment utilisé dans des scripts **PYTHON** d'administration afin de prévenir les personnes concernées de possibles anomalies dans les traitements, ou tout simplement leur adresser un compte rendu.

Pour la gestion de la réception, je vous renvoie à la documentation du module que nous allons utiliser ici : **smtplib**.

```
import smtplib
```

Le but ici est de vous donner les bases, non pas pour créer un client mail, mais pour savoir implémenter l'envoi de rapports par mail depuis vos scripts.

V-M-1 - Données nécessaires

La première chose à faire est de préparer les données.

Pour envoyer un mail, vous aurez besoin des éléments suivants :

=>adresse d'un serveur **SMTP** ;

=>adresse d'expéditeur ;

=>adresse de destinataire ;

=>objet de mail ;

=>corps de mail ;

=>nom et mot de passe d'utilisateur pour le serveur **SMTP ;**

=>paramétrage de connexions au serveur **SMTP**.

V-M-2 - Exemple

Un exemple, valant mieux qu'un long discours, place au code :

```
1. #!/usr/bin/PYTHON
2. # -*- coding: utf-8 -*-
3.
4.
5.
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
6.
7. import smtplib
8. from email.MIMEText import MIMEText
9.
10.
11.
12.
13. def sendmail():
14.     user = 'user'
15.     passwd = 'pass'
16.
17.     msg = MIMEText('message du mail de test', _charset = 'utf8')
18.     msg['From'] = 'testo@gmail.com'
19.     msg['To'] = 'dest@yahoo.fr'
20.     msg['Subject'] = 'Mail de test'
21.
22.     smtp = smtplib.SMTP('smtp.gmail.com:587')
23.     smtp.starttls()
24.     smtp.login(user,passwd)
25.
26.     smtp.sendmail(msg['From'], msg['To'], msg.as_string())
27.
28.     smtp.close()
29.
30.
31.
32.
33. sendmail()
```

Un peu d'explications maintenant.

Ligne 14 et 15, nous déclarons simplement notre nom d'utilisateur et notre mot de passe pour nous identifier sur le serveur désiré.

De la ligne 17 à la ligne 20, nous définissons les paramètres de notre message.

Tout d'abord nous créons un type **MIMEtext**. Il s'agit d'un format de texte adapté pour les envois mails avec **SMTP**. Il faut le voir ici comme un conteneur.

Ce **MIMEtext** contiendra le corps texte de notre mail. Nous précisons ici que nous utilisons le format d'encodage UTF-8.

Ensuite, nous précisons les divers paramètres : From, To et Subject.

Ligne 22 à 24, nous nous connectons avec noter user au serveur **SMTP** (ici, celui de gmail), sur le port 587, en TLS.

Ligne 26, nous envoyons notre mail proprement dit, et enfin ligne 28, nous nous déconnectons du serveur **SMTP**.

V-N - Le port série

Le module à utiliser est **pyserial**. L'ensemble de la documentation est disponible ici : **http://pyserial.sourceforge.net/index.html**

On peut se poser la question de l'utilité d'un tel module. Dans les faits, beaucoup de matériels USB ne sont en fait que du matériel série ou parallèle avec un pont USB. Savoir gérer correctement ces ports se révèle par conséquent utile.

V-N-1 - Paramétrage

Voyons voir les éléments principaux de ce module. Voici un code présentant l'utilisation d'un **port série** sous Linux, de l'ouverture à la fermeture

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
1. import serial
2. ser = serial.Serial('/dev/ttyS0', 9600, timeout=0.008)
3. ser.write ("hello,world!!!")
4. x=ser.read()
5. ser.close()
```

Analysons cet exemple. Pour commencer, nous importons le module **pyserial**, ligne 1. Ligne 2, nous déclarons notre port: le port utilisé, le débit souhaité, et le timeout (temps à attendre un octet avant de passer à la suite).

Concernant le port série, il s'agit ici de la notation Linux, sur une machine de dev DEBIAN.

Ligne 3, l'écriture sur le port, se passe de tout commentaire, tout comme la lecture ligne 4, ou encore la fermeture du port ligne 5.

Si vous n'avez besoin que d'émettre et recevoir sur le port série, cette configuration vous suffira amplement.

Pour ceux qui auraient besoin de quelque chose de plus complet, nous allons approfondir. Tout d'abord le constructeur tel que défini sur le site officiel :

__init__(port=None, baudrate=9600, bytesize=EIGHTBITS, parity=PARITY_NONE, stopbits=STOPBITS_ONE, timeout=None, xonxoff=False, rtscts=False, writeTimeout=None, dsrdtr=False, interCharTimeout=None)

Nous pouvons d'ores et déjà constater que le port série est fortement configurable dans les moindres détails.

Quelques précisions sur certains de ces paramètres : Bytesize peut prendre quatre valeurs différentes (FIVEBITS, SIXBITS, SEVENBITS, EIGHTBITS), Parity cinq valeurs (PARITY_NONE, PARITY_EVEN, PARITY_ODD PARITY_MARK, PARITY_SPACE) et stopbits trois valeurs différentes (STOPBITS_ONE, STOPBITS_ONE_POINT_FIVE, STOPBITS_TWO).

Côté débit, les valeurs standard sont 50, 75, 110, 134, 150, 200, 300, 600, 1200, 1800, 2400, 4800, 9600, 19200, 38400, 57600, 115200.

Enfin, vous trouverez ci-dessous l'essentiel de la documentation traduite en français

Méthode	Utilité
Open()	Ouvrir le port
Close()	Fermer le port
read(size=1)	Lire un nombre d'octets égal à size (1 par défaut). Retourne le nombre d'octets lus
write(data)	Écrit des données sur le port. Retourne le nombre d'octets écrits
inWaiting()	Retourne le nombre d'octets en attente dans le buffer de réception
setBreak (**level=True**)	Sert à contrôler le TXD
setRTS (**level=True**)	Sert à contrôler le RTS
setDTR (**level=True**)	Sert à contrôler le DTR
getCTS ()	Retourne un booléen indiquant l'état du cts
getDSR ()	Retourne un booléen indiquant l'état du DSR
getRI ()	Retourne un booléen indiquant l'état du RI
getCD ()	Retourne un booléen indiquant l'état du CD
name	Retourne le nom du port ouvert
baudrate	Retournent les différentes valeurs de la configuration du port

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

bytesize	
parity	
stopbits	
timeout	
writeTimeout	
xonxoff	
rtscts	
dsrdtr	
interCharTimeout	
BAUDRATES	Retournent les différentes valeurs valides pour les paramètres. Idéal pour remplir une liste déroulante
BYTESIZES	
PARITIES	
STOPBITS	
serial.tools.list_ports.comports()	Renvoie une liste des ports disponibles (depuis la V2.6)

V-N-2 - Exemple

```
#! /usr/bin/env PYTHON
# -*- coding: utf-8 -*-



import serial



def mon_port_serie():
    """
        Émet une chaine de test puis indique le paramétrage
        A utiliser en mode root, pour les droits sur ttyS0
    """

    ser = serial.Serial('/dev/ttyS0', 9600, timeout=0.008)
    ser.write ("Test emission")
    print "Chaine emise sur port ", ser.name, " à ", ser.baudrate, "Bauds"
    ser.close()



if __name__ == '__main__':
    mon_port_serie()
```

V-O - Le port parallèle

Pour utiliser le **port parallèle** d'un PC nous utilisons le module **pyparallel**. Dans le code nous utiliserons un **import parallel.**

Le port parallèle est ici utilisé dans sa version originelle, c'est-à-dire unidirectionnel (et non à la norme IEEE1284).

Il existe d'autres modules pour piloter le port parallèle, mais celui-ci est le plus utilisé.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

V-O-1 - Paramétrage

Nous disposons donc d'un bus d'un octet en sortie, plus trois sorties indépendantes (Data Strobe, Auto Feed, Initialize). Côté entrées nous disposons de Select, Paper Out, et Acknoledge.

Voici l'exemple fourni sur le site officiel :

```
import parallel
p = parallel.Parallel()      # open LPT1
p.setData(0x55)
```

Le code se passe de commentaire au vu de la simplicité. Outre le **setData()**, il existe d'autres méthodes :

Methode	Utilité
setDataStrobe (**level**)	Configure la sortie Data Strobe
setAutoFeed (**level**)	Configure la sortie Auto Feed
setInitOut (**level**)	Configure la sortie Initialize
getInSelected ()	Récupère l'état de l'entrée Select
getInPaperOut ()	Récupère l'état de l'entrée Paper Out
getInAcknowledge ()	Récupère l'état de l'entrée Acknoledge

Ici, level vaut 1 ou 0.

V-O-2 - Exemple

```
#! /usr/bin/env PYTHON
# -*- coding: utf-8 -*-



import parallel



def mon_port_parallele():
    """
        Emet une chaine de test puis indique le parametrage
        A utiliser en mode root, pour avoir les droits
    """

    p = parallel.Parallel()      # open LPT1
    p.setData(0x55)
    print "Donnée 0x55 emise. Paper out à ", p.getInPaperOut()



if __name__ == '__main__':
    mon_port_parallele()
```

V-P - Le réseau: les bases

Le réseau quel que soit le langage utilisé est un vaste sujet. Nous ne nous attarderons ici qu'aux bases. Cela vous permettra d'établir une connexion sur un réseau, d'émettre et de recevoir des données.

Pour des opérations plus complexes, je vous invite à rechercher sur le Net, ou dans les sites répertoriés en fin de ce livre afin de trouver nombre d'exemples adaptés à vos besoins spécifiques.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

V-P-1 - Le module socket

Le module le plus usité pour le réseau est le module **socket**

```
import socket
```

Ce module est valable pour tous les OS modernes. Pour ceux qui ne seraient pas familiers avec ce concept, rappelons qu'un **socket** peut être résumé simplement en l'association d'une adresse et d'un port.

V-P-2 - Ouverture d'un port

L'ouverture d'un **port** réseau se passe en réalité en trois étapes. Pour commencer, on définit le port que nous désirons utiliser. En général, on désire passer des adresses IP, et utiliser le port en type STREAM. La ligne suivante est donc généralement à copier-coller directement.

```
sock= socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

Ensuite, nous définissons le timeout du port. Ce dernier est un float, que l'utilisateur définit comme bon lui semble.

```
sock.settimeout(3.0)
```

Enfin, nous ouvrons le port Ethernet en passant comme paramètres l'adresse réseau du destinataire, et le port sur lequel nous dialoguons.

```
sock.connect ((HOST,PORT))
```

Ici HOST vaudra par exemple 192.168.1.25 et PORT 9100, si nous dialoguons avec la machine dont l'adresse IP est 192.168.1.25, sur le port 9100. Notons que HOST pourrait tout aussi bien être une chaîne de caractères

V-P-3 - Envoi de données

L'envoi de données ne représente rien de bien sorcier ici. Il suffit d'utiliser la méthode **send()**, avec comme paramètre la chaîne de caractères.

```
sock.send('beep\r')
```

V-P-4 - Réception de données

Tout comme pour l'émission, la réception est très facile. La méthode à utiliser est **recv()**, qui prend comme paramètre le nombre maximum d'octets à lire en une fois

```
data=sock.recv(128)
```

Précisons qu'une fois la ligne précédente interprétée, si rien n'est reçu, passé le timeout, le programme poursuit sa route.

V-P-5 - Fermeture d'un port

Tout comme pour le port série, il suffit d'appeler ici la méthode **close** pour fermer un port.

```
sock.close()
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

V-P-6 - Exemple

```
#! /usr/bin/env PYTHON
# -*- coding: utf-8 -*-



import serial
import socket



HOST= '10.5.150.11'
PORT= 9100



def conversion():
    """
        Convertit une trame reseau en trame serie
    """

    ser = serial.Serial('/dev/ttyS0', 9600, timeout=0.008)

    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.settimeout(3.0)
    sock.connect ((HOST,PORT))

    while True:
        data = sock.recv(10)
        ser.write (data)
        sock.send("ACK")

    ser.close()
    sock.close()



if __name__ == '__main__':
    conversion()
```

V-Q - La webcam avec OpenCV

V-Q-1 - Utilisation basique

Pour utiliser la **webcam** avec **PYTHON**, via **OpenCV**, vous avez besoin de trois éléments : un objet de capture pour vous connecter au flux vidéo, un objet de frame pour réaliser un snapshot à un instant t du flux vidéo, et enfin un conteneur fenêtre (**GTK**, **KDE**, ou **OpenCV**, cas que nous verrons) afin d'afficher la vidéo.

Bien entendu, nous commencerons par l'import d'**OpenCV :**

```
import cv2.cv as cv
```

Remarque : selon les versions, l'import peut ne pas se faire de la même façon.

V-Q-1-a - Connexion au flux vidéo

Comme cité précédemment, la première chose à faire est de se connecter au flux vidéo. Pour cela nous allons créer un objet dédié à une caméra/webcam :

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
ma_caméra = cv.CaptureFromCAM(0)
```

le paramètre passé lors de la création de l'objet est un index permettant de définir quelle caméra utiliser. Lorsqu'il n'y a qu'une seule caméra, il faut passer 0 en paramètre.

Remarque : il existe la possibilité de remplacer le flux vidéo d'une webcam par le flux vidéo d'un fichier. Il ne faut alors pas utiliser cv.CaptureFromCAM(index), mais cv.CaptureFromFile(fichier).

V-Q-1-b - Snapshot du flux

Bien que nous puissions afficher directement le flux vidéo à l'écran, cette solution est rarement retenue.

En effet, passer par une série de snapshot possède un certain nombre d'avantages telles l'application d'un traitement sur image, la possibilité de sauvegarder certains snapshots en direct…

Nous allons donc utiliser un objet nommé « **frame** ». Dans le monde vidéo, une frame est tout simplement une image. On ne parle donc pas de 25 images/seconde, mais de 25 frames/seconde.

```
ma_frame = cv.QueryFrame(ma_caméra)
```

Lorsque nous créons cette **frame**, nous lui passons en paramètre le flux vidéo auquel elle est liée.

La **frame** est ainsi l'équivalent d'un snapshot à un instant t du flux vidéo.

V-Q-1-c - Affichage des snapshots

Enfin, pour afficher la vidéo, il faut une fenêtre conteneur. Dans vos futurs développements, vous inclurez cela sans nul doute dans une fenêtre **GTK** ou **KDE**.

Il existe aussi la possibilité toute simple de l'afficher à l'écran.

```
cv.NamedWindow("Test_Webcam")
```

Cela indique à **PYTHON** que nous utiliserons une fenêtre **OpenCV** pour l'affichage qui se nommera Test_Webcam.

Il ne reste ensuite plus qu'à afficher le conteneur en lui passant en paramètres la fenêtre à utiliser ainsi que la frame à afficher, puis tourner en boucle.

V-Q-1-d - Exemple d'affichage d'un flux webcam

```
1. #! /usr/bin/env PYTHON
2. # -*- coding: utf-8 -*-
3.
4. import cv2.cv as cv
5.
6. if __name__=='__main__':
7.     ma_caméra = cv.CaptureFromCAM(0)
8.     cv.NamedWindow("Test_Webcam")
9.     while True:
10.         ma_frame = cv.QueryFrame(ma_caméra)
11.         cv.ShowImage("Test_Webcam", ma_frame)
12.
13.         if (cv.WaitKey(10) % 0x100) == 113:
14.             break
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Lignes 13 et 14, il s'agit d'une méthode que nous verrons plus loin. Cela permet de lire le clavier et de sortir du programme lorsque la touche « q » est appuyée.

V-Q-2 - Utilisation avancée

Comme vous vous en doutez sûrement, **OpenCV** ne se contente pas de ces fonctionnalités basiques. Nous en verrons un peu plus, plus loin, mais nous allons ici nous concentrer sur les trois objets de base que nous serons régulièrement amenés à utiliser.

V-Q-2-a - Les images

Le premier de ces objets est l'image. En effet, avec **OpenCV**, il faut différencier image et **frame**.

SI la **frame** est bien une image capturée à partir du flux vidéo, on ne peut réaliser aucun traitement dessus. Elle ne peut servir qu'à un affichage direct (comme vu précédemment), ou comme source pour un traitement d'imagerie.

Ces traitements sont nombreux, et nous en verrons quelques-uns, mais l'outil de base est un objet image.

```
mon_image = cv.CreateImage(cv.GetSize(ma_frame), cv.IPL_DEPTH_8U, 1)
```

N° param	Nom	Type	Valeur par défaut	Rôle/ description
1	SIZE	CVSIZE	-	Taille de l'image (largeur et hauteur)
2	DEPTH	INT	-	Type d'image
3	CHANNELS	INT	-	Nombre de canaux

Le premier paramètre est un tuple indiquant la largeur et la hauteur de l'image. La plupart du temps pour éviter tous soucis, nous réalisons un **GetSize** depuis la source.

Le second argument est le type d'image : 8, 16, 32 ou 64 bits. Cela joue sur l'image finale. Par exemple, une image en nuance de gris sera en 8 bits. À noter qu'il s'agit ici de constante **OpenCV**.

Paramètre	Description
IPL_DEPTH_8U	Image 8 bits non signée
IPL_DEPTH_8S	Image 8 bits signée
IPL_DEPTH_16U	Image 16 bits non signée
IPL_DEPTH_16S	Image 16 bits signée
IPL_DEPTH_32S	Image 32 bits signée
IPL_DEPTH_32F	Image 32 bits float simple précision
IPL_DEPTH_64F	Image 64 bits float double précision

Remarque : beaucoup de traitements que nous verrons par la suite nécessitent des images de type cv.IPL_DEPTH_8U.

Enfin le dernier paramètre est le nombre de canaux. Entendez ici le nombre de couleurs : 1 pour une image en nuance de gris (grayscale), 3 pour une image classique RGB (couleur), 4 pour une image RGB avec canal alpha (transparence).

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Remarque : si votre logiciel nécessitait de charger une photo, par exemple, afin d'y appliquer des traitements via OpenCV, vous pouvez utiliser cv.LoadImage(Fichier, -1).

À noter que la valeur du pixel (x,y) d'une image est accessible via **img[x,y]**, que la largeur de l'image est **image.width** et sa hauteur **image.height**.

V-Q-2-b - Les enregistreurs vidéo

OpenCV offre la possibilité d'enregistrer les frames sous la forme d'une vidéo de manière très simple.

Il faut tout d'abord créer l'objet dédié.

```
mon_rec = cv.CreateVideoWriter(" ma_video.avi ", mon_codec, 25,cv.GetSize(ma_frame), 0)
```

N° param	Nom	Type	Valeur par défaut	Rôle/ description
1	FILENAME	CHAR	-	Nom du fichier de sortie
2	FOURCC	INT	-	TYPE de CODEC à utiliser
3	FPS	DOUBLE	-	Nombre de frames/seconde
4	FRAME_SIZE	CVSIZE	-	Taille des frames vidéo
5	IS_COLOR	INT	1	Si <> 0, mode couleur, grayscale sinon

Le premier paramètre est le chemin et le nom de la vidéo. Ici la vidéo sera enregistrée dans le dossier courant.

Le second paramètre est un objet **CV_FOURCC**. Il s'agit d'un objet auquel on passe en paramètres les quatre lettres du codec désiré.

```
mon_codec = cv.CV_FOURCC('D','I','V','X')
```

Les **codecs** les plus courants sont les suivants :

Paramètre	Description
PIM1	MPEG1
MJPG	Motion JPEG
DIVX	MPEG4
H264	H264
U263	H263
I263	H263 entrelacé
FLV1	FLV1

Le troisième paramètre est le nombre d'images désirées par seconde.

Le quatrième paramètre est la taille de la vidéo. Une fois de plus, on se calera sur la taille de la frame pour éviter tout problème.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Le cinquième et dernier paramètre permet d'indiquer si nous désirons un enregistrement en noir et blanc (0) ou en couleur (1).

Une fois l'objet enregistreur créé, il suffit d'y faire appel via une méthode **OpenCV**.

```
cv.WriterFrame(mon_rec, ma_frame)
```

Dans notre cas d'exemple, au bout de 25 appels à cette méthode, le film possédera donc une seconde de vidéo.

Remarque : dans le cas où vous ne désireriez pas enregistrer une vidéo, mais simplement une image, il faut utiliser la méthode cv.SaveImage(fichier, Image).

V-Q-2-c - Les polices d'écriture

Le troisième et dernier objet complémentaire que nous verrons ici est le texte. Il peut être très utile de saisir du texte sur une image pour identifier la date et l'heure, voire le lieu.

Cependant, actuellement seule une police d'écriture est supportée.

Tout comme pour l'enregistrement vidéo, cela aura lieu en deux étapes : création de l'objet puis appel d'une méthode **OpenCV**.

```
ma_police = cv.InitFont(cv.CV_FONT_HERSHEY_SIMPLEX, 1,1,0,1,8)
```

N° param	Nom	Type	Valeur par défaut	Rôle/ Description
1	FONTFACE	INT	-	Nom de la police
2	HSCALE	DOUBLE	-	Échelle horizontale
3	VSCALE	DOUBLE	-	Échelle verticale
4	SHEAR	DOUBLE	0	Inclinaison
5	THICKNESS	INT	1	Épaisseur des traits
6	LINETYPE	INT	8	Type de traits

Le premier paramètre est la taille et le type de la police.

Paramètre	Description
CV_FONT_HERSHEY_SIMPLEX	Sans serif, taille normale
CV_FONT_HERSHEY_PLAIN	Sans serif, petite taille
CV_FONT_HERSHEY_DUPLEX	Sans serif, taille normale
CV_FONT_HERSHEY_COMPLEX	Taille normale avec serif
CV_FONT_HERSHEY_TRIPLEX	Taille normale avec serif
CV_FONT_HERSHEY_COMPLEX_SMALL	Petite taille avec serif
CV_FONT_HERSHEY_SCRIPT_SIMPLEX	Effet « écriture à la main »
CV_FONT_HERSHEY_SCRIPT_COMPLEX	Effet « écriture à la main »

Les paramètres 2 et 3 sont les ratios en horizontal et vertical du texte par rapport à sa taille normale (100 % = 1.0).

Le quatrième paramètre est le degré d'inclinaison de l'italique. Sans italique, ce paramètre vaut 0. La valeur 1.0 indique une inclinaison d'environ 45 °.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Le cinquième paramètre sert à gérer le paramètre « GRAS » du texte. Sa valeur par défaut est de 1. Il s'agit d'un entier.

Le dernier paramètre est également un entier, avec comme valeur par défaut 8. Cela permet d'indiquer si le texte doit être souligné ou non.

Une fois l'objet créé selon notre besoin, il ne reste plus qu'à incruster le texte dans l'image. Pour cela nous utiliserons la méthode **PutText()**.

```
cv.PutText(ma_frame, mon_texte, emplacement_texte, ma_police, 1)
```

Nous passons plusieurs paramètres à cette méthode.

Tout d'abord, la frame (ou l'image au choix) impactée. Viennent ensuite le texte à afficher, puis l'emplacement du coin inférieur gauche du texte (en pixels).

Suit notre objet police et un paramètre booléen. SI ce dernier vaut 1 (ou True), la référence (le pixel (0,0)) se situe au coin supérieur gauche, sinon au coin inférieur gauche.

V-Q-2-d - Utilisation du clavier

OpenCV vous laisse la liberté d'interagir avec le clavier, afin de déclencher des traitements, de sortie du programme…

Pour cela, il faut utiliser la méthode **WaitKey(tps_en_ms)**. Pendant x ms, la méthode écoutera alors le clavier à la recherche d'une touche appuyée.

Le code renvoyé est un entier. Il faut effectuer un masque (% 0x100) avant de le comparer à une valeur décimale (ex. : 113 pour « q »).

Remarque : le WaitKey peut ne pas fonctionner si vous n'avez pas créé de fenêtre conteneur OpenCV. Dans le cas d'une intégration dans une IHM GTK ou autre, pensez à utiliser votre IHM pour interagir avec l'utilisateur.

V-Q-3 - Paramétrage de la webcam

Il est possible de paramétrer ou de lire le paramétrage de la webcam via les méthodes **SetCaptureProperty()** et **GetCaptureProperty()**.

Les paramètres utiles accessibles sont les suivants :

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Paramètre	Description
CV_CAP_PROP_POS_MSEC	Position dans le film en millisecondes
CV_CAP_PROP_POS_FRAMES	Position dans le film en nombre de frames
CV_CAP_PROP_POS_AVI_RATIO	Position dans le film en pourcentage
CV_CAP_PROP_FRAME_WIDTH	Largeur du flux vidéo
CV_CAP_PROP_FRAME_HEIGHT	Hauteur du flux vidéo
CV_CAP_PROP_FPS	Nombre d'images/seconde du flux vidéo
CV_CAP_PROP_FOURCC	Code du codec du flux vidéo
CV_CAP_PROP_FRAME_COUNT	Nombre total de frames (fichier vidéo uniquement)
CV_CAP_PROP_BRIGHTNESS	Luminosité de l'image
CV_CAP_PROP_CONTRAST	Contraste de l'image
CV_CAP_PROP_SATURATION	Saturation de l'image
CV_CAP_PROP_HUE	Teinte de l'image
CV_CAP_PROP_GAIN	Gain de l'image
CV_CAP_PROP_EXPOSURE	Exposition de l'image
CV_CAP_PROP_CONVERT_RGB	Image N&B ou RGB

Tous ces paramètres ne sont pas forcément accessibles selon le périphérique utilisé.

V-Q-3-a - SetCaptureProperty

Il s'agit de la méthode de paramétrage. D'un emploi très simple, elle prend trois paramètres : l'objet de capture, la propriété à modifier, et la nouvelle valeur.

```
cv.PutText(ma_frame, mon_texte, emplacement_texte, ma_police, 1)
```

V-Q-3-b - GetCaptureProperty

La méthode pour lire le paramétrage, est également très simple d'emploi. Elle prend en paramètres l'objet de capture, ainsi que la propriété désirée.

```
nb_fps = cv.GetCaptureProperty(ma_caméra,  CV_CAP_PROP_FPS)
```

V-Q-4 - Traitements sur les frames

Comme explicité précédemment, le fait de travailler avec des frames plutôt qu'avec un flux vidéo brut va permettre d'appliquer des traitements aux images.

Vu le nombre important de possibilités avec **OpenCV**, nous nous limiterons aux principales méthodes.

V-Q-4-a - Les types de données OpenCV

Vous trouverez ci-après les principaux types de données que vous trouverez dans **OpenCV** et dans les pages qui suivent.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Paramètre	Description
CHAR	Grand classique, caractères alphanumériques
CVARR	Type le plus utilisé. Toute image et frame en OpenCV est un CVARR, un simple tableau de données.
CVMEMSTORAGE	Espace de stockage servant aux calculs dynamiques
CVPOINT	Tuple contenant des coordonnées x et y
CVSIZE	Taille d'un rectangle ou d'une image
CVSCALAR	Tuple contenant de un à quatre doubles
DOUBLE	Grand classique également, nombre à virgule
INT	Autre grand classique, des entiers

V-Q-4-b - AbsDiff

Cette méthode permet d'effectuer une soustraction entre deux images puis de passer l'ensemble des pixels en valeur absolue.

Ainsi, les pixels n'ayant pas changé donneront en sortie un pixel blanc, et ceux ayant changé seront en grayscale.

À l'issue de l'application de cette méthode, nous récupérons donc une image dont les pixels non blancs correspondent aux pixels différents entre les deux images.

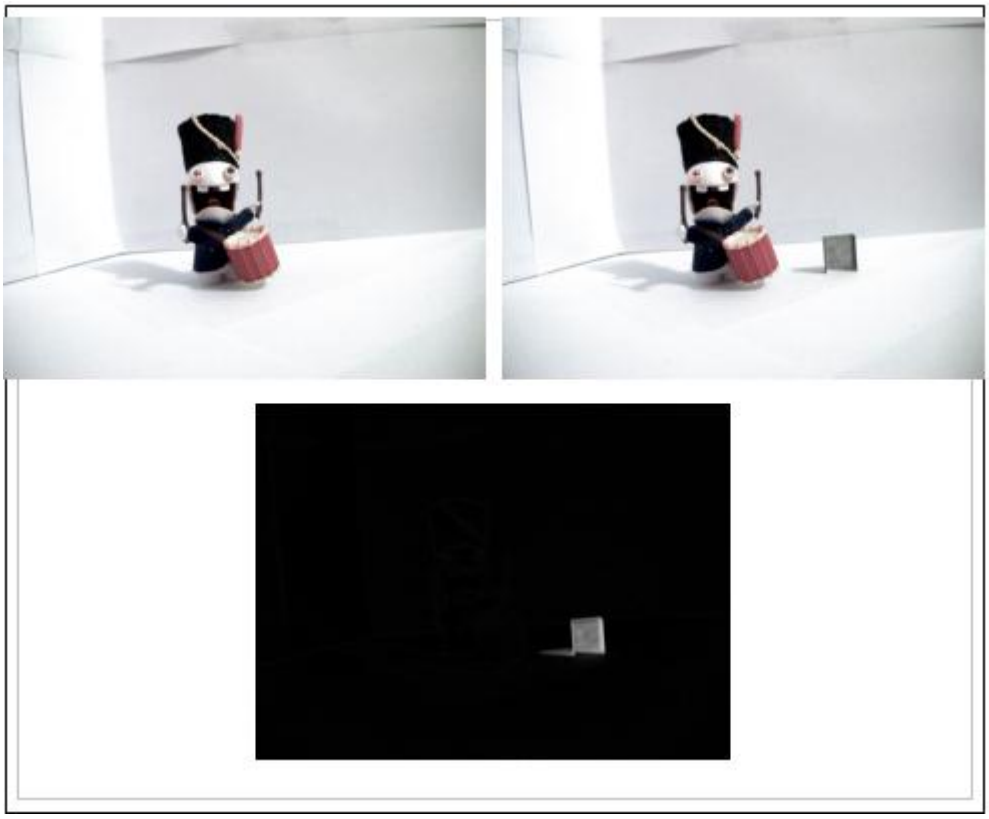
```
cv.AbsDiff(img_1, img_2, img_cible)
```

N° param	Nom	Type	Valeur par défaut	Rôle/ description
1	SRC1	CVARR	-	Image source 1
2	SRC2	CVARR	-	Image source 2
3	DST	CVARR	-	Image de sortie

Sans ambiguïté, nous voyons que les deux premiers paramètres sont les images/**frames** servant à la soustraction, et le dernier paramètre l'image où nous stockerons le résultat.

L'illustration ci-après montre le résultat que l'on peut obtenir grâce à un **ABSDIFF**. Bien entendu, il faudra d'abord effectuer diverses opérations dont des **CvtColor**, mais le résultat final est là.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/



Application du ABSDIFF

V-Q-4-c - CvtColor

Cette méthode permet de convertir une image d'un espace de couleur à un autre, ou plus généralement, une image couleur en niveaux de gris ou grayscale.

```
cv.cvtColor(frame_source, frame_cible, cv.CV_RGB2GRAY)
```

N° param	Nom	Type	Valeur par défaut	Rôle/ description
1	SRC	CVARR	-	Image source
2	DST	CVARR	-	Image de sortie
3	CODE	INT	-	Type d'encodage

Cette méthode est extrêmement simple, et convertit ici une image couleur en une image grayscale. Le dernier paramètre est une constante **OpenCV** stipulant le type de conversion désirée.

D'autres types de conversion que grayscale existent, mais beaucoup moins usitées. Aussi nous ne verrons que la **HSV**.

La HSV pour Hue/Saturation/Value ou teinte saturation valeur est un type d'image qui nous servira plus loin pour la détection de couleur. Seule la constante est à remplacer. Nous utiliserons alors **CV_BGR2HSV** ou CV_RGB2HSV, la différence consistant en l'inversion des canaux bleu et rouge.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/



Avant/après un CvtColor en grayscale

V-Q-4-d - Dilate

L'opération de dilatation permet de diminuer la finesse d'une image.

Attention : la référence ici n'est pas le pixel noir, mais le pixel blanc. Aussi pourriez-vous avoir la sensation que la méthode de dilate érode et que la méthode d'érosion dilate.

Prenez une image et un curseur carré. À l'intérieur du curseur la méthode va analyser les pixels et placer l'ensemble des pixels à la valeur la plus élevée disponible dans le curseur (rappel, la référence est le pixel blanc, soit 0 ; donc plus un pixel sera proche de 0, plus il sera considéré comme élevé).



Avant/après étape de dilatation

L'intérêt de cette méthode est de supprimer l'ensemble des petits défauts parasites, et d'obtenir une image à la fois représentative, mais également plus brute, plus dense et donc plus simple à analyser.

```
cv.Dilate(img_source, img_cible, None, Nb_dilate)
```

N° param	Nom	Type	Valeur par	Rôle/ description

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

			défaut	
1	SRC	CVARR	-	Image source
2	DST	CVARR	-	Image de sortie
3	ELEMENT	IplConv Kernel	NULL	Taille du curseur.
4	ITERATIONS	INT	1	Nombre d'itérations

Les deux premiers paramètres sont classiques.

Le troisième sera la plupart du temps à None. Il s'agit de la dimension du curseur à utiliser. Par défaut, via la valeur None, il sera de 3*3 pixels.

Le dernier paramètre enfin indique le nombre de fois que l'on doit appliquer la méthode de dilatation.

V-Q-4-e - Erode

L'opération d'érosion, comme son nom l'indique, consiste à affiner une image.

Le concept est l'exact opposé de celui de la dilatation, à savoir qu'à l'intérieur du curseur carré, le pixel central prend la valeur du pixel le moins élevé du curseur.

Nous obtenons ainsi une image affinée et là aussi potentiellement débarrassée des pixels parasites isolés.

Appliqué après un **Dilate**, un **Erode** permet d'obtenir en sortie une image brute de décoffrage, simplifiée et plus rapide à analyser.



Avant/après étape d'érosion

```
cv.Erode(img_source, img_cible, None, nb_erode)
```

N° param	Nom	Type	Valeur par	Rôle/ description

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

			défaut	
1	SRC	CVARR	-	Image source
2	DST	CVARR	-	Image de sortie
3	ELEMENT	IplConv Kernel	NULL	Taille du curseur.
4	ITERATIONS	INT	1	Nombre d'itérations

Les paramètres ont la même fonction que pour la fonction **Dilate**.

V-Q-4-f - Combinaison : Ouverture

Le but d'une ouverture est de supprimer les petits objets isolés ou pixels.

Cette opération ne fonctionne que si l'objet est blanc sur fond noir, étant donné que la référence est le pixel blanc.

Le résultat est obtenu en effectuant une opération **Dilate**, suivie d'une opération **Erode**.



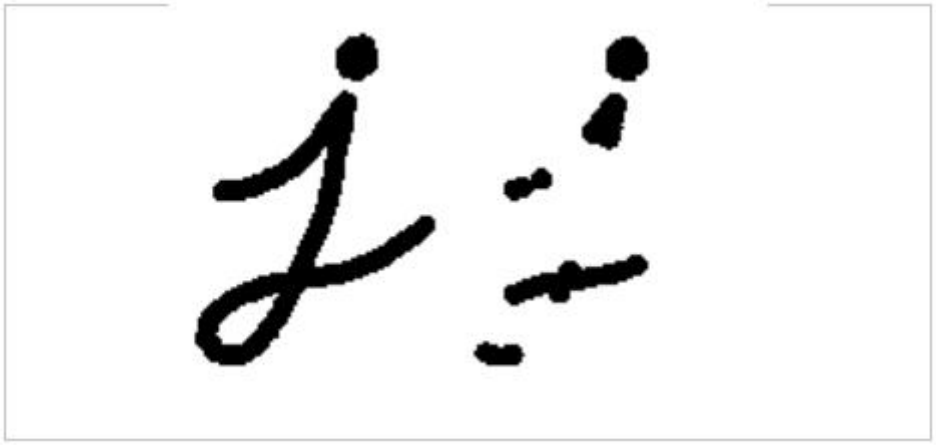
Avant/après une opération d'ouverture

V-Q-4-g - Combinaison : Fermeture

L'opération de fermeture vise à supprimer des taches noires pouvant exister et parasiter notre corps blanc sur fond noir.

Elle est obtenue en effectuant d'abord une opération **Erode**, suivie d'une opération **Dilate**.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/



Avant/après une opération de fermeture

V-Q-4-h - Combinaison : Gradient

L'opération de gradient sert à déterminer les contours d'une forme.

Elle est obtenue en faisant l'opération suivante: **Dilate - Erode**



Avant/après une opération de gradient

V-Q-4-i - Combinaison : Top Hat

L'opération Top Hat vise à isoler grossièrement notre forme.

Elle est obtenue en faisant : image - ouverture

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/



Avant/après un Top Hat

V-Q-4-j - Combinaison : Black Hat

L'opération de Black Hat permet, elle, d'inverser nos valeurs de pixels, en effectuant : fermeture - image



Avant/après un Black Hat

V-Q-4-k - ContourArea

Il s'agit d'une des deux méthodes (avec **DrawContours**) que l'on peut exécuter directement après un **FindContours**.

Le seul paramètre à passer est le retour du **FindContours**. La méthode nous renvoie alors la somme de toutes les aires détectées dans l'image.

Remarque : la référence est le pixel blanc, et non le noir.

```
surface = cv.ContourArea(contours)
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

V-Q-4-l - DrawContours

La seconde méthode exécutable après un **FindContours**. Elle permet tout simplement de tracer les contours détectés par la méthode **Findcontours**.

```
cv.DrawContours(ma_frame, NewContours, (0,0,255),(0,255,0),1,2,8)
```

N° param	Nom	Type	Valeur par défaut	Rôle/ description
1	IMG	CVARR	-	Image source (encodée en 8 bits)
2	CONTOURS	CVARR	-	Image de sortie
3	EXTERNAL_ COLOR	CVSCALAR	-	Couleur de ligne du contour externe
4	HOLE_COLOR	CVSCALAR	-	Couleur de ligne du contour interne
5	MAX_LEVEL	INT	-	Niveau de contour à dessiner. À 1, tous les contours sont dessinés
6	THICKNESS	INT	1	Épaisseur du trait. Si inférieur à 0 ou égal à cv.CV_FILLED, alors l'intérieur du contour est rempli
7	LINETYPE	INT	8	Type de trait

V-Q-4-m - EqualizeHist

Cette opération vise à optimiser l'histogramme de l'image pour permettre le meilleur traitement possible de l'image.

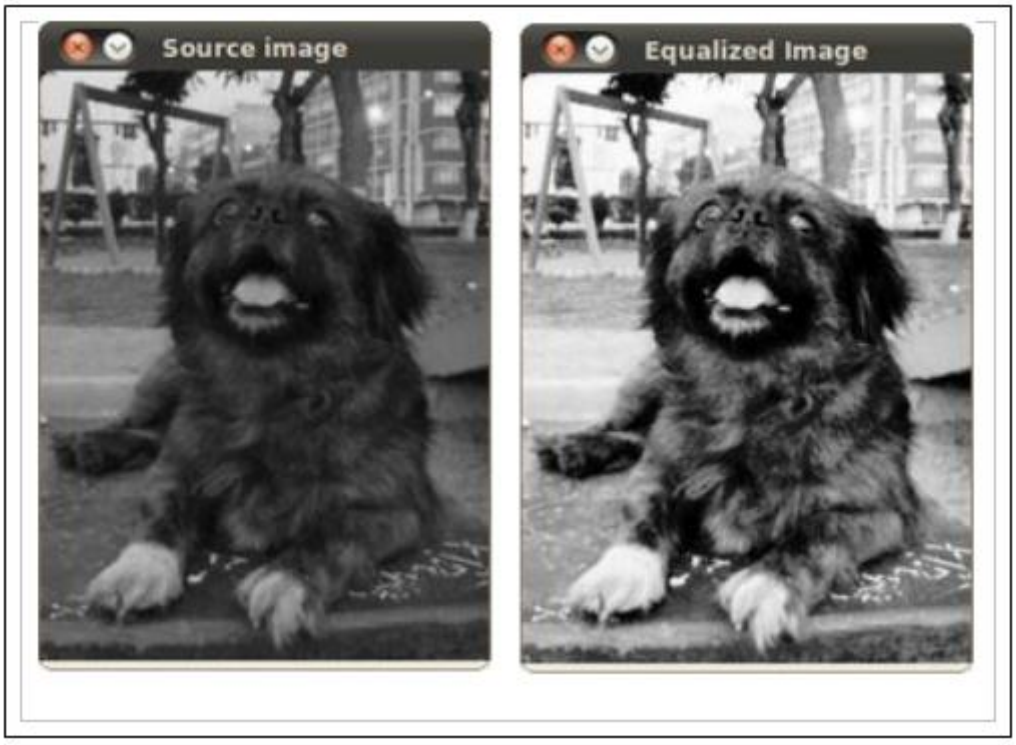
Pour rappel, l'histogramme d'une image correspond à la répartition de la teinte, du contraste, de la luminosité…

```
cv.EqualizeHist(img_source, img_cible)
```

N° param	Nom	Type	Valeur par défaut	Rôle/ description
1	SRC	CVARR	-	Image source (encodée en 8 bits)
2	DST	CVARR	-	Image de sortie

Rien de bien compliqué ici comme vous pouvez le constater.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/



Avant/après un EqualizeHist

V-Q-4-n - FindContours

Il s'agit d'une méthode permettant de trouver les contours d'une image, en lui fournissant simplement en entrée une image binaire.

```
contours = cv.FindContours(grey,storage,cv.CV_RETR_EXTERNAL,\
                           cv.CV_CHAIN_APPROX_SIMPLE)
```

N° param	Nom	Type	Valeur par défaut	Rôle/ description
1	IMAGE	CVARR	-	Image source binaire (encodée en 8 bits)
2	STORAGE	CVMEM STORAGE	-	Espace de calcul dynamique
3	MODE	INT	CV_RETR _LIST	Mode de détection de contours
4	METHOD	INT	CV_CHAIN_APPROX_S	Méthode de détection des contours

MODE peut prendre différentes valeurs :

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Paramètre	Description
CV_RETR_EXTERNAL	Ne recherche que les contours externes
CV_RETR_LIST	Recherche tous les contours sans distinction
CV_RETR_CCOMP	Recherche tous les contours puis les organise par priorité
CV_RETR_TREE	Recherche tous les contours, les trie, puis créé un fichier descriptif

METHOD peut également prendre différentes valeurs :

Paramètre	Description
CV_CHAIN_APPROX_NONE	Détecte l'intégralité des pixels composant un contour
CV_CHAIN_APPROX_SIMPLE	Détecte les contours en simplifiant ces derniers à des vecteurs. Utilise alors les coordonnées des points des extrémités composant les vecteurs
CV_CHAIN_APPROX_TC89_L1	Utilisation d'un algorithme de Chin basé sur la détection des points dominants dans l'image
CV_CHAIN_APPROX_TC89_KCOS	Autre algorithme de Chin

Vous verrez plus loin comment utiliser ces fonctions dans la partie sur les détections de contours avancés.

V-Q-4-o - HaarDetectObject

Cette méthode permet d'utiliser ce qu'on appelle des cascades Haar afin d'effectuer des détections diverses au sein d'une image.

C'est notamment via cette méthode que l'on peut réaliser de la détection de nez, des yeux, de visage, de main…

```
face = cv.HaarDetectObjects(img_src, haarcasc, storage, 1.2, 2, 0, (20, 20))
```

N° param	Nom	Type	Valeur par défaut	Rôle/ description
1	IMAGE	CVARR	-	Image source
2	CASCADE	CVHAAR	-	Algorithme de détection à utiliser
3	STORAGE	CVMEM STORAGE	-	Espace de mémoire alloué pour le calcul
4	SCALE_ FACTOR	DOUBLE	1.1	Spécifie le diviseur appliqué à l'image à chaque recherche
5	MIN_ NEIGHBORS	INT	3	Indique la précision de recherche à utiliser. Plus le chiffre sera

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

				élevé, plus la recherche sera fine
6	FLAGS	INT	0	Non utilisé, laisser à 0
7	MIN_SIZE	CVSIZE	(0,0)	Taille limite à laquelle la recherche doit s'arrêter

Une petite précision ici. La recherche part avec la taille d'origine de l'image. Si rien n'est trouvé, alors le traitement applique une division de l'image par **SCALE_FACTOR**, et ce jusqu'à atteindre **MIN_SIZE**.

Aussi, en cas de problème de fluidité, vous pouvez jouer sur ces deux paramètres.

V-Q-4-p - InRangeS

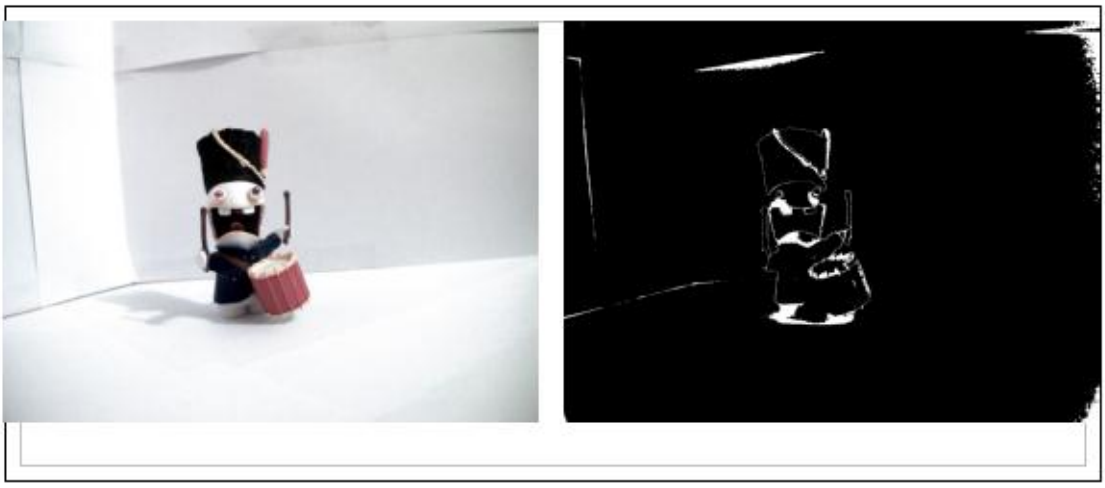
Derrière ce nom de méthode énigmatique se cache une fonction permettant de traiter en masse une image.

Je m'explique : il suffit de passer une image en entrée puis d'indiquer une limite basse de pixels, et une limite hausse de pixels. L'ensemble des pixels dont la valeur se situe entre ces deux seuils sera blanc dans l'image de sortie. Les autres pixels seront noirs.

Cette fonction sert principalement pour isoler des couleurs, chose que nous verrons plus loin en détail.

```
cv.InRangeS(img_source, cv.Scalar(h,s,v), cv.Scalar(h,s,v), img_cible)
```

N° param	Nom	Type	Valeur par défaut	Rôle/ description
1	SRC	CVARR	-	Image source
2	LOWER	CVSCALAR	-	Seuil bas (en mode HSV)
3	UPPER	CVSCALAR	-	Seuil haut (en mode HSV)
4	DST	CVARR	-	Image de sortie



Résultat d'un InRangeS

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

V-Q-4-q - Load

Load permet comme son nom l'indique de charger un fichier. Nous verrons plus loin comment l'utiliser pour la détection de visage.

```
cv.Load(filename)
```

N° param	Nom	Type	Valeur par défaut	Rôle/ description
1	FILENAME	CHAR	-	Fichier source à charger

V-Q-4-r - MatchTemplate

MatchTemplate est une méthode qui vous permet de rechercher une image au sein d'une autre image.

```
resultat = cv.MatchTemplate(img, template, method)
```

N° param	Nom	Type	Valeur par défaut	Rôle/ description
1	IMAGE	CVARR	-	Image d'entrée (8 ou 32 bits)
2	TEMPL	CVARR	-	Image à rechercher (même type que l'image d'entrée, et d'une taille inférieure ou égale)
3	METHOD	INT	-	Type de recherche

Comme on peut le constater ici, nous récupérons une image en retour de l'appel dans résultat.

Résultat est en réalité une image un peu particulière de par sa construction. Pour commencer, elle doit obligatoirement être de type **IPL_DEPTH_32F**. Ensuite, en admettant que la résolution de img soit W*H et celle de template de w*h, alors sa taille doit être strictement de (W-w+1)*(H-h+1).

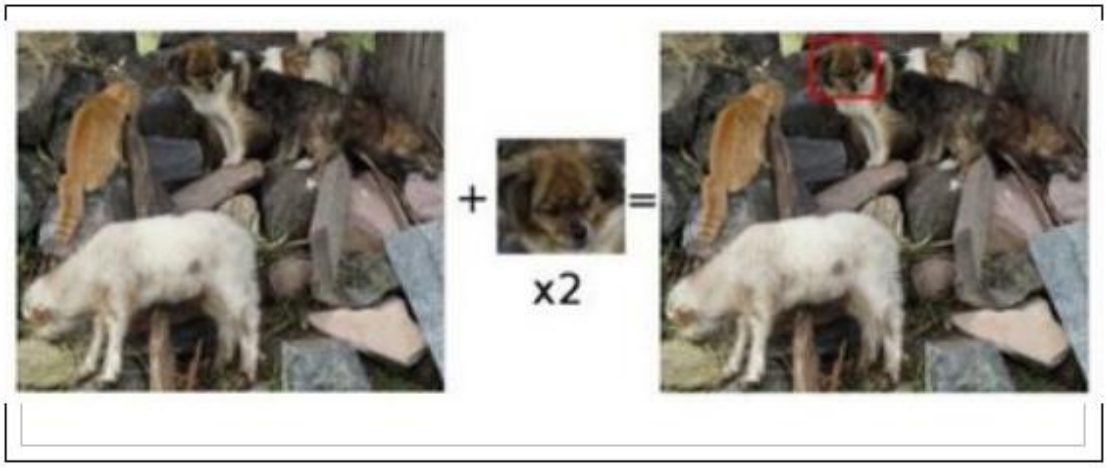
```
result = cv.CreateImage(((W-w+1),(H-h+1)),cv.IPL_DEPTH_32F, 1)
```

Remarque : pour rappel, la largeur d'une image est image.width et sa hauteur image.height.

Concernant les paramètres, précisons qu'il existe cinq types de méthodes différentes à essayer selon vos besoins.

Paramètre
CV_TM_SQDIFF_NORMED
CV_TM_CCORR
CV_TM_CCORR_NORMED
CV_TM_CCOEFF
CV_TM_CCOEFF_NORMED

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/



Résultat d'un MatchTemplate

V-Q-4-s - MemoryStorage

Cette méthode créé un espace de stockage dynamique utile pour les calculs effectués par certaines méthodes **OpenCV**.

L'unique paramètre correspond à la taille allouée. La valeur par défaut, 0, correspond à un espace de 64 K.

```
storage = cv.CreateMemStorage(0)
```

V-Q-4-t - MinMaxLoc

Cette méthode permet d'obtenir les coordonnées de points significatifs depuis une image binaire. Elle est très souvent utilisée après un **MatchTemplate.**

Remarque : pour rappel, la référence est le pixel blanc.

```
min_val, max_val, min_loc, max_loc = cv.MinMaxLoc(resultat)
(x,y) = min_loc
(x1,y1) = max_loc
```

N° param	Nom	Type	Valeur par défaut	Rôle/ description
1	IMAGE	CVARR	-	Image d'entrée (un canal)

En sortie, nous récupérons la valeur la plus faible puis la valeur la plus élevée. Ces deux retours sont rarement utiles dans les faits.

Suivent deux tuples : min_loc et max_loc.

Dans notre cas, x et y représenteront les coordonnées du coin supérieur gauche d'un rectangle et x1 et y1 les coordonnées du coin inférieur droit.

V-Q-4-u - RunningAvg

Il s'agit ici de tenir compte de l'évolution d'une scène saisie au fil du temps.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Chaque nouvelle image capturée est additionnée à ce qu'on appelle un accumulateur, puis on fait la moyenne de cette somme pour chaque pixel. L'image résultante est alors injectée dans l'accumulateur.

Il en résulte une image qui garde pendant un certain temps des traces des variations de l'image dans le temps.

Pour vous donner une image plus parlante, comparer cela avec un avion en vol. Quand vous le regardez dans le ciel, ses moteurs laissent des traînées de nuages derrière eux, dessinant le chemin de l'avion. Au bout d'un certain temps ces nuages se dissipent à l'extrémité de la traînée.

Le concept est ici exactement le même.

S'il existe différentes méthodes concernant les accumulateurs, c'est celle du **RunningAvg** qu'il faut retenir.

```
cv.RunningAvg(img_source, accu, ratio)
```

N° param	Nom	Type	Valeur par défaut	Rôle/ description
1	IMAGE	CVARR	-	Image d'entrée
2	ACC	CVARR	-	Image servant d'accumulateur
3	ALPHA	DOUBLE	-	Poids de l'image d'entrée

Les paramètres passés sont l'image à ajouter à l'accumulateur, l'accumulateur lui-même et un ratio.

Pour information, l'accumulateur n'est rien d'autre qu'une image. Le ratio quant à lui est compris entre 0.0 et 1.0. Plus ce ratio sera élevé, et plus la nouvelle image aura d'importance dans le calcul.

acc(x,y) = (1-ratio)*acc(x,y) + ratio * img(x,y)

On voit tout de suite l'importance qu'a ce ratio dans le temps durant lequel les traces resteront en mémoire.

V-Q-4-v - Smooth

Effectuer un « **Smooth** », c'est effectuer un lissage ou un moyennage.

Sur une image, cela revient, pour un pixel donné à prendre les x pixels autour, à en calculer la moyenne et à appliquer cette moyenne au pixel donné.

Quel intérêt cela peut-il avoir ? Prenez une image N&B contenant des pixels parasites noirs sur le fond blanc. Effectuer ce type de traitement va transformer les pixels parasites noirs en gris très clair.

En d'autres termes, lisser une image revient à atténuer les défauts. Nous pouvons considérer cela comme un nettoyage de l'image. Vous comprendrez tout l'intérêt de ce type de traitement à la lecture du **Threshold**.

```
cv.Smooth(img_source,img_cible,cv.CV_BLUR,p1,p2)
```

N° param	Nom	Type	Valeur par	Rôle/ description

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

			défaut	
1	SRC	CVARR	-	Image d'entrée
2	DST	CVARR	-	Image de sortie
3	SMOOTHTYPE	INT	CV_GAUSSIAN	Type de lissage
4	PARAM1	INT	3	Paramètre 1 de lissage
5	PARAM2	INT	0	Paramètre 2 de lissage

Les deux premiers paramètres ne posent, ici, pas de souci.

Le troisième est le type de lissage désiré. Les principaux à retenir sont les suivants.

Paramètre	Description
CV_BLUR	Moyennage linéaire sur un rectangle de p1 pixels par p2 pixels.
CV_GAUSSIAN	Moyennage gaussien sur un rectangle de p1 pixels par p2 pixels
CV_MEDIAN	Moyennage sur un carré de p1 pixels de côté

P1 et p2 représentent la taille du curseur de traitement à utiliser sur l'image, en pixels.



Application d'un Smooth

V-Q-4-w - Threshold

Appliqué sur une image grayscale, le **threshold** permet d'obtenir une image dite binaire.

Il s'agit ni plus ni moins que d'une image ne possédant que des pixels de deux couleurs : blanc ou non (noir en général).

En grayscale, chaque pixel possède une valeur comprise entre 0 et 255.

À partir d'un seuil passé en paramètre, et compris entre 0 et 255, le **threshold** générera une image binaire où les pixels d'un côté du seuil seront blancs et ceux de l'autre seront d'une valeur définie par l'utilisateur.

```
cv.Threshold(img_source, img_cible, seuil, val, cv.CV_THRESH_BINARY_INV)
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

N° param	Nom	Type	Valeur par défaut	Rôle/ description
1	SRC	CVARR	-	Image d'entrée
2	DST	CVARR	-	Image de sortie
3	TRESHOLD	DOUBLE	-	Valeur de seuil
4	MAXVALUE	DOUBLE	-	Valeur à utiliser, selon paramètre 5
5	TRESHOLDTYPE	INT	-	Type de sortie

Les trois premiers paramètres se passent de commentaires.

Le quatrième paramètre, val, est la valeur désirée pour les pixels non blancs. S'il est possible d'utiliser des nuances de gris, en général, nous passons toujours 255 comme valeur afin d'obtenir un pixel noir.

Le dernier paramètre définit le fonctionnement du **Threshold**.

Paramètre	Description
THRESH_BINARY	Si pixel > seuil, alors pixel = val, 0 sinon
THRESH_BINARY_INV	Si pixel > seuil, alors pixel = 0, val sinon
THRESH_TRUNC	Si pixel > seuil, alors pixel = seuil
THRESH_TOZERO	Si pixel > seuil, alors pixel = pixel, 0 sinon
THRESH_TOZERO_INV	Si pixel > seuil, alors pixel = 0, pixel = pixel sinon

V-Q-5 - Ajout d'élément sur une image/frame

V-Q-5-a - Rectangle

Pour dessiner un rectangle sur une image **OpenCV** c'est très simple via la méthode Rectangle.

```
cv.Rectangle(img, pt1, pt2, cv.CV_RGB(red, green, blue), epaisseur, type_ligne)
```

N° param	Nom	Type	Valeur par défaut	Rôle/ description
1	IMG	CVARR	-	Image d'entrée
2	PT1	CVPOINT	-	Coordonnées coin supérieur gauche
3	PT2	CVPOINT	-	Coordonnées coin inférieur droit
4	COLOR	CVSCALAR	-	Couleur des traits
5	THICKNESS	INT	1	Épaisseur du trait
6	LINETYPE	INT	8	Type de trait

Après avoir précisé à quelle image/**frame**, nous désirons appliquer le dessin, nous passons les coordonnées du coin supérieur gauche sous forme d'un tuple (x,y) (pt1) puis du coin inférieur droit (pt2) du rectangle.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Nous précisons ensuite la couleur via la méthode **cv.CV_RGB** qui nous permet de simplifier le choix de la couleur.

Épaisseur est de préférence à mettre à 1 ; -1 si vous souhaitez que le rectangle soit rempli.

type_ligne permet de préciser l'épaisseur du trait souhaité pour le traçage du rectangle.

Dans les faits, la nuance est subtile entre épaisseur et type_ligne et peu importante. Je vous conseille de gérer l'épaisseur du trait via type_ligne et de n'utiliser épaisseur que pour indiquer si le cercle doit être vide (valeur positive) ou non (valeur négative).

V-Q-5-b - Cercle

Dessiner un cercle n'est pas plus difficile qu'un rectangle.

```
cv.Circle(img, centre, rayon, cv.CV_RGB(red, green, blue), epaisseur, type_ligne)
```

N° param	Nom	Type	Valeur par défaut	Rôle/ description
1	IMG	CVARR	-	Image d'entrée
2	CENTER	CVPOINT	-	Coordonnées du centre
3	RADIUS	INT	-	Rayon
4	COLOR	CVSCALAR	-	Couleur des traits
5	THICKNESS	INT	1	Épaisseur du trait
6	LINETYPE	INT	8	Type de trait

Comme nous pouvons le voir, ici rien de sorcier. Nous passons d'abord l'image/**frame** impactée, puis un tuple contenant les coordonnées du centre, le rayon du cercle, sa couleur puis ses épaisseurs.

Sur ce dernier point, quelques précisions. Le paramètre épaisseur est relatif au tracé lui-même alors que le type de ligne correspond à l'extérieur du tracé.

V-Q-5-c - Ligne

Pour finir, le traçage d'une ligne.

```
cv.Line(img, pt1, pt2, cv.CV_RGB(red, green, blue), epaisseur, type_ligne)
```

N° param	Nom	Type	Valeur par	Rôle/ description

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

			défaut	
1	IMG	CVARR	-	Image d'entrée
2	PT1	CVPOINT	-	Coordonnées de départ
3	PT2	CVPOINT	-	Coordonnées d'arrivée
4	COLOR	CVSCALAR	-	Couleur du trait
5	THICKNESS	INT	1	Épaisseur du trait
6	LINETYPE	INT	8	Type de trait

En vous basant sur les rectangles et les cercles, je pense que le commentaire des paramètres est superflu.

V-Q-6 - Détection de mouvement

La détection de mouvement est un grand classique dans les exercices vidéo, au même titre que la détection de couleur.

Dans les faits comment cela se passe-t-il ?

On considère qu'il y a mouvement à partir du moment où nous constatons une différence entre deux scènes consécutives.

Au niveau d'une caméra/webcam, cela se traduit par un changement de pixels. Mais comment détecter cela, en informatique ? Il suffit de comparer les deux images.

Nous avons vu une fonction pour réaliser cela : **AbsDiff**.

En réalisant la soustraction des deux images, nous récupérerons une image blanche contenant les pixels ayant changé.

Mais cela peut s'avérer alors encore difficile, surtout si vos images sont en couleur.

La solution dans ce cas là est de ne pas réaliser **l'AbsDiff** sur vos images couleur, mais sur des images en grayscale.

L'image résultante de la comparaison peut alors être binarisée.

Rappelez-vous, cela passe par la méthode **Threshold**, et consiste à comparer les niveaux des pixels à un seuil puis selon leur position par rapport à ce dernier, à les passer en pixel blanc ou noir.

Il ne vous reste plus qu'à comparer la proportion de pixels noirs rapport aux pixels blancs pour déterminer si l'image semble avoir détecté un mouvement digne d'attention.

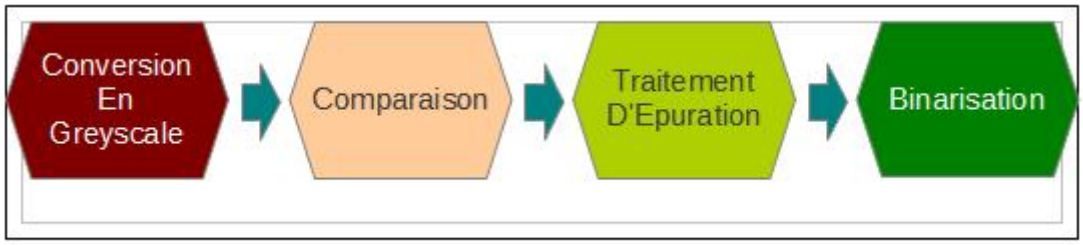
Le problème que vous constaterez est que parfois des petits pixels parasites s'invitent et influent trop sur le résultat final. Il faut alors jouer avec des méthodes telles **Smooth**, **Erode** et **Dilate** pour optimiser l'image pour traitement.

Comme vous le constatez, détecter un mouvement est moins difficile qu'il n'y paraît.

Bien entendu, cela est ici simplifié, mais le principe est viable.

Résumons tout cela par un schéma simple.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/



Chaîne de traitement pour la détection de mouvement

V-Q-6-a - Exemple

```
#! /usr/bin/env PYTHON
# -*- coding: utf-8 -*-



import cv2.cv as cv
import time



if __name__=='__main__':
    """
        Detection de mouvement, appui sur 'q' pour quitter
    """
    capture = cv.CaptureFromCAM(0)
    cv.NamedWindow("Test_Webcam")
    while True:
        frame = cv.QueryFrame(capture)
        grey1 = cv.CreateImage(cv.GetSize(frame), cv.IPL_DEPTH_8U, 1)
        grey2 = cv.CreateImage(cv.GetSize(frame), cv.IPL_DEPTH_8U, 1)
        res = cv.CreateImage(cv.GetSize(frame), cv.IPL_DEPTH_8U, 1)
        cv.CvtColor(frame, grey1, cv.CV_RGB2GRAY)
        time.sleep(0.001)
        frame = cv.QueryFrame(capture)
        cv.CvtColor(frame, grey2, cv.CV_RGB2GRAY)
        cv.AbsDiff(grey1, grey2, res)
        cv.Smooth(res, res, cv.CV_BLUR, 5,5)
        cv.Threshold(res,res,15,255,cv.CV_THRESH_BINARY_INV)
        cv.ShowImage("Test_Webcam", res)

        if (cv.WaitKey(10) % 0x100) == 113:
            break
```

V-Q-7 - Reconnaissance des couleurs

Après avoir vu la détection de mouvements, nous allons monter un peu en difficulté : la détection de couleur.

Beaucoup d'entre vous se disent sans doute qu'il suffit de choisir sa couleur et de la traquer.

J'avoue que c'est également ce que je me disais avant d'attaquer la théorie et la pratique ; mais malheureusement cela est un peu plus compliqué.

En effet, une image telle que nous la connaissons est codée en RGB. Autrement dit, quelle que soit la couleur choisie, à l'exception du rouge/bleu/vert pur, il y aura toujours une composante d'une autre couleur venant parasiter le résultat.

Nous allons donc devoir changer de type d'encodage. Nous nous dirigerons vers du HSV (Hue/Saturation/Value, ou en français Teinte/Saturation/Luminosité).

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Pourquoi ce format ? À cause de sa simplicité dans ce type d'application.

En HSV, la teinte représente la couleur désirée, la saturation représente l'intensité de la couleur, et la luminosité la brillance de la couleur.

En d'autres termes, le seul paramètre dont nous avons à nous soucier dans ce cas est la teinte.

Comme le montre l'échelle de teintes ci-après, il est simple de choisir sa couleur.

Remarque : OpenCV utilise des demies valeurs sur le paramètre Teinte. Autrement dit, pensez à diviser les valeurs issues de l'échelle ci -après par 2.



Échelle de teintes issues de Wikipédia (Huescale.svg)

Ou une fois converties en valeur **OpenCV**.

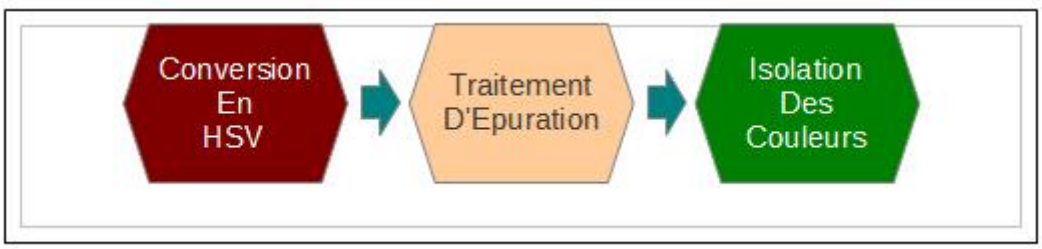
0 à 15	Rouge
15 à 45	Jaune
45 à 75	Vert
75 à 105	Cyan
105 à 135	Bleu
135 à 165	Magenta
165 à 180	Rouge

Bien entendu, il faut par la suite ajuster le tir par l'expérimentation afin de toujours diminuer les parasites et/ou appliquer un traitement d'ouverture pour améliorer le résultat de sortie.

La saturation et la luminosité seront en général prises entre 0 et 255.

Nous utiliserons la méthode **InRangeS** afin d'isoler notre couleur recherchée et générer l'image finale.

Remarque : le seuil bas doit toujours contenir la valeur la plus petite et ne peut être inférieur à 0. Le seuil haut doit toujours contenir la valeur la plus haute et ne peut être supérieur à 180.



Chaîne de traitement d'une détection de couleur

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

V-Q-7-a - Exemple

```
#! /usr/bin/env PYTHON
# -*- coding: utf-8 -*-

import cv2.cv as cv

if __name__=='__main__':
    """
        Permet de suivre la couleur bleu, appui sur 'q' pour quitter
    """
    ma_camera = cv.CaptureFromCAM(0)
    cv.NamedWindow("Test_Webcam")
    while True:
        ma_frame = cv.QueryFrame(ma_camera)
        ma_frame2 = cv.CreateImage(cv.GetSize(ma_frame), 8, 3)
        ma_frame3 = cv.CreateImage(cv.GetSize(ma_frame2), 8, 1)
        cv.CvtColor(ma_frame, ma_frame2, cv.CV_BGR2HSV)
        #Recherche du bleu
        cv.InRangeS(ma_frame2,cv.Scalar(90, 0, 0), \
                cv.Scalar(130, 255, 255), ma_frame3)
        cv.ShowImage("Test_Webcam", ma_frame3)

        if (cv.WaitKey(10) % 0x100) == 113:
            break
```

V-Q-8 - Détection de contours

Montons encore d'un petit cran pour atteindre la détection de contours.

Si vous vous rappelez bien, nous avons vu cela un peu plus tôt. On appelle cela un **GRADIENT**. Il s'agit tout simplement d'une opération **d'Erode** soustraite à une opération de **Dilate**.

Pour effectuer la différence, il suffit d'utiliser un **AbsDiff**. Nous récupérons alors en sortie une image ne contenant que les contours des formes, en blanc sur fond noir.

V-Q-8-a - Exemple basique

```
#! /usr/bin/env PYTHON
# -*- coding: utf-8 -*-




import cv2.cv as cv
import time



if __name__=='__main__':
    capture = cv.CaptureFromCAM(1)
    cv.SetCaptureProperty(capture, cv.CV_CAP_PROP_FRAME_WIDTH , 1200)
    cv.SetCaptureProperty(capture, cv.CV_CAP_PROP_FRAME_HEIGHT, 1200)
    cv.NamedWindow("Test_Webcam")
    cv.ResizeWindow("Test_Webcam", 200,200)
    while True:
        frame = cv.QueryFrame(capture)
        grey1 = cv.CreateImage(cv.GetSize(frame), cv.IPL_DEPTH_8U, 1)
        grey2 = cv.CreateImage(cv.GetSize(frame), cv.IPL_DEPTH_8U, 1)
        grey3 = cv.CreateImage(cv.GetSize(frame), cv.IPL_DEPTH_8U, 1)
        res = cv.CreateImage(cv.GetSize(frame), cv.IPL_DEPTH_8U, 1)
        cv.CvtColor(frame, grey1, cv.CV_RGB2GRAY)
        cv.CvtColor(frame, grey2, cv.CV_RGB2GRAY)
        cv.Erode(grey1, grey1, None, 3)
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
        cv.Dilate(grey2, grey2, None, 3)
        cv.AbsDiff(grey1, grey2, grey3)
        cv.ShowImage("Test_Webcam", grey3)

        if (cv.WaitKey(10) % 0x100) == 113:
            break
```

V-Q-8-b - Exemple avancé

Sur **OpenCV**, il existe une façon encore plus simple d'effectuer une détection de contours.

```
#! /usr/bin/env PYTHON
# -*- coding: utf-8 -*-



import cv2.cv as cv



if __name__=='__main__':
    ma_camera = cv.CaptureFromCAM(1)
    cv.NamedWindow("Test_Webcam")
    cv.SetCaptureProperty(ma_camera, cv.CV_CAP_PROP_FRAME_WIDTH , 1200)
    cv.SetCaptureProperty(ma_camera, cv.CV_CAP_PROP_FRAME_HEIGHT, 1200)
    while True:
        ma_frame = cv.QueryFrame(ma_camera)
        ma_frame2 = cv.CreateImage(cv.GetSize(ma_frame), 8, 3)
        ma_frame3 = cv.CreateImage(cv.GetSize(ma_frame2), 8, 1)
        cv.CvtColor(ma_frame, ma_frame2, cv.CV_BGR2HSV)
        #Recherche du rouge
        cv.InRangeS(ma_frame2,cv.Scalar(175, 0, 0),cv.Scalar(180, 255, 255), \
                                ma_frame3)
        #Traitement d'image
        cv.Erode(ma_frame3, ma_frame3, None, 3)
        cv.Dilate(ma_frame3, ma_frame3, None, 5)
        cv.Smooth(ma_frame3, ma_frame3, cv.CV_BLUR, 5,5)
        #Detection de contours
        storage = cv.CreateMemStorage(0)
        contours = cv.FindContours(ma_frame3, storage, \
                    cv.CV_RETR_EXTERNAL, cv.CV_CHAIN_APPROX_SIMPLE)
        NewContours = contours
        cv.DrawContours(ma_frame, NewContours, (0,0,255),(0,255,0),2,2,8)
        #IHM
        cv.ShowImage("Test_Webcam", ma_frame)

        if (cv.WaitKey(10) % 0x100) == 113:
            break
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/



Résultat d'une détection de contours avec traçage

V-Q-9 - Détection d'une forme

La détection d'une forme précise est quelque chose d'également assez simplifié dans **OpenCV**.

Il suffit juste d'avoir une image/frame à analyser, et la forme/image que l'on recherche dedans.

V-Q-9-a - Recherche d'une image dans l'image

Voici un petit code d'exemple :

```
1. #! /usr/bin/env PYTHON
2. # -*- coding: utf-8 -*-
3.
4.
5.
6.
7. import cv2.cv as cv
8.
9.
10.
11.
12. image = cv.LoadImage('test0.jpg')
13. reference = cv.LoadImage('test1.jpg')
14.
15. image_size = cv.GetSize(image)
16. reference_size = cv.GetSize(reference)
17. result_size = [ s[0] - s[1] + 1 for s in zip(image_size, reference_size) ]
18.
19. result = cv.CreateImage(result_size, cv.IPL_DEPTH_32F, 1)
20.
21. cv.MatchTemplate(image, reference, result, cv.CV_TM_CCORR_NORMED)
22.
23. min_val, max_val, min_loc, max_loc = cv.MinMaxLoc(result)
24. cv.Rectangle(image,max_loc,min_loc,(0, 0, 255), 3,8,0)
25. cv.SaveImage("test2.png", image)
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Voilà, 25 petites lignes pour tout faire. L'ensemble des éléments constituant ce code ont tous été vus en détail.

Lignes12 et 13 nous chargeons nos images.

Lignes 15 à 17, nous déterminons la taille précise que doit avoir l'image de sortie. Il s'agit d'une autre façon de coder le calcul à effectuer, tel que vu en .

Ligne 19, nous créons notre image de sortie, avec un seul canal, pour ne fonctionner qu'en greyscale.

Ligne 21, nous recherchons notre référence dans l'image à l'aide de la méthode **CV_TM_CCORR_NORMED**.

Ligne 23, nous récupérons différentes valeurs dont les coordonnées précises de détection permettant de tracer le rectangle, ligne 24.

Enfin, ligne 25, nous sauvegardons le résultat dans une nouvelle image.

V-Q-10 - Détection de visage

La détection de visage et de certains de ses éléments (yeux, bouche…) fait partie de ces choses assez complexes.

Avec **OpenCV**, cela se simplifie grandement. Nous allons essayer de détailler cela au mieux, mais Internet saura combler les questions qui subsisteront.

Comme dans tout traitement, la première chose à faire est de passer notre image en gris sur 8 bits.

Il faut ensuite créer un espace de stockage qui permettra de stocker les calculs de détection. Pour cela, on utilise **CreateMemStorage(0)**.

On va ensuite égaliser l'histogramme de l'image greyscale via un EqualizeHist.

Puis nous allons définir l'algorithme de détection que nous allons utiliser. Nous utiliserons par exemple haarcascade_frontalface_alt.xml pour le visage.

Cet algorithme est chargé via la méthode **Load.**

Enfin, il reste à détecter ce que l'on recherche à l'aide de l'algorithme via la méthode **cv.HaarDetectObjects**.

V-Q-10-a - Exemple

```
#! /usr/bin/env PYTHON
# -*- coding: utf-8 -*-



import sys
import cv2.cv as cv



def detect(image):
    image_size = cv.GetSize(image)
    grayscale = cv.CreateImage(image_size, 8, 1)
    cv.CvtColor(image, grayscale, cv.CV_RGB2GRAY)
    storage = cv.CreateMemStorage(0)
    cv.EqualizeHist(grayscale, grayscale)
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
    cascade = cv.Load('./haarcascade_frontalface_alt.xml')
    faces = cv.HaarDetectObjects(image, cascade, cv.CreateMemStorage(0), \
                            1.2, 2, 0, (20, 20))
    if faces:
        print 'face detected!'
        for i in faces:
             cv.Rectangle(image,(i[0][0], i[0][1]),(i[0][0] + i[0][2], i[0][1] + i[0][3]),\
                        (0, 255, 0), 3,8,0)



if __name__ == '__main__':
    cv.NamedWindow('caméra')
    capture = cv.CreatecaméraCapture(1)
    while 1:
        frame = cv.QueryFrame(capture)
        detect(frame)
        cv.ShowImage('caméra', frame)
        k = cv.WaitKey(10)
        if (cv.WaitKey(10) % 0x100) == 113:
            break
```

V-Q-11 - Création d'un algorithme de détection

Les **algorithmes Haar**, quoique complexes, peuvent être conçus maison.

Nous allons voir ici les grandes lignes des étapes permettant d'aboutir à la génération d'un fichier **XML** servant pour une détection **Haar**. Si certains points vous semblent flous, je vous invite alors à rechercher sur Internet le terme Haartraining.

Vous pourrez trouver notamment, plus d'informations sur la mesure de performance des algorithmes générés entre autres.

Cependant, avant de commencer, sachez que cette procédure est longue et fastidieuse. Ne partez pas sur l'idée de le concevoir en une nuit.

V-Q-11-a - Création des positifs

Il s'agit là de la première étape. Nous allons créer ce que l'on appelle des positifs. Il s'agit ni plus ni moins que d'images contenant l'objet que nous souhaitons savoir détecter, pris sous différents angles, et éventuellement également différents types d'éclairage. Plus vous en aurez, plus la détection sera efficace.

Remarque : à titre d'information, des algorithmes tels que celui permettant de détecter les visages ont été conçus avec plusieurs milliers de positifs et mis plusieurs semaines à générer le fichier XML de détection de visages.

Ces images serviront plus loin à générer le fichier vecteur.

Concernant le besoin d'avoir un fond neutre (couleur unie) ou non (par exemple une pièce avec ses meubles), cela ne semble pas avoir d'importance particulière.

Cependant, si un fond neutre vous simplifiera la vie plus loin, un fond non neutre permettra également, dans certains cas, d'améliorer la détection de l'objet désiré.

Mon conseil est de privilégier un fond neutre, mais d'augmenter la quantité de positifs et de négatifs pour compenser.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

V-Q-11-b - Création des négatifs

Complément des positifs, les négatifs sont des images ne contenant absolument pas l'objet que nous souhaitons détecter.

Idéalement, il faut la même quantité de négatifs que de positifs. Cela permettra d'apprendre à l'algorithme (si l'on peut utiliser cette expression) à ne pas détecter l'objet.

V-Q-11-c - Marquage des positifs

Étape suivante indispensable, le marquage consiste à préciser dans les positifs où se trouve l'objet que nous souhaitons détecter, afin que l'algorithme sache quoi chercher.

Cela se fait avec l'utilitaire **ObjectMarker** (à charger sur le net, puis à compiler : objectmarker.cpp ; ou charger l'équivalent **PYTHON**).

Pour le lancer depuis un terminal, il suffit de taper

```
ObjectMarker fichier_sortie.txt <chemin ou se trouvent les positifs>
```

Dans cet exemple, fichier_sortie.txt est généré par le traitement. Il contiendra pour chaque image, la référence de l'image, ainsi que les coordonnées de l'objet à détecter.

Il s'agit là d'une étape longue et fastidieuse, mais nécessaire. N'hésitez pas à vous appliquer. Plus cette étape sera faite avec soin, plus fiable sera la détection finale.

Pour marquer l'objet, sur l'image qui apparaît, imaginer un rectangle encadrant l'objet à détecter, et cliquer à l'emplacement du coin supérieur gauche, puis à l'emplacement du coin inférieur droit.

Attention : seuls ces deux coins doivent être utilisés. En effet, le traitement ne sait pas gérer les autres coins pour générer le fichier de sortie.

Si jamais le rectangle tracé vous semble inadéquat, il suffit de recommencer l'opération en cliquant sur le coin supérieur gauche puis le coin inférieur droit.

Si le rectangle vous convient, il faut alors taper sur espace pour le valider, puis de taper sur <ENTREE> pour sauvegarder et passer à l'image suivante.

Une fois la totalité des marquages terminés, il suffit de taper sur <ESCAPE> pour sauvegarder le fichier de sortie et fermer ObjectMarker.

Le fichier de sortie contiendra entre autres le nom de l'image, les coordonnées des coins du rectangle sélectionné, et la largeur et hauteur de ce dernier.

V-Q-11-d - Création du fichier vecteur

Une fois les positifs marqués, il faut empaqueter l'ensemble des images dans un fichier vectoriel, appelé fichier vecteur.

Pour ce faire, nous utilisons un utilitaire fourni par **OpenCV** qui s'appelle opencv-createsamples.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Nous lui fournirons comme paramètres le chemin du fichier généré à l'étape précédente (via l'option -info) ; le chemin de sortie, ainsi que le nom désiré pour le fichier vecteur (via l'option -vec) ; la hauteur désirée de l'échantillon, pour la future détection, ainsi que sa largeur (via les options respectives -h et -w).

```
opencv-createsamples -info ./output.txt -vec ./pos.vec -w 50 -h 50
```

V-Q-11-e - Création du classificateur

L'ultime étape. Je ne saurais dire si le terme de « classificateur » correspond à une traduction juste. Je pense que nous parlerons d'algorithme de **Haar**.

Car c'est cela que nous allons voir ici : sa création.

Nous allons pour cela utiliser un autre outil fourni par **OpenCV :** opencv-haartraining.

```
Opencv-haartraining -data ./haar_algo -vec ./pos.vec -bg ./neg.txt \
                           -npos 1000 -nneg 1000 -nstages 20
```

L'option -data permet de spécifier où générer et comment appeler notre algorithme de sortie (un dossier et un fichier XML seront générés).

L'option -vec permet d'indiquer où se trouve le fichier vecteur.

L'option -bg est un simple fichier texte contenant un listing des négatifs. Le format sera le suivant :

```
/negative/im00.jpg
./negative/im01.jpg
./negative/im02.jpg
...
```

Les options -npos et -nneg servent respectivement à indiquer le nombre de positifs et de négatifs.

L'option -nstage, enfin, permet de stipuler au générateur d'algorithmes le nombre d'entraînements qu'il doit exécuter. Plus ce nombre sera élevé, meilleure sera la détection, mais plus longue sera la génération de l'algorithme.

Attention : la génération de l'algorithme peut prendre jusqu'à plusieurs semaines, en fonction du nombre d'images et de la complexité de l'algorithme final.

V-R - Les Threads

Un **thread** est une tâche de fond dans laquelle nous pouvons implémenter certaines actions précises.

Prenons un exemple concret : les messageries instantanées. Si nous respectons une programmation séquentielle, la messagerie attendrait que vous saisissiez un message, puis l'enverrait, puis attendrait une réponse avant de vous autoriser à saisir un nouveau message.

Cela ne serait pas pratique. Pour pallier cela, nous pouvons donc utiliser plusieurs threads. Le premier se chargera uniquement de prendre en compte votre saisie et de l'envoyer tout en émettant un écho sur l'écran, et le second se contentera d'attendre un message avant de vous l'afficher.

L'avantage ici réside dans le fait que les **threads** ne s'exécutent pas de manière séquentielle, mais simultanée, en parallèle. Cela peut permettre de gagner beaucoup de temps et de fluidité dans une application.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Attention : il existe deux types de threads. Les premiers gèrent des processus totalement indépendants, mais nécessitant des actions parallèles. Les seconds gèrent des processus exécutés en parallèle, mais accédant aux mêmes ressources (par exemple, un champ texte). Les threads présentés ci-après correspondent au premier type.

V-R-1 - Création

Pour créer un **thread**, il faut commencer par importer le module dédié

```
import threading
```

Ensuite, il suffit d'appeler le constructeur **threading.Thread()**. Ce constructeur prend plusieurs paramètres, cinq pour être plus précis :

=>group ;

=>target ;

=>name ;

=>args ;

=>kwargs.

Le premier, group, sert lorsque l'on désire créer des threads pour les regrouper par ensembles. S'agissant d'une fonctionnalité avancée, nous le positionnerons toujours à None.

Le second, target, est la fonction/procédure (juste le nom sans parenthèses) que doit gérer le **thread**.

name est tout simplement le nom que nous désirons donner au **thread**. Si vous ne désirez pas lui en donner, vous pouvez saisir none.

args est un tuple contenant l'ensemble des paramètres pour la fonction/procédure donnée dans target.

kwargs est un dictionnaire pour passer de manière nommée les arguments pour la fonction/procédure donnée dans target.

Remarque : il est possible dans certains cas de passer une partie des paramètres dans args et une autre dans kwargs.

```
mon_thread = threading.Thread(None, ma_fonction, nom_thread, (P1,P2),{})
```

V-R-2 - Lancement

Pour lancer un **thread**, rien de très compliqué. Il suffit d'utiliser la méthode **start()** après avoir créé le **thread**

```
mon_thread = threading.Thread(...)
mon_thread.start()
```

V-R-3 - Arrêt

Pour arrêter un **thread**, la méthode la plus simple et la plus directe (même si pas forcément la meilleure) consiste à utiliser

```
mon_tread = threading.Thread(...)
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
mon_thread.start()
...
mon_thread._Thread__stop()
```

V-R-4 - Pause

Un **thread** ne se met pas en pause. Il peut par contre être temporairement arrêté, puis relancé, tel que vu précédemment.

V-R-5 - Appel toutes les x secondes

Il existe une possibilité alternative aux **threads**. En effet, s'il s'agit uniquement de lancer périodiquement une fonction (MAJ IHM par exemple), on peut utiliser le module **gobject**.

```
import gobject
```

On utilisera alors sa méthode timeout_add, laquelle prend comme premier paramètre le nombre de millisecondes au bout desquelles il faut lancer la fonction passée en second paramètre. On récupère alors un entier indiquant si l'appel et l'exécution se sont bien déroulés.

```
Gobject.timeout_add(1000,ma_fonction) #Lancement de ma_fonction toutes les s
```

V-R-6 - Exemple

```
#! /usr/bin/env PYTHON
# -*- coding: utf-8 -*-



import threading
import time



def fonction_a():
    while True:
        print "fonction_a"
        time.sleep(0.5)



def fonction_b():
    while True:
        print "fonction_b"
        time.sleep(0.5)



def mon_thread():
    """
        Demarre 2 thread qui effectuent des print
    """
    a = threading.Thread(target=fonction_a)#version simplifiée pour
    b = threading.Thread(target=fonction_b)# fonction sans paramètre

    a.start()
    time.sleep(0.5)
    b.start()
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
if __name__ == '__main__':
    mon_thread()
```

V-S - Les PDF : Reportlab

Reportlab est un module **PYTHON** puissant vous permettant de générer proprement des fichiers **PDF**.

Nul besoin de présenter le format **PDF**. C'est devenu un standard utilisé par bon nombre d'administrations et de sociétés à travers le monde.

Nous allons voir ici les bases de ce module. Bien entendu, comme d'habitude, si cela ne vous suffisait pas, je vous invite à aller consulter la documentation complète sur le site dédié, et à faire quelques recherches sur Internet.

V-S-1 - Principes de fonctionnement

Afin de pouvoir utiliser correctement **Reportlab**, et avant même de voir les bases de ce module, il est indispensable de prendre connaissance de ses principes de fonctionnement.

Il ne faut pas imaginer en effet que le module va nous permettre d'écrire comme dans un traitement de texte type Writer.

Reportlab part d'une feuille totalement vierge, à créer, et s'appelant **canvas**, sur laquelle nous allons venir déposer différents types d'objets (texte, image…) afin de créer un template, ou plus simplement un fichier tel qu'on le désire.

Le fonctionnement se rapprocherait donc plutôt ici de Draw que de Writer.

De plus, comme dans Draw, si l'on dispose un élément hors de la page, aucune alerte ne sera levée. De fait, il est primordial de savoir précisément la disposition désirée afin d'éviter toute mauvaise surprise.

À l'image de Draw, vous devrez gérer l'ordre d'insertion afin d'éviter tout conflit de disposition : le dernier élément inséré est placé au-dessus des autres.

Il faut également savoir que cela implique que le code soit exécuté de manière procédurale. En clair, si à un moment, vous demandez une rotation de 90 °, cela veut dire que tant que vous ne changerez pas cette rotation, tout le code interprété sera tourné de 90 °.

Une fois ces quelques principes assimilés, l'utilisation de Reportlab vous semblera plus naturelle. Pour ceux qui trouveraient ces principes encore un peu flous, je ne peux que les inviter à travailler un peu avec Libre Office Draw afin d'illustrer concrètement les principes explicités ici.

V-S-2 - Les bases

V-S-2-a - Canvas, format de page et unités

Reportlab, sait gérer les différents formats de page standard existants. Vous avez également la possibilité de définir de toutes pièces la dimension de la page voulue.

Pour des raisons de lisibilité de code et de simplicité de codage, je vous recommande la seconde option. En précisant en commentaire le format utilisé (A4, letter…), le code n'en sera que plus lisible.

```
from reportlab.pdfgen import canvas
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
from reportlab.lib.units import *

canvas = canvas.Canvas("hello.pdf", pagesize=(210.0 * mm,297.0 * mm))     #A4
```

Comme nous le voyons dans notre exemple, créer un **canvas** est très simple. Le principal est de connaître le nom du PDF à générer, et les dimensions à utiliser.

Les dimensions passées permettent d'ailleurs de définir si l'on fonctionne en portrait ou bien en paysage.

Enfin, le dernier point non négligeable ici concerne les unités. Les principales disponibles sont le cm, le mm et le inch (pouce).

Dans notre cas, fonctionnant en système **ISO**, je vous recommande d'utiliser le mm.

Attention : pour Reportlab, le point d'origine est le coin inférieur gauche.

V-S-2-b - Texte

V-S-2-b-i - Polices d'écriture

Par défaut, seules 14 polices d'écriture sont disponibles avec Reportlab.

Police
Courier
Courier-bold
Courier-oblique
Courier-BoldOblique
Helvetica
Helvetica-Bold
Helvetica-Oblique
Helvetica-BoldOblique
Times-Roman
Times-Bold
Times-Italic
Times-BoldItalic
Symbol
ZapfDingbats

Il est cependant possible, je vous rassure, d'utiliser d'autres polices. Pour cela deux solutions : connaître le chemin d'accès sur le poste, ou la mettre à disposition avec votre programme.

Le premier choix est assez cavalier, car cela impose que vous soyez sûr que l'utilisateur final disposera de la police sur son poste. Aussi, je vous recommande plutôt la solution N° 2.

```
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont

pdfmetrics.registerFont(TTFont('FreeSans', './FreeSans.ttf'))
canvas.setFont("FreeSans", 36)
```

Dans cet exemple, nous décidons d'utiliser la police d'écriture libre FreeSans, équivalent d'Arial. Cette police, nous en mettons un exemplaire (fichier TTF) à disposition dans le même répertoire que notre script.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

La déclaration auprès de **Reportlab** prend en premier paramètre le nom sous lequel on désire avoir accès à la police dans le code, et en second paramètre le chemin d'accès à la police désirée.

V-S-2-b-ii - Simple ligne texte

Pour dessiner une simple ligne texte, il faut passer par la série de méthodes **DrawString**.

Fonction	Coordonnées fournies
drawString	Gauche
drawCentreString	Centre
drawRightString	Droite

Lors de l'appel à une de ces méthodes, il faut passer en paramètres, les coordonnées de référence, correspondant à l'endroit où insérer le texte sur la ligne, puis le texte lui-même.

```
mon_canvas.drawString(10*mm, 10*mm, " Hello World ")
```

V-S-2-c - Paragraphe

On peut se poser la question de l'utilité des paragraphes et/ou leurs différences avec de simples zones texte.

Eh bien, pour simplifier et résumer les choses, disons qu'une zone texte ne gère pas les retours à la ligne, les paragraphes, si.

De plus, les paragraphes peuvent être préparamétrés, via les styles, pour être tout de suite disponibles quand on en a besoin.

V-S-2-c-i - Les styles

Comme leur nom l'indique, les styles permettent de définir le comportement d'un paragraphe complet.

Ainsi, quand nous créons un objet style, nous récupérons de base un certain nombre de styles prédéfinis auxquels nous pouvons rajouter des styles que nous créerons nous-mêmes.

Les styles de base sont : BodyText, Bullet, Code, Definition, Heading1, heading2, Heading3, Italic, Normal, Title.

Je vous invite à les essayer tous pour vous faire votre propre idée de chacun.

```
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.enums import *
...
types_style = getSampleStyleSheet()
mon_style = types_style['BodyText']
```

Voici comment avoir accès au style de base.

Concernant l'ajout d'un style, il faut avant tout savoir de quoi se compose un style de paragraphe. Nous allons voir ici les principaux, ceux qui vous seront vraiment utiles pour créer vos styles.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Composante de style	Description
name	Nom de la police
alignment	Justification du texte. TA_LEFT, TA_RIGHT, TA_CENTER, TA_JUSTIFY
backColor	Couleur de fond : Color(R,G,B)
firstLineIndent	Indentation de la 1re ligne du paragraphe. Précisez l'unité
fontName	Nom de la police d'écriture retenue
fontSize	Taille (en points) de la police. Ne pas préciser d'unité
spaceAfter	Espace après le paragraphe. Précisez l'unité
spaceBefore	Espace avant le paragraphe. Précisez l'unité
textColor	Couleur du texte : Color(R,G,B)

```
types_style.add(ParagraphStyle(name='mon_style_perso''
        alignment = TA_JUSTIFY,
        backColor = None,
        firstLineIndent = 10 * mm,
        fontName = 'FreeSans',
        fontSize = 14,
        spaceAfter = 5 * mm,
        spaceBefore = 5 * mm,
        textColor = Color(0,0,0)
    ))
```

V-S-2-c-ii - Création d'un paragraphe

La création d'un paragraphe est très simple et se passe de commentaire.

```
from reportlab.platypus import Paragraph
...
mon_para =Paragraph('This is a very silly example', mon_style)
```

V-S-2-c-iii - Ajout d'un paragraphe à un canvas

L'ajout d'un paragraphe à un **canvas** se déroule en deux étapes : on commence d'abord par définir la taille du paragraphe via une méthode **wrap**, puis on l'ajoute au canvas à la position désirée.

Attention : au niveau du wrap, c'est surtout la largeur qui compte, et prévaut sur la hauteur.

```
mon_para.wrapOn(mon_canvas, 80*mm, 10*mm)
mon_para.drawOn(canvas, 10*mm, 150*mm)
```

V-S-2-d - Couleur

V-S-2-d-i - Principe

Pour **Reportlab**, la couleur se définit en système RGB. Cependant nous ne fonctionnons pas ici sur un système allant de 0 à 255 pour chaque composante, mais de 0.0 à 1.0.

Cela peut être interprété comme un pourcentage. Pour vous simplifier la vie, je vous invite à notifier la valeur de la composante sur 255, puis à effectuer une division par 255 dans l'appel à la couleur afin de retomber sur un pourcentage.

```
(240.0/255.0)
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Remarque : Pour rappel, pour avoir du gris, il faut R=G=B.

V-S-2-d-ii - Couleur de ligne

Pour choisir la couleur des lignes, on utilise la méthode **setStrokeColorRGB()**.

```
mon_canvas.setStrokeColorRGB(255.0/255, 0.0, 0.0)
```

V-S-2-d-iii - Couleur de remplissage

Pour définir la couleur de remplissage d'un élément dessin, on utilise la méthode **setFillColorRGB(R,G,B)**, ou R, G, B sont des floats.

```
mon_canvas.setFillColorRGB(0,0,196.0/255.0)
```

V-S-2-d-iv - Transparence

Comme nous le verrons plus loin, **Reportlab** sait gérer la transparence.

Si l'on désire remplir un élément dessin avec une couleur gérant en partie le canal alpha (canal de la transparence), il ne faut pas utiliser la méthode précédente, mais la suivante :

```
from reportlab.lib.colors import Color
...
mon_canvas.setFillColor(Color(255,0,0,alpha=0.5))
```

Attention : même si la couleur est indiquée en RGB, on voit qu'ici nous fonctionnons sur un système allant de 0 à 255.

Remarque : le canal alpha est une implémentation récente. Si vous avez un message indiquant un nombre erroné d'arguments, c'est que votre version de Reportlab est trop ancienne.

V-S-2-d-v - Couleur de texte

Changer la couleur de texte est similaire à changer la couleur de remplissage d'un élément dessin.

```
mon_canvas.setFillColorRGB(0,0,196.0/255.0)
```

V-S-2-d-vi - Définir une couleur

Vous pouvez définir vos propres couleurs aisément :

```
from reportlab.lib.colors import Color
...
red = Color(R,G,B)     #R, G, B, compris entre 0 et 255
```

V-S-2-e - Rotation

Une rotation s'effectue via la méthode **rotate()**, en passant en argument la valeur de la rotation en degré. La rotation s'effectue dans le sens trigonométrique.

```
mon_canvas.rotate(90)
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

V-S-2-f - Tableau

Nous allons maintenant aborder le sujet des tableaux dans les fichiers PDF.

V-S-2-f-i - Tableau de base

Pour créer un tableau basique, il faut peu de choses : avoir importé le bon module, et avoir un tableau de données.

```
from reportlab.platypus import Table, TableStyle
...
data = [['00', '01', '02', '03', '04'],['10', '11', '12', '13', '14'],
    ['20', '21', '22', '23', '24'],['20', '21', '22', '23', '24'],
    ['30', '31', '32', '33', '34']]
mon_tab = Table(data)
```

V-S-2-f-ii - Les styles de tableaux

Les styles de tableaux vous permettent d'appliquer du paramétrage avec une finesse pouvant descendre à la cellule.

```
mon_tab.setStyle(TableStyle([('BACKGROUND',(1,1),(-2,-2),colors.green),  \
                        ('TEXTCOLOR',(0,0),(1,-1),colors.red)]))
```

Comme vous pouvez le constater, nous indiquons quatre éléments à la fois.

Le premier élément est le style sur lequel nous désirons intervenir.

Viennent ensuite les éléments 2 et 3. Ils correspondent aux cellules visées par la modification que nous souhaitons effectuer.

Le premier tuple (X, Y) est la cellule de départ (on compte à partir de 0 depuis le coin supérieur gauche).

Le second tuple correspond à la cellule de fin. Toutes les cases comprises dans ce rectangle ainsi défini seront affectées par la modification.

Ce second tuple est particulier, car il y a deux façons de procéder : on peut donner les coordonnées précises (ex. : (3,2)) ou des lignes/colonnes à exclure, comme dans notre exemple. Dans les faits, je vous recommande la première solution, bien plus lisible.

Attention : dans un tableau Reportlab nous tournons en rond. Comprenez que quand vous atteignez la dernière colonne, vous recommencez à la première. Ainsi lorsqu'on vous dit -2, alors que vous êtes à la colonne 1, cela vous positionne sur la dernière colonne.

Enfin, le dernier élément est le paramètre à utiliser pour le style.

Voyons maintenant plus en détail les principaux différents styles sur lesquels nous pouvons intervenir :

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

STYLE	Description
FONTNAME	Prend en paramètre le nom d'une police définie dans Reportlab (voir partie sur le texte)
FONTSIZE	Indique la taille (en points) de la police. Ne pas préciser d'unité.
TEXTCOLOR	Choisir la couleur de police. Définissez vos propres couleurs, tel que vu précédemment
ALIGNMENT	Permet la justification du texte au sein d'une cellule : LEFT, RIGHT, CENTER
BACKGROUND	Permet de définir une couleur de fond
VALIGN	Permet de choisir l'alignement vertical dans une cellule : TOP, MIDDLE, BOTTOM (par défaut)

V-S-2-f-iii - Taille de cellules

La taille des cellules se paramètre à la création du tableau. La méthode que nous avons vue jusqu'à présent est la méthode automatique, et la plus simple.

Il existe cependant une façon un peu plus complexe, mais plus paramétrable :

```
from reportlab.platypus import Table, TableStyle
...
data = [['00', '01', '02', '03', '04'],['10', '11', '12', '13', '14'],
    ['20', '21', '22', '23', '24'],['20', '21', '22', '23', '24'],
    ['30', '31', '32', '33', '34']]
mon_tab = Table(data, colWidths=[50*mm,40*mm,30*mm,20*mm,10*mm], \
           rowHeights = 10*mm)
```

V-S-2-f-iv - Les bordures

Les bordures sont un sujet très intéressant et important. Voici les principaux types de bordures :

Paramètre	Description
GRID	Contours externes de toutes les cellules
BOX	Contours externes de la sélection
INNERGRID	Lignes internes de la sélection
LINEBELOW	Bordure inférieure de la cellule
LINEABOVE	Bordure supérieure de la cellule
LINEBEFORE	Bordure gauche de la cellule
LINEAFTER	Bordure droite de la cellule

Pour utiliser ces paramètres, il faut également procéder de la même façon que pour les styles.

```
mon_tab.setStyle(TableStyle([('BOX',(1,1),(-2,-2),1*mm,colors.green)]))
```

Le premier élément est ici le paramètre à configurer.

Les éléments 2 et 3 sont les coordonnées des cellules de début et de fin de sélection.

Le 4e élément est l'épaisseur du trait.

Le dernier élément est la couleur du trait.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

V-S-2-f-v - Les images dans les tableaux

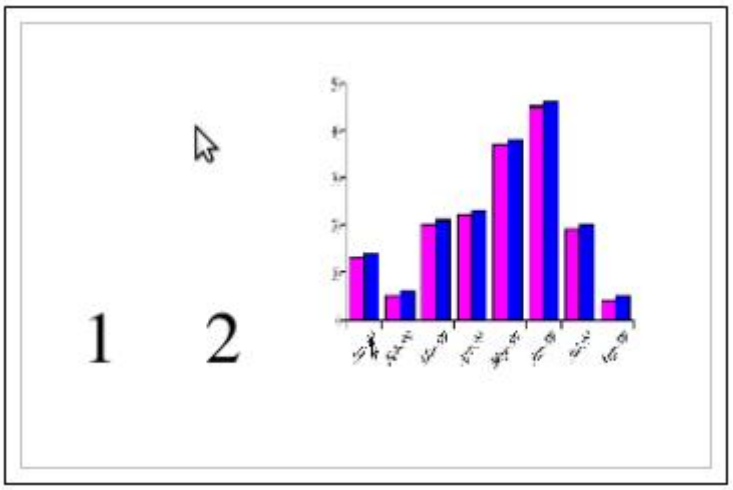
L'import d'images n'a rien de bien compliqué au sein d'un tableau. Cependant, de manière simple, vous devrez choisir, au niveau d'une cellule, entre image et texte.

Vous avez cependant la possibilité d'insérer non pas du texte, mais des paragraphes avec l'image ([mon_image, paragraphe] par exemple).

```
from reportlab.platypus import Image
...
mon_image = Image('./test.png')
mon_image.drawHeight = 15*mm
mon_image.drawWidth = 15*mm
data = [['1','2',[mon_image]]
```

Nous pouvons voir ici que nous définissons notre image, puis paramétrons sa hauteur et sa largeur.

Ensuite nous l'insérons tout simplement dans notre base de données, qui servira à la création du tableau.



Le résultat du code

V-S-2-f-vi - Insertion d'un tableau dans un canvas

Pour insérer notre tableau dans un canvas, nous allons utiliser deux méthodes : **wrapOn** et **drawOn**.

La première méthode prend en paramètres le canvas cible et les dimensions maximales allouées au tableau, autrement dit, la taille maximale autorisée pour le tableau.

La seconde méthode, elle, prend en paramètres le canvas, et le positionnement, en X et Y, du tableau dans le canvas.

```
mon_tab.wrapOn(canvas, 210*mm, 29.7*mm)
mon_tab.drawOn(canvas,10*mm, 96*mm)
```

V-S-2-g - Dessin

Excepté la ligne, les différents éléments dessin présentés ici possèdent deux paramètres : stroke et fill.

Fill a pour valeur par défaut 0. Il permet de préciser si l'élément dessin doit être rempli d'une couleur (1, couleur par défaut : noir) ou non (0).

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Stroke a pour valeur par défaut 0. Il permet de préciser si l'élément dessin, même rempli d'une couleur doit être semi-transparent ou non.

V-S-2-g-i - Ligne

Pour dessiner une ligne, il suffit d'appeler la méthode **line()**, en lui passant en paramètres les coordonnées de début,de fin et l'épaisseur désirée, avec l'unité.

```
mon_canvas.line(10*mm, 10*mm, 100*mm, 100*mm)
```

V-S-2-g-ii - Cercle

Nous allons utiliser pour ce faire la méthode **circle()**.

```
mon_canvas.circle(x_centre, y_centre, rayon, stroke = 0, fill = 0)
```

Nous pouvons également tracer des ellipses :

```
mon_canvas.ellipse(x1_centre, y1_centre, x2_centre, y2_centre, fill = 0)
```

V-S-2-g-iii - Rectangle

Pour dessiner un rectangle, on utilise la méthode **rect()**, en passant comme arguments les coordonnées du coin inférieur gauche, sa largeur, sa hauteur, et en précisant s'il doit être rempli d'une couleur ou non.

```
mon_canvas.rect(10*mm, 10*mm, 100*mm, 100*mm, fill = 1)
```

Vous pouvez également dessiner un rectangle avec des bords arrondis, via la méthode **roundrect()**.

```
mon_canvas.roundRect(10*mm, 10*mm, 100*mm, 100*mm, 5*mm, fill = 0)
```

L'avant-dernier paramètre correspond ici au rayon des arrondis.

V-S-2-g-iv - Épaisseur de lignes

L'épaisseur des lignes des différents éléments dessin se paramètre via la méthode **setLineWidth()**. Cette méthode est à appeler avant de réaliser le tracé proprement dit.

```
mon_canvas.setLineWidth(2*mm)
mon_canvas.rect(1*mm,1*mm,10*mm,20*mm,fill = 0)
```

V-S-2-g-v - Type de ligne

Vous pouvez choisir votre type de ligne très simplement. **Reportlab** fonctionne sur le principe de portion, concernant le type de ligne.

En clair, pour une portion donnée, vous indiquez comment le trait doit se comporter. On commence par ON, puis OFF et ainsi de suite.

Vous pouvez ainsi définir une simple alternance (ex. : 50% ON, 50% OFF) ou quelque chose de plus complexe (ex. : ON, OFF, ON, OFF, ON)

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Reportlab prendra la totalité comme représentant 100 % de la portion et recalculera les valeurs saisies en pourcentage.

```
mon_canvas.setDash(6,4)     #60%ON, puis 40% OFF
mon_canvas.setDash(4,1,5)     #40% ON, 10% OFF, 50% ON
```

Remarque : pour que les exemples ci-dessus soient plus parlants, nous avons fait en sorte que la somme égale 10, mais ce n'est nullement une obligation. Vous pouvez très bien saisir setDash(5,7,9,3) par exemple.

V-S-2-h - Image

L'import d'images est relativement simple également :

```
mon_canvas.drawImage(" mon_image.png ", x,y, width=100*mm,height=20*mm)
```

V-S-2-i - Graphique

Attention : chaque type de graphique possède son propre module, et nécessite donc son propre import.

V-S-2-i-i - Support de graphique

Les graphiques en **Reportlab** ont besoin de leur propre support, qui sera par la suite intégré au canvas.

```
from reportlab.graphics.shapes import Drawing
...
mon_draw = Drawing(400, 200) #support de 400 pixels de large et 200 de haut
```

Remarque : essayez de faire en sorte que votre support soit toujours plus grand que votre graphique, même si dans les faits, vous n'aurez pas toujours d'erreur.

Une fois notre graphique dessiné, il ne nous restera plus qu'à l'ajouter au support via la méthode **add :**

```
mon_draw.add(mon_graph)
```

V-S-2-i-ii - Histogramme

Un histogramme se crée avec la méthode **VerticalBarChart()**. Il faut ensuite une liste contenant les données du graphique sous la forme de tuples. Chaque tuple représente un jeu de données.

Une fois n'est pas coutume, place au code que nous allons commenter pour expliquer et clarifier les choses :

```
1. from reportlab.graphics.charts.barcharts import VerticalBarChart
2. from reportlab.lib.colors import Color
3.
4. data = [ (13, 5, 20, 22, 37, 45, 19, 4), (14, 6, 21, 23, 38, 46, 20, 5) ]
5. bc = VerticalBarChart()
6. bc.x = 50 *mm
7. bc.y = 50*mm
8. bc.height = 125 *mm
9. bc.width = 300 *mm
10. bc.data = data
11. bc.strokeColor = colors.black
12. bc.valueAxis.valueMin = 0
13. bc.valueAxis.valueMax = 50
14. bc.valueAxis.valueStep = 10
15. bc.categoryAxis.labels.boxAnchor = 'ne'
16. bc.categoryAxis.labels.dx = 8
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
17. bc.categoryAxis.labels.dy = -2
18. bc.categoryAxis.labels.angle = 30
19. bc.categoryAxis.categoryNames = ['Jan-99','Feb-99','Mar-99', \
20.              'Apr-99','May-99','Jun-99','Jul-99','Aug-99']
21. violet = Color(255,0,255)
22. bleue = Color(0,0,255)
23. bc.bars[0].fillColor = violet
24. bc.bars[1].fillColor = bleue
```

Jusqu'à la ligne 5, rien de neuf. Nous notons que data possède deux tuples. Le diagramme représentera donc deux jeux de données.

Ligne 6 à 9, nous définissons l'emplacement de notre graphique sur le support, ainsi que ses dimensions.

Ligne 10, nous définissons la source de données.

Ligne 11, nous définissons la couleur du trait de l'histogramme.

Ligne 12 à 14, nous définissons notre axe y : son min, son max, et son pas.

Ligne 15, nous définissons le point d'accroche des labels. Les valeurs possibles correspondent aux points cardinaux et à leurs composantes : n, s, w, e, ne, nw, se, sw.

Ligne 16 et 17, nous paramétrons l'offset de positionnement du label sur le graphe.

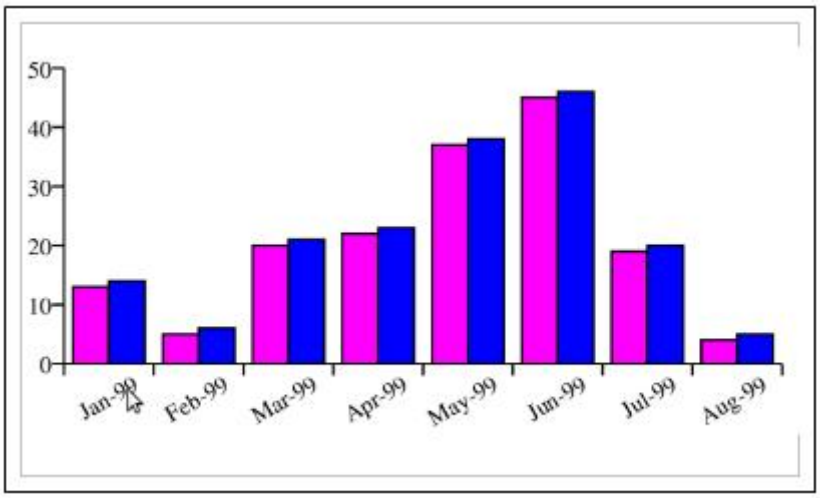
Ligne 18, nous réglons l'inclinaison des labels de l'histogramme.

Les lignes 15 à 18 servent donc à bien paramétrer les labels de l'histogramme.

Ligne 19 et 20, nous « labellisons » l'axe X, en passant une liste contenant les labels. Il faut autant de labels en X qu'il y a de données dans la liste alimentant l'histogramme.

Ligne 21 et 22, nous définissons deux couleurs.

Ligne 23 et 24, nous colorons nos histogrammes aux couleurs choisies.



L'histogramme généré

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

V-S-2-i-iii - Linéaire

Nous allons utiliser la méthode **HorizontalLineChart()**.

Comme précédemment, nous allons expliquer le fonctionnement au travers d'un code. Vous verrez un certain nombre de similitudes avec l'histogramme.

```
1. from reportlab.graphics.charts.linecharts import HorizontalLineChart
2. ...
3. data = [ (13, 5, 20, 22, 37, 45, 19, 4), (5, 20, 46, 38, 23, 21, 6, 14) ]
4. lc = HorizontalLineChart()
5. lc.x = 50 *mm
6. lc.y = 50 *mm
7. lc.height = 125 *mm
8. lc.width = 300 *mm
9. lc.data = data
10. lc.joinedLines = 1
11.
12. lc.categoryAxis.categoryNames = ['Jan','Feb','Mar','Apr','May','Jun','Jul','Aug']
13. lc.categoryAxis.labels.boxAnchor = 'n'
14. lc.valueAxis.valueMin = 0
15. lc.valueAxis.valueMax = 60
16. lc.valueAxis.valueStep = 15
17. lc.lines[0].strokeWidth = 2 *mm
18. lc.lines[1].strokeWidth = 1.5 *mm
19. violet = Color(255,0,255)
20. bleue = Color(0,0,255)
21. lc.lines[0].strokeColor = violet
22. lc.lines[1].strokeColor = bleue
```

Ligne 3, nous retrouvons notre source de données.

Ligne 4, nous créons notre courbe.

Ligne 5 à 8, nous paramétrons sa position, ainsi que ses dimensions.

Ligne 9, nous définissons sa source de données.

Ligne 10, une nouveauté : nous précisons que nous désirons que les points de la source de données soient liés.

Ligne 12, nous définissons les labels de l'axe X.

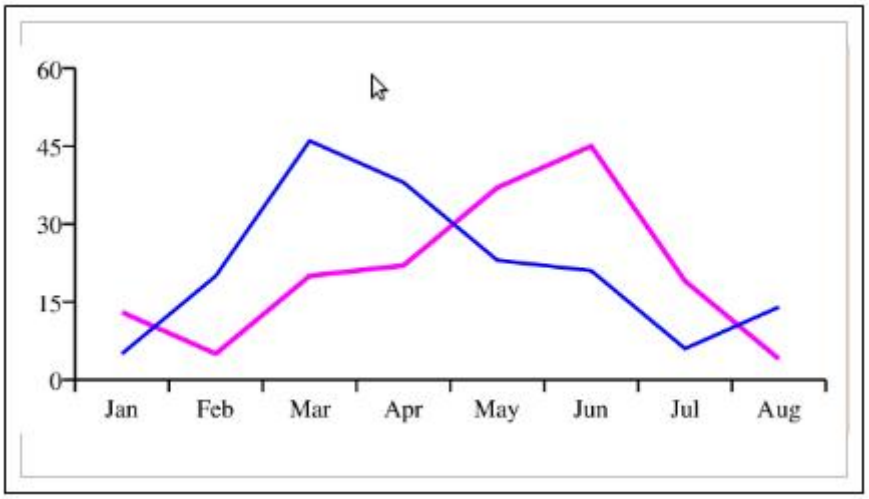
Ligne 13, nous définissons l'ancrage des labels de l'axe X.

Ligne 14 à 16, nous paramétrons l'axe Y.

Ligne 17 et 18, nous paramétrons l'épaisseur des traits de chaque courbe.

Enfin, comme pour l'histogramme, ligne 19 à 22, nous colorons nos courbes.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/



La courbe obtenue

V-S-2-i-iv - Camembert

Le dernier classique en graphique est le célèbre graphique camembert. Nous le créons via la méthode **Pie()**.

Là aussi nous passerons par un code, et vous noterez des similitudes avec les graphiques précédemment passés en revue.

```
1. from reportlab.graphics.charts.piecharts import Pie
2. ...
3. pc = Pie()
4. pc.x = 65
5. pc.y = 15
6. pc.width = 70 *mm
7. pc.height = 70 *mm
8. pc.data = [10,20,30,40,50,60]
9. pc.labels = ['a','b','c','d','e','f']
10. pc.slices.strokeWidth=0.5 *mm
11. pc.slices[3].popout = 10 *mm
12. pc.slices[3].strokeWidth = 2 *mm
13. pc.slices[3].strokeDashArray = [2,2]
14. pc.slices[3].labelRadius = 1.75
15. bleue = Color(0,0,255)
16. pc.slices[3].fontColor = bleue
17. pc.slices[3].fillColor = bleue
```

Ligne 3, nous créons notre graphique.

Ligne 4 à 7, nous paramétrons sa position dans le support, ainsi que ses dimensions.

Ligne 8, nous lui passons ses données. Vous aurez maintenant compris qu'il s'agit d'une simple liste qui peut être créée sur le moment ou plus en amont dans le code.

Reportlab considère l'ensemble des données communiquées comme représentant 100 %, ou 360 °. Il effectue ensuite un ratio sur les données passées.

Ligne 9, nous passons les labels respectifs.

Ligne 10, nous paramétrons l'épaisseur des traits du graphique.

Ligne 11 à 17, nous paramétrons plus en détail la donnée en indice 3, autrement dit en 4e position: « d ».

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

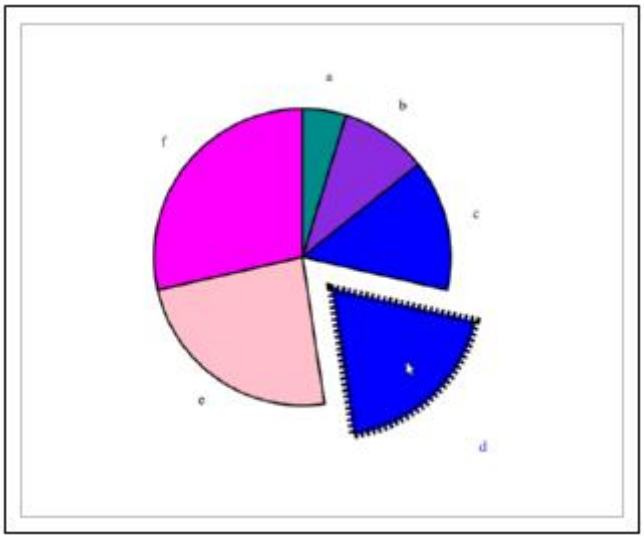
Ligne 11, nous séparons la part de camembert « d » de 10 mm par rapport au centre. Cela est généralement utilisé afin d'effectuer une mise en valeur.

Ligne 12, nous modifions la largeur du trait de son contour.

Ligne 13, nous transformons le contour en trait discontinu.

Ligne 14, il s'agit d'un ratio par rapport au rayon du camembert indiquant où placer le label ; dans notre cas « d ». Ici, il sera placé à une distance (du centre), de 1.5 fois le rayon du camembert. Il ne s'agit pas d'une dimension.

Ligne 15 à 17, nous colorons la part de camembert « d » de la couleur désirée.



Le graphique final

V-S-2-i-v - Intégration du support dans un canvas

Pour intégrer un support, et donc le graphique qu'il abrite, dans un canvas, il faut utiliser une nouvelle méthode: le **renderPDF()**.

```
from reportlab.graphics import renderPDF
...
renderPDF.draw(mon_draw, mon_canvas, x*mm, y*mm)
```

Comme on peut le constater, le premier paramètre est le support de graphique, le second paramètre le **canvas** accueillant le graphique, puis la position du support graphique sur le **canvas**.

V-S-2-j - Changer de page

Pour fermer une page, on utilise la méthode **showPage()**.

```
mon_canvas.showPage()
```

Attention : à chaque changement de page, tous les paramétrages sont remis à zéro (couleur, police…).

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

V-S-2-k - Métadonnées PDF

Les métadonnées sont ces informations que l'on peut afficher dans tout lecteur PDF, et généralement nommées « propriétés du document ». Nous allons voir ici comment définir les principales.

V-S-2-k-i - Auteur

Il faut utiliser la méthode **setAuthor() :**

```
mon_canvas.setAuthor(" toto ")
```

V-S-2-k-ii - Titre

Il faut utiliser la méthode **setTitle() :**

```
mon_canvas.setTitle(" titre ")
```

V-S-2-k-iii - Sujet

Il faut utiliser la méthode **setSubject() :**

```
mon_canvas.setSubject(" test pdf ")
```

V-S-2-l - Sauvegarde

La sauvegarde d'un PDF, s'effectue à travers son canvas. Il est important de noter que tant que l'appel de la sauvegarde n'a pas été effectué, le PDF n'existe qu'en RAM. Toute coupure de courant supprimerait ainsi le contenu complet du PDF.

```
mon_canvas.save()
```

V-S-3 - Exemple

```
#!/usr/bin/PYTHON
# -*-coding:utf-8 -*



from reportlab.pdfgen import canvas
from reportlab.lib.units import *
from reportlab.graphics.charts.barcharts import VerticalBarChart
from reportlab.lib.colors import Color
from reportlab.lib import colors
from reportlab.graphics.shapes import Drawing
from reportlab.graphics import renderPDF



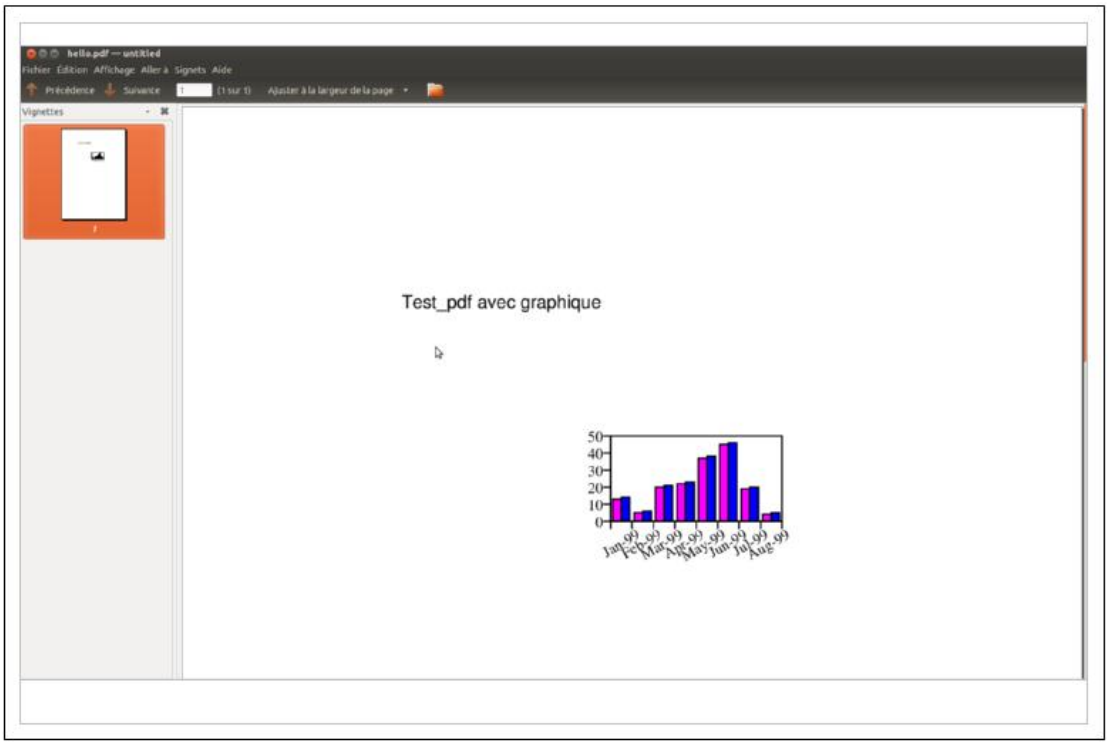
canvas = canvas.Canvas("hello.pdf", pagesize=(210.0 * mm,297.0 * mm))
canvas.drawString(50*mm, 250*mm, " Test_pdf avec graphique ")
mon_draw = Drawing(50*mm, 200*mm)


data = [ (13, 5, 20, 22, 37, 45, 19, 4), (14, 6, 21, 23, 38, 46, 20, 5) ]
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
bc = VerticalBarChart()
bc.x = 50 *mm
bc.y = 50*mm
bc.height = 20 *mm
bc.width = 40 *mm
bc.data = data
bc.strokeColor = colors.black
bc.valueAxis.valueMin = 0
bc.valueAxis.valueMax = 50
bc.valueAxis.valueStep = 10
bc.categoryAxis.labels.boxAnchor = 'ne'
bc.categoryAxis.labels.dx = 8
bc.categoryAxis.labels.dy = -2
bc.categoryAxis.labels.angle = 30
bc.categoryAxis.categoryNames = ['Jan-99','Feb-99','Mar-99', \
                        'Apr-99','May-99','Jun-99','Jul-99','Aug-99']
violet = Color(255,0,255)
bleue = Color(0,0,255)
bc.bars[0].fillColor = violet
bc.bars[1].fillColor = bleue


mon_draw.add(bc)
renderPDF.draw(mon_draw, canvas, 50*mm, 150*mm)
canvas.save()
```



Rendu de l'exemple pdf

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

VI - Les interfaces graphiques



Wikimedia Commons,GTK.png

À la recherche de modules dédiées **IHM**, nous trouvons de nombreux résultats.

Les deux plus connus sont **GTK** et **QT**. Nous nous concentrerons ici sur **GTK**.

Ces deux modules sont à la base des IHM Linux les plus connues, GNOME et KDE.

Remarque : si GTK est sous licence libre, QT dispose de diverses licences. Attention donc à vos DEV avec cette dernière à choisir la bonne version.

VI-A - PYGTK

GTK, pour Gimp ToolKit, est une bibliothèque graphique conçue à l'origine pour le célèbre logiciel GIMP. Par la suite adoptée par le projet GNOME, GTK lui est régulièrement (par erreur) associé.

D'un aspect un peu brut, diront certains, **GTK** et **PYGTK** (son pendant **PYTHON**) n'en demeurent pas moins fonctionnels à 100 % et suffisent à la plupart des besoins des développeurs.

Dans les faits, son interface simple et dénuée de toute fioriture inutile, telle que celles que l'on voit régulièrement dans les nouveaux OS (et qui consomment quantité de ressources), lui permet de remplir son office de façon optimale.

Nous verrons dans ce sous-chapitre l'essentiel de la bibliothèque. Pour de plus amples informations sur les widgets (y compris ceux qui ne seront pas vus ici) et leurs méthodes associées, je vous renvoie vers la page web dédiée: taper « **PYGTK** <NomDuWidget> » dans votre moteur de recherches préféré.

Remarque : il existe certains modules complémentaires permettant de rajouter des widgets à ceux de base.

```
import pygtk
import gtk
```

Attention : une fois un widget créé, il ne faudra pas oublier d'utiliser la méthode show() afin de l'afficher dans le conteneur. De même, pour le cacher, on utilisera la méthode hide(), et pour le détruire totalement, la méthode destroy().

```
widget.show()
widget.hide()
widget.unparent()
widget.destroy()
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Concernant les fenêtres, on utilisera les méthodes run() et destroy(). Pour les fenêtres utilisant des boutons, on n'oubliera pas également de récupérer la valeur du bouton dans une variable (la fenêtre main est un cas à part et n'utilise ni run() ni destroy()).

```
retour_bouton = fenetre.run()
fenetre.show()
fenetre.hide()
fenetre.destroy()
```

VI-A-1 - Les fenêtres

VI-A-1-a - Main

Il s'agit de la fenêtre principale de l'application, qu'on peut créer tout simplement

```
ma_fenetre = gtk.Window(type=gtk.WINDOW_TOPLEVEL)
```

Notez le paramètre type. Sa valeur par défaut est **gtk.WINDOW_TOPLEVEL**. De fait nous pouvons également créer la fenêtre principale sans préciser type.

```
ma_fenetre = gtk.Window()
```

Remarque : si type vaut gtk.WINDOW_POPUP alors cela générera une fenêtre particulière, généralement utilisée pour les Splash Screen ou encore les fenêtres About, même si un constructeur spécifique existe pour ces dernières.

On peut ensuite préciser certains paramètres tel que le titre

```
ma_fenetre.set_title("Ceci est mon titre")
```

On peut également préciser si la fenêtre est à maximiser ou non, ainsi que sa taille, sa position ou encore si elle peut être redimensionnable ou non.

```
#Taille de la fenêtre
ma_fenetre.set_default_size(L, H)
ma_fenetre.resize(L, H)

#Position
ma_fenetre.set_position(gtk.WIN_POS_CENTER) #au centre
ma_fenetre.move(X, Y)

#Maximiser une fenêtre
ma_fenetre.maximize()

#rendre une fenêtre redimensionnable ou non
ma_fenetre.set_resizable(True)
result = ma_fenetre.get_resizable()
ma_fenetre.set_size_request(650,450) # A utiliser avec un set_resizable(False)
```

Il reste ensuite les principales méthodes des fenêtres à connaître. L'ajout des widgets et/ou conteneurs se fait avec la méthode **add()**.

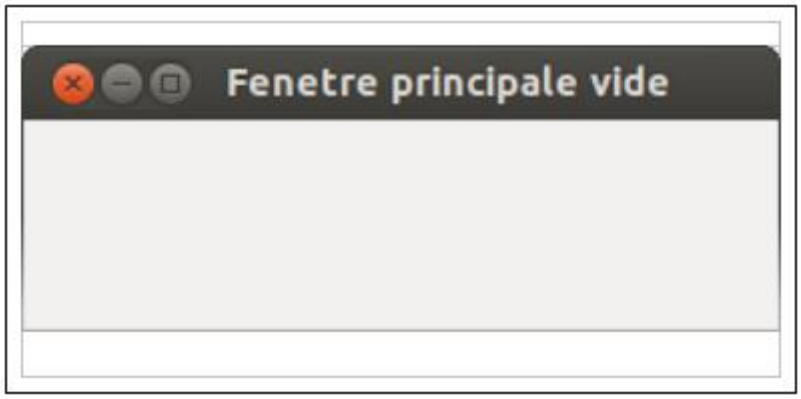
```
ma_fenetre.add(mon_conteneur)
```

Il ne reste ensuite qu'à préciser à l'interpréteur l'action à accomplir lorsque l'on clique sur la croix de fermeture de la fenêtre, puis afficher la fenêtre à l'utilisateur, et enfin lancer le programme complet

```
ma_fenetre.connect("destroy", gtk.main_quit)
ma_fenetre.show() #show_all() existe également
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
gtk.main() #lance le programme complet
```



Une fenêtre principale vide

VI-A-1-b - About

PYGTK vous propose comme n'importe quelle autre interface graphique des fenêtres « À propos de ». Cependant, ici tout est préconfiguré.

De fait, vous avez juste à passer les paramètres pour obtenir une fenêtre « À propos de » de qualité professionnelle.

```
ma_fenetre = gtk.AboutDialog()
```

Il ne reste alors plus qu'à configurer chaque paramètre.

```
ma_fenetre.set_name(" Nom de mon programme ")
ma_fenetre.set_version(" 0.0.1 ")
ma_fenetre.set_copyright(" Copyleft ")
ma_fenetre.set_comments(" mon commentaire ")
ma_fenetre.set_license(" Licence GPL ")
ma_fenetre.set_website(" http://diablotronic.bzh.bz ")
ma_fenetre.set_website_label(" texte de mon lien ")
ma_fenetre.set_authors(liste_programmeurs)
ma_fenetre.set_documenters(Liste_auteurs_docs)
ma_fenetre.set_translator_credits(" listes des traducteurs ")
ma_fenetre.set_logo(logo) #logo est un pixbuf, voir ci-dessous
logo = PYGTK.gdk.pixbuf_new_from_file("mon_logo.png")
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/



Une fenêtre À propos de (http://developer.gnome.org)

VI-A-1-c - Info / erreur / question / Attention

Pour tous ces types de fenêtres, il n'y a qu'un seul constructeur, mais avec un paramétrage différent : **MessageDialog()**.

Ce constructeur prend cinq paramètres

```
ma_fenetre = gtk.MessageDialog(parent, flags, type, buttons, message_format)
```

Le premier permet de préciser si la fenêtre de dialogue dépend d'une autre fenêtre ou non. Sa valeur par défaut est None. En général, on précisera la fenêtre principale de notre programme.

Le second paramètre permet de configurer le comportement du parent vis-à-vis de la fenêtre de dialogue.

Constante	Description
gtk.DIALOG_MODAL	Tant que la fenêtre de dialogue est ouverte, on n'a pas accès à la fenêtre parent
gtk.DIALOG_DESTROY_WITH_PARENT	La fenêtre de dialogue sera détruite en même temps que le parent
gtk.DIALOG_MODAL| gtk.DIALOG_DESTROY_WITH_PARENT	Combinaison des deux comportements précédemment décrits
0	Aucun (valeur par défaut)

Le paramètre suivant, type, permet de paramétrer le type de la fenêtre de dialogue affichée à l'utilisateur.

Constante
gtk.MESSAGE_INFO

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

(valeur par défaut)
gtk.MESSAGE_WARNING
gtk.MESSAGE_QUESTION
gtk.MESSAGE_ERROR

Viennent ensuite les boutons à afficher. Ici, vous ne créez pas vos boutons, ils sont prédéfinis et vous choisissez juste ceux que vous voulez parmi une liste de six configurations.

Constante	Description
gtk.BUTTONS_NONE	Aucun bouton (peut alors servir de substitut à un splash screen) (valeur par défaut)
gtk.BUTTONS_OK	Affiche juste un bouton « Valider »
gtk.BUTTONS_CLOSE	Affiche juste un bouton « Fermer »
gtk.BUTTONS_CANCEL	Afficher un bouton « Annuler »
gtk.BUTTONS_YES_NO	Afficher un couple de boutons « Oui » et « Non »
gtk.BUTTONS_OK_CANCEL	Affiche un bouton « Valider » et un bouton « Annuler »

Chacun de ces boutons est associé à des constantes **GTK** de type **RESPONSE**. Si cette constante vaut True, alors le bouton a été cliqué.

Constante
gtk.RESPONSE_YES
gtk.RESPONSE_NO
gtk.RESPONSE_CANCEL
gtk.RESPONSE_CLOSE
gtk.RESPONSE_OK
gtk.RESPONSE_ACCEPT
gtk.RESPONSE_APPLY

Remarque : vous pouvez avoir accès aux constantes en recherchant sur Internet « PYGTK PYGTK-constants ».

Le dernier paramètre, lui, est le texte à afficher dans la fenêtre de dialogue.

Il existe deux méthodes de gestion de la fenêtre : **run()** et **destroy()**, permettant respectivement de lancer et de détruire la fenêtre.

```
retour = ma_fenetre.run()
ma_fenetre.destroy()
```

Comme on le voit sur l'exemple, on récupère quelque chose lorsqu'on lance la fenêtre. Il s'agit du bouton cliqué par l'utilisateur dans la fenêtre de dialogue. Il suffit alors de comparer la variable retour avec une des constantes **GTK**.



Un exemple de fenêtre de dialogue (http://developer.gnome.org)

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

VI-A-1-d - Print

La fenêtre d'impression classique, telle que nous la connaissons, existe toute faite en **PYTHON**.

On peut de plus la configurer, comme bon nous semble, via une constante **GTK**.

Constante	Description
gtk.PRINT_OPERATION_ACTION_PRINT_DIALOG	Fenêtre classique
gtk.PRINT_OPERATION_ACTION_PRINT	Impression directe
gtk.PRINT_OPERATION_ACTION_PREVIEW	Aperçu
gtk.PRINT_OPERATION_ACTION_EXPORT	Exporter vers un fichier (nécessite un export-filename)

Il faut commencer par créer un conteneur d'impression, puis lancer l'affichage.

```
1. import gtk
2. fenetre = gtk.PrintOperation()
3. result = fenetre.run(gtk.PRINT_OPERATION_ACTION_PRINT_DIALOG)
```

Ligne 3, on peut voir qu'on récupère le résultat de l'opération pour savoir quoi faire.

Ce résultat peut prendre plusieurs valeurs. Il suffit de comparer la valeur de result avec les constantes **GTK** suivantes :

Constante	Description
gtk.PRINT_OPERATION_RESULT_ERROR	Erreur
gtk.PRINT_OPERATION_RESULT_APPLY	Bouton imprimer sélectionné
gtk.PRINT_OPERATION_RESULT_CANCEL	Bouton annuler sélectionné
gtk.PRINT_OPERATION_RESULT_IN_PROGRESS	Impression non terminée

```
if result = gtk.PRINT_OPERATION_RESULT_APPLY
```

Le travail ne s'arrête cependant pas là. En effet, nous n'avons fait que la moitié des choses. Une fois que l'opérateur a paramétré son impression, il reste à imprimer. Cela se fait de la manière suivante :

```
import gtkunixprint
filename = 'mon_fichier.txt'
ma_print = fenetre.get_selected_printer()
params = fenetre.get_settings()
config = fenetre.get_page_setup()
printjob = gtkunixprint.PrintJob('Print %s' % filename, ma_print, params, config)
printjob.set_source_file(filename)
printjob.send(print_cb)
pud.destroy()
```

Ce code ne fera pas l'objet de commentaires. Précisons tout de même qu'il ne fonctionnera que sous Linux/UNIX. Sous Windows, basez-vous sur :

```
import win32api
win32api.ShellExecute (0, "print", filename, None, ".", 0)
```

Afin de satisfaire votre soif de connaissance concernant ces quelques lignes de code, je vous renvoie vers Internet.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

VI-A-1-e - File

Au niveau des fenêtres de gestion de fichiers, vous avez plusieurs choix : sélection de fichier(s) à ouvrir, sélection de fichiers pour la sauvegarde, sélection de dossiers, création de dossiers.

Pour toutes ces fenêtres, un constructeur unique.

```
ma_fenetre = gtk.FileChooserDialog(title=None, parent=None, \
                            action=gtk.FILE_CHOOSER_ACTION_OPEN,\
                    buttons=None, backend=None)
```

Comme on peut le constater, il faut jusqu'à cinq paramètres.

Title sera le titre que portera la fenêtre lorsqu'elle s'affichera à l'utilisateur.

Parent, backend sont pour une utilisation sur mesure et seront généralement à None. Pour plus d'informations sur ce paramètre, je vous renvoie vers la documentation officielle.

Action correspond au type de fenêtre souhaitée.

Constante	Description
gtk.FILE_CHOOSER_ACTION_OPEN	Choisir un ou plusieurs fichier(s) à ouvrir
gtk.FILE_CHOOSER_ACTION_SAVE	Choisir un fichier pour l'enregistrement
gtk.FILE_CHOOSER_ACTION_SELECT_FOLDER	Choisir un dossier où effectuer une action
gtk.FILE_CHOOSER_ACTION_CREATE_FOLDER	Permet de créer un dossier dans un dossier donné

Enfin buttons est un tuple qui contient la liste des boutons désirés avec les signaux associés : (bouton1, signal1, bouton2, signal2…). Les boutons et les signaux sont identiques à ceux vus précédemment pour les fenêtres d'informations.

Du côté des méthodes, nous allons voir les principales.

```
ma_fenetre.set_select_multiple(True)
ma_fenetre.get_select_multiple()
```

Permet de déterminer si on peut sélectionner plusieurs fichiers (True) ou non.

```
ma_fenetre.get_filename()
```

Permet de récupérer le nom du fichier sélectionné (dans le cas d'un seul fichier possible). À None si aucune sélection.

```
ma_fenetre.get_filenames()
```

Permet de récupérer les noms des fichiers sélectionnés (dans le cas d'une sélection multiple). À None si aucune sélection.

```
ma_fenetre.set_filename(" monfichier.txt ")
```

Permet de définir le nom du fichier par défaut.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
ma_fenetre.set_do_overwrite_confirmation(True)
```

Permet de demander une confirmation avant tout enregistrement.

```
ma_fenetre.get_current_folder()
```

Permet d'obtenir le chemin complet du dossier sélectionné, ou dans lequel se trouve le fichier sélectionné.

Nous avons également la possibilité de créer des filtres de fichiers pour la fenêtre de sélection de fichiers.

```
mon_filtre = gtk.FileFilter()
mon_filtre.set_name("fichiers texte")
mon_filtre.add_pattern("*.txt")
ma_fenetre.add_filter(filtre)
```

Enfin pour finir, on lance la fenêtre.

```
retour = ma_fenetre.run()
```

Ensuite, il faut analyser la variable retour et exécuter du code en fonction du choix de l'utilisateur.

```
if retour == gtk.RESPONSE_OK:
    dossier = ma_fenetre.get_current_folde()
    fichier = ma_fenetre.get_filename()
    chemin = dossier + fichier
```

À la fin, on n'oublie pas de détruire la fenêtre.

```
ma_fenetre.destroy()
```

VI-A-2 - Les conteneurs

En **PYGTK**, il existe trois grands types de conteneurs. Les **Hbox** divisent l'espace de manière horizontale. Les **Vbox** divisent l'espace de manière verticale. Enfin les tableaux divisent l'espace en plusieurs zones, à la façon d'un damier, ou d'un tableau d'où leur nom.

Chacune des zones créées peut ainsi être utilisée de manière indépendante pour la répartition des différents widgets.

VI-A-2-a - Hbox / Vbox

La création d'une **Hbox** prend deux paramètres. Le premier, homogeneous, permet de faire en sorte que toutes les zones aient la même taille (True) ou non (False). Le second paramètre, spacing, permet de préciser le nombre de pixels que l'on désire avoir en séparation, entre chaque zone.

La création d'une **Vbox** prend les mêmes paramètres

```
ma_hbox = gtk.HBox(homogeneous=True, spacing=2)
ma_vbox = gtk.VBox(homogeneous=True, spacing=2)
```

Il ne reste ensuite plus qu'à charger nos widgets dans nos conteneurs. Pour cela nous utilisons la méthode **pack_start** (pour charger le conteneur depuis la 1re zone) ou **pack_end** (pour le charger depuis la dernière zone).

Ces méthodes prennent quatre arguments. Le premier est le nom du widget à placer dans la zone. Le second, expand, va dépendre de la valeur de homogeneous. Si ce dernier est à True, alors expand ne sera pas pris en compte. Par contre, dans le cas contraire, l'ensemble des widgets dont expand sera à True se partagera l'espace disponible de la

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

fenêtre. Le troisième paramètre, fill, permet d'indiquer si le widget va occuper tout l'espace qui lui est alloué ou non. Le dernier, padding, permet de définir un espace (en pixels) entre le widget et la bordure de l'espace qui lui est alloué.

```
ma_hbox.pack_start(mon_widget, expand=True, fill=True, padding=2)
```

Remarque : si vous divisez un espace en trois zones, le pack_end mettra le 1er widget dans la zone 1, le 2d widget dans la zone 2, et le troisième dans la zone 3. Seul un widget peut exister par zone. Mais rien ne vous empêche de mettre un conteneur dans un autre afin de pouvoir mettre plusieurs widgets dans une zone initiale.

Attention : les méthodes pack de ces conteneurs, ne permettent pas de commencer à les remplir par une case au choix.

VI-A-2-b - Boîte à boutons

Les boîtes à boutons existent en mode vertical et horizontal. Elles ont le même rôle que les **Hbox** et **Vbox**, mais sont dédiées à la mise en place des boutons.

```
ma_hbuttonbox = gtk.HButtonBox()
ma_vbuttonbox = gtk.VButtonBox()
```

Une fois la boîte créée, il suffit de préciser le type de mise en page grâce à une des constantes prédéfinies.

Constante	Description
gtk.BUTTONBOX_SPREAD	Permet de répartir les boutons de manière équitable dans l'espace dédié (la plus usitée)
gtk.BUTTONBOX_EDGE	Permet de placer les boutons à l'extrême de l'espace
gtk.BUTTONBOX_START	Pour aligner les boutons sur la gauche ou le haut
gtk.BUTTONBOX_END	Pour aligner les boutons sur le bas ou la droite

```
ma_hbuttonbox.set_layout(gtk.BUTTONBOX_SPREAD)
```

Pour ajouter des boutons, on utilisera là aussi les méthodes **pack_start()** et **pack_end()**, tel que vu pour les **Hbox** et **Vbox**.

VI-A-2-c - Tableau

Le constructeur des tableaux prend trois paramètres : le nombre de lignes (rows), le nombre de colonnes (columns), et homogeneous que nous venons de voir avec le **Hbox**.

```
mon_tableau = gtk.Table(rows=1, columns=1, homogeneous=False)
```

Attention : la numérotation des colonnes et des lignes, pour la suite, commence à 0, en haut à gauche, tels des axes inversés. Le (0,0) correspond alors au coin supérieur gauche du tableau.

L'avantage du conteneur tableau est que l'on peut attribuer un widget, non pas à un des espaces créés, comme c'est le cas avec **Hbox** ou **Vbox**, mais sur plusieurs espaces.

De plus, on peut placer chaque widget à n'importe quelle place, et dans n'importe quel ordre.

Pour ajouter un widget à notre tableau, on utilise la méthode **attach()**.

```
mon_tableau.attach(mon_widget, left_attach, right_attach, top_attach, \
        bottom_attach, xoptions=gtk.EXPAND|gtk.FILL,\
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
        yoptions=gtk.EXPAND|gtk.FILL, xpadding=0, ypadding=0)
```

Voyons maintenant les différents paramètres de cette méthode.

Avant tout, rappelons que nous commençons à compter à partir de 0, en partant du coin supérieur gauche. Il faut aussi savoir qu'on ne parle pas dans ce cas de ligne 0 ou de ligne 1, mais de ligne contenue entre la borne 0 et la borne 1 ou de ligne contenue entre la borne 1 et la borne 2.

•left_attach : N° de la borne à gauche du widget.

•right_attach : N° de la borne à droite du widget.

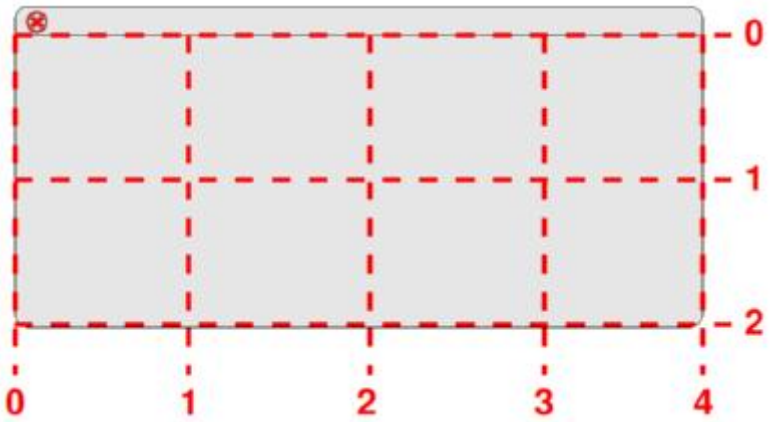
•top_attach : N° de la borne au-dessus du widget.

•bottom_attach : N° de la borne en dessous du widget.

•xpadding, ypadding : espace en pixels entre chaque zone, en x et en y.

•xoptions, yoptions : permet de définir le comportement du widget en x et en y. Pour cela, trois variables (tableau ci-après) sont disponibles et associables indifféremment, via un pipe « | », selon le résultat souhaité.

Constante	Description
gtk.EXPAND	Le widget prendra le maximum de place disponible dans le tableau
gtk.SHRINK	Le widget prendra le minimum de place possible
gtk.FILL	Le widget prendra toute la place qui lui est allouée



Le découpage d'un conteneur Tableau

Remarque : il est possible de superposer plusieurs widgets sur la même zone, puis de choisir lequel on désire afficher.

VI-A-2-d - Fixed

Le conteneur **fixed** permet de positionner les éléments avec des coordonnées. Très pratique, cela signifie néanmoins que votre application doit avoir une taille fixe pour tourner, si vous ne désirez pas avoir de mauvaises surprises en cas de redimensionnement.

```
mon_conteneur = gtk.Fixed()
```

Une fois créées, on retiendra deux principales méthodes liées : l'insertion, et le déplacement des widgets.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
mon_conteneur.put(widget, X, Y)
mon_conteneur.move(widget, X, Y)
```

VI-A-2-e - ScrolledWindow

Les **scrolledwindow** sont des fenêtres possédant des barres de défilement.

Ce type de fenêtre ne peut accepter qu'un seul enfant, qui sera la plupart du temps un conteneur (voir partie suivante).

VI-A-2-e-i - Création

La création d'une **scrolledwindow** est triviale :

```
ma_scroll = gtk.ScrolledWindow()
```

VI-A-2-e-ii - Paramétrage

Parmi les paramétrages, nous retrouvons la possibilité d'afficher ou non les barres de défilement :

```
ma_scrol.set_policy(HorizontalScrol, VerticalScrol)
```

ici, HorizontalScrol et VerticalScrol, ne peuvent réellement prendre que deux valeurs : **gtk.POLICY_ALWAYS** (affiche toujours le scroll), ou **gtk.POLICY_AUTOMATIC** (affiche le scroll uniquement quand nécessaire).

VI-A-2-e-iii - Ajout de l'enfant

Comme dit plus haut, on ne peut ajouter qu'un enfant. Cela s'effectue de la manière suivante :

```
ma_scrol.add_with_viewport(mon_conteneur)
```

L'ancrage de l'enfant a lieu sur le coin supérieur gauche de la **ScrolledWindow**.

VI-A-3 - Les widgets

VI-A-3-a - Boutons à cliquer

Les boutons permettent à l'utilisateur de déclencher des actions. À la création d'un bouton, on a la possibilité de lui passer trois paramètres

paramètre	Description
label	Texte affiché dans le bouton. Par défaut à None
stock	Icône à afficher en plus du texte, par défaut à None. Les icônes disponibles seront vues un peu plus loin en
use_underline	Chaque lettre précédée d'un « _ » sera soulignée dans le texte du bouton. Par défaut à True

La méthode pour créer un bouton est **Button()**

```
mon_bouton = gtk.Button('_Bouton', stock=None, use_underline=True)
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Dans l'exemple ci-dessus, nous utilisons tous les paramètres, mais avec leur valeur par défaut. De fait, nous pouvons nous abstenir de les préciser pour arriver au même résultat.

```
mon_bouton = gtk.Button('_Bouton')
```

Une fois créé, il faut préciser l'action associée au bouton. Pour cela, on utilise la méthode **connect()** et l'un des trois signaux suivants :

Signal	Description
pressed	Quand le bouton est pressé
released	Quand le bouton est relâché
clicked	Quand le bouton est cliqué (pressé, puis relâché)

Bien que pour certains, la subtilité ne soit pas forcément évidente, dans les faits, cela pourra parfois vous servir. Cependant, la plupart du temps on se contentera de **clicked**.

```
mon_bouton.connect(" clicked ", gtk.main_quit) #bouton pour sortir du software
mon_bouton.connect(" clicked ", ma_fonction)
```

Remarque : lors de l'utilisation de la méthode connect(), l'appel de la fonction est particulier. En effet, les arguments doivent être passés à la suite du nom de la fonction. De plus, le premier argument d'une fonction appelée par un connect() sera toujours widget (le widget appelant) (sauf si on est dans une classe, le 1er argument sera self, et le second widget). Résumons donc par l'exemple

```
def ma_fonction(widget, arg1, arg2): #ou def ma_fonction(self,widget,arg1,arg2):
    &#8230;
&#8230;
mon_bouton.connect(" clicked ", ma_fonction, arg1, arg2)
```

VI-A-3-b - Boutons à commuter

Les boutons à commuter ne peuvent prendre que deux états (enfoncé ou non).

```
mon_bouton_commut = gtk.ToggleButton(label= "Bouton 2 ")
```

On peut définir l'état par défaut du bouton.

```
mon_bouton_commut.set_active(True)
```

On peut tester l'état du bouton.

```
&#8230;
if mon_bouton_commut.get_active():
    ...
```



Les boutons à commutation (http://developer.gnome.org)

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

VI-A-3-c - Boutons à cocher

Les **checkbox**, ou cases à cocher permettent à l'utilisateur d'activer ou non certains labels. Il peut s'agir aussi bien d'un questionnaire, que d'une configuration.

Le constructeur est très simple. Le paramètre use_underline est à **True** par défaut, nous ne le préciserons donc pas.

```
ma_case = gtk.CheckButton(" Texte de la case')
```

Il existe derrière deux méthodes permettant de changer de manière logicielle l'état de la case, et également de le récupérer. On peut également déclencher certaines actions lorsqu'on active/désactive la case avec la méthode **connect**

```
ma_case.set_active(True)
ma_case.get_active() #vaut True si la case est cochee
ma_case.connect(" clicked ", ma_fonction)
```



Un bouton à cocher (http://developer.gnome.org)

VI-A-3-d - Boutons radio

Les **cases radio** sont comparables aux **cases à cocher** à ceci près, qu'il est possible de créer des groupes et que chaque groupe ne peut posséder plus d'une case active.

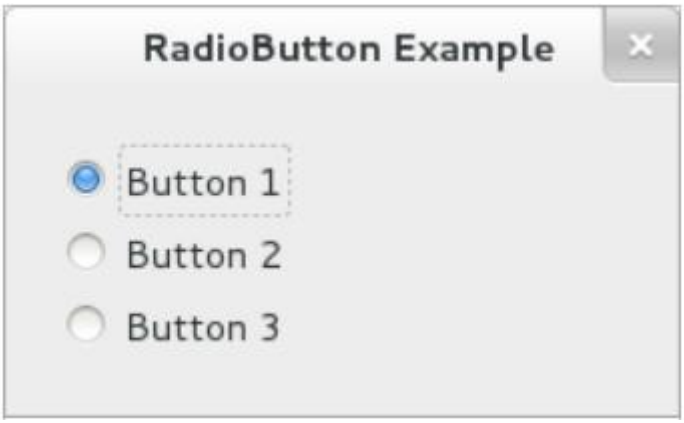
Une **case radio** est construite sur le même principe qu'une case à cocher, mais prend un paramètre supplémentaire, passé en 1re position : son groupe. En fait de groupe, nous passons le 1er élément du groupe aux éléments suivants

```
mon_radiob1 = gtk.RadioButton(None, "1er choix")
mon_radiob2 = gtk.RadioButton(mon_radiob1, "2d choix")
mon_radiob3 = gtk.RadioButton(mon_radiob1, "3e choix")
```

Tout comme les cases à cocher, on peut en cas de clic, faire appel à une fonction

```
mon_radiob1.connect(" clicked ", ma_fonction )
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/



Quelques boutons radio (http://developer.gnome.org)

VI-A-3-e - Combobox

Les **combobox**, ou listes déroulantes permettent à l'utilisateur d'effectuer un choix parmi une liste prédéfinie.

```
mon_combo = gtk.combo_box_new_text()
```

Une fois la **combo** créée, on peut ajouter ou supprimer simplement des éléments.

```
1. mon_combo.append_text(" second élément ")
2. mon_combo.insert_text(0, " premier element ")
3. mon_combo.remove(1) # retire " second element "
```

Comme on peut le voir ligne 3, on utilise non pas le texte, mais un index pour se repérer dans une **combobox**.

Ainsi pour savoir quel élément est actif, il est préférable d'avoir une liste ou un dictionnaire associé. Grâce aux méthodes associées, on pourra alors facilement faire un lien.

```
mon_combo.get_active()
mon_combo.set_active(0) #prend l'index comme parametre
```

Remarque : l'index commence à 0. De plus, l'index -1 signifie « affichage vierge » ; en clair aucune sélection.

Enfin, il est possible de savoir quand l'élément choisi change.

```
mon_combo.connect('changed', ma_fonction)
```

VI-A-3-f - Les onglets

Les onglets, ou **Notebook**, sont très pratiques en IHM.

Lorsqu'on a beaucoup d'informations à afficher à l'utilisateur, plutôt que de faire de multiples fenêtres, nous avons juste à créer un **Notebook**.

Ce dernier se comporte un peu comme un de ces annuaires, avec des signets qui dépassent en extrémité de page : vous sélectionnez votre signet, et vous arrivez sur une page dédiée.

Le principe utilisé ici sera le même.

```
mon_nb = gtk.Notebook()
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Jusque-là pas de souci, je pense. Alors, passons aux choses sérieuses.

Il faut tout d'abord choisir où positionner nos onglets. Quatre choix possibles :

```
mon_nb.set_tab_pos(POS_LEFT)    #a gauche
mon_nb.set_tab_pos(POS_RIGHT)    #a droite
mon_nb.set_tab_pos(POS_TOP)    #en haut (par défaut)
mon_nb.set_tab_pos(POS_BOTTOM)    #en bas
```

Il faut ensuite ajouter des onglets à notre **notebook**. Par défaut, il est créé sans aucun signet.

```
1. mon_nb.append_page(widget, signet_texte)
2. mon_nb.prepend_page(widget, signet_texte)
3. mon_nb.insert_page(widget, signet_texte, pos)
4. mon_nb.remove_page(num_onglet)
```

La ligne 1 effectue un ajout d'onglet à la suite de ceux existants.

La ligne 2 effectue l'ajout d'onglet avant ceux déjà existants.

Dans les deux cas, le 1er paramètre correspond au conteneur ou widget inséré dans l'onglet, et signet_texte au label de l'onglet. Il est ainsi impossible de créer un signet totalement vide. Il faut au moins un conteneur dans l'onglet.

Ligne 3, nous créons également un onglet, mais en précisant cette fois sa position parmi ceux déjà existants. Le 1er onglet est le 0.

Ligne 4, nous enlevons un onglet grâce à son numéro.

Outre cela, nous pouvons également gérer l'onglet actif aisément :

```
1. onglet_actif = mon_nb.get_current_page()
2. mon_nb.set_current_page(num_onglet)
3. mon_nb.next_page()
4. mon_nb.prev_page()
5. mon_nb.set_scrollable(True)
```

Ligne 1, nous récupérons l'onglet actif (le 1er est 0 pour rappel).

Ligne 2, nous définissons quel onglet doit être affiché à l'utilisateur.

Ligne 3 et 4, nous nous déplaçons au sein des onglets du Notebook.

Ligne 5 enfin, nous précisons qu'en cas d'un nombre d'onglets trop important pour qu'ils soient tous affichés, le **Notebook** doit afficher une **scrollbar**.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/



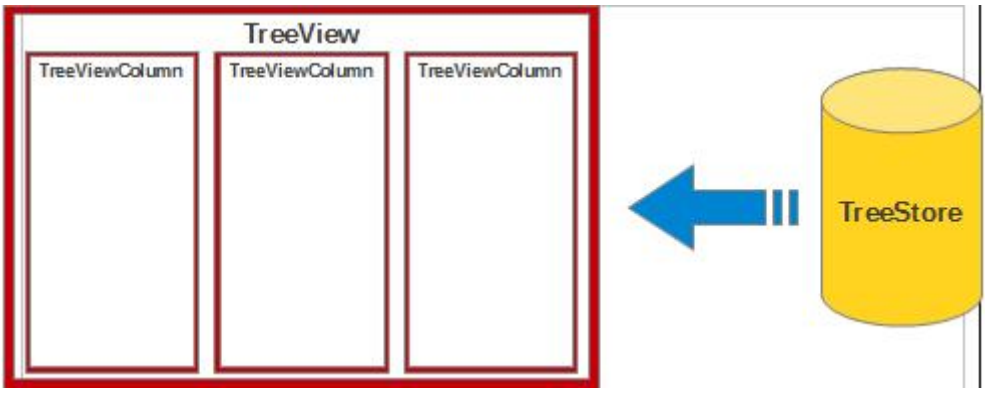
Exemple de notebook (http://developer.gnome.org)

VI-A-3-g - Les tableaux de données

Les tableaux de données sont une composante souvent utile en IHM.

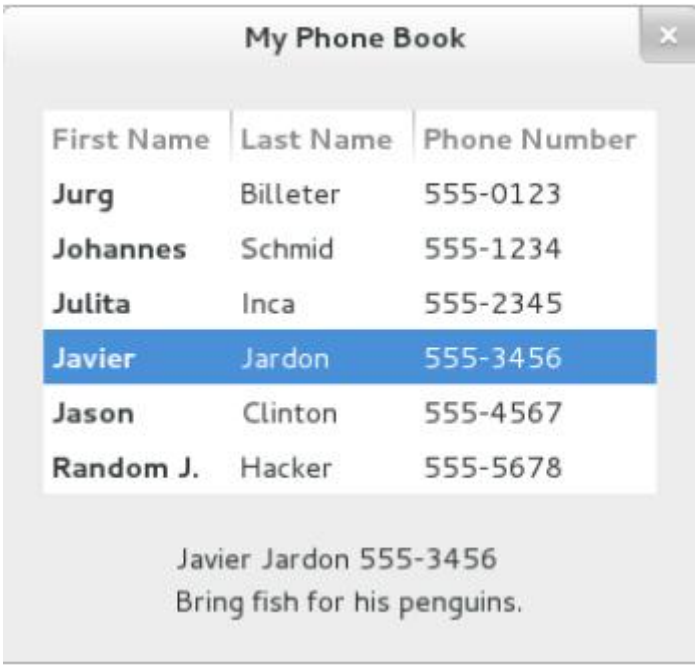
Quoiqu'en premier lieu un peu rebutant, vous verrez que créer en **PYGTK** n'est pas si compliqué.

Un tableau de données en PYGTK repose sur trois éléments : Le **TreeStore**, le **TreeView** et le **TreeViewColumn**.



Principe d'un tableau de données GTK

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/



Exemple de rendu final (http://developer.gnome.org)

VI-A-3-g-i - Les TreeStores

Il s'agit de l'objet contenant les données qui seront affichées dans le tableau. Il est comparable à une BDD spéciale pour le tableau. Il est comparable à une liste contenant elle-même d'autres listes, de façon à former une matrice.

Remarque : toutes les lignes d'un TreeStore doivent avoir le même nombre d'éléments.

On le crée en lui passant en paramètres les types des différents éléments qu'il contiendra.

```
mon_ts = gtk.TreeSore(int, float, str, str)
```

Ici nous venons de créer un **TreeStore** dont les lignes sont composées de quatre éléments : le premier élément est un entier, le second un float, et les deux derniers des chaînes de caractères.

Remarque : les autres possibilités à connaître sont : long, boolean.

Nous pouvons interagir avec les données d'un **TreeStore :**

```
mon_ts.append(None, [entier1, float1, string1, True])
```

Ici nous ajoutons des données au **TreeStore**. Le premier paramètre est le parent (en général None), et le second une liste contenant les éléments de la nouvelle ligne à ajouter.

Pour les actions suivantes, cela va se compliquer légèrement. En effet, il faudra passer par un **TreeIter**. Il s'agit simplement d'un tuple désignant l'accès à une donnée du **TreeStore**, ou à une de ses lignes.

Comparons, temporairement, nos **TreeStores** à des dossiers. Les dossiers peuvent à la fois contenir d'autres dossiers, et des données.

Sur le même principe, les **TreeStores** peuvent contenir d'autres **TreeStores**. Pour accéder à une ligne, nous utilisons ce qu'on appelle un **PATH** ou chemin.

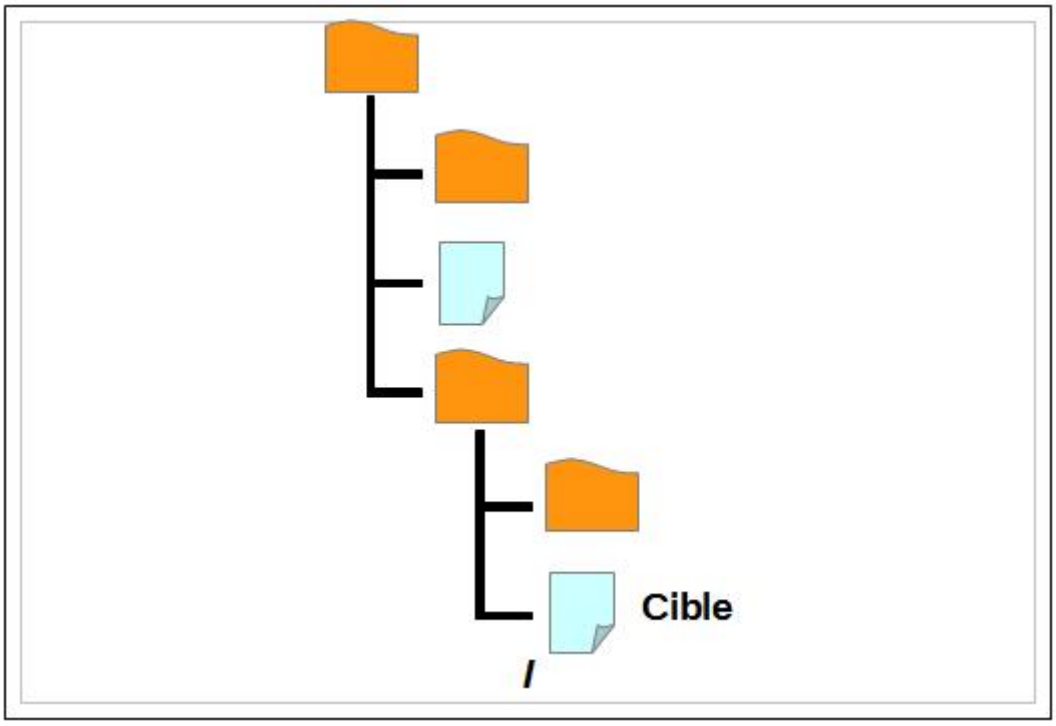
Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

La référence ici est le 0, une fois n'est pas coutume. Chaque entier correspond au nœud à rechercher dans le « dossier ». La chose à bien retenir est qu'on lit les éléments dans l'ordre où ils ont été insérés.

Le **PATH** pour accéder au fichier cible est alors (2,1). Si nous interprétons ce chemin, une fois dans notre **TreeStore** (dossier père dans l'exemple), nous devons rentrer dans le 3e élément (le 2d sous-dossier), afin d'atteindre notre cible qui est le 2d élément du sous-dossier.

Dans un **TreeIter** basique, où nous n'avons pas de « sous-dossier », le PATH est simplement le numéro de l'élément cible. Ainsi, le fichier placé directement dans le dossier maître de l'exemple est (1,).

Prenons un exemple visuel :



Principe du PATH d'un TreeStore

Une fois notre **PATH** identifié, nous récupérons notre iter ainsi :

```
mon_treeiter = mon_ts.get_iter(PATH)
```

À partir de là, nous pouvons continuer sur nos interactions sur les **TreeStores**.

```
1. mon_ts.remove(mon_treeiter)
2. mon_ts.set_value(mon_treeiter, ma_colomne, valeur)
```

Ligne 1, nous enlevons une ligne de données.

Ligne 2, nous modifions la valeur d'un champ, en passant en paramètre le **TreeIter** permettant d'accéder à la bonne ligne, puis le numéro de la colonne impactée, et enfin la nouvelle valeur.

D'autres possibilités sont effectives sur les **TreeStores**. Si vous êtes amené à les considérer, je vous invite à lire plus amplement la documentation en ligne.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

VI-A-3-g-ii - Les TreeViews

Les **TreeView** servent d'interface entre le **TreeStore** (la base de données) et les **TreeViewColumns** (que nous verrons juste après), servant à afficher lesdites données.

On crée un **TreeView** très simplement en l'associant à un **TreeStore :**

```
mon_tv = gtk.TreeView(mon_TreeStore)
```

Les **TreeView** possèdent beaucoup de méthodes et de propriétés. Voici les principales qui vous serviront :

```
1. mon_tv.set_rules_hint(True)
2. mon_tv.append_column(ma_colonne)
3. mon_tv.remove_column(ma_colonne)
4. mon_tv.insert_column(ma_colonne, position)
5. ma_colonne = mon_tv.get_column(n)
6. mon_treeiter = mon_tv.get_active_iter()
7. mon_tv.set_active_iter(mon_treeiter)
```

Ligne 1, nous indiquons que nous désirons un affichage au format tableau classique.

Ligne 2, nous ajoutons une colonne (point suivant) à notre **TreeView**.

Ligne 3, nous retirons notre colonne du **TreeView**.

Ligne 4, nous insérons notre colonne à une certaine position dans notre **TreeView** (on commence à 0).

Ligne 5, nous récupérons le **TreeViewColumn** placé à la position n dans le **TreeView**.

Ligne 6, nous récupérons le **TreeIter** correspondant à la ligne active dans le tableau.

Ligne 7, nous surlignons la ligne désignée par le **TreeIter**.

Voilà pour faire simple. Mais avant de passer aux **TreeViewColumns**, nous allons apporter quelques précisions sur la sélection dans un **TreeView**.

Cette dernière passe par un **TreeSelection**. Un **TreeSelection** est un objet créé en même temps que le **Treeview** et qui le renseigne sur les diverses actions liées à la sélection.

Cependant, en utilisant le **TreeView** pour nous interfacer avec, tel que vu lignes 6 et 7 précédemment, nous perdons nombre de possibilités. Nous allons combler ce manque maintenant.

Pour récupérer le **TreeSelection** associé au **TreeView**, on utilise :

```
mon_tselect = mon_tv.get_selection()
```

Le type de sélection se fait via **set_mode() :**

```
mon_tselect.set_mode(mode)
mode_actif = mon_tvselect.get_mode()     # Renvoie le mode actif
```

Les modes disponibles sont les suivants :

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Mode	Description
gtk.SELECTION_NONE	Aucune sélection autorisée
gtk.SELECTION_SINGLE	Sélection possible d'une seule ligne par clic
gtl.SELECTION_BROWSE	Sélection d'une seule ligne par survol du pointeur
gtk.SELECTION_MULTIPLE	Sélections multiples autorisées

Si vous souhaitez cacher les entêtes du tableau, il faudra procéder ainsi :

```
mon_tv.set_headers_visible(False)
```

De manière logicielle il est possible de (dé)sélectionner tout ou partie du tableau.

```
mon_tselect.select_path(PATH)
mon_tselect.unselect_path(PATH)
mon_tselect.select_iter(mon_TreeIter)
mon_tselect.unselect_iter(mon_TreeIter)
mon_tselect.select_all()    #Necessite d'etre en mode selection multiple
mon_tselect.unselect_all()    #Necessite d'etre en mode selection multiple
mon_tselect.select_range(START_PATH, END_PATH)
mon_tselect.unselect_range(START_PATH, END_PATH)
```

On peut aussi savoir si une ligne précise est sélectionnée ou non :

```
est_actif = mon_tselect.path_is_selected(PATH)
est_actif = mon_tselect.iter_is_selected(mon_TreeIter)
```

Pour connaître le nombre de lignes sélectionnées :

```
mon_tselect.count_selected_rows()
```

Enfin, le principal, pour récupérer les lignes sélectionnées :

```
1. mon_modele, mon_treeiter = mon_tselect.get_selected()
2. mon_modele, mon_treeiter = mon_tselect.get_selected_rows()
```

La ligne 1 est à utiliser dans les modes de sélection unique, la ligne 2 dans le mode de sélections multiples.

Remarque : les modèles sont une façade des TreeView que je n'ai volontairement pas évoqués, car cela complexifie considérablement l'approche, et n'est pas indispensable à l'usage des TreeView. Vous êtes obligé de le récupérer lors de l'appel de ces dernières méthodes, mais n'êtes nullement obligé de vous en servir par la suite. N'hésitez pas à consulter la documentation officielle PYGTK sur les TreeModel si votre curiosité vous le demande.

VI-A-3-g-iii - Les TreeViewColumn

Le dernier élément. Il permet d'afficher les données à l'utilisateur. Là aussi, la création est simple. Il ne prend qu'un seul paramètre qui s'avère être le titre de la colonne qu'il portera à l'écran.

ma_colonne **=** gtk**.**TreeViewColumn**(**'Colonne 1'**)**

parmi les méthodes les plus usuelles, nous remarquerons les suivantes :

```
1. ma_colonne.set_sizing(gtk.TREE_VIEW_COLUMN_AUTOSIZE)
2. ma_colonne.set_fixed_width(60)
3. ma_colonne.set_resizable(True)
4. ma_colonne.set_expand(True)
5. mon_tv = ma_colonne.get_tree_view()
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
6. ma_colonne.set_title('Colonne 0')
7. ma_colonne.set_clickable(True)
8. ma_colonne.set_alignment(0.5)
```

Ligne 1, nous définissons le comportement que doit avoir la largeur de la cellule vis-à-vis de la largeur de son contenu. Les paramètres possibles sont les suivants :

Paramètre	Description
gtk.TREE_VIEW_COLUMN_AUTOSIZE	La colonne prend automatiquement la bonne largeur
gtk.TREE_VIEW_COLUMN_FIXED	La largeur de la colonne est paramétrée en pixels par le programmeur (voir ligne2)
gtk.TREE_VIEW_COLUMN_GROW_ONLY	La taille de la colonne est définie par le modèle

Il existe également la méthode **get_sizing()**.

Ligne 2, nous définissons la largeur de la colonne en pixels. Il existe aussi la méthode **get_fixed_width()**.

Attention : le paramètre de largeur doit être strictement supérieur à 0 afin de ne pas générer d'erreurs.

Ligne 3, nous indiquons à **GTK** que nous autorisons l'utilisateur à modifier manuellement la largeur de la colonne. Utiliser la méthode **get_resizable()** pour connaître ce paramétrage.

Ligne 4, nous indiquons à **GTK** que la colonne doit prendre le maximum de place qu'elle peut prendre. Là aussi, la méthode inverse existe : **get_expand()**.

Ligne 5, nous récupérons le nom du **TreeView** dont dépend notre **TreeViewColumn.**

Ligne 6, nous modifions le titre de la colonne.

Ligne 7, nous transformons l'entête, contenant le titre, de la colonne en bouton. Le signal à utiliser alors est « clicked ».

Ligne 8, nous centrons le titre de notre colonne.

Remarque : l'ajout/insert/retrait de colonnes se gère au niveau du TreeView.

Pour la suite, nous allons compliquer légèrement, une fois de plus, les choses. En effet, même si cela n'est pas obligatoire, vous avez la possibilité de passer par ce qu'on appelle un **CellRendererText**.

Il s'agit d'un objet permettant des actions supplémentaires sur les colonnes. Par exemple, vous pouvez changer la couleur de fond, masquer les données…

```
1. mon_crt = gtk.CellRendererText()
2. mon_crt.set_property('cell-background', 'cyan')
3. mon_crt.set_property("'visible', False)
4. ma_colonne.pack_start(mon_crt, True)
5. ma_colonne.add_attribute(mon_crt, 'text', 0)
```

Ligne 1, nous créons simplement notre **CellRendererText**.

Ligne 2, nous paramétrons la couleur de fond. Les valeurs possibles sont entre autres : cyan, red, yellow, magenta, green, blue.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Pour définir votre propre couleur, utilisez : mon_crt.set_property('cell-background-gdk', gtk.gdk.Color(R,G,B,P)

Ici, RGB est compris entre 0.0 et 1.0. P est simplement un index.

Ligne 3, nous précisons que nous voulons cacher les données.

Ligne 4, nous positionnons le **CellRendererText** dans notre **TreeViewColumn**. Le 2d paramètre à True est expand. Nous sommes ainsi assurés d'occuper tout l'espace disponible.

Ligne 5, nous configurons notre **CellRendererText** par rapport à notre **TreeViewColumn**. Le 1er argument est le **CellRendererText**, le 2d le type de **CellRenderer** (toujours « text »), et enfin le numéro de colonne auquel il doit être rattaché.

VI-A-3-h - Label

Les labels permettent d'afficher du texte non modifiable par l'utilisateur.

```
mon_label = gtk.Label("Hello_world")
```

Le texte, une fois établi est librement modifiable via une simple méthode

```
mon_lable.set_text(" nouveau texte ")
```

Vous avez également la possibilité de choisir l'emplacement du texte

```
mon_label.set_justify(gtk.JUSTIFY_CENTER)
```

On remarque ici l'utilisation d'une constante **PYGTK**. Les autres choix possibles sont **gtk.JUSTIFY_LEFT** (qui est la valeur par défaut), **gtk.JUSTIFY_RIGHT** et **gtk.JUSTIFY_FILL**.

Remarque : même si cela n'est pas vu ici, il existe la possibilité de choisir sa police, couleur, style de texte… via du code HTML dans la méthode set_text. Regardez sur Internet du côté de la méthode set_use_markup pour plus d'informations.

VI-A-3-i - Textbox

VI-A-3-i-i - Monoligne

Les monolignes sont ce qu'on appelle des entrées. Le constructeur portera donc ce nom et prendra comme paramètre la longueur de texte qu'il doit supporter. Par défaut, paramétré à 0 (infini).

```
mon_entree = gtk.Entry(Longeur)
```

Parmi ses méthodes, les plus usitées sont les suivantes :

```
mon_entree.set_text(" Texte ") #renseigne le champ
mon_entree.get_text() #recupere le texte du champ
mon_entree.set_alignment(0,5) #Place le texte au centre (max=1 à droite,min=0)
mon_entree.set_visibility(True) #remplace l&#146;écho des caractères par des points
mon_entree.set_max_length(10) #definit la taille max
mon_entree.get_max_length() #renvoie la taille max du champ
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

VI-A-3-i-ii - Multilignes

Les textbox multilignes ne sont, à première vue pas si évidentes à maîtriser.

Si je dis cela, c'est qu'il ne suffit pas de créer un simple objet pour en disposer.

En effet, une textbox multiligne est en réalité un assemblage de trois objets : un **TextView**, un **TextBuffer**, un **TextTagTable**.

Le **TextTagTable** peut être vu comme un miniespace de stockage, abritant les données que l'utilisateur saisira et lira.

Le **TextBuffer** est l'objet **PYGTK** contenant le texte qui aura été saisi, et qui sera affiché à l'utilisateur. Le **TextBuffer** utilise comme aire de stockage un **TextTagTable**. Il s'agit donc simplement d'un outil d'accès à des données.

Pour l'afficher à l'utilisateur, nous utilisons un **TextView**. Cet objet se contente de faire interface entre l'utilisateur et le **TextBuffer**.

Voilà pour la partie explication fonctionnelle. Le plus dur est fait. Pour la partie pratique, les choses se simplifient.

Il faut savoir que **PYGTK** nous permet une manipulation partiellement transparente de ces objets.

Ainsi, la création d'un simple **Textview** entraîne la création automatique d'un **TextBuffer** et d'un **TextTagTable** associés. De plus, en général, nous n'avons nullement besoin d'intervenir sur le **TextTagTable**, simplifiant encore les choses.

Finalement donc, nous interviendrons la plupart du temps au niveau du **TextView** pour paramétrer notre interaction avec l'utilisateur, et sur le **TextBuffer** pour gérer le texte.

```
1. mon_tv = gtk.TextView()
2. mon_buf = mon_tv.get_buffer()
```

Faisons simple pour commencer: la création des objets.

Ligne 1, nous créons notre **textview**, et ligne 2, nous récupérons la référence au buffer lié.

Commençons la revue en détail par le **TextView**.

Vous avez la possibilité d'autoriser ou non l'édition du texte affiché, et de connaître si cela est possible :

```
mon_tv.set_editable(True)     #True ou False
edition = mon_tv.get_editable()
```

Vous avez également la possibilité de masquer le curseur :

```
mon_tv.set_cursor_visible(True)     #True ou False
```

Vous pouvez aussi paramétrer le type de renvoi à la ligne automatique :

```
mon_tv.set_wrap_mode(gtk.WRAP_NONE) #Sans retour à la ligne
mon_tv.set_wrap_mode(gtk.WRAP_WORD) #Retour à la ligne au mot près
```

Enfin, reste le paramétrage de la justification :

```
mon_tv.set_justification(gtk.JUSTIFY_LEFT)
mon_tv.set_justification(gtk.JUSTIFY_RIGHT)
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
mon_tv.set_justification(gtk.JUSTIFY_CENTER)
justif = mon_tv.get_justification      #On recupere la justification parametree
```

Remarque : les TextView ne savent pas gérer les scrolls (barres de défilement). En cas de besoin, il vous faut donc passer par une fenêtre ScrolledWindow.

Maintenant, place à la revue en détail du **TextBuffer**.

Comme dit précédemment, il s'agit de l'objet contenant le texte à afficher sur le **TextView**.

Vous pouvez grâce à lui connaître le nombre de lignes ou encore de caractères qu'il contient :

```
mon_buf.get_line_count()
mon_buf.get_char_count()
```

De manière tout aussi pratique, vous pouvez initialiser le texte :

```
mon_buf.set_text(mon_texte)
```

Le plus pratique maintenant : récupérer du texte. Cette partie se complique légèrement, mais rien d'inquiétant je vous rassure.

En effet, la méthode récupérant le texte demande de préciser le caractère de début et de fin, d'où l'utilité de connaître le nombre total de caractères dans le texte. Le dernier paramètre passé permet de préciser que l'on veut récupérer la totalité des caractères de la fourchette fournie.

```
mon_texte = mon_buf.get_text(start, end, True)
```

Finissons maintenant par l'effacement. Là aussi, il faut passer une fourchette de caractères :

```
mon_buf.delete(start, end)
```

Voici pour les explications d'une textbox multiligne. Le but ici n'est clairement pas de créer un éditeur de texte. D'autres paramétrages sont évidemment possibles, mais pour ces derniers plus complexes, et moins usuels, je vous renvoie vers la documentation officielle.

VI-A-3-j - Compteur

Les compteurs sont de petits widgets permettant de choisir une valeur numérique à l'aide de deux petits curseurs.

Pour créer un widget compteur, il faut avant créer un objet appelé **Adjustment**.

```
adjustment = gtk.Adjustment(value,lower,upper,step_incr,page_incr,page_size)
```

Comme on peut le voir, plusieurs paramètres sont ici passés. Ils vont permettre de paramétrer précisément, par la suite, notre compteur. L'ensemble de ces paramètres sont par défaut à 0 (pouvant alors signifier infini pour certains paramètres)

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

paramètre	Description
value	Valeur initiale
lower	Minimum possible
upper	Maximum possible
step_incr	Pas d'incrémentation (ou de décrémentation) lors de l'appui sur un bouton avec le clic gauche
page_incr	Pas d'incrémentation (ou de décrémentation) lors de l'appui sur un bouton avec le clic droit
page_size	Non utilisé

Une fois ce paramétrage réalisé, il faut créer notre compteur

```
mon_compteur = gtk.SpinButton(adjustment, climb_rate, digits)
```

Ici, trois paramètres.

paramètre	Description
adjustment	Notre objet créé précédemment
climb_rate	La vitesse d'avance quand on reste appuyé sur un des deux boutons (en %: de 0.0 à 1.0)
digits	Nombre de chiffres à utiliser pour l'affichage

Vous pouvez reconfigurer ce compteur également à la volée.

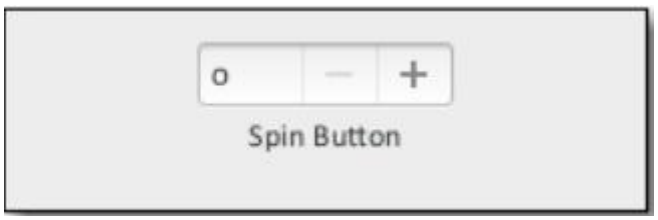
```
mon_compteur.configure(adjustment, climb_rate, digits)
```

Vous pouvez aussi récupérer l'ajustement.

```
mon_compteur.get_adjustment()
```

Enfin, vous pouvez récupérer la valeur simplement avec un get.

```
mon_compteur.get_value()
```



Un compteur (http://developer.gnome.org)

VI-A-3-k - Progressbar

Les **barres de progression** sont présentes un peu partout dans les logiciels afin de montrer le niveau d'avancement d'une tâche, ou encore pour représenter graphiquement certaines valeurs.

```
ma_pb = gtk.ProgressBar() #creation d'une pb à 0%
```

Ensuite, il est possible de changer l'orientation (et de fait le sens de la progression) grâce à la méthode set_orientation(). Les valeurs possibles sont les suivantes

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Constante	Description
gtk.PROGRESS_LEFT_TO_RIGHT	De gauche à droite (par défaut)
gtk.PROGRESS_RIGHT_TO_LEFT	De droite à gauche
gtk.PROGRESS_BOTTOM_TO_TOP	De bas en haut
gtk.PROGRESS_TOP_TO_BOTTOM	De haut en bas

```
ma_pb.set_orientation( = gtk.ProgressBar(PYGTK.PROGRESS_LEFT_TO_RIGHT)
```

Il existe ensuite différentes façons de faire progresser une **ProgressBar**. La première méthode (la plus simple) consiste à lui indiquer quelle valeur elle doit représenter

```
ma_pb.set_fraction(0.5) #50%
ma_pb.set_fraction(1.0) #100%
```

La seconde méthode consiste à définir un pas d'avance, et à envoyer des impulsions

```
ma_pb.set_pulse_step(0.01) #pas de 1%
ma_pb.pulse() #chaque execution de cette ligne ajoute 1%
```

Par défaut, la **ProgressBar** n'affiche aucun texte. Il est cependant possible de le lire et de le définir.

```
ma_pb.set_text(" ma progressbar ")
retour = ma_pb.get_text()
```

VI-A-3-l - Calendrier

Il peut être utile de présenter à l'utilisateur un **calendrier**, ne serait-ce que pour s'assurer que la date qu'il va renseigner sera conforme à nos attentes. Pour cela, le widget calendrier est très utile.

```
mon_calendrier = gtk.Calendar()
```

Ici aussi, rien de très sorcier. Parmi les méthodes les plus utiles, nous retrouvons les suivantes

```
mon_calendrier.select_month(mois, annee) #affiche le calendrier
mon_calendrier.select_day(jour) #selectionne le jour dans le calendrier (de 1 à 31)
mon_calendrier.mark_day( jour) #met le jour en gras
mon_calendrier.get_date() #retourne la sélection user en tuple (an, mois, jour)
```

VI-A-3-m - Image

L'affichage d'images sous **PYGTK** est très simple. Il suffit de créer un widget Image, puis de le remplir avec une image. Il ne reste ensuite plus qu'à mettre le widget dans un conteneur.

```
mon_image =  gtk.Image()
mon_image.set_from_file("./mon_image.png ")
mon_image.show()
```

Il faut savoir aussi que l'on peut insérer une image dans un bouton pour remplacer une icône par exemple (attention alors à la taille de l'image), en s'en servant comme d'un conteneur.

```
mon_bouton.add(mon_image)
```

Remarque : il est également possible de mettre l'image avec un label dans une Hbox et de mettre la Hbox dans le bouton.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

VI-A-3-m-i - Image en fond de fenêtre

Nous allons voir ici comment mettre une image en fond de fenêtre et faire en sorte que l'image s'adapte à la taille de la fenêtre, même si cette dernière est redimensionnée.

Nous allons commenter le code suivant, qui vous permettra de réaliser cette action. Nous verrons certaines méthodes dont nous ne détaillerons que le strict nécessaire. Cela dépasserait le cadre des simples éléments de base, mais une petite recherche sur Internet vous aidera.

À partir de ce code, vous pourrez déduire par vous-même comment faire dans le cas d'une fenêtre de taille fixe.

```
1. #! /usr/bin/env PYTHON
2. # -*- coding: utf-8 -*-
3. import gtk
4.
5. def draw_pixbuf(widget, event):
6.     path = './logo.jpg'
7.     allocation = widget.get_allocation()
8.     pixbuf = gtk.gdk.pixbuf_new_from_file(path)
9.     pixbuf = pixbuf.scale_simple(allocation.width, allocation.height,  \
10.             gtk.gdk.INTERP_TILES)
11.
12.     widget.window.draw_pixbuf(widget.style.bg_gc[gtk.STATE_NORMAL], \
13.                     pixbuf, 0, 0, 0,0)
14.
15.      widget.window.invalidate_rect( allocation, False )
16.
17.     del pixbuf
18.
19.
20. window = gtk.Window()
21. window.set_title('Drawing Test')
22. window.set_size_request(64,48)
23. window.connect('destroy',gtk.main_quit)
24. hbbox = gtk.HBox()
25. window.add(hbbox)
26. hbbox.connect('expose-event', draw_pixbuf)
27. button = gtk.Button()
28. hbbox.pack_start(button, True, False, 50)
29. window.show_all()
30.
31. gtk.main()
```

Ligne 1 et 3, vous reconnaîtrez des lignes classiques, dont nous passerons l'explication.

Ligne 20 à 23, nous créons et configurons notre fenêtre principale.

Ligne 24, nous créons un conteneur **Hbox**, et l'ajoutons à notre fenêtre principale ligne 25.

Ligne 27 et 28, nous créons et ajoutons un bouton à notre conteneur, puis ligne 29 et 31, nous affichons et démarrons notre logiciel.

Maintenant, attaquons la partie intéressante de ce code. Tout d'abord la ligne 26. Nous connectons notre conteneur à la fonction draw_pixbuf lorsque l'événement « **expose-event** » est déclenché.

Remarque : il existe de nombreux événements possibles (fenêtre détruite, pointeur souris qui bouge, bouton de souris pressé ou relâché, double ou triple clic, touche de clavier pressée ou relâchée, souris qui rentre ou sort d'une fenêtre…). Rendez-vous sur cette page pour les retrouver : **http://pygtk.org/docs/pygtk/class-gtkwidget.html**.

Cet événement est généré lorsqu'un événement quelconque (déplacement, modification de taille…) a lieu sur le conteneur ou la fenêtre concernée.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Dans notre code, quand cela se produit nous appelons donc draw_pixbuf, fonction déclarée de la ligne 5 à 17.

Pour commencer, ligne 6, nous déclarons le chemin de l'image de fond.

Ligne 7, nous effectuons un get_allocation, de la classe widget. Cela permet de récupérer la largeur et la hauteur du widget (dans notre cas, allocation.height et allocation.width).

Ligne 8, nous créons un **pixbuf** à partir de notre image. Le pixbuf est un type d'image servant beaucoup en interne dans GTK.

Ligne 9 et 10, nous redimensionnons notre image à la taille du conteneur. Les paramètres sont la largeur, la hauteur, et le type de rendu. Ici, la constante **gtk.gdk.INTERP_TILES** est le meilleur compromis entre performance et rendu.

Ligne 12 et 13, nous dessinons notre pixbuf dans notre conteneur. Les quatre derniers paramètres sont des coordonnées, qui seront quasi exclusivement toujours à 0.

Ligne 15, nous effectuons une opération d'invalidation. Sans cette opération, votre dessin sera redimensionné, mais également superposé avec tous les dessins intermédiaires du redimensionnement. Je vous invite à essayer pour en constater les effets concrets.

Ligne 17 enfin, nous effaçons notre objet pixbuf, qui une fois inséré en fond de conteneur, ne sert plus.

VI-A-3-n - Cadre

Les **cadres**, comme leur nom l'indique, servent à encadrer un lot de widgets en leur donnant un nom précis.

```
mon_cadre = gtk.Frame(label= "Mon cadre ")
```

Peu de choses à connaître pour s'en servir. On peut changer le label en cours de code, jouer sur la position du texte par rapport au **cadre** (les paramètres X et Y varient alors entre 0.0 et 1.0, par défaut à 0.0), et enfin jouer sur le cadre lui-même, grâce aux constantes suivantes.

Constante	Description
SHADOW_ETCHED_IN	Par défaut, trait du contour en creux
SHADOW_ETCHED_OUT	Trait du contour en relief
SHADOW_IN	Cadre de type creux
SHADOW_OUT	Cadre de type relief
SHADOW_NONE	Sans relief

```
mon_cadre = gtk.Frame(label= "Mon cadre ")mon_cadre.set_label(" mon nouveau titre")
mon_cadre.set_label_align(X, Y)
mon_cadre.set_shadow_type(SHADOW_ETCHED_IN)
```

Comme d'habitude, on n'oubliera pas d'exécuter la méthode show sur notre widget afin de l'afficher.

Remarque : s'il est un widget, le cadre joue également le rôle de conteneur.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/



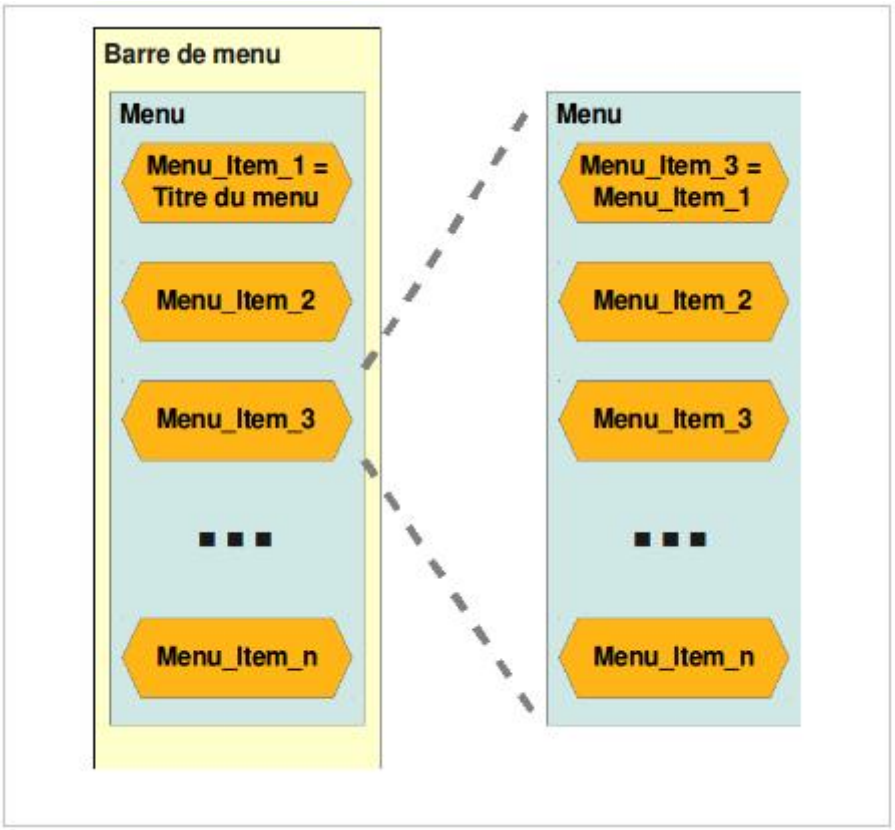
Un exemple de cadre (http://developer.gnome.org)

VI-A-3-o - Menu

Les **menus** permettent à l'utilisateur d'accéder à nombre de fonctions et de paramétrages au sein d'une application.

Selon l'application, on pourra se contenter d'une barre d'outils, ou l'on devra au contraire implémenter un menu complet

VI-A-3-o-i - Structure d'un menu



Fonctionnement des menus PYGTK

L'illustration ci-dessus résume le fonctionnement des menus sous **PYGTK**.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Un menu **PYGTK** n'est qu'un assemblage de **menus** et de **menu_Items**, le tout rattaché à une barre de menu. Par menu_items nous entendons tout texte interprété du point de vue utilisateur comme une entrée menu.

Tout sous-menu est un menu à part entière.

Le premier menu_items d'un menu est également le titre du menu.

VI-A-3-o-ii - Menu texte

```
1. mon_menu = gtk.MenuBar()
2. menu_fichier = gtk.Menu()
3. m_fichier = gtk.MenuItem("_Fichier")
4. m_fichier.set_submenu(menu_fichier)
5. exit = gtk.MenuItem("_Exit")
6. exit.connect("activate", gtk.main_quit)
7. menu_fichier.append(exit)
8. mb.append(m_fichier)
```

Ligne 1, nous créons notre barre de menus. C'est elle qui contiendra l'ensemble des menus.

Ligne 2, nous créons notre menu Fichier. Il est alors vide, sans aucune donnée.

Ligne 3, nous créons notre premier menu_items, Fichier. Le « _ » placé avant le F dans le label indique à **PYGTK** qu'il doit souligner le F. Cela peut servir à indiquer à l'utilisateur un raccourci clavier par exemple.

Ligne 4, nous connectons notre menu_items à notre menu afin d'en faire le titre du menu.

Ligne 5, nous créons un nouveau menu_items, et le configurons linge 6.

Ligne 7, nous l'ajoutons au menu Fichier.

Ligne 8, nous ajoutons notre menu à la barre de menu.

Remarque : une fois assimilé, le fonctionnement de création d'un menu n'est pas si complexe qu'en prime abord. Il est important de bien noter que le premier élément inséré dans le conteneur de menu sera l'élément que l'utilisateur considérera comme le nom du menu. L'ordre a donc son importance.

VI-A-3-o-iii - Sous-menu

Pour éviter qu'un menu ne devienne trop brouillon, nous pouvons effectuer des regroupements par genre. Cela donnera à l'utilisateur l'impression de disposer de menus et de sous-menus par thème.

Comme vu précédemment, un sous-menu n'est rien d'autre qu'un menu inséré dans un autre menu. Nous allons donc créer plusieurs menus, et les insérer les uns dans les autres afin de créer nos menus et sous-menus.

```
1. mon_menu = gtk.MenuBar()
2. menu_fichier = gtk.Menu()
3. m_fichier = gtk.MenuItem("_Fichier")
4. m_fichier.set_submenu(menu_fichier)
5. mb.append(m_fichier)
6.
7. menu_choix = gtk.Menu()
8. m_choix = gtk.MenuItem("_Choix")
9.
10. choix1 = gtk.MenuItem("Choix 1")
11. choix2 = gtk.MenuItem("Choix 2")
12. choix3 = gtk.MenuItem("Choix 3")
13.
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
14. m_choix.set_submenu(menu_choix)
15. menu_choix.append(choix1)
16. menu_choix.append(choix2)
17. menu_choix.append(choix3)
18.
19. menu_fichier.append(m_choix)
```

Nous pouvons voir, ligne 1 à 5, la création du menu principal.

Lignes 7 à 17, nous avons la création de notre menu2, qui va devenir notre sous-menu.

Enfin, ligne 19, nous intégrons notre menu 2 au menu 1 et en faisons donc le sous-menu.

VI-A-3-p - Barre d'outils

Les **barres d'outils** sont les barres contenant généralement des icônes et vous permettant d'avoir un accès rapide à certaines fonctions.

```
ma_toolbar = gtk.Toolbar()
ma_toolbar.set_style(gtk.TOOLBAR_ICONS)
b1 = gtk.ToolButton(gtk.STOCK_NEW, 'Nouveau')
b2 = gtk.ToolButton(gtk.STOCK_QUIT, 'Quitter')
separator = gtk.SeparatorToolItem()
ma_toolbar.insert(b1, 0)
ma_toolbar.insert(separator, 1)
ma_toolbar.insert(b2, 3)
b2.connect("clicked", gtk.main_quit)
```

Dans cet exemple, ligne 1, nous créons notre barre d'outils.

Ligne 2, nous lui précisons que nous ne voulons que des icônes dans cette barre d'outils. Nous aurions pu également utiliser « **gtk.TOOLBAR_TEXT** » pour indiquer que nous ne voulions que du texte, ou encore « **gtk.TOOLBAR_BOTH** » pour indiquer que nous voulions du texte et des icônes.

Ligne 3 et 4, nous créons nos boutons dédiés à la toolbar, en précisant le type d'icônes utilisées. Dans notre exemple, il s'agit d'icônes **PYGTK** prédéfinies. Nous précisons également le texte correspondant aux boutons.

Ligne 5, nous créons un séparateur de boutons pour la lisibilité de la toolbar.

Lignes 6 à 8, nous insérons nos objets dans la toolbar, en assignant à chacun une position. Le compteur commence à gauche, à partir de 0.

Ligne 9 nous connectons un de nos boutons à une fonction. Un **ToolButton** se configure comme un bouton classique.

VI-A-3-q - Barre de statut

La barre de statut est une petite barre mince, située en bas de la fenêtre d'un programme et servant à donner des informations à l'utilisateur : aide à l'utilisation, avancée d'un traitement, espace mémoire…

La création et la manipulation d'une barre de statut sont très simples :

```
1. ma_statusbar = gtk.Statusbar()
2. ma_statusbar.push(ID_TEXT, mon_texte)
3. ma_statusbar.pop(ID_TEXT)
```

Ligne 1, nous créons notre barre de statut.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Ligne 2, nous l'alimentons. Deux paramètres sont passés ici : un ID, et un texte. Nous pouvons passer un nombre important de textes à une barre de statut. L'ID permet de les différencier les uns des autres.

Ligne 3, nous supprimons un texte donné grâce à son ID.

VI-A-3-r - Les curseurs

Les **curseurs**, qu'ils soient verticaux ou horizontaux, permettent à l'utilisateur de configurer facilement des paramètres numériques, tout comme les compteurs.

Leur création est extrêmement aisée :

```
cur_v = gtk.VScale()
cur_h = gtk.HScale()
```

Les paramétrages pour les deux sont identiques. Pour la suite nous utiliserons donc le curseur horizontal en exemple.

```
1. cur_h.set_range(0, 100)
2. cur_h.set_increments(1, 5)
3. cur_h.set_digits(0)
4. cur_h.set_size_request(150, 25)
5. cur_h.connect("value-changed", cur_h_changed)
```

Ligne 1, nous paramétrons le min et le max de notre curseur.

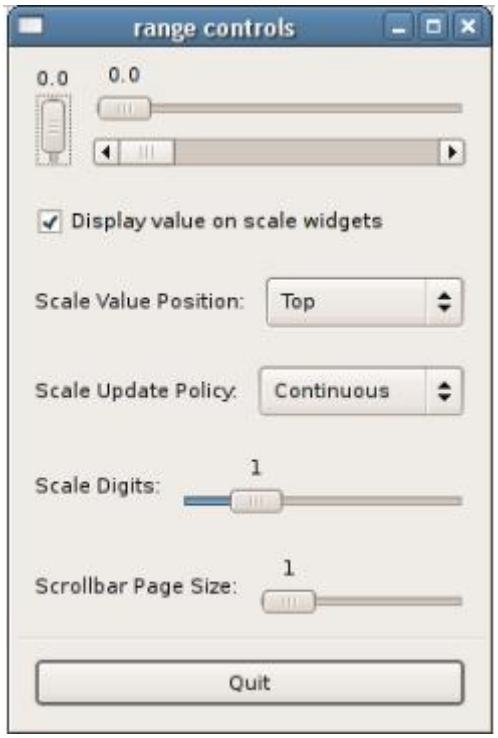
Ligne 2, nous paramétrons le pas par mouvement du curseur sur sa barre, via la souris (1er paramètre) et lorsque l'utilisateur utilise les flèches clavier ou clique sur la barre de défilement, sur le côté du curseur (2^{d} paramètre).

Ligne 3, indique le nombre de décimales désirées dans la valeur du curseur.

Ligne 4, nous paramétrons la taille du widget curseur en pixels (largeur puis hauteur).

Ligne 5, enfin, nous connectons le changement de la valeur du curseur à une fonction/procédure.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/



Assemblage de Combobox et de Scale (http://developer.gnome.org)

VI-A-3-s - Bulles d'aide

Pour chacun des widgets de **PYGTK**, on a la possibilité de créer des **infobulles**. Ces infobulles sont elles-mêmes considérées comme des widgets.

```
mon_infob = gtk.Tooltips()
mon_infob.set_tip(mon_widget, " texte ", None)
```

VI-A-3-t - Les stocks (icônes)

Les **stocks** sont des icônes prédéfinies dans **PYGTK**. Selon la version de **PYGTK** que vous utilisez, vous en disposerez de plus ou moins. Vous trouverez ici le nom et l'icône de ces stocks

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Constante	icône
gtk.STOCK_ABOUT	
gtk.STOCK_ADD	
gtk.STOCK_APPLY	
gtk.STOCK_BOLD	
gtk.STOCK_CANCEL	
gtk.STOCK_CDROM	
gtk.STOCK_CLEAR	
gtk.STOCK_CLOSE	
gtk.STOCK_COLOR_PICKER	
gtk.STOCK_CONVERT	
gtk.STOCK_CONNECT	
gtk.STOCK_COPY	
gtk.STOCK_CUT	
gtk.STOCK_DELETE	
gtk.STOCK_DIALOG_AUTHENTICATION	
gtk.STOCK_DIALOG_ERROR	

gtk.STOCK_DIALOG_INFO	
gtk.STOCK_DIALOG_QUESTION	
gtk.STOCK_DIALOG_WARNING	
gtk.STOCK_DIRECTORY	
gtk.STOCK_DISCONNECT	
gtk.STOCK_DND	
gtk.STOCK_DND_MULTIPLE	
gtk.STOCK_EDIT	
gtk.STOCK_EXECUTE	
gtk.STOCK_FILE	
gtk.STOCK_FIND	
gtk.STOCK_FIND_AND_REPLACE	
gtk.STOCK_FLOPPY	
gtk.STOCK_FULLSCREEN	
gtk.STOCK_GOTO_BOTTOM	

gtk.STOCK_GOTO_FIRST	
gtk.STOCK_GOTO_LAST	
gtk.STOCK_GOTO_TOP	
gtk.STOCK_GO_BACK	
gtk.STOCK_GO_DOWN	
gtk.STOCK_GO_FORWARD	
gtk.STOCK_GO_UP	
gtk.STOCK_HARDDISK	

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

gtk.STOCK_SORT_ASCENDING	
gtk.STOCK_SORT_DESCENDING	
gtk.STOCK_SPELL_CHECK	
gtk.STOCK_STOP	
gtk.STOCK_STRIKETHROUGH	
gtk.STOCK_UNDELETE	
gtk.STOCK_UNDERLINE	
gtk.STOCK_UNDO	
gtk.STOCK_UNINDENT	
gtk.STOCK_YES	
gtk.STOCK_ZOOM_100	
gtk.STOCK_ZOOM_FIT	
gtk.STOCK_ZOOM_IN	
gtk.STOCK_ZOOM_OUT	

VI-A-3-u - Exemple

```
#!/usr/bin/PYTHON
# -*-coding:utf-8 -*



import pygtk
import gtk



def appui_but(widget):
    val = lab_nb.get_text()
    if val.isdigit():
        calcul = int(val) * int(val)
        lab_square.set_text(str(calcul))
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
    else:
        err_win = gtk.MessageDialog(None,gtk.DIALOG_MODAL,\
                gtk.MESSAGE_ERROR, gtk.BUTTONS_OK, \
                "Valeur saisie non numerique")
        err_win.run()
        err_win.destroy()



mw = gtk.Window()
mw.set_title("Calcul d'un carre (3 chiffres)")
mw.set_size_request(300,100)
mw.connect("destroy",gtk.main_quit)

nb = gtk.Frame("Nombre")

square = gtk.Frame("Carre")

but = gtk.Button("Calcul du carre")
but.connect("clicked", appui_but)

lab_nb = gtk.Entry(3)
lab_nb.set_alignment(0.5)
lab_square = gtk.Label("0")
nb.add(lab_nb)

square.add(lab_square)

table = gtk.Table(rows=2, columns=2, homogeneous=True)
table.attach(nb, 0,1,0,1)
table.attach(square,1,2,0,1)
table.attach(but,0,2,1,2, xoptions=gtk.SHRINK, yoptions=gtk.SHRINK)

mw.add(table)

table.show()
but.show()
square.show()
nb.show()
lab_nb.show()
lab_square.show()
mw.show()

gtk.main()
```

VII - La 3D



Wikimedia Commons,Nuvola_apps_edu_miscellaneous_3dCube.svg

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

VII-A - Le format STL

Le format **STL** est un format très répandu dans l'univers du prototypage. Il s'agit d'un simple fichier ASCII ou binaire permettant de définir la forme d'un objet en 3D en le décrivant sous la forme d'un assemblage de triangles.

Chacun de ces triangles est formé à partir de trois points 3D (appelé vertex), et d'une normale, généralement perpendiculaire à la surface du triangle, permettant d'indiquer au logiciel comment la lumière doit interagir avec cette surface.

VII-A-1 - STL ASCII

Un fichier STL commence par le mot-clé solid, suivi du nom de l'objet ; et il se finit avec le mot-clé endsolid suivi du nom de l'objet.

Entre les deux, nous retrouvons la définition de chaque triangle à suivre, de ce type :

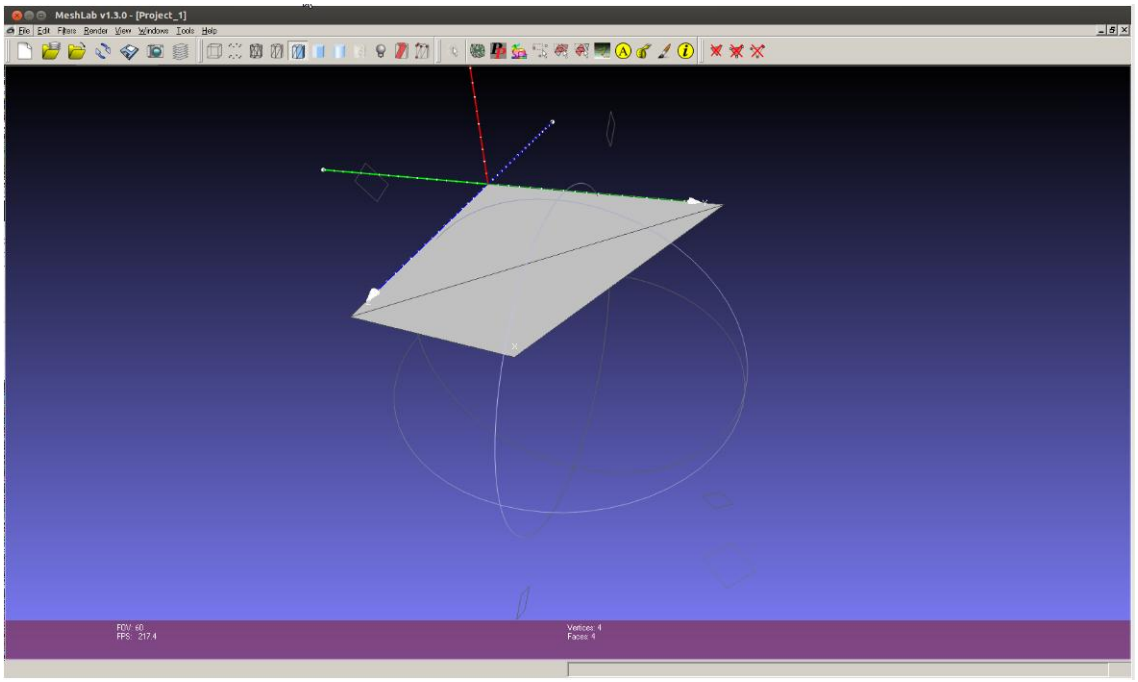
```
facet normal ni nj nk
  outer loop
    vertex v1x v1y v1z
    vertex v2x v2y v2z
    vertex v3x v3y v3z
  endloop
endfacet
```

Remarque : les espaces sont importants dans la définition d'un fichier STL ASCII. Il faut respecter le décalage tel que dans l'exemple ci-dessous et ne surtout pas utiliser de tabulation pour remplacer les espaces.

Imaginons maintenant un objet très simple type pyramide, donc quatre triangles uniquement. Voici à quoi pourrait ressembler son fichier ASCII STL :

```
solid Pyramid
  facet normal 0.000000e+00 0.000000e+00 -1.000000e+00
    outer loop
      vertex 0.000000e+00 0.000000e+00 0.000000e+00
      vertex 1.000000e+00 0.000000e+00 0.000000e+00
      vertex 0.000000e+00 1.000000e+00 0.000000e+00
    endloop
  endfacet
  facet normal -1.000000e+00 0.000000e+00 0.000000e+00
    outer loop
      vertex 0.000000e+00 0.000000e+00 0.000000e+00
      vertex 0.000000e+00 1.000000e+00 0.000000e+00
      vertex 0.000000e+00 0.000000e+00 1.000000e+00
    endloop
  endfacet
  facet normal 0.000000e+00 -1.000000e+00 0.000000e+00
    outer loop
      vertex 0.000000e+00 0.000000e+00 0.000000e+00
      vertex 1.000000e+00 0.000000e+00 0.000000e+00
      vertex 0.000000e+00 0.000000e+00 1.000000e+00
    endloop
  endfacet
  facet normal 1.000000e+00 1.000000e+00 1.000000e+00
    outer loop
      vertex 1.000000e+00 0.000000e+00 0.000000e+00
      vertex 0.000000e+00 1.000000e+00 0.000000e+00
      vertex 0.000000e+00 0.000000e+00 1.000000e+00
    endloop
  endfacet
endsolid Pyramid
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/



Une pyramide STL dans MESHLAB

Outre sa simplicité, ce type de format est extrêmement pratique pour réaliser de la 3D avec Open GI, car ce dernier fonctionne lui aussi sur le principe de Vertex.

VII-A-2 - STL Binaire

Le format STL Binaire présente l'avantage de gérer en plus de la forme une couleur.

Ce format possède également un entête permettant de renseigner des informations sur le fichier.

Si dans les faits, le STL binaire est probablement le plus répandu, il n'est cependant pas le plus simple à ouvrir.

Voici comment est formé un fichier STL Binaire :

>80 octets d'entête ;

>un mot de quatre octets pour indiquer le nombre de triangles

>puis pour chaque triangle

>>trois mots de quatre octets pour la normale,

>>trois mots de quatre octets pour le vertex 1,

>>trois mots de quatre octets pour le vertex 2,

>>trois mots de quatre octets pour le vertex 3,

>>deux octets pour la couleur (si utilisé).

Il n'y a aucun retour à la ligne dans le fichier. Les octets se suivent les uns à la suite des autres.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

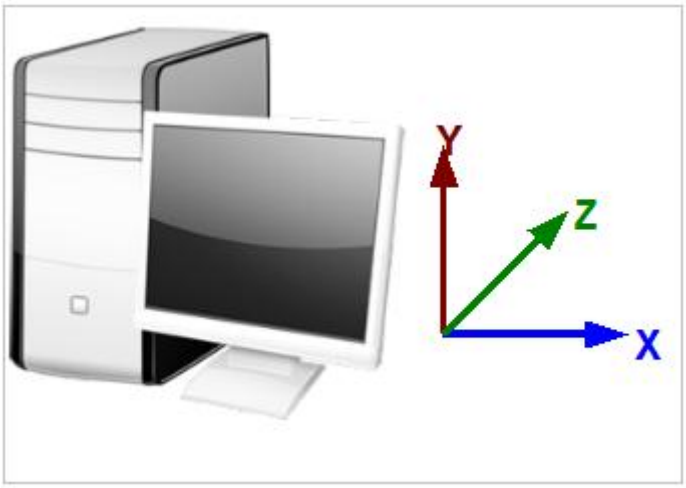
VII-B - OpenGL et PYTHON: les bases

Nous allons voir ici l'ensemble des bases nécessaires pour pratiquer l'**OpenGL**. Nous détaillerons autant que possible les principaux éléments indispensables.

VII-B-1 - Le repère XYZ

En image de synthèse, le **repère** XYZ n'est pas celui que nous avons l'habitude d'utiliser en physique. En effet, si nous positionnons X à l'horizontal de gauche à droite, et Y en vertical de bas en haut, z devrait venir vers nous.

En imagerie, la convention veut qu'en réalité z s'éloigne de nous. Il faut donc bien en prendre pleinement conscience afin de ne limiter les potentielles erreurs.



Le repère OpenGL

En **OpenGL** (et en infographie en général), l'axe Z est également connecté à ce qu'on appelle communément le Z-Buffer.

Ce **Z-Buffer** permet au moteur 3D d'appréhender la notion de profondeur et évite donc des superpositions hasardeuses.

L'exemple typique est un cube. Sans **Z-Buffer**, les faces sont affichées dans l'ordre où vous les créez. Autrement dit, si vous commencez par la face de devant et finissez par la face de derrière, la face de derrière apparaîtra en réalité devant la face avant.

Pour éviter ce quiproquo et ces interprétations hasardeuses, le **Z-Buffer** indique au moteur comment interpréter les instructions dans la globalité.

VII-B-2 - Notions de base

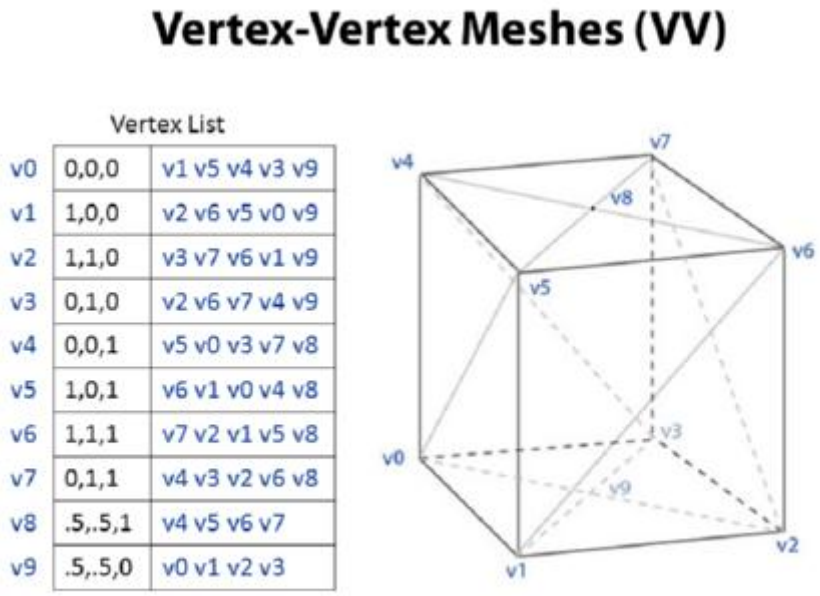
Dans cette partie, nous allons nous concentrer sur les éléments indispensables à l'utilisation courante **d'OpenGL**.

VII-B-2-a - Les vertex

Dans le monde de l'infographie 3D, un **vertex** (ou vertice), correspond simplement à un point dans l'espace.

Ce point possède des coordonnées qui lui sont propres et sert en général à créer un polygone 2D ou 3D dans l'espace.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/



Exemple de vertex formant un cube (wiki. Commons, vertex-vertex meshes (VV).png)

VII-B-2-b - Les polygones

Comme vu à l'instant, un **polygone** est défini par plusieurs vertex (un minimum de trois).

Cet assemblage de vertex permet de définir une forme dans l'espace. Chaque polygone peut lui-même être décomposé en une infinité de polygones plus petits. Cela peut parfois être utile, surtout lorsque l'on travaille sur des fichiers STL.

VII-B-2-c - Les formes de base

L'ajout d'une forme de base est relativement simple en **OpenGL**.

Une forme est décrite à l'intérieur d'un bloc **glBegin**()/**glEnd**(). Il faut un bloc par type de forme à afficher.

Au sein de chaque bloc, le type de forme est passé en paramètre au **glBegin()**. Les plus utiles sont les suivants :

Constante	Description
GL_POINTS	Chaque vertex est un point unique
GL_LINES	Un ensemble de deux vertex forme une ligne. Fonctionne par groupe de deux vertex
GL_LINE_STRIP	Une ligne est formée par deux vertex. Si trois vertex sont déclarés, cela forme deux lignes avec un point commun. Fonctionne par groupe de deux vertex minimum.
GL_TRIANGLES	Trois vertex forment un triangle. Fonctionne par groupe de trois vertex
GL_QUADS	Quatre vertex forment un carré. Fonctionne par groupe de quatre4 vertex
GL_POLYGON	Tous les vertex déclarés forment un polygone

Remarque : pour le format STL, on fonctionnera avec le mode GL_TRIANGLES.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Une fois le polygone déclaré, on le ferme avec la méthode **glEnd()**.

```
glBegin(GL_TRIANGLES)
&#8230;
glEnd()
```

Attention : un bloc glBegin/glEnd doit être déclaré pour chaque objet 3D que l'on désire afficher.

VII-B-2-c-i - Les points

Élément de base, un point se déclare avec la fonction **glVertex2d**.

```
glBegin(GL_POINTS)
glVertex3d(x, y, z)
glEnd()
```

Remarque : les coordonnées d'un vertex sont des floats.

Nous pouvons définir la taille d'un point de la manière qui suit :

```
glPointSize( 3.0 )        #Definition de la taille du point
```

VII-B-2-c-ii - Les lignes

Si nous assemblons l'ensemble des éléments déjà vus, nous devinons aisément qu'une ligne se dessine grâce à un bloc du type :

```
glBegin(GL_LINES)
glVertex3d(x1, y1, z1)
glVertex3d(x2, y2, z2)
glEnd()
```

Nous pouvons définir la largeur d'une ligne de la manière qui suit :

```
glLineWidth( 3.0 )    #Definition de la largeur du point
```

VII-B-2-c-iii - Les triangles

Les triangles sont les éléments de base pour un fichier STL. C'est pourquoi nous allons les voir ici.

Il n'y a rien de plus compliqué qu'avec les lignes. Un triangle se dessine avec le code suivant :

```
glBegin(GL_LINES)
glVertex3d(x1, y1, z1)
glVertex3d(x2, y2, z2)
glVertex3d(x3, y3, z3)
glEnd()
```

On peut jouer sur l'épaisseur des lignes constituant le périmètre du triangle, et sur la taille des vertex le définissant.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

VII-B-2-d - État de surface d'un polygone

VII-B-2-d-i - Les couleurs

Les couleurs, en **OpenGL**, fonctionnent en RGB. Une couleur peut être attribuée à chaque vertex, et **OpenGl** se charge alors de réaliser les dégradés de couleurs pour une surface formée par plusieurs vertex.

La couleur se configure avec **glColor3ub**

```
glColor3ub(R, G, B)
```

Comme on peut le constater dans notre exemple, on commence par définir la couleur, puis les vertex associés à cette couleur.

VII-B-2-d-ii - Les textures

Il est pratique de pouvoir coloriser nos formes 3D, mais il peut être encore plus pratique de pouvoir utiliser des textures.

Nous allons voir ici comment utiliser des textures basiques. Entendez par là qu'il ne s'agit que d'une simple image en PNG.

Elles seront configurées pour s'adapter à la forme.

La première chose à savoir gérer est l'activation/désactivation de la texture. Cela s'effectue avec **glEnable**(**GL_TEXTURE_2D**) et **glDisable**(**GL_TEXTURE_2D**).

Il faut ensuite importer notre texture depuis notre image avec **gluint** ma_texture = loadTexture("ma_texture.png")

Enfin, il ne reste plus qu'à l'appliquer où on le désire.

Attention : une seule texture peut être appliquée par bloc. Il faut donc changer de bloc à chaque forme si la texture est différente.

L'application de la texture sur notre objet est un peu plus complexe. Il faut tout d'abord préciser à **OpenGL** que nous désirons utiliser une texture sur notre bloc avec :

```
glBindTexture(GL_TEXTURE_2D, ma_texture)
```

Il faut ensuite simplement faire correspondre des coordonnées de la texture à des coordonnées du polygone grâce à **glTexCoord2d()**

Cela implique que vous connaissiez précisément les vertex de la texture et du polygone.

```
glBindTexture(GL_TEXTURE_2D, ma_texture)
glBegin(GL_TRIANGLES)
glTexCoord2d(texture_x1, texture_y1)
glVertex3d(x1, y1, z1)
glTexCoord2d(texture_x2, texture_y2)
glVertex3d(x2, y2, z2)
glTexCoord2d(texture_x3, texture_y3)
glVertex3d(x3, y3, z3)
glEnd()
```

Remarque : concernant la texture, vous êtes libre de prendre les coordonnées que vous désirez dans l'image. Quoi qu'il arrive, OpenGL déformera l'image pour s'adapter à votre demande.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

VII-B-2-e - La caméra

Au sein d'une application de visualisation 3D, la gestion de la caméra est un élément très important.

C'est par son intermédiaire que l'utilisateur visualise la scène 3D. Aussi de mauvais paramétrages, selon l'usage final de l'application, pourraient avoir de sérieuses répercussions.

VII-B-2-e-i - Les différents types de caméra

Il existe différents types de caméra pour visualiser une scène. Les principaux sont : **freefly** et **trackball**

La différence intervient lorsque l'on se déplace au sein d'une scène 3D.

L'exemple le plus typique est celui d'un simple cube. Lorsque nous nous déplaçons autour, celui-ci peut correspondre à deux mouvements différents :

=>l'objet tourne sur lui-même autour d'un axe et la caméra ne bouge pas (caméra trackball) ;

=>l'objet reste fixe et c'est la caméra qui tourne autour du cube (caméra freefly).

Selon le type de caméra utilisée, effectuer une rotation de l'objet selon un axe défini n'aura pas du tout le même rendu visuel à l'écran. Si on ne prend pas garde, non seulement l'objet en 3D va tourner, mais également les axes.

De fait, demander une rotation de l'objet selon un axe qui n'est plus là où on l'attend provoque un résultat surprenant.

Par défaut, c'est généralement le mode trackball qui est utilisé. Le Mode freefly se configure par l'intermédiaire du repositionnement de la caméra (voir point suivant).

L'utilisation du mode Freefly est recommandée pour avoir un mode de visualisation de type **ARCBALL**.

Il s'agit d'un type d'interaction programme/utilisateur, dans lequel, le programme fait généralement apparaître (mais pas nécessairement) trois cercles à l'utilisateur autour de l'objet. Le but est d'obtenir une utilisation intuitive.

Ainsi lorsque l'utilisateur déplacera son curseur vers la droite de l'écran, l'objet tourne vers la gauche (ou vers la droite selon la configuration souhaitée).

Dans les faits, réaliser une telle interface par des outils de rotation en OpenGL est complexe. C'est pourquoi il est plutôt recommandé de positionner la caméra autour de l'objet.

VII-B-2-e-ii - Positionnement

Le **positionnement de la caméra** est important pour la visualisation d'une scène 3D. Cela n'est plus à démontrer.

```
gluLookAt(x1, y1, z1, x2, y2, z2, vx, vy, vz)
```

Le premier jeu de coordonnées correspond au point où sera fixée la caméra.

Le second jeu de coordonnées correspond au point visé par la caméra.

Attention : n'oubliez pas que sur les axes des objets, l'axe Z est inversé. Z2 est donc en réalité -z2.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

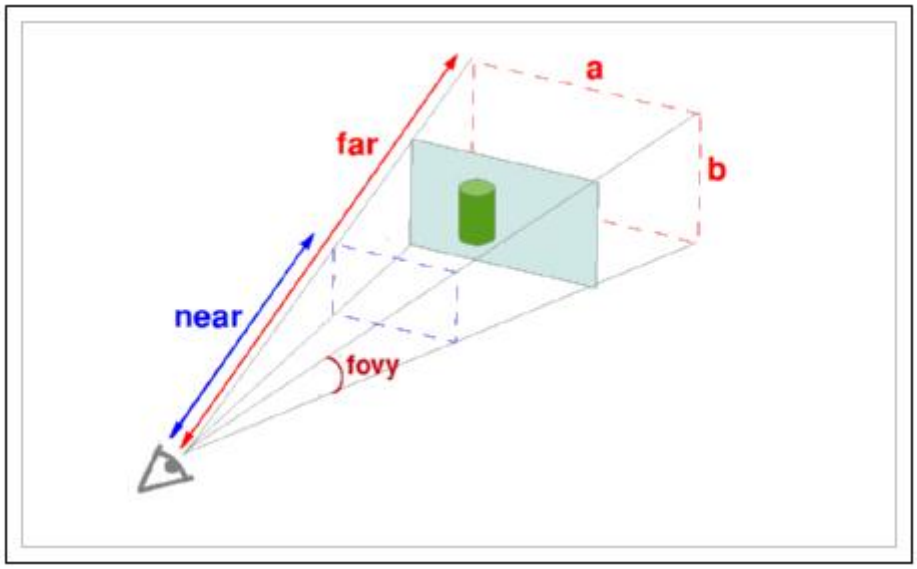
Le vecteur fourni, enfin, indique l'inclinaison de la caméra. Par exemple, dans un plan XYZ, où Z, positif, représente la verticale, on aura Vxyz(0,0,1). Si on veut regarder la scène à l'envers, on aura Vxyz(0,0,-1).

Ce **gluLookAt** s'appelle dans le DrawGLScene, juste après la réinitialisation de l'affichage, et juste avant de redessiner les objets dans la scène.

VII-B-2-e-iii - Zoom, translation, et rotation

Le zoom peut s'effectuer de deux façons possibles : via le **gluPerpective** ou via le **glTranslatef**.

```
gluPerspective(fovy,ratio,near,far)
```



Fonctionnement du gluPerspective

fovy est l'angle d'ouverture de la caméra, autrement dit l'angle de vision. En général, on saisit 45 (pour 45 °). Plus cet angle sera petit, plus l'impression de zoom sera importante.

ratio correspond au rapport a/b. En général ce ratio vaut 4/3 ou 16/9 selon les écrans.

near et far définissent les limites de vision. En deçà et au-delà de ces distances, aucun objet n'apparaîtra à l'écran. Essayer les valeurs 0.1 et 100 pour commencer.

Le **glTranslatef** est à double usage. En effet, si on fixe la caméra dans l'alignement d'un axe, on peut s'en servir comme outil de zoom.

Cela n'est cependant pas son usage initial. Le **glTranslatef** permet d'effectuer des translations selon un vecteur passé en paramètre :

```
glTranslatef(X,Y,Z)
```

Concernant le zoom, si on est aligné sur un axe, une translation positive ou négative aura un effet de ZoomIn/ZoomOut.

Remarque : avec le LookAt, nous pouvons également effectuer un zoom. Il suffit dans ce cas précis de déplacer la caméra plus ou moins loin de notre cible.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Enfin, la rotation. On utilise **glRotated**. Cette méthode prend en paramètre un angle en degrés (sens trigonométrique) et un vecteur définissant l'axe de rotation.

```
glRotated(alpha, X,Y,Z)
```

Attention : lorsque vous combinez des opérations de rotation/translation, le résultat peut changer selon l'ordre d'exécution.

VII-B-3 - Fenêtre

VII-B-3-a - Les indispensables

VII-B-3-a-i - Le constructeur de la classe

Il ne s'agit pas ici d'un indispensable réel, mais plus d'une recommandation.

Pour nous, cela constituera un moyen simple de tester le code que nous réaliserons.

Il prendra la forme suivante :

```
if __name__ == '__main__':
    try:
        GLU_VERSION_1_2
    except:
        print "Need GLU 1.2 to run this demo"
        sys.exit(1)
    main()
    glutMainLoop()
```

Ce code ne doit normalement plus vous poser de questions importantes. Le constructeur est exécuté dès que nous lançons le module en autonome.

Un test permet de s'assurer de la bonne version minimale de **GLUT**. Cas échéant, un message d'erreur est affiché et nous sortons en erreur.

Sinon nous appelons notre main.

VII-B-3-a-ii - Le main

Le main contient l'ensemble des éléments qui nous permettront de préciser à **GLUT** comment il doit démarrer et quelles sont les procédures/fonctions qui gèrent les différents évènements.

Un petit exemple :

```
def main():
    global window
    glutInit(sys.argv)
    glutInitDisplayMode(GLUT_RGBA|GLUT_DOUBLE|GLUT_ALPHA|GLUT_DEPTH)
    glutInitWindowSize(640, 480)
    glutInitWindowPosition(0, 0)
    window = glutCreateWindow("Mon test a moi")
    glutDisplayFunc(DrawGLScene)
    glutReshapeFunc(ReSizeGLScene)
    glutKeyboardFunc(keyPressed)
    glutSpecialFunc(specialkey)
    glutSpecialUpFunc(specialkey2)
    glutMotionFunc(movemouse)
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
    glutMouseFunc(buttmouse)
    InitGL(640, 480)
```

Cet exemple ne sera pas commenté cette fois, car nous allons voir chaque élément dans les points qui vont suivre.

VII-B-3-a-iii - L'InitGL

L'**InitGL** est la fonction qui s'occupe, comme son nom l'indique, d'initialiser OpenGL, et plus précisément l'instance **d'OpenGL** à laquelle nous faisons appel.

Elle est appelée à la fin du main et prend en paramètres la largeur et la hauteur de la fenêtre souhaitée.

Elle va permettre de paramétrer un certain nombre d'éléments que nous allons voir ici. Ci-dessous une portion de code que nous allons détailler pour rendre plus concrète cette fonction particulière :

```
1. def InitGL(Width, Height):
2.     glClearColor(0.0, 0.0, 0.0, 0.0)
3.     glClearDepth(1.0)
4.     glDepthFunc(GL_LESS)
5.
6.     glEnable(GL_DEPTH_TEST)
7.     glShadeModel(GL_SMOOTH)
8.
9.     glMatrixMode(GL_PROJECTION)
10.     glLoadIdentity()
11.     gluPerspective(45.0, float(Width)/float(Height), 0.1, 100.0)
12.     glMatrixMode(GL_MODELVIEW)
```

Ligne 2, nous retrouvons une fonction qui nous permet de modifier la couleur du fond de la scène, ici à noir. Les paramètres sont respectivement R, G, B, alpha, compris entre 0 et 1

Ligne 3, nous réinitialisons le buffer de profondeur en lui donnant une valeur par défaut. Le buffer de profondeur est indispensable pour gérer les alignements de pièces dans l'espace.

Ligne 4, nous sommes toujours en train de configurer le buffer de profondeur. Nous lui indiquons que tout ce qui est inférieur à la valeur passée ligne 3, doit être pris en compte dans la gestion de la profondeur.

Ligne 6, cela concerne toujours le buffer de profondeur. Nous activons un lot de fonctions de calcul GL.

Ligne 7, nous précisons à **OpenGL** que nous désirons que les faces d'un polygone soient pleines. Si on ne veut pas qu'elles soient remplies, il faut faire appel à **glShadeModel**(**GL_FLAT**).

Ligne 9, nous passons en mode projection. Cela correspond au **gluPerspective** vu précédemment. On va ainsi paramétrer la caméra. Nous réinitialisons le paramétrage ligne 10, et ligne 11 nous configurons la caméra.

La réinitialisation effectuée ligne 10 est extrêmement importante, car sans cela, les transformations et paramétrages s'accumulent et s'effectuent les uns après les autres. Il faut donc effectuer un **glLoadIdentity**() avant chaque nouveau jeu de transformation et/ou de paramétrage.

Ligne 12, enfin, nous repassons sur la gestion des objets mêmes.

VII-B-3-a-iv - Le ReSizeGLScene

C'est la fonction qui est appelée lorsque l'on change la taille de la fenêtre afin de redessiner l'ensemble des éléments à la bonne échelle.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Dans l'exemple suivant, nous détaillerons succinctement les différentes procédures qui seront examinées plus tard :

```
1. def ReSizeGLScene(Width, Height):
2.     if Height == 0:    #Eviter une division par 0.
3.         Height = 1
4.
5.     glViewport(0, 0, Width, Height)
6.     glMatrixMode(GL_PROJECTION)
7.     glLoadIdentity()
8.     gluPerspective(45.0, float(Width)/float(Height), 0.1, 100.0)
9.     glMatrixMode(GL_MODELVIEW)
```

Quelques explications. Tout d'abord on peut constater que les paramètres passés sont les nouvelles dimensions de la fenêtre.

En ligne 2 et 3, nous nous assurons d'avoir une hauteur minimale afin d'éviter toute erreur de type division par 0 ligne 8

En ligne 5, nous utilisons **glViewport()**. Cette procédure sert à définir la position de la fenêtre (coin supérieur gauche), ainsi que sa nouvelle taille.

En ligne 6, nous utilisons **glMatrixMode()**. Cette procédure permet de fixer la matrice devant accueillir la redéfinition du contenu graphique.

En ligne 7, **gLoadIdentitty()**. Cela permet de recharger la matrice dans la fenêtre pour redessiner les objets.

Ligne 8, **gluPerspective()**. Nous redéfinissons ici la position de la caméra. Un angle de vision de 45 °, le ratio de la fenêtre, et pour finir les distances entre le devant et l'arrière de la scène.

Enfin, ligne 9, nous recommutons le **glMatrixMode()**.

VII-B-3-a-v - Le DrawnGLScene

Le **DrawGLSCene** est la fonction qui s'occupe de dessiner l'ensemble de la partie 3D.

```
1. def DrawGLScene():
2.     global rtri, rquad, yrtri, yrquad, zoom,dquad,dyquad, transf, \
3.             astx, lasty, alpha, xc, yc, zc
4.
5.     glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
6.     glLoadIdentity()
7.
8.     gluLookAt(xc,yc,zc,xa,ya,za,vx,vy,vz)
9.
10.     glBegin(GL_QUADS)
11.     glColor3f(0.0,1.0,0.0)
12.     glVertex3f( 1.0, 1.0,-1.0)
13.     glVertex3f(-1.0, 1.0,-1.0)
14.     glVertex3f(-1.0, 1.0, 1.0)
15.     glVertex3f( 1.0, 1.0, 1.0)
16.
17.     glColor3f(1.0,0.5,0.0)
18.     glVertex3f( 1.0,-1.0, 1.0)
19.     glVertex3f(-1.0,-1.0, 1.0)
20.     glVertex3f(-1.0,-1.0,-1.0)
21.     glVertex3f( 1.0,-1.0,-1.0)
22.
23.     glColor3f(1.0,0.0,0.0)
24.     glVertex3f( 1.0, 1.0, 1.0)
25.     glVertex3f(-1.0, 1.0, 1.0)
26.     glVertex3f(-1.0,-1.0, 1.0)
27.     glVertex3f( 1.0,-1.0, 1.0)
28.
29.     glColor3f(1.0,1.0,0.0)
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
30.     glVertex3f( 1.0,-1.0,-1.0)
31.     glVertex3f(-1.0,-1.0,-1.0)
32.     glVertex3f(-1.0, 1.0,-1.0)
33.     glVertex3f( 1.0, 1.0,-1.0)
34.
35.     glColor3f(0.0,0.0,1.0)
36.     glVertex3f(-1.0, 1.0, 1.0)
37.     glVertex3f(-1.0, 1.0,-1.0)
38.     glVertex3f(-1.0,-1.0,-1.0)
39.     glVertex3f(-1.0,-1.0, 1.0)
40.
41.     glColor3f(1.0,0.0,1.0)
42.     glVertex3f( 1.0, 1.0,-1.0)
43.     glVertex3f( 1.0, 1.0, 1.0)
44.     glVertex3f( 1.0,-1.0, 1.0)
45.     glVertex3f( 1.0,-1.0,-1.0)
46.     glEnd()
47.
48.     time.sleep(0.002)
49.     glutSwapBuffers()
```

Voici un exemple de DrawGLScene. Cette fonction est définie dans le main avec **glutDisplayFunc(DrawGLScene)**.

Ligne 5, nous appelons **glClear**. Cette fonction permet de réinitialiser les buffers passés en paramètres. Nous réinitialisons ici les buffers gérant la couleur et la profondeur.

Ligne 6, nous réinitialisons la scène.

Ligne 8, nous paramétrons la caméra. Nous fonctionnons ici en mode freefly. Aussi chaque paramètre du **glLookAt** est une variable.

Ligne 10 à 46, nous dessinons un cube.

La ligne 48 requiert une attention toute particulière. Cette fonction, déjà vue quelques chapitres plus tôt permet de forcer le programme à faire des pauses. Sans cela, si vous bougez votre cube, votre système tournera à 100 % uniquement pour votre **OpenGL**.

En modulant cette valeur, vous pouvez alléger la charge de votre processeur, dans une certaine limite. En effet, si vous montez trop en temps de pause, cela jouera sur la fluidité de votre programme.

Enfin, en ligne 49, nous mettons à jour l'affichage.

Remarque : l'affichage en OpenGL est basé sur une série de buffers étant en réalité des matrices. Dans cet exemple, tant que nous n'avons pas appelé la ligne 49, l'affichage ne montre pas le cube que nous avons dessiné : nous n'avons rien à l'écran.

VII-B-3-b - Insertion dans une frame PYGTK

Il s'agit là d'un point très souvent difficile à appréhender, mais qui peut se révéler extrêmement pratique.

Sans insertion de la fenêtre **OpenGL** dans notre interface **GTK**, un programme peut vite se révéler pénible à utiliser.

Pour notre démonstration, nous utiliserons le module **pygtkglext**.

VII-B-3-b-i - Principe

Le principe ici est très simple avec le module sélectionné. Nous créons notre IHM **GTK** dans un coin, et notre code **OpenGL** dans un autre.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Pygtkglext va alors venir effectuer l'interface entre les deux en permettant à la fenêtre OpenGL d'être considérée comme un widget graphique à part entière par **PYGTK**

VII-B-3-b-ii - Mise en œuvre

Dans les faits, cela se complique légèrement par rapport à la théorie, mais rien de sorcier non plus, je vous rassure.

Nous allons ensemble passer en revue le code ci-dessous :

```
#! /usr/bin/env PYTHON
# -*- coding: utf-8 -*-

from OpenGL.GL import     *
from OpenGL.GLU import *
import gtk
import gtk.gdkgl as gdkgl
import gtk.gtkgl as gtkgl

def DrawGL (area, event):
    zone = area.get_gl_drawable ()
    gl_context = area.get_gl_context ()

    if not zone.gl_begin (gl_context):
        return False

    allocation = area.get_allocation ()

    glViewport (0, 0, allocation.width, allocation.height)
    glClear (GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)

    glMatrixMode (GL_MODELVIEW)
    glLoadIdentity ()

    glMatrixMode (GL_PROJECTION)
    glLoadIdentity ()
    gluPerspective (45., float(allocation.width)/float(allocation.height), 2., 0.)
    glTranslate (0, 0, -2)

    glBegin (GL_TRIANGLES)
    glColor3ub(0, 0, 255)
    glVertex ( 0.5, -0.5, 0.0)
    glColor3ub(0, 255, 0)
    glVertex (0.0,  0.5, 0.0)
    glColor3ub(255, 0, 0)
    glVertex (-0.5, -0.5, 0.0)
    glEnd ()

    zone.swap_buffers ()

    return True

if __name__ == '__main__':
    window = gtk.Window()
    window.set_title("Exemple: OpenGL dans PYGTK")
    window.connect('destroy', gtk.main_quit)

    main_box = gtk.HBox(homogeneous=False, spacing=0)
    window.add(main_box)

    config = gdkgl.Config(mode=(gdkgl.MODE_RGB|gdkgl.MODE_DOUBLE|\
                                gdkgl.MODE_DEPTH))
    area = gtkgl.DrawingArea(config)
    area.set_size_request(width=320,height=240)
    area.connect('expose-event', DrawGL)
    main_box.pack_start(area)

    area.show()
    main_box.show()
    window.show()
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
61.     gtk.main()
```

Les 8 premières lignes ne devraient pas vous poser de problèmes. On est sous Linux, et on importe les modules dont nous avons besoin.

Ligne 10, nous avons notre fonction DrawGL qui va se charger de dessiner un triangle. Nous reviendrons dessus plus tard.

Ligne 43, nous avons notre main.

Nous commençons par créer une fenêtre **GTK**, et un conteneur Hbox.

C'est lignes 51 à 56, que les choses vont commencer à devenir intéressantes.

Tout d'abord, ligne 51, nous paramétrons le futur affichage **d'OpenGL :** Couleur RGB, double buffers, et test de profondeur pour **OpenGL**.

Ligne 53, nous créons une zone dédiée **OpenGL**, en lui passant la configuration souhaitée par paramètre.

Ligne 54, nous paramétrons la taille de base par défaut de notre zone spéciale **OpenGL**.

Ligne 55, nous appellerons la fonction DrawGL, à chaque événement (ouverture de l'application, clic de souris, frappe au clavier…)

Ligne 56, nous ajoutons notre zone spéciale à notre conteneur.

Lignes 58 à 61 enfin, nous retrouvons le fonctionnement d'une fenêtre **GTK** normale.

Nous allons maintenant revenir un peu à la fonction DrawGL. Elle récupère deux arguments : la zone de dessin, et l'événement qui l'a appelée.

Dans notre cas, nous ne tiendrons pas compte de l'événement appelant dans la suite du programme.

Ligne 11, nous commençons par nous approprier la zone de dessin.

Ligne 12, nous récupérons le contexte de la zone **OpenGL**. Le contexte correspond à un objet **GTK** contenant certaines informations liées, entre autres, au drag et drop.

Ligne 14, nous interrogeons **OpenGL** pour savoir s'il est prêt à recevoir nos instructions.

Ligne 17, nous récupérons la taille de notre objet, comme vu dans le .

Lignes 19 et 20, nous définissons la position et la taille de la fenêtre.

Lignes 22, 23, et 25 à 28, nous paramétrons la matrice d'affichage **OpenGL**.

Remarque : les lignes 19 à 28 correspondent au ReSizeGLScene.

Lignes 30 à 37, vous reconnaîtrez sûrement maintenant le code permettant de dessiner un triangle multicolore.

Lignes 39 à 41 enfin, nous commutons les buffers et retournons True pour valider que tout s'est bien déroulé.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

VII-B-4 - Gestion de la souris

La déclaration des fonctions de gestion de la souris s'effectue dans le main.

VII-B-4-a - Les boutons et la molette

Pour gérer les boutons et la molette de la souris, nous utilisons la fonction **glutMouseFunc(ma_procédure)**.

La procédure est du type : **def ma_procédure(button, state,mouse_x,mouse_y.**

Nous voyons ici quatre paramètres. Le premier est le type de bouton concerné, le second est l'état du bouton puis le troisième et le quatrième sont les coordonnées du curseur au moment de la détection.

Les boutons sont prédéfinis et sont principalement :

Constante
GLUT_LEFT_BUTTON
GLUT_RIGHT_BUTTON
GLUT_WHEEL_UP
GLUT_WHEEL_DOWN

Les deux derniers concernent la molette. Ils sont à définir au début du programme :

```
GLUT_WHEEL_UP = Constant('GLUT_WHEEL_UP', 3)
GLUT_WHEEL_DOWN = Constant('GLUT_WHEEL_DOWN', 4)
```

Le paramètre state peut prendre lui deux états :

État
GLUT_DOWN
GLUT_UP

```
def buttmouse(butt, state,x,y):
    global flag_mouse_left, flag_mouse_right, zoom

    if (butt==GLUT_LEFT_BUTTON) and (state==GLUT_DOWN):
        flag_mouse_left=1
    elif (butt==GLUT_LEFT_BUTTON) and (state==GLUT_UP):
        flag_mouse_left=0

    if (butt==GLUT_RIGHT_BUTTON) and (state==GLUT_DOWN):
        flag_mouse_right=1
    elif (butt==GLUT_RIGHT_BUTTON) and (state==GLUT_UP):
        flag_mouse_right=0

    if (butt == GLUT_WHEEL_UP): #gestion de la molette de la souris
        zoom=zoom+1
        DrawGLScene()

    if (butt == GLUT_WHEEL_DOWN):
        zoom=zoom-1
        if zoom < 0:
            zoom = 0
        DrawGLScene()
```

L'exemple ci-dessus montre la gestion des boutons et de la molette pour le zoom.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

VII-B-4-b - Le déplacement de la souris

Le déplacement de la souris se gère simplement. Dans le main il faut utiliser une méthode permettant de définir quelle est la fonction s'occupant du mouvement de la souris :

```
glutMotionFunc(movemouse)
```

Cette fonction récupérera d'office les coordonnées X et Y de la souris sur l'écran.

La déclaration de la fonction devra donc être du type :

```
def movemouse(x,y):
```

Pour la suite la meilleure méthode consiste à comparer la position active du curseur à la précédente. Si cela dépasse un certain seuil, alors il faut effectuer une action, sinon ne rien faire.

En effet, tout mouvement de la souris, même d'un pixel activera cette fonction.

En général, au sein de cette fonction, on effectue des calculs de rotation et/ou de déplacement avant de rappeler le DrawGLScene.

VII-B-5 - Gestion du clavier

La déclaration des fonctions de gestion du clavier s'effectue dans le main.

VII-B-5-a - Touches ALT, SHIFT, CTRL

Ces trois touches sont prédéfinies dans **GLUT**. Ces définitions sont vraies quand la touche est pressée et maintenue :

Constante
GLUT_ACTIVE_SHIFT
GLUT_ACTIVE_CTRL
GLUT_ACTIVE_ALT

Remarque : GLUT_ACTIVE_SHIFT est vrai également quand le verrouillage majuscule est actif. De plus si on appuie sur la touche SHIFT alors que le verrouillage majuscule est actif, alors GLUT_ACTIVE_SHIFT sera faux.

VII-B-5-b - Touches alphanumériques

Les touches classiques du clavier se gèrent avec la fonction **glutKeyboardFunc(ma_procédure)** où ma_procédure représente la procédure que nous définissons pour gérer les touches du clavier.

Cette procédure aura la forme suivante : **def ma_fonction(*args):**

Le paramètre de cette procédure permettra de déterminer la touche appuyée. Ainsi pour savoir si nous avons appuyé sur la touche « a », il suffit de comparer arg[0] avec 'a'.

```
def keyPressed(*args):
    global zoom
    # Si on appuie sur ESCAPE, alors on sort de l'application
    if args[0] == ESCAPE:
        glutDestroyWindow(window)
        sys.exit()
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
    elif args[0] == 'z':
        zoom=zoom+2
        if zoom>10:
            zoom=2 #
        DrawGLScene()
```

L'exemple ci-dessus montre par exemple la gestion de la touche **ESCAPE** et de la touche z pour gérer le zoom.

VII-B-5-c - Touches spéciales

Les touches spéciales sont principalement des touches prédéfinies et concernent surtout les touches telles que les Fx et les flèches de direction.

Pour les gérer, nous utilisons la fonction **glutSpecialFunc(ma_procédure)** où ma_procédure représente la procédure qui sera appelée.

Cette procédure sera de la forme : **def ma_procédure(key, mouse_x, mouse_y):**

Trois paramètres sont passés à cette procédure. Le premier est la touche pressée, le second et troisième paramètres sont les coordonnées 2D de l'endroit où se trouvait la souris dans la fenêtre au moment où la touche a été pressée.

Cette fonction détecte la pression de la touche. Il existe autrement la fonction **glutSpecialUpFunc(ma_procédure)** qui détecte, elle, le relâchement des touches spéciales.

Remarque : la pression et le relâchement ne se distinguent que si l'on presse normalement une touche. En cas d'appui prolongé d'une touche, aucune différence n'est visible.

Les touches spéciales prédéfinies sont les suivantes :

Constante
GLUT_KEY_F1
GLUT_KEY_F2
GLUT_KEY_F3
GLUT_KEY_F4
GLUT_KEY_F5
GLUT_KEY_F6
GLUT_KEY_F7
GLUT_KEY_F8
GLUT_KEY_F9
GLUT_KEY_F10
GLUT_KEY_F11
GLUT_KEY_F12
GLUT_KEY_LEFT
GLUT_KEY_UP
GLUT_KEY_RIGHT
GLUT_KEY_DOWN
GLUT_KEY_PAGE_UP
GLUT_KEY_PAGE_DOWN
GLUT_KEY_HOME
GLUT_KEY_END
GLUT_KEY_INSERT

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

De plus, les touches escape, backspace et delete sont aussi des touches spéciales, mais sont à déclarer au début du programme (non prédéfinies) :

ESCAPE = chr(27)

BACKSPACE = chr(8)

DELETE = chr(127)

VII-B-6 - Exemple

Dans cet exemple tout simple, nous allons afficher un cube et gérer les divers événements liés.

Nous resterons volontairement basique au niveau du code, et ne respecterons pas la totalité des règles classiques afin que le code, déjà un peu complexe, soit accessible à tous.

```
#! /usr/bin/env PYTHON
# -*- coding: utf-8 -*-



from OpenGL.GL import *
from OpenGL.GLUT import *
from OpenGL.GLU import *
import sys
import time



ESCAPE = chr(27)
TURN= '\060'
zz=chr(122) #z
GLUT_WHEEL_UP = Constant('GLUT_WHEEL_UP', 3)
GLUT_WHEEL_DOWN = Constant('GLUT_WHEEL_DOWN', 4)

window = 0
flag=0

zoom=10.0
xc=2.
yc=2.
zc=2.



def InitGL(Width, Height):
    glClearColor(1.0, 1.0, 1.0, 0.0)
    glClearDepth(1.0)
    glDepthFunc(GL_LESS)
    glEnable(GL_DEPTH_TEST)
    glShadeModel(GL_SMOOTH)

    glMatrixMode(GL_PROJECTION)
    glLoadIdentity()

    gluPerspective(45.0, float(Width)/float(Height), 0.1, 100.0)

    glMatrixMode(GL_MODELVIEW)


def ReSizeGLScene(Width, Height):
    if Height == 0:
        Height = 1
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
    glViewport(0, 0, Width, Height)
    glMatrixMode(GL_PROJECTION)
    glLoadIdentity()
    gluPerspective(45.0, float(Width)/float(Height), 0.1, 100.0)
    glMatrixMode(GL_MODELVIEW)



def DrawGLScene():
    global zoom, xc, yc, zc

    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glLoadIdentity()

    gluLookAt(zoom*xc,zoom*yc,zoom*zc,0,0,0,0,0,1)

    glBegin(GL_QUADS)

    glColor3f(0.0,1.0,0.0)
    glVertex3f( 1.0, 1.0,-1.0)
    glVertex3f(-1.0, 1.0,-1.0)
    glVertex3f(-1.0, 1.0, 1.0)
    glVertex3f( 1.0, 1.0, 1.0)

    glColor3f(1.0,0.5,0.0)
    glVertex3f( 1.0,-1.0, 1.0)
    glVertex3f(-1.0,-1.0, 1.0)
    glVertex3f(-1.0,-1.0,-1.0)
    glVertex3f( 1.0,-1.0,-1.0)

    glColor3f(1.0,0.0,0.0)
    glVertex3f( 1.0, 1.0, 1.0)
    glVertex3f(-1.0, 1.0, 1.0)
    glVertex3f(-1.0,-1.0, 1.0)
    glVertex3f( 1.0,-1.0, 1.0)

    glColor3f(1.0,1.0,0.0)
    glVertex3f( 1.0,-1.0,-1.0)
    glVertex3f(-1.0,-1.0,-1.0)
    glVertex3f(-1.0, 1.0,-1.0)
    glVertex3f( 1.0, 1.0,-1.0)

    glColor3f(0.0,0.0,1.0)
    glVertex3f(-1.0, 1.0, 1.0)
    glVertex3f(-1.0, 1.0,-1.0)
    glVertex3f(-1.0,-1.0,-1.0)
    glVertex3f(-1.0,-1.0, 1.0)

    glColor3f(1.0,0.0,1.0)
    glVertex3f( 1.0, 1.0,-1.0)
    glVertex3f( 1.0, 1.0, 1.0)
    glVertex3f( 1.0,-1.0, 1.0)
    glVertex3f( 1.0,-1.0,-1.0)
    glEnd()

    time.sleep(0.002)

    glutSwapBuffers()



def buttmouse(butt, state,x,y):
    global zoom

    if (butt == GLUT_WHEEL_UP):
        zoom=zoom+1
        DrawGLScene()

    if (butt == GLUT_WHEEL_DOWN):
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

```
        zoom=zoom-1
        if zoom < 1:
            zoom = 1
        DrawGLScene()



def main():
    global window

    glutInit(sys.argv)

    glutInitDisplayMode(GLUT_RGBA | GLUT_DOUBLE | GLUT_ALPHA | GLUT_DEPTH)

    glutInitWindowSize(640, 480)

    glutInitWindowPosition(0, 0)

    window = glutCreateWindow("Mon test a moi")

    glutDisplayFunc(DrawGLScene)

    glutReshapeFunc(ReSizeGLScene)

    glutMouseFunc(buttmouse)

    InitGL(640, 480)



if __name__ == '__main__':
    try:
        GLU_VERSION_1_2
    except:
        print "Need GLU 1.2 to run this demo"
        sys.exit(1)
    main()
    glutMainLoop()
```

VII-C - Concepts mathématiques 2D

Travailler dans un plan 3D peut aisément se rapporter à travailler sur 2 plans 2D. C'est pourquoi, avant de se lancer dans les plans 3D, il est impératif de maîtriser les bases des plans 2D.

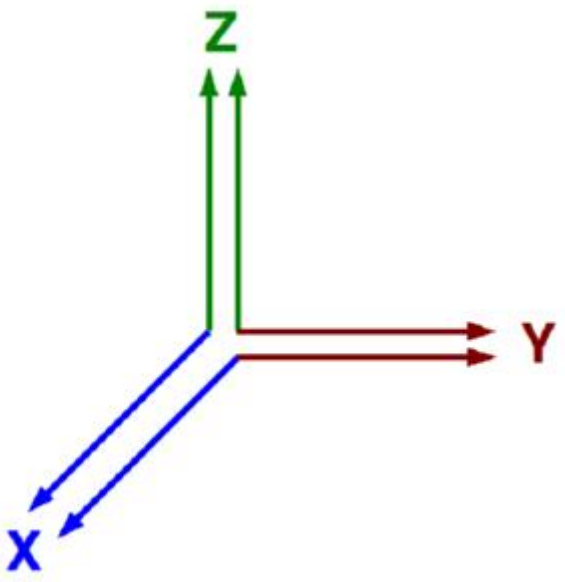
Ainsi, par la suite, en cas de difficultés sur un plan 3D, il est aisé de repasser sur des plans 2D pour modifier la vision de notre problème.

VII-C-1 - Les plans 2D

Tout le monde connaît les **plans 2D**. Nombre de jeux vidéo et d'éléments dans la vie courante sont basés sur ce principe.

Dans les faits, un **plan 3D** (XYZ) n'est rien d'autre qu'un ensemble de 3 plans 2D (XY, XZ, YZ).

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/



Assemblage des plans

Il ne faut donc jamais perdre de vue cela, car un point en 3D, défini par ses coordonnées, peut de fait également être vu comme étant la fusion de trois points de trois plans 2D différents.

A(x,y,z) = fusion (A(x,y), A(x,z), A(y,z))

VII-C-2 - La trigonométrie

À partir du moment où l'on se met à manipuler des **objets 3D**, et dans la mesure où un objet 3D est un assemblage d'**objets 2D**, la maîtrise de la **trigonométrie** de base est impérative.

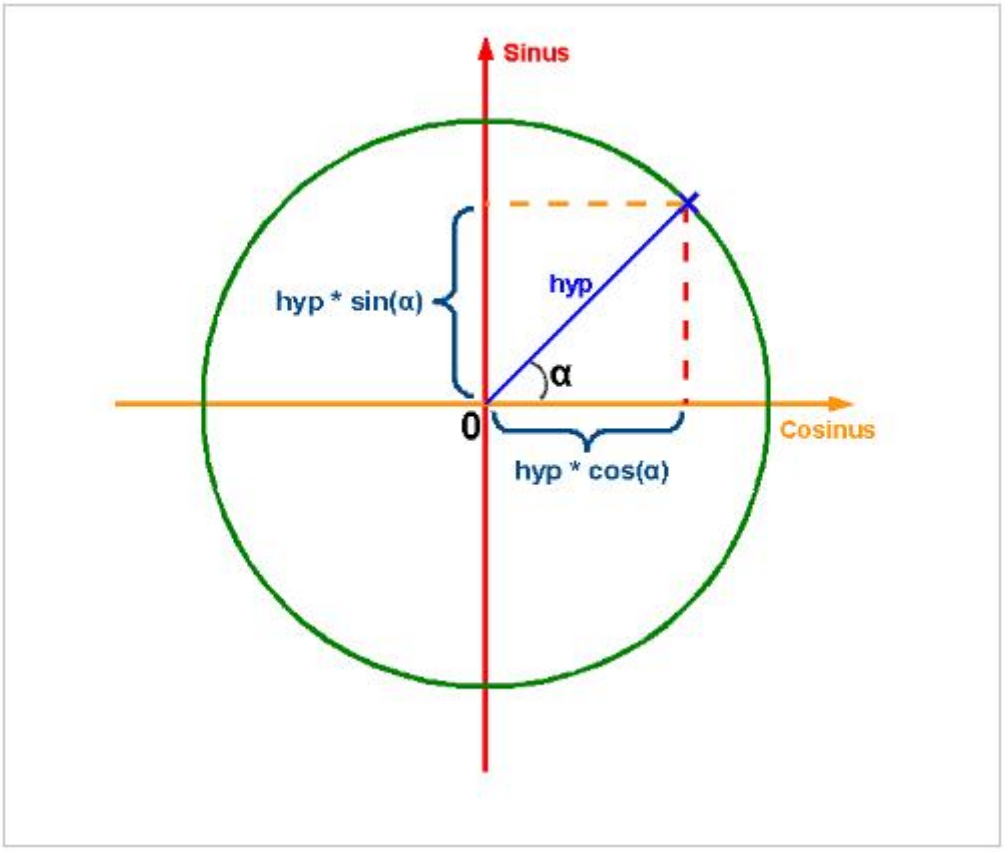
Cela vous permettra de manipuler aisément (rotation par exemple) des objets en 3D.

VII-C-2-a - Cercle trigonométrique

Le cercle trigonométrique est la base de toute rotation dans l'espace.

La rotation en 3D fera l'objet d'un descriptif plus loin. Nous allons ici parler des rotations en 2D, servant de base pour la 3D.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/



Le cercle trigonométrique

Une rotation est simple à effectuer quand on maîtrise ce principe : les coordonnées de tout point tournant autour d'une origine définie sont inscrites sur un même cercle, le cercle trigonométrique.

Dans notre illustration, les axes cosinus et sinus pourraient être remplacés par les axes X et Y.

Ainsi, si l'on connaît les coordonnées d'un point, et que l'on sait que la pièce tourne autour de l'origine de x°, il faut suivre la procédure suivante :

- à partir des coordonnées du point, détermination de l'angle initial ;
- addition de l'angle initial et de l'angle de rotation ;
- détermination des nouvelles coordonnées grâce aux cosinus et sinus (respectivement coordonnées en X et Y)

La seule chose à connaître est l'origine pour la rotation, qui est la plupart du temps simplement XY(0,0).

VII-D - Concepts mathématiques 3D

VII-D-1 - Formules

VII-D-1-a - Équation d'une droite en 3D

En 3D, une droite ne possède pas d'équation au sens propre de type « y = ax +b +c ». Elle est cependant définie par un ensemble de trois équations paramétriques :

$x = x_0 + \Phi tx$;

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

$y = y_0 + \Phi ty$;

$z = z_0 + \Phi tz$où x_0,y_0,z_0 sont les coordonnées d'un point de AB.

Soit $A(x_1,y_1,z_1)$ et $B(x_2,y_2,z_2)$ les deux points définissant notre droite. Nous avons :

$tx = x_2 - x_1$

$ty = y_2 - y_1$

$tz = z_2 - z_1$

Par exemple : A(2,1,3) et B(4,-2,1)

$tx = 4 - 2 = 2$

$ty = -2 -1 = -3$

$tz = 1 - 3 = -2$

Les équations paramétriques de la droite AB sont donc :

$x = x_1 + 2\Phi$

$y = y_1 - 3\Phi$

$z = z_1 - 2\Phi$

VII-D-1-b - Équation d'un plan en 3D

VII-D-1-b-i - Méthode 1

L'équation d'un plan 3D est de type ax + by + cz + d = 0

Un plan est défini par trois vertex. On obtient donc :

$L1ax_1 + by_1 + cz_1 + d = 0etf_1 = \frac{x_2}{x_1}$

$L2ax_2 + by_2 + cz_2 + d = 0$

$L3ax_3 + by_3 + cz_3 + d = 0f_2= \frac{x_3}{x_1}$

Avec finalement :

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

a = $\frac{(-by_1 - cz_1 - d)}{x_1}$

b = $\frac{c*(z_2 - f_1*z_1) + d(1 - f_1)}{f_1*y_1 - y_2}$

c = $-d\left[\frac{(1-f_2)(f_1*y_1 - y_2) + (y_3 - f_2*y_1)(1-f_1)}{(z_3 - f_2*z_1)(f_1*y_1 - y_2) + (z_2 - f_1*z_1)(y_3 - f_2*y_1)}\right]$

N.B. Prendre d = 1, puis calculer c, puis b, puis a

Attention : si $x_1 = 0$ cela pose problème pour f_1 et f_2. Échanger l'ordre des points pour avoir un $x_1 \neq 0$.

Il faut également utiliser un algorithme similaire au suivant pour déterminer le signe de f_1 et f_2 :

si $x_1 \neq 0$

si $x_1 > 0$

si $x_2 > 0$

$f_1 = \frac{+x_2}{x_1}$

sinon

$f_1 = \frac{-x_2}{x_1}$

sinon

si $x_2 > 0$

$f_1 = \frac{-x_2}{x_1}$

sinon

$f_1 = \frac{+x_2}{x_1}$

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

VII-D-1-b-ii - Méthode 2

La méthode 1 est à connaître, mais n'est cependant pas la plus simple. En effet, la plus simple est la méthode dite des produits mixtes qui nous fournit les équations suivantes :

$$\underset{\sim}{a} = (Y_B - Y_A)(Z_C - Z_A) - (Z_B - Z_A)(Y_C - Y_A)$$

$$\underset{\sim}{b} = -((X_B - X_A)(Z_C - Z_A) - (Z_B - Z_A)(X_C - X_A))$$

$$\underset{\sim}{c} = (X_B - X_A)(Y_C - Y_A) - (Y_B - Y_A)(X_C - X_A)$$

$$\underset{\sim}{d} = -(a * X_A + b * Y_A + c * Z_A)$$

$$\underset{\sim}{z} = \frac{(-a * X - b * Y - d)}{c}$$

VII-D-1-b-iii - Cas particuliers

Il existe quelques cas particuliers

>Plan parallèle à l'axe $\vec{Z}$; $\underset{\sim}{a}x + \underset{\sim}{b}y + d = 0$
>Plan parallèle à l'axe $\vec{Y}$; $\underset{\sim}{a}x + \underset{\sim}{c}z + d = 0$
>Plan parallèle à l'axe $\vec{X}$; $\underset{\sim}{b}y + \underset{\sim}{c}z + d = 0$

De plus si l'origine est comprise dans le plan, alors cela simplifie encore les équations puisque d = 0.

Enfin, le calcul d'un plan 3D dont l'origine est dans le plan se fait avec :

$$\underset{\sim}{a} = \frac{(y_2 * z_1 - y_1 * z_2)}{(x_2 * y_1 - y_2 * x_1)}$$

$$\underset{\sim}{b} = \frac{(-z_1 - a * x_1)}{y_1}$$

$$\underset{\sim}{c} = 1$$

$$\underset{\sim}{d} = 0$$

VII-D-1-c - Calcul de la normale au plan 3D

La **normale** au plan 3D est un vecteur perpendiculaire au plan, et servant dans les logiciels 3D principalement à indiquer comment réfléchir la lumière.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

À partir d'un plan formé par les points $A(x_1,y_1,z_1)$, $B(x_2,y_2,z_2)$ et $C(x_3,y_3,z_3)$, le plan formé par ces trois points peut être défini selon les deux vecteurs

$$\overrightarrow{AB}\ (x_2-x_1,y_2-y_1,z_2-z_1) \text{ et } \overrightarrow{AC}\ (x_3-x_1,y_3-y_1,z_3-z_1).$$

La normale se calcule ainsi :

$Nx = (y_2 - y_1)*(z_3 - z_1) - (z_2 - z_1) * (y_3 - y_1)$

$Ny = (z_2 - z_1)*(x_3 - x_1) - (x_2 - x_1) * (x_3 - x_1)$

$Nz = (x_2 - x_1) * (y_3 - y_1) - (y_2 - y_1) * (x_3 - x_1)$

Avec la norme de la normale

$$\vec{N} = \sqrt{(x^2+y^2+z^2)}$$

VII-D-1-d - Calcul des coordonnées d'un point en 3D

Pour calculer les coordonnées d'un point en 3D, à partir d'une équation de plan auquel il appartient :

$$\underset{\sim}{x} = \frac{(-b*y-c*z-d)}{a}$$

$$\underset{\sim}{y} = \frac{(-a*x-c*z-d)}{b}$$

$$\underset{\sim}{z} = \frac{(-a*x-b*y-d)}{c}$$

À partir des équations paramétriques d'une droite :

$x = x0 + \Phi tx$

$y = y0 + \Phi ty$

$z = z0 + \Phi tz$

Il suffit de connaître x, y ou z afin de déterminer Φ. À partir de là, on peut trouver le reste des coordonnées du point.

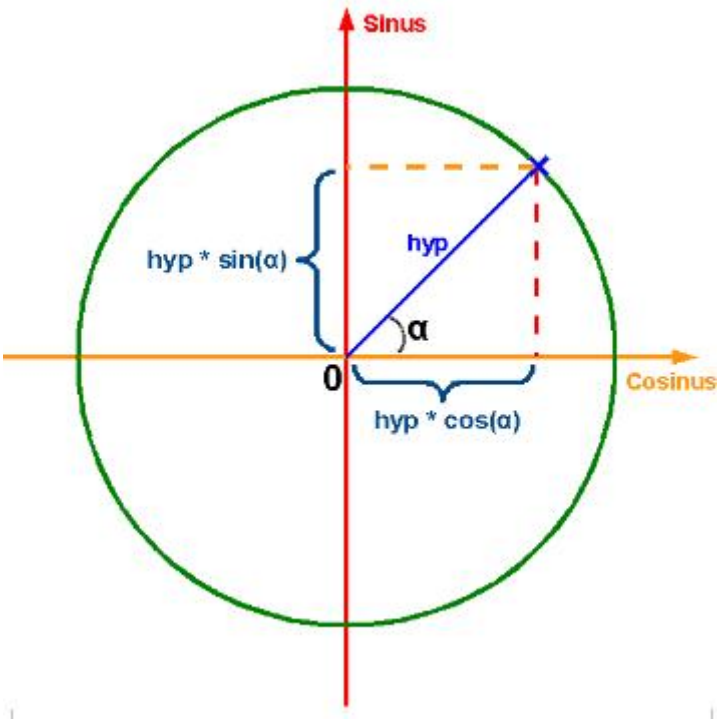
VII-D-1-e - Rotation d'un point en 3D

Quand un point tourne en 3D, il peut tourner selon trois jeux d'axes : XY, XZ ou YZ.

Pour chacun de ces jeux, tout calcul n'est que trigonométrie de base constitué de sinus et de cosinus.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

En résumé donc, une rotation 3D est simplement un assemblage de trois rotations 2D.



Le cercle trigonométrique

Pour toute rotation en 3D, il faut donc pour chaque jeu d'axes, se rapporter à un plan 2D et simplement recalculer les coordonnées du point.

Vous connaissez systématiquement les coordonnées sur les deux axes. Vous pouvez donc déterminer la valeur de l'hypoténuse (h en pointillés sur l'illustration), puis par déduction la valeur de l'angle.

$$hyp^2 = \sqrt{a^2 + b^2} \quad \text{et} \quad \cos(\alpha) = \frac{b}{hyp}$$

Comme vous connaissez également la valeur angulaire de la rotation, vous pouvez donc en déduire les nouvelles coordonnées du point.

Pour une rotation de #° :

$$\beta = \alpha + \Upsilon \quad \text{donc} \quad a = hyp * \sin(\beta) \quad \text{et} \quad b = hyp * \cos(\beta)$$

VII-D-1-f - Un point appartient-il à un triangle ?

Quand un point appartient à un triangle, quand nous parcourons les droites du triangle, ce point se trouve toujours du même côté des droites.

Si nous considérons notre triangle ABC, un point M se situe dans ce triangle si les t_0, t_1 et t_2 sont du même signe.

$t_0 = (x_2 - x_1) * (y_M - y_1) - (x_M - x_1) * (y_2 - y_1)$

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

$t_1 = (x_3 - x_2) * (y_M - y_2) - (x_M - x_2) * (y_3 - y_2)$

$t_2 = (x_1 - x_3) * (y_M - y_3) - (x_M - x_3) * (y_1 - y_3)$

VII-E - Quelques techniques 3D

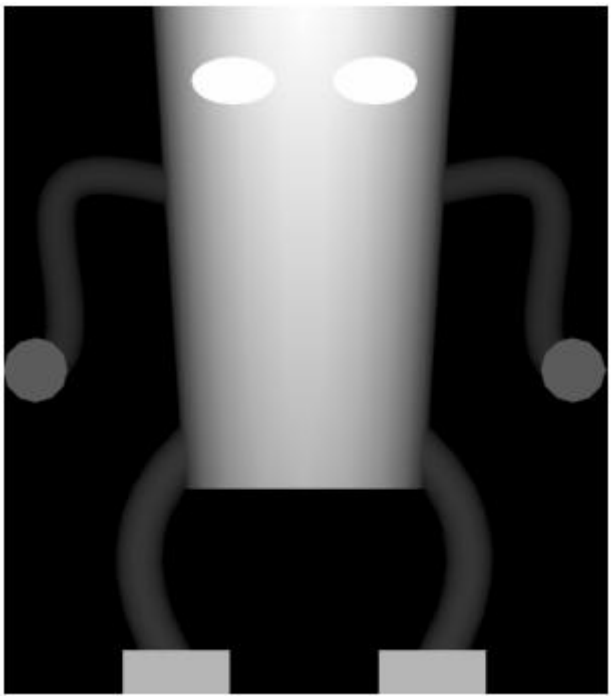
VII-E-1 - Depthmaps

Un **depthmaps** est une image totalement en nuances de gris ou chaque nuance de gris correspond à une profondeur différente. Le noir représente le 0 et le blanc le niveau le plus haut, ou l'inverse au choix.

Lorsque l'on désire traduire un depthmaps pour une machine, on précise alors la hauteur de l'objet final, ainsi que sa taille (hauteur et largeur).

Le logiciel se charge alors de calculer la hauteur correspondante à chaque nuance de gris, ainsi que la position de chaque pixel, puis effectue un lissage global pour obtenir un produit fini.

En absence de ce lissage, le produit final aurait un aspect relativement « pixellisé ».



Un depthmaps issu d'un fichier STL

VII-E-2 - Stéréogramme

Le terme de **stéréogramme** définit toute image qui permet à un être vivant, par n'importe quelle technique, de visualiser une image en 3D.

Les exemples les plus connus de stéréogrammes sont probablement les images en nuances de bleu et de rouge livrées avec une paire de lunettes adaptées.

Ces images sont conçues assez simplement sur le principe des yeux humains.

Prenez une feuille de papier et dessinez dessus un trait, puis sur ce trait placez deux points distancés de 10 cm.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Ensuite, posez votre appareil photo sur le point de gauche et prenez une photo. Puis prenez une autre photo après avoir posé l'appareil sur le point de droite.

Vous obtenez deux photos. La première correspond à ce que voit normalement l'œil gauche humain et la seconde l'œil droit humain.

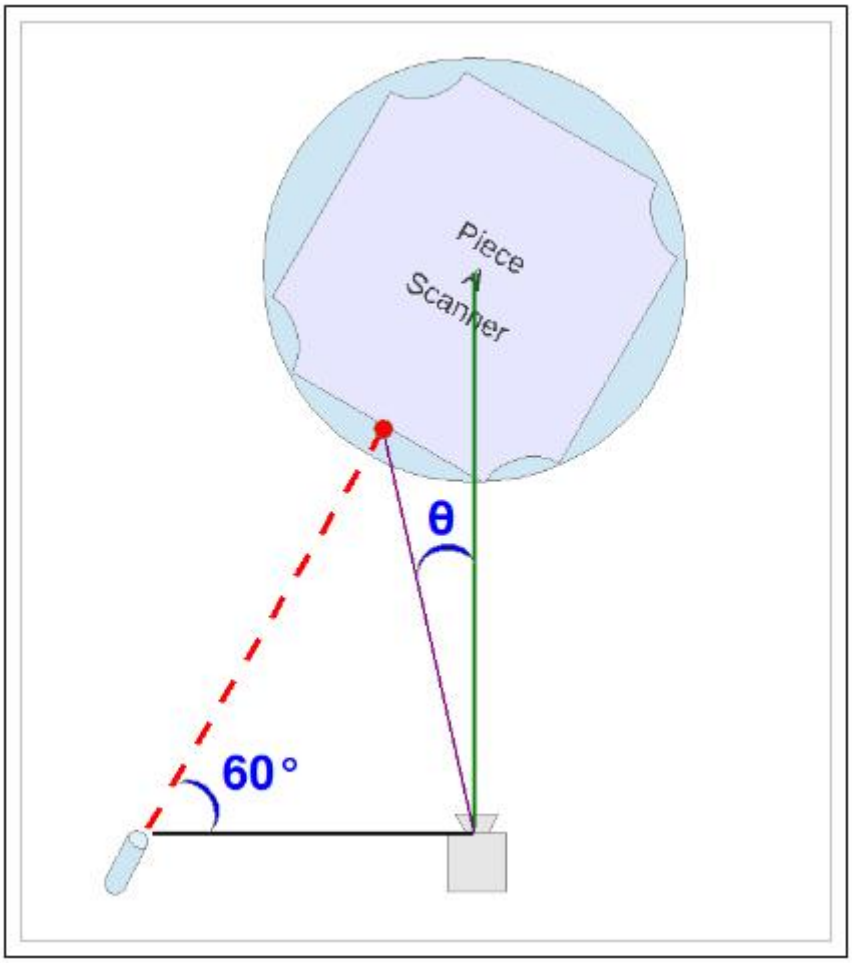
Via le traitement de ces deux images, vous êtes alors capable de concevoir un stéréogramme.

Avec un traitement informatique adapté, vous pouvez même reconstituer un objet en 3D à partir de ces deux images.

VII-E-3 - Scanner 3D laser

Il est possible de scanner simplement un objet en 3D afin d'en déterminer son fichier STL équivalent par exemple.

Pour cela, il est nécessaire d'avoir un plateau tournant, une webcam, et une ligne laser.



Principe du scanner laser 3D

Le principe est le suivant : on connaît la distance entre la webcam et le centre du plateau tournant, de même que la distance entre la webcam et la ligne laser et enfin l'angle de la ligne laser, qui pointe vers le centre du plateau représentant l'origine.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

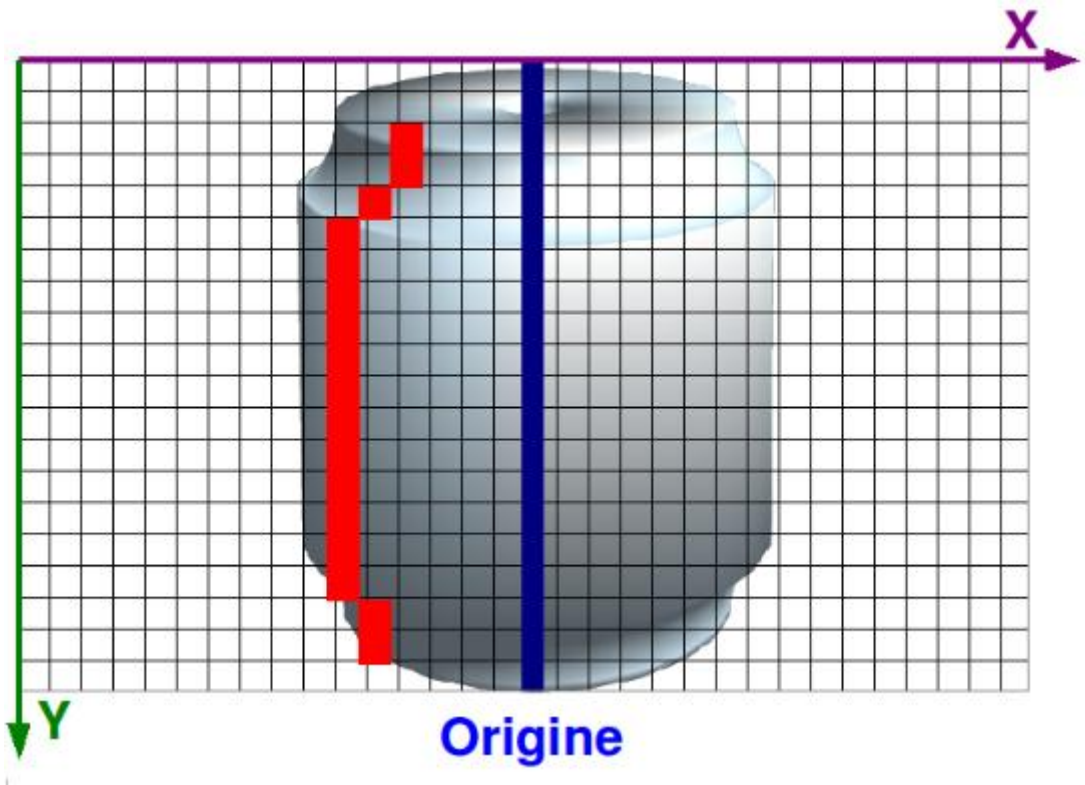
Vu depuis la webcam, on constate que la ligne laser dessine le contour de l'objet. Si on fait tourner l'objet sur lui-même, on peut alors déterminer sa forme en 3D. Cela est encore plus flagrant dans le noir. En passant l'image en noir et blanc, on obtient alors une ligne blanche sur fond noir.

Au niveau de l'analyse de la forme, cela est très simple. Notre webcam capture en réalité non pas un film, mais une suite d'images. Ces images possèdent une certaine résolution. Il faut alors considérer cette résolution comme un quadrillage.

Si on aligne le centre de l'axe horizontal de la webcam avec le centre du plateau rotatif, alors chaque pixel, à droite ou à gauche, représente un certain angle. Il faut connaître ou mesurer l'angle de vision de la webcam pour déterminer la valeur angulaire d'un pixel.

Un pixel vaudra donc : $$pixel = \frac{angle\ vision\ webcam}{resolution\ native\ en\ X}$$

Le quadrillage vertical de pixels (en Y) permet donc de déterminer la forme de l'objet.

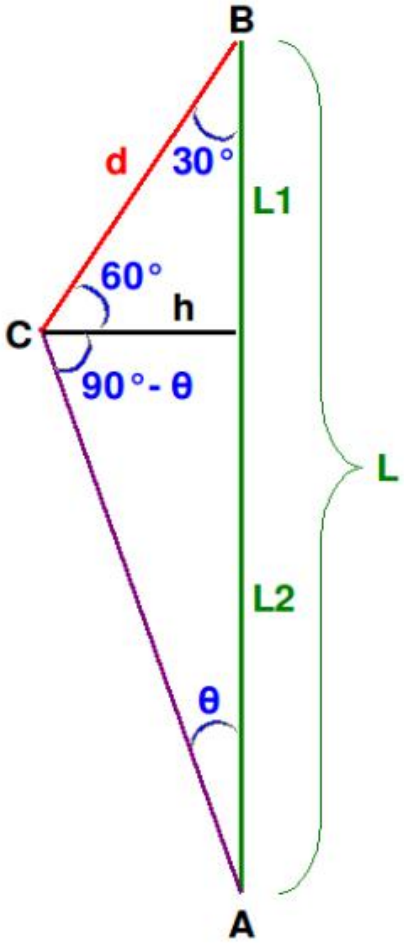


Matrice de points

De fait, la précision d'un scanner d'un tel type dépend beaucoup de la qualité de la webcam. Plus la résolution sera haute, plus précis sera le scanner.

À partir de là, tout n'est alors que mathématique.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/



$$d=\sqrt{L1^2+h^2}$$

$$\tan(60)=\frac{L1}{h}$$

$$\tan(90-\theta)=\frac{L2}{h}=\frac{L-L1}{h}$$

$$L1=\frac{L*\tan(60)}{\tan(60)+\tan(90-\theta)}$$

$$h=\frac{L-L1}{\tan(90-\theta)}$$

$$d=\sqrt{L1^2+h^2}$$

Le résultat final est un fichier de points 3D qui, via un logiciel type **MESHLAB**, ou un petit traitement maison, peut facilement devenir un fichier STL ou un ficher DEPTHMAPS. La référence de chaque valeur de d est le centre du plateau rotatif. Dans un plan 3D, cela correspondra à l'origine.

Remarque : pour avoir de bons résultats, il est préférable d'avoir une ligne laser de chaque côté de la webcam, de scanner l'objet dans un sens avec la première ligne laser, puis de le scanner dans l'autre sens avec la seconde ligne.

VIII - Gestion d'un projet



Wikimedia Commons,Gnome-mime-application-x-gnumeric.png

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Tout **projet** (**PJ**), quel qu'il soit, a besoin d'être bien géré pour aboutir.

Nous allons ici voir les principaux aspects de la gestion d'un projet afin de vous permettre, quel que soit votre objectif, de l'atteindre de manière la plus structurée possible.

VIII-A - Organisation

VIII-A-1 - Description du PJ

La première chose à faire lorsque vous démarrez un projet est de réfléchir à ce que vous désirez vraiment. À ce stade il s'agit principalement d'une **description fonctionnelle**, néanmoins la plus détaillée possible.

Utilisez le document d'annexe pour vous aider :

« 01 - Specifications_Fonctionnelles_PYTHON ».

Ce document est disponible également en téléchargement au format LibreOffice sur le site de l'auteur.

VIII-A-2 - Structure du projet

Une fois l'étape de la description de votre projet effectuée, il faut réfléchir à la structure même de votre projet : regroupement de fonctions/procédures en packages et modules, répartition des différentes fonctions, les classes à créer, modules et/ou fonctions déjà existants…

Pour résumer, il s'agit d'une **traduction fonctionnelle/technique** de l'étape précédente.

Utilisez le document d'annexe pour vous aider :

« 02 - Specifications_Techniques_PYTHON ».

Ce document est disponible également en téléchargement au format LibreOffice sur le site de l'auteur.

L'expérience démontre qu'un projet mal décrit et/ou structuré peut aboutir, et ce même en fin de développement, à la réécriture partielle ou totale de code. De fait, il est important de passer du temps sur ces deux premières étapes afin de, par la suite, limiter les problèmes de développement.

VIII-A-3 - Priorisation

Maintenant que vous avez terminé la description et la structure de votre projet, il va falloir prioriser les divers éléments.

En effet, tout comme dans le monde réel, il faut savoir dans quel ordre évoluer.

Pour prendre un exemple concret, imaginez un immeuble, où chaque étage représente un module différent. Il n'est pas vraiment recommandé de commencer par le module « Étage3 ». Il ne pourrait exister seul. Par contre, le module « Rez de chaussée », lui, pourrait exister seul. Il s'agit de la brique de base.

Pour bien gérer un projet, il faut respecter la même logique et commencer par les briques de base avant d'évoluer progressivement vers les briques de sommet.

Bien entendu, il est possible pour certains cas particuliers de développer plusieurs modules en parallèle sans conflit ou problèmes particuliers.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Cet ordre que nous suivrons s'appelle la **priorisation**. Pour la symboliser, nous noterons, en commençant à 0, un indice de priorité dans le tableau précédent en face de chaque module, code, fonctions… que nous aurons à développer.

Utilisez le document d'annexe pour vous aider :

« 03 - Priorisation_PYTHON ».

Ce document est disponible également en téléchargement au format LibreOffice sur le site de l'auteur.

VIII-A-4 - Planification

Toutes les étapes précédentes passées, il ne vous reste plus qu'à **planifier votre développement** à l'aide de la documentation préalablement renseignée.

S'il est vrai que cela est indispensable dans le milieu professionnel, dans le cas d'un développement personnel, voire communautaire, cela peut parfois être moins significatif.

Quoi qu'il en soit, c'est une chose que je vous encourage à faire, même à titre privé, ne serait-ce que pour, à terme, avoir une vision générale du développement.

Pour planifier votre projet, le mieux est d'utiliser ce qu'on appelle un **diagramme de Gantt**. Il s'agit d'un graphique permettant visuellement d'identifier le temps alloué et passé à une tâche. Chaque tâche étant associée à une ressource, autrement dit un développeur, il est facile de suivre l'évolution d'un projet jour après jour.

De plus, chaque ressource pouvant être associée à un coût/heure ou coût/jour, un chef de projet, en entreprise par exemple, pourra rapidement avoir une vision globale du projet qu'il gère.

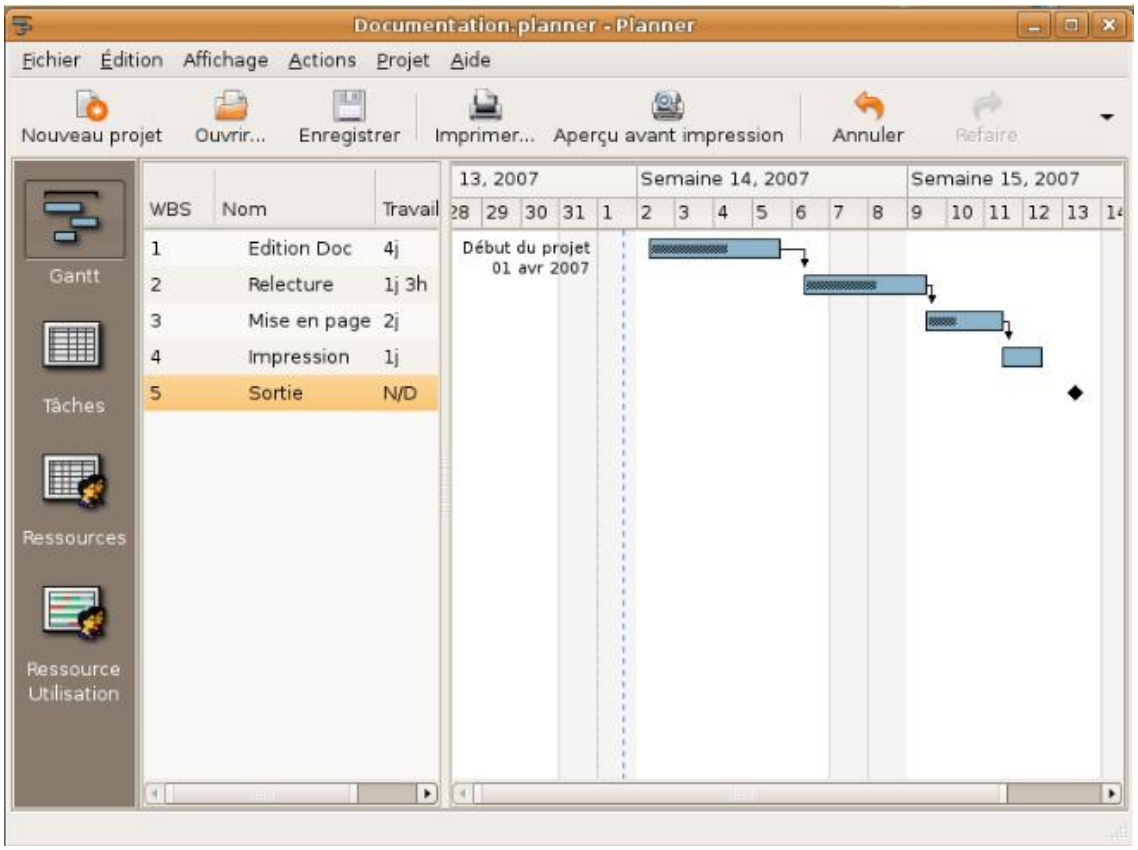
Dans notre cas, je vais vous présenter deux logiciels : **Planner**, qui est idéal pour les développements en solo ou en petite équipe, et **Calligraplan** qui s'adresse plutôt à des équipes de taille moyenne et importante.

VIII-A-4-a - Planner

L'outil idéal pour un développeur solo, ou une petite équipe. Vous créez vos ressources, vos projets, vos tâches, et vous êtes prêt à planifier vos projets.

Cet outil se limite au strict minimum, mais le fait bien. Le strict nécessaire est suffisamment paramétrable pour un entrepreneur indépendant, et vous avez facilement l'ensemble des informations dont vous avez besoin, même si vous avez besoin de compiler l'ensemble des informations à la mano pour réaliser un rapport complet ou extrapoler les données brutes.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/



L'interface de Planner

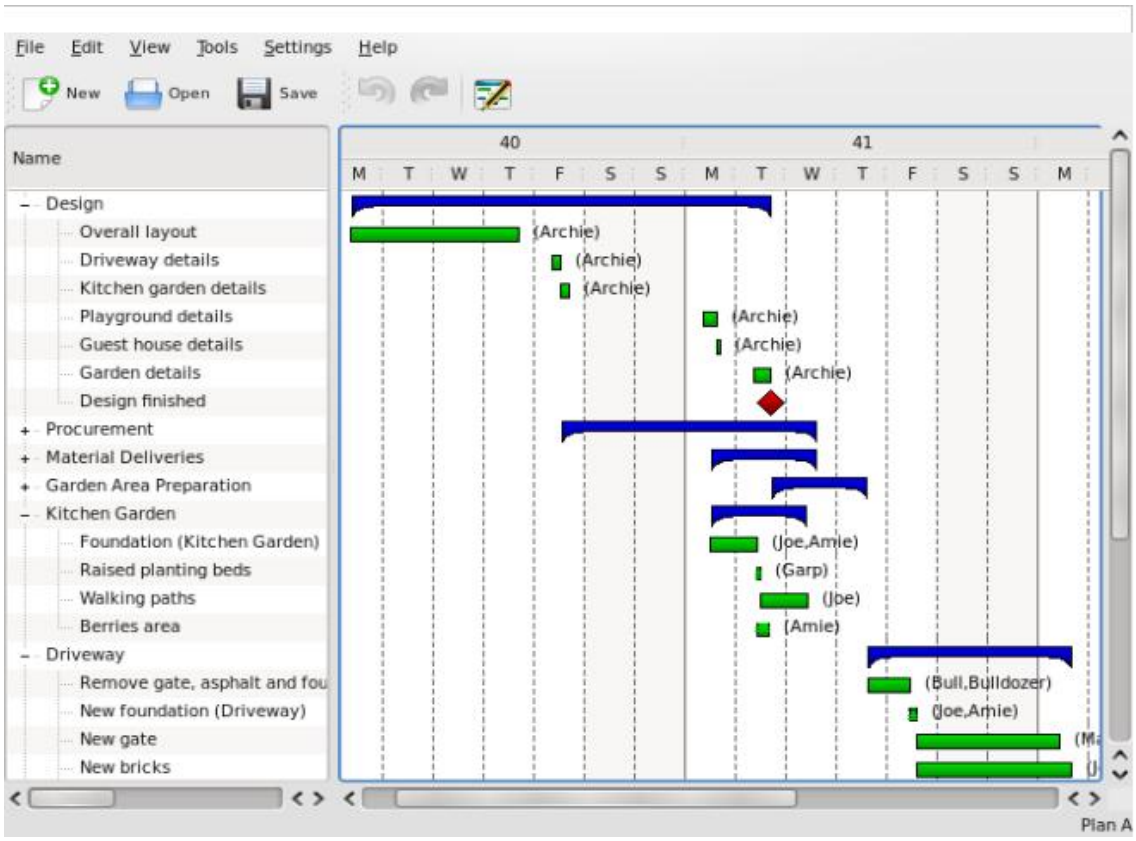
VIII-A-4-b - Calligraplan

Calligraplan est un logiciel similaire à Planner. Cependant, contrairement à ce dernier, il permettra de générer des synthèses, ce qui peut s'avérer très utile.

De plus, l'interface n'est pas la même. Si Planner est basé sur PYGTK, Calligraplan est lui basé sur KDE.

Enfin, comme précisé précédemment, le nombre de fonctionnalités avancées proposées par Calligraplan, rapport à celles de Planner, le prédestine plus à un usage de users avancés ou d'un groupe de développeurs un minimum élevé.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/



L'interface de Calligraplan

VIII-B - Viabilité et pérennité

Lorsque vous développez, il faut penser en permanence à la **viabilité** et à la **pérennité de votre code**.

Par viabilité, il faut comprendre code simple, fonctionnel et aisément maintenable, et par pérennité, code aisément réutilisable.

Le manquement à ces deux principes peut compromettre un développement complet, une refonte, et/ou un colmatage de sécurité ou de fonctionnement.

VIII-B-1 - Code

Un code doit en permanence être viable. Si vous codez n'importe comment, vous n'arriverez pas vous-même à vous replonger dans votre code quelques mois plus tard.

Pour cette raison, il est primordial de respecter les règles énoncées précédemment, car la plupart des développeurs **PYTHON** les suivront.

C'est notamment ce respect du principe de code viable qui fait que l'on puisse autant échanger.

VIII-B-2 - Tests

La partie des **tests** dans un projet est extrêmement importante. En effet, c'est cette dernière qui déterminera si votre code est conforme ou non à ce qui était attendu.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Dans les faits, on conseille souvent que les tests soient rédigés avant la réalisation, surtout si les testeurs et les développeurs sont les mêmes personnes.

Le but d'anticiper les tests est de ne pas être influencé par le code que l'on pourrait avoir développé ou participé à développer.

Les tests d'un projet peuvent se définir en trois phases : deux techniques visant à tester le code même, et une fonctionnelle.

VIII-B-2-a - Test unitaire

Il s'agit du test de plus bas niveau, de la plus petite unité, autrement dit au plus proche du code.

Le test unitaire consiste à tester une fonction ou une procédure précise. Cela permet de s'assurer qu'une fois en cours d'exécution, chaque petite pièce de l'ensemble du code se comportera tel que désiré.

VIII-B-2-b - Test d'intégration

Niveau supérieur au test unitaire, le test d'intégration vise à vérifier le bon fonctionnement d'un ensemble de fonctions/procédures/classes/… et ce, à différents niveaux.

Ainsi vous testerez d'abord un ensemble de fonctions/procédures d'une classe, puis un ensemble de classes… jusqu'à votre logiciel complet.

Il ne s'agit pas de vérifier ici si en interne du code tout se déroule correctement, mais de s'assurer que la sortie est conforme à ce que l'on attend. Et cela même si en interne le fonctionnement n'est pas celui escompté.

En effet, cette dernière vérification est laissée au soin des tests unitaires.

VIII-B-2-c - Test de validation ou fonctionnel

Le test de plus haut niveau. Ici, on se moque du code. Tout ce qui compte est le fonctionnement global de l'application. Il s'agit d'un test de fonctionnement d'un point de vue utilisateur.

Le logiciel réagit-il comme attendu ? L'interface est-elle conforme ? Les fichiers générés sont-ils au bon format ?…

Toutes ces questions de « surface » de fonctionnement sont gérées par les tests fonctionnels.

VIII-B-2-d - Construction d'un test

Chacun de ces tests se construit différemment. Le seul point commun est que, comme explicité au début, il faut écrire chaque test avant de concevoir. Cela vous aidera à coder au mieux et au plus proche du cahier des charges.

Ainsi, pour un test unitaire, vous devez décrire ce que vous attendez en entrée, ce que vous attendez en sortie, et le fonctionnement attendu en interne.

De même, pour un test d'intégration, vous devrez décrire les mêmes éléments : entrée, sortie, fonctionnement interne.

Enfin, pour le test fonctionnel, il faut purement vous appuyer sur le cahier des charges. Ce dernier est censé décrire de manière assez complète le côté fonctionnel attendu.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Une fois tout cela établi, le plus dur est fait. Il ne vous reste plus éventuellement qu'à vous rédiger des petites fiches de tests et à passer ces derniers pour vérifier que tout est OK.

Vous trouverez en annexe et sur le site de l'auteur un exemple de fiche pour chaque type de tests.

VIII-B-2-e - Logiciel dédié : Unittest

Côté **PYTHON**, vous avez la possibilité d'automatiser l'ensemble des tests techniques (unitaires et intégration).

Pour cela nous allons utiliser le module **UNITTEST**. Ce module vous permettra d'écrire un ou plusieurs modules dédiés.

Nous verrons ici comment utiliser les fonctions basiques de ce module.

VIII-B-2-e-i - Fonctionnement de base

De quoi peut bien se composer un test ? Eh bien dans notre cas, ce dernier prendra l'apparence d'un module à part entière qui sera placé dans le même dossier (ou package) que notre code à tester.

Je vous conseille de nommer votre module de test d'un nom explicite ; par exemple TestUnitaire_p_multiplication.py.

Le but est clairement que vous identifiez d'un simple coup d'œil ce qui est testé dans ce module de test. N'oubliez pas qu'avec **PYTHON**, « Explicite est mieux qu'implicite ».

Une fois un nom explicite choisi, il ne reste plus qu'à remplir le module de test. Nous imaginerons ici que le module à tester s'appelle division, et que sa seule fonction interne s'appelle également division.

La première chose à faire est d'importer le module **Unittest**, puis de créer la boucle principale.

```
import unittest
&#8230;
if __name__ == " __main__ ":
    unittest.main()
```

Il faut ensuite importer le module que l'on désire tester.

```
from division import *
```

Enfin, il faut créer les tests mêmes. Cela passe par la création d'une classe, héritant du module Unittest.

```
class TestDivision(unittest.TestCase):
    ...
```

Cette classe contiendra nos procédures de test.

```
def test_division(self):
    retour = division(9,3)
    self.assertEqual(retour, 3)
```

Attention : la classe de test doit obligatoirement hériter de la classe unittest.TestCase. Par convention, son nom commencera toujours par Test. De même, toutes les procédures de tests devront commencer par test_ (attention à la casse). Cela permet à Unittest de savoir qu'il s'agit bien d'une procédure de test.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

VIII-B-2-e-ii - Possibilités offertes

Avant de voir plus en avant la réalisation d'un test, étudions ici les différentes possibilités.

À l'issue de l'exécution de votre portion de code, il vous faudra effectuer une opération de comparaison afin de déterminer si le fonctionnement est correct ou non.

Pour cela, il existe différentes opérations possibles.

La plus classique est le test d'égalité ou de presque égalité. Le mot-clé diffère selon le type d'élément à comparer.

Mot-clé	Type d'objet	Type d'égalité
assertEqual	Indifférent	**=**
assertAlmostEqual	indifférent	**≈**
assertListEqual	Liste	**=**
assertTupleEqual	Tuple	**=**
assertDictEqual	Dict	**=**

La possibilité suivante est bien entendu, comme en mathématiques, le supérieur ou inférieur avec possibilité d'égalité.

mot-clé	opération
assertGreater	**>**
assertGreaterEqual	**#**
assertLess	**<**
assertLessEqual	**#**

Voici pour les principaux types de comparaison. Bien entendu, il y en a d'autres. Je vous invite à étudier plus en détail la documentation officielle pour en apprendre plus.

VIII-B-2-e-iii - Test simple

Nous allons repartir sur l'exemple vu dans le fonctionnement de base. Imaginons que notre module « division » à tester soit le suivant.

```
def division(p1 = 0, p2 = 1):
    if (p2 <> 0):
        resultat = p1 / p2
    else:
        resultat = 0
    return resultat

if __name__ = " __main__ ":
    division()
```

Notre module de test ressemblerait alors, comme vu précédemment, à ceci.

```
1. import unittest
2. from division import *
3.
4. class TestDivision(unittest.TestCase):
5.     def test_division(self):
6.         retour = division.division(9,3)
7.         self.assertEqual(retour, 3)
8.
9. if __name__ == " __main__ ":
10.     unittest.main()
```


Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Comme nous pouvons le voir ici, ligne 7, nous testons le retour de la fonction division. Si le retour vaut bien 3, alors le test sera positif, négatif sinon.

Pour lancer le fichier de test, rien de plus simple. Dans un terminal, lancez-le comme n'importe quel fichier **PYTHON** (pour rappel : **PYTHON** <fichier.py>).

Vous pouvez également implémenter une redirection du résultat vers un fichier texte.

Pour votre information, vous verrez ci-dessous le résultat d'un test positif et d'un test négatif.

VIII-B-2-e-iv - Tests multiples

Maintenant que nous avons vu comment faire un test de base, nous allons voir comment les enchaîner.

En effet, il serait bien pénible de devoir les lancer un à un.

Eh bien, pour effectuer de multiples tests, il suffit de les coder les uns à la suite des autres. Cela n'est pas plus compliqué.

Chaque fonction de test définie dans la classe de tests sera exécutée, tout simplement.

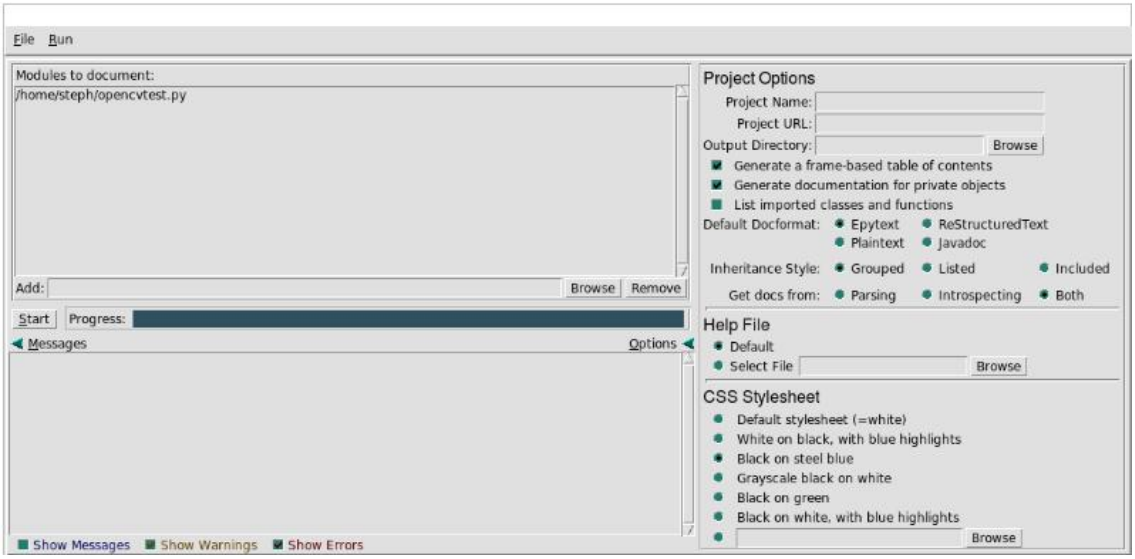
VIII-B-3 - Documentation

VIII-B-3-a - Epydoc

Une bonne documentation est indispensable pour la pérennité de votre code. C'est elle qui vous permettra, plus tard, de réutiliser certains packages et/ou modules dans d'autres projets.

Il ne faut ainsi jamais se limiter à une vision à court terme, comme le font beaucoup de personnes et d'entreprises, mais au contraire, se projeter dans le temps et se demander comment rendre notre code le plus générique possible pour une future réutilisation.

Une fois cela fait, et vos docstrings réalisées clairement, vous avez fait 90 % du travail. Les 10 % restants seront assurés par un outil fort pratique nommé **EPYDOC**.



L'interface d'EPYDOCGUI

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

EPYDOC est très simple à utiliser, même si son interface, simple, mais un peu brute, peut ne pas être attirante de prime abord. Une fois installé, il suffit d'ouvrir un terminal et de saisir « **epydocgui »**. Vous avez aussi la possibilité de vous créer un raccourci.

Dans la fenêtre qui s'ouvre, via le bouton BROWSE, il faut sélectionner vos fichiers à documenter, ou vos dossiers packages, voir le dossier de votre projet. Une fois votre sélection validée, il ne reste plus qu'à cliquer sur START, et attendre que la boîte à messages vous indique que le travail est terminé.

Vous trouverez alors au même niveau que vos fichiers/dossiers un dossier nommé HTML. Ce dossier contient un fichier index qu'il vous suffit de lancer pour avoir accès à la documentation complète, générée par **EPYDOC**.

Attention : EPYDOC ne générera de la documentation pour vos fichiers .py qu'à la stricte condition qu'il n'utilise que des tabulations pour l'indentation.

Remarque : en cliquant sur option sur la fenêtre principale de EPYDOCGUI, vous avez la possibilité de changer le thème de la documentation (en bas à droite).

VIII-B-4 - Fichier de log

Un **fichier de log** est un fichier qui garde en mémoire l'activité du logiciel, traçant les opérations, mais également les erreurs.

En cas d'anomalie provoquant le crash de votre application, vous disposez ainsi des informations nécessaires au débogage.

Si votre fichier de log est bien construit, vous aurez alors la classe, fonction, données… ayant provoqué le crash.

À l'aide de ces informations, vous serez alors capable de reproduire le crash, d'en déterminer la cause, et d'y remédier.

Les fichiers de log, quoique parfois considérés comme superflus sont réellement utiles et permettent d'améliorer en permanence les logiciels.

Il ne faut donc pas que vous hésitiez à les implémenter au sein de vos applications.

Je vous conseille d'ailleurs de créer une classe dédiée dont les principales procédures seront l'ouverture/création du fichier de log, l'écriture dans ce dernier, et la fermeture propre du fichier de log.

VIII-C - Versionning

Le **versionning** est une façon simple de différencier les différentes versions d'un logiciel.

Comme l'explique l'article dédié à « version d'un logiciel » de Wikipédia, il existe différentes façons de référencer les versions d'un logiciel. Je vous invite à lire cet article très bien documenté.

Nous allons ici détailler une seule façon de faire, mais très classique, souvent appelée système de version XYZ.

Ce système fonctionne sur trois chiffres, chacun ayant sa propre signification : logiciel Version X.Y.Z

Z représente une correction de bogue(s) ;

Y représente un ajout/suppression de fonctionnalité(s) ;

X représente une évolution majeure ou bien une refonte partielle, voire totale du logiciel.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

En règle générale, on dit que X est la version majeure, Y la version mineure et Z la version de correction.

La numérotation commencera toujours à 0 et chaque évolution d'une version entraînera la remise à zéro des niveaux inférieurs. Nous passerons par exemple, de la version 1.10.5 à la version 2.0.0.

De plus, il faut savoir que X ne vaudra 0 que pour des versions dites alpha ou bêta, c'est-à-dire des versions de logiciels toujours en développement, encore non aboutis et non recommandés pour utilisations courantes.

La première version stable sera toujours la 1.0.0

Remarque : si une version X peut ne pas être compatible avec une version X-1, une version X.Y doit absolument être compatible avec une version X.Y-1. Si ce n'est pas le cas, alors on changera de version majeure X.

VIII-D - Les licences libres

Qu'il s'agisse de tutoriels, de notices, de documentations, ou des logiciels, le choix judicieux d'une licence adaptée est primordial.

En effet, le choix d'une **licence** et son application sur vos œuvres même sur un simple tutoriel peut vous éviter certains soucis juridiques et même vous protéger contre des abus ou de potentielles accusations.

Nous verrons ici, la licence Creative Commons, parfaitement adaptée aux écrits relatifs à l'informatique (livres, tutoriels ou documentations) et la licence GPL pour les logiciels.

VIII-D-1 - Licences logicielles

Il y a deux questions importantes à prendre en compte pour les licences logicielles :

=>permettent-elles de faire des logiciels libres ? ;

=>obligent-elles la redistribution en licence libre ?

Remarque : on parle souvent de copyleft dans le cadre de la redistribution. D'après Wikipédia, « Le copyleft est l'autorisation donnée par l'auteur d'un travail soumis au droit d'auteur (œuvre d'art, texte, programme informatique ou autre) d'utiliser, d'étudier, de modifier et de copier son œuvre, dans la mesure où cette autorisation reste préservée. »

Outre ces questions, on regarde aussi les différences par rapport à la licence libre de référence : la GNU GPL V3.

Remarque : il existe une différence entre libre et open source. Un logiciel libre sous licence GPL oblige les contributeurs à rendre accessibles leurs modifications. Un logiciel open source n'obligera pas forcément cette divulgation, telle que la licence Apache. Sur le fond, il s'agit surtout, avec le terme « Open source » de faire moins peur aux investisseurs qu'avec le terme « Libre ». En effet, ce terme peut avoir mauvaise réputation dans le milieu professionnel. Dans les faits, même s'il existe une petite différence, « Libre » ou « Open Source » peuvent être employés indifféremment.

VIII-D-1-a - GPL

La licence **GPL**, est directement opposée au copyright. Quand ce dernier interdit toute copie, modification et redistribution d'une œuvre, la GPL l'autorise.

Parmi les principales obligations imposées par la GPL, on retrouvera le libre accès aux codes source et la conservation de la licence, quelle que soit l'action effectuée vis-à-vis du code source (modification, redistribution…).

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

La GPL garantit quatre libertés aux utilisateurs :

=>la liberté d'exécuter le programme, pour tous les usages ;

=>la liberté d'étudier le fonctionnement du programme et de l'adapter à ses besoins ;

=>la liberté de redistribuer des copies du programme (ce qui implique la possibilité aussi bien de donner que de vendre des copies) ;

=>la liberté d'améliorer le programme et de distribuer ces améliorations au public, pour en faire profiter toute la communauté.

De plus, même si l'erreur est souvent commise, il y a une différence entre gratuit et libre. En effet, gratuit concerne le tarif, le prix du logiciel, et libre concerne son code source.

De fait, un logiciel peut être libre, mais néanmoins payant. La licence GPL permet donc à l'utilisateur de facturer des copies du logiciel, à la condition qu'il livre le code source.

Dans les faits, les logiciels libres sont la plupart du temps gratuits et on trouvera plutôt la facturation sur des services que sur les logiciels.

Pour placer votre œuvre sous licence GPL, il faut mettre le bloc suivant au début de chaque fichier source.

Nom de votre programme + ce qu'il fait>

Copyright (C) <année sur quatre chiffres> <Nom des auteurs>

This file is part of <Nom de votre programme>.

<Nom de votre programme> is free software: you can redistribute it and/or modify

it under the terms of the GNU General Public License as published by

the Free Software Foundation, either version 3 of the License, or

(at your option) any later version.

<Nom de votre programme> is distributed in the hope that it will be useful,

but WITHOUT ANY WARRANTY; without even the implied warranty of

MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the

GNU General Public License for more details.

You should have received a copy of the GNU General Public License

along with <Nom de votre programme>. If not, see <http://www.gnu.org/licenses/>

Bien entendu, pour **PYTHON**, il faut laisser en ligne 1 et 2 le chemin d'accès à l'interpréteur et le type d'encodage. On placera donc les termes de la licence GPL à partir de la ligne 4 (lignes 1 et 2 pour l'interpréteur et l'encodage, ligne 3 en saut de ligne).

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Il ne faut pas oublier également de placer dans le dossier de votre logiciel un dossier contenant au minimum la licence GPL complète au format texte.

VIII-D-2 - Licences Documentaires

VIII-D-2-a - Creative Commons

La licence Creative Commons est un ensemble de six licences distinctives permettant à des auteurs de mettre leurs œuvres à disposition des personnes tierces de façon simplifiée.

Dans le cas de l'informatique, plutôt appliquée aux documentations ou livres dédiés au sujet, ces licences CC présentent beaucoup d'avantages.

Tout d'abord, ces licences copyleft ont une vraie valeur légale. Ensuite, elles vous garantissent la paternité de vos œuvres. Enfin, vous restez propriétaire de vos œuvres.

Ces licences sont basées sur quatre éléments, dont l'association donne naissance à six licences différentes.

Element	Nom	Description
BY	**Attribution**	Les tiers vous reconnaissent la paternité de l'œuvre. Lorsqu'ils rediffusent l'œuvre par leurs propres moyens, la paternité doit vous être attribuée. Vous êtes le seul propriétaire légitime de l'œuvre.
NC	**Pas d'utilisation commerciale**	Sauf à retenir l'option ND, vous autorisez la reproduction, la diffusion et la modification de vos œuvres à des fins exclusivement non commerciales. Vous êtes libre d'accorder une autorisation exceptionnelle sur demande.
SA	**Partage à l'identique**	Vous autorisez la reproduction, la diffusion et la modification de vos œuvres sous la condition que la nouvelle œuvre qui en découle utilise exactement la même licence Creative Commons. Vous êtes libre d'autoriser un changement de licence sur demande.
ND	**Pas de modification**	Vous autorisez la reproduction et la diffusion de votre œuvre stricto à l'identique. Vous êtes libre d'accorder une autorisation exceptionnelle sur demande

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Les licences disponibles sont les suivantes (définitions issues du site Creative Commons FR) :

Licence	Description
BY	Le titulaire des droits autorise toute exploitation de l'œuvre, y compris à des fins commerciales, ainsi que la création d'œuvres dérivées, dont la distribution est également autorisée sans restriction, à condition de l'attribuer à son auteur en citant son nom. Cette licence est recommandée pour la diffusion et l'utilisation maximale des œuvres.
BY-ND	Le titulaire des droits autorise toute utilisation de l'œuvre originale (y compris à des fins commerciales), mais n'autorise pas la création d'œuvres dérivées.
BY-NC-ND	Le titulaire des droits autorise l'utilisation de l'œuvre originale à des fins non commerciales, mais n'autorise pas la création d'œuvres dérivées.
BY-NC	Le titulaire des droits autorise l'exploitation de l'œuvre, ainsi que la création d'œuvres dérivées, à condition qu'il ne s'agisse pas d'une utilisation commerciale (les utilisations commerciales restant soumises à son autorisation).
BY-NC-SA	Le titulaire des droits autorise l'exploitation de l'œuvre originale à des fins non commerciales, ainsi que la création d'œuvres dérivées, à condition qu'elles soient distribuées sous une licence identique à celle qui régit l'œuvre originale.
BY-SA	Le titulaire des droits autorise toute utilisation de l'œuvre originale (y compris à des fins commerciales) ainsi que la création d'œuvres dérivées, à condition qu'elles soient distribuées sous une licence identique à celle qui régit l'œuvre originale. Cette licence est souvent comparée aux licences « copyleft » des logiciels libres. C'est la licence utilisée par Wikipédia.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

IX - Déploiement



Wikimedia Commons,Package-x-generic.svg

Le choix d'un bon **déploiement** est essentiel pour le succès d'une application.

En effet, les utilisateurs désirent plus que jamais de la simplicité, et on peut difficilement s'imaginer retourner en arrière au temps où, jadis, il fallait tout faire en ligne de commande.

Nous allons ici voir les paquets DEBIAN. Le choix de ce type de paquets plutôt qu'un autre tient en plusieurs réponses.

Tout d'abord, il est souvent considéré comme l'un des paquets les plus difficiles à générer, et comme dit l'adage, qui peut le plus peut le moins.

Ensuite, les désormais distributions connues Ubuntu, et Linux Mint sont basées sur la distribution DEBIAN.

Enfin, la distribution DEBIAN est très utilisée dans le monde industriel et professionnel.

IX-A - Création d'un .deb manuel

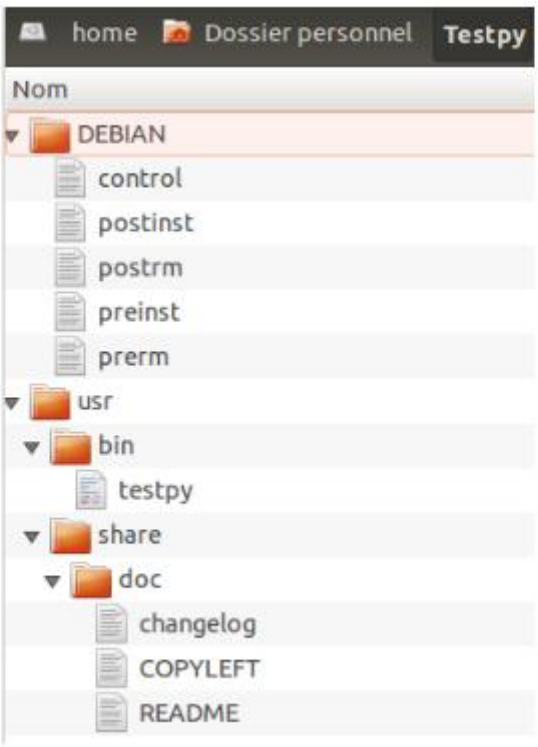
La création d'un package DEBIAN de base n'est pas aussi complexe qu'on peut le laisser entendre, spécialement, lorsque notre programme **PYTHON** consiste en un seul fichier.

Cette méthode est ici à titre informatif, car il existe désormais des méthodes plus simples pour créer nos paquets. Nous allons donc voir une création basique.

Si votre curiosité est plus forte, Internet vous fournira de plus amples informations.

Dans notre cas d'un seul et unique fichier, voici ce que l'on obtient comme structure pour un paquet DEBIAN.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/



Composition d'un package DEBIAN

Remarque : il faut scrupuleusement respecter la casse.

Quelques explications seront, je le pense, les bienvenues.

Tout d'abord le dossier DEBIAN. Il contient les informations pour l'installation. Le fichier INDISPENSABLE : control.

Ce fichier possède la structure suivante :

control

```
1  Package: Nom_du_programme(ex:monSoftware)
2  Version: version_du_programme(ex:1.0.0)
3  Section: sections_ou_mettre_le_programme
4  Priority: optional
5  Architecture: all
6  Depends: les_dependances_de_votre_programme(ex:matplotlib (>= 1.2), opencv (>=2.0))
7  Maintainer: votre_nom <votre_mail>
8  Description: une_description_courte_de_votre_programme
9  Homepage: adresse_web_de_votre_site
```

Contenu du fichier CONTROL d'un paquet DEBIAN

Vous pouvez recopier textuellement le contenu.

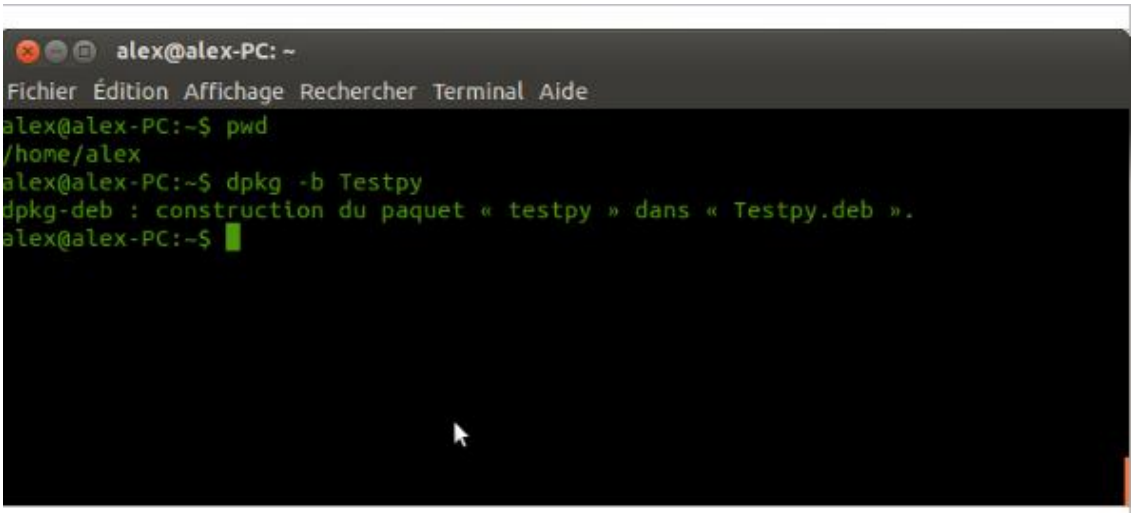
Viennent ensuite les fichiers Preinst et Postinst, qui sont des shells permettant d'exécuter certaines actions avant et après l'installation. De même, Prerm et Postrm servent lors de la désinstallation.

Dans usr/bin, nous trouvons notre fichier **PYTHON**, sans extension. Cette absence d'extension permettra par la suite de n'avoir à saisir que le nom du programme pour qu'il se lance.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

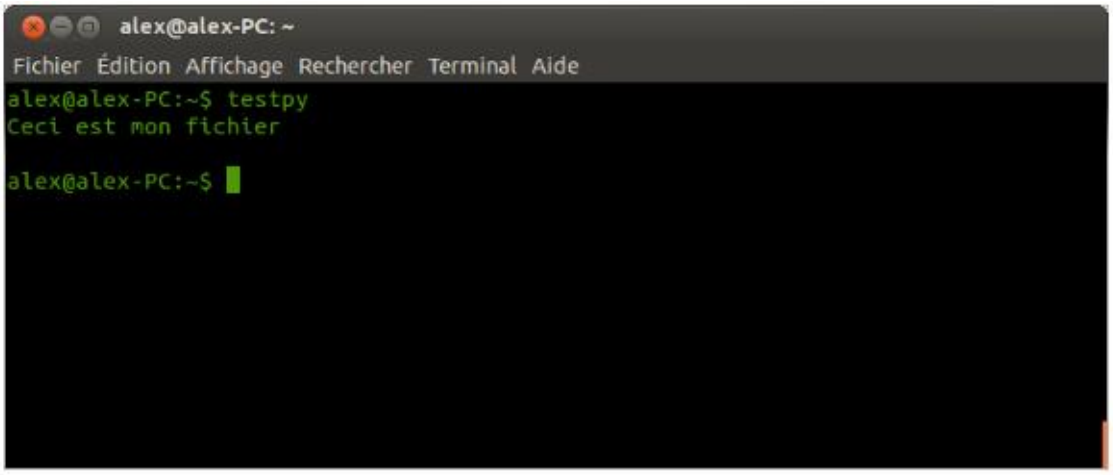
Dans usr/doc, nous trouvons quelques fichiers informatifs : README avec l'utilité qu'on lui connaît tous, changelog indiquant les évolutions des différentes versions, et COPYLEFT contenant la licence.

Une fois notre structure prête, il suffit de se placer là où se trouve le dossier (dans notre cas, /home/dossier personnel) puis d'exécuter la requête ***dpkg -b <nom_du_dossier>***



Génération d'un paquet DEBIAN via dpkg

Le fichier .deb est alors créé dans votre dossier home. Vous pouvez l'installer comme bon vous semble. Ensuite un simple appel avec le nom du fichier contenu dans bin suffit à exécuter votre programme.



Test du soft une fois installé via le paquet DEBIAN

IX-B - Création d'un .DEB automatique

DEBREATE est un outil graphique permettant de créer simplement et facilement un paquet DEBIAN.

Divisé en neuf étapes, avec un passage simple de l'une à l'autre grâce aux flèches bleues en haut à droite, ce logiciel dispose d'une traduction disponible dans plusieurs langues, dont le français, le rendant ainsi particulièrement agréable d'utilisation, et rendant superflue l'utilisation des lignes de commande.

De plus, **DEBREATE** permet de renseigner la totalité des informations dans un paquet DEBIAN, ainsi que la structure finale. Bref, un outil plus qu'utile.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Voyons son fonctionnement plus en détail.

IX-C - Utilisation du logiciel Alien

Nous venons de voir comment créer un .deb permettant ainsi de diffuser simplement notre programme sur l'ensemble des distributions basées sur DEBIAN, telles Ubuntu ou encore LinuxMint, pour ne citer que les plus connues.

Cependant, le monde Linux est très vaste, et au-delà de DEBIAN (.deb), il existe également des distributions basées sur Red Hat (.rpm), sur Slackware (.tgz), ou encore sur Solaris (.pkg) par exemple.

Bien que cette liste ne soit pas exhaustive, elle couvre déjà un minimum de distribution. La plupart de ces distributions possèdent en effet un certain nombre de dérivées.

On pourrait ainsi se dire qu'utiliser un logiciel pour générer un paquet pour chacune de ces distributions résoudrait le problème.

Mais à chaque mise à jour, même mineure, vous passeriez un temps important pour régénérer les paquets et vous finiriez probablement par abandonner le support de certaines distributions.

Place donc à la solution : **ALIEN**.

Il s'agit d'une commande permettant de réaliser des conversions de packages (et elle vous servira aussi parfois pour convertir un paquet n'étant pas destiné à votre distribution).

Ainsi, une fois notre .deb créé, il suffit d'y faire appel pour créer l'ensemble des paquets pour les autres distributions, permettant ainsi un déploiement simplifié sur les distributions Linux.

Attention : une conversion vers un autre système de paquets peut parfois être défectueuse. N'oubliez pas de tester votre paquet avant de le mettre à disposition.

Alien étant un outil en ligne de commande, il est extrêmement simple d'emploi.

En effet, une fois installé sur le système, on se contentera des fonctionnalités de base.

Une fois **Alien** installé sur votre système donc, il suffit d'ouvrir un terminal et de vous placer dans le dossier où se trouve le paquet à modifier.

Une fois cela fait, il ne reste plus qu'à appeler la commande Alien, en précisant le type de conversion désirée et le nom du paquet à changer.

Les options sont les suivantes :

Option	Description
-d	Transforme un paquet quelconque en paquet DEBIAN
-r	Transforme un paquet quelconque en paquet Red Hat
-t	Transforme un paquet quelconque en paquet Slackware
-p	Transforme un paquet quelconque en paquet Solaris

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Par exemple, pour convertir un paquet DEBIAN de votre création pour Red Hat, vous taperez dans votre terminal la commande suivante :

```
alien -r mon_paquet.deb
```

X - Ressources



Wikimedia Commons,Book icon 1.png

Langage	PYTHON
http://www.PYTHON.org/ http://docs.python.org/3/	

Aide/Info	Developpez.com
http://PYTHON.developpez.com/	

Aide/Info	Le site du zero
http://www.siteduzero.com/	

Aide/Info	ODT
http://fr.wikipedia.org/wiki/OpenDocument	

Module	ezodf
https://pypi.python.org/pypi/ezodf/0.2.1	

Module	FTPLib
http://docs.PYTHON.org/2/library/ftplib.html	

Module	GLUT
http://www.opengl.org/resources/libraries/glut/	

Module	LXML
http://lxml.de/	

Module	Math
http://docs.PYTHON.org/2/library/math.html	

Module	Matplotlib
http://matplotlib.org/	

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Module	Numpy
http://www.numpy.org/	

Module	OpenCV
http://docs.opencv.org/	

Module	OS
http://docs.PYTHON.org/2/library/os.html	

Module	PIL
http://www.PYTHONware.com/products/pil/	

Module	PostGreSQL
http://www.postgresql.fr/	

Module	PYGTK
http://www.PYGTK.org/docs/PYGTK/	

Module	Pypi
https://pypi.python.org/pypi	

Module	Pyserial et Pyparallel
http://pyserial.sourceforge.net/ http://pyserial.sourceforge.net/pyparallel.html	

Module	ReportLab
http://www.reportlab.com/software/opensource/	

Module	socket
http://docs.PYTHON.org/2/library/socket.html	

Module	SQLite
http://www.sqlite.org/	

Module	SMTPLib
http://docs.PYTHON.org/2/library/smtplib.html	

Module	Sys
http://docs.PYTHON.org/2/library/sys.html	

Module	Thread
http://docs.PYTHON.org/2/library/threading.html	

Module	Time
http://docs.PYTHON.org/2/library/time.html	

Module	ZIPFile
http://docs.PYTHON.org/2/library/zipfile	

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

Livre	Apprendre à programmer avec PYTHON
De Gérard SWINNEN, ISBN-13: 978-2212134346	

Livre	Apprenez à programmer en PYTHON
De Vincent Le Goff, ISBN-13: 979-1090085039	

Logiciel	Debreate
http://debreate.sourceforge.net/?page=download	

Logiciel	Geany
http://www.geany.org/	

Logiciel	Libre Office
http://fr.libreoffice.org/	

Logiciel	Meshlab
http://meshlab.sourceforge.net/	

Ressources	Lot d'icône Open Source
http://openiconlibrary.sourceforge.net/	

Ressources	Wikimedia Commons
http://commons.wikimedia.org/wiki/Accueil	

Ajouter les annexes.

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

XI - Annexes

XI-A - Spécifications fonctionnelles

SPECIFICATIONS FONCTIONNELLES	PJ PYTHON
PROJET:	

Société:	Date (début):	Date (fin):
AUTEUR(S):		

Description du projet et du contexte

<Société>	1/2

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

SPECIFICATIONS FONCTIONNELLES	PJ PYTHON 
PROJET:	

Fonctionnalités souhaitée (Règles de Gestion)
F<N° sur 3 chiffres> - <Nom de la fonction> <Description>

<Société>	2/2

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

XI-B - Spécifications techniques

SPECIFICATIONS TECHNIQUES	PJ PYTHON
PROJET:	

Société:	Date (début):	Date (fin):
AUTEUR(S):		

Ref. Fonctionnelle	Ref. Technique Associée	Existant?	Nom du composant si existant
Fxxx	Txxx		

<Société> 1/2

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

SPECIFICATIONS TECHNIQUES	PJ PYTHON 
PROJET:	

Description Technique
T<N° sur 3 chiffres> - <Nom du composant> <Description>

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

XI-C - Priorisation

PRIORISATION	PJ PYTHON
PROJET:	

Société:	Date (début):	Date (fin):
AUTEUR(S):		

Priorisation des tâches				
Ref Technique	Charge Estimée	Jalon (Date limite)	Ressource	Indice de Priorité
Txxx				

<Société>	1/1

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

XI-D - Tests unitaires

Tests Unitaires	PJ PYTHON
PROJET:	

Société:	Date:	Testeur:	Contrôleur:

Composant	Description du test	Résultat attendu	OK / KO	Capture Ecran
-	-	1)	1) OK)	NO IMAGE AVAILABLE
-	-	1)	1) OK)	NO IMAGE AVAILABLE
-	-	1)	1) OK)	NO IMAGE AVAILABLE

«Société» 1/*

XI-E - Tests intégration

Tests Intégration	PJ PYTHON
PROJET:	

Société:	Date:	Testeur:	Contrôleur:

Ref. Technique	Description du test	Résultat attendu	OK / KO	Capture Ecran
Txxx	-	1)	1) OK)	NO IMAGE AVAILABLE
Txxx	-	1)	1) OK)	NO IMAGE AVAILABLE
Txxx	-	1)	1) OK)	NO IMAGE AVAILABLE

«Société» 1/*

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

XI-F - Tests de validation

Tests Validation	PJ PYTHON
PROJET:	

Société:	Date:	Testeur:	Contrôleur:

Ref. Fonctionnelle	Description du test	Résultat attendu	OK / KO	Capture Ecran
[illegible]	-	1)	1) OK/KO	NO IMAGE AVAILABLE
[illegible]	-	1)	1) OK/KO	NO IMAGE AVAILABLE
[illegible]	-	1)	1) OK/KO	NO IMAGE AVAILABLE

«Société» "Projet"

XI-G - Code ASCII

CODE ASCII

Decimal	Hexa	ASCII	Decimal	Hexa	ASCII	Decimal	Hexa	ASCII	Decimal	Hexa	ASCII
0	0x00	ctrl+@	32	0x20	Espace	64	0x40	@	96	0x60	`
1	0x01	ctrl+A	33	0x21	!	65	0x41	A	97	0x61	a
2	0x02	ctrl+B	34	0x22	"	66	0x42	B	98	0x62	b
3	0x03	ctrl+C	35	0x23	#	67	0x43	C	99	0x63	c
4	0x04	ctrl+D	36	0x24	$	68	0x44	D	100	0x64	d
5	0x05	ctrl+E	37	0x25	%	69	0x45	E	101	0x65	e
6	0x06	ctrl+F	38	0x26	&	70	0x46	F	102	0x66	f
7	0x07	ctrl+G	39	0x27	'	71	0x47	G	103	0x67	g
8	0x08	ctrl+H	40	0x28	(	72	0x48	H	104	0x68	h
9	0x09	ctrl+I	41	0x29	)	73	0x49	I	105	0x69	i
10	0x0A	ctrl+J	42	0x2A	*	74	0x4A	J	106	0x6A	j
11	0x0B	ctrl+K	43	0x2B	+	75	0x4B	K	107	0x6B	k
12	0x0C	ctrl+L	44	0x2C	,	76	0x4C	L	108	0x6C	l
13	0x0D	ctrl+M	45	0x2D	-	77	0x4D	M	109	0x6D	m
14	0x0E	ctrl+N	46	0x2E	.	78	0x4E	N	110	0x6E	n
15	0x0F	ctrl+O	47	0x2F	/	79	0x4F	O	111	0x6F	o
16	0x10	ctrl+P	48	0x30	0	80	0x50	P	112	0x70	p
17	0x11	ctrl+Q	49	0x31	1	81	0x51	Q	113	0x71	q
18	0x12	ctrl+R	50	0x32	2	82	0x52	R	114	0x72	r
19	0x13	ctrl+S	51	0x33	3	83	0x53	S	115	0x73	s
20	0x14	ctrl+T	52	0x34	4	84	0x54	T	116	0x74	t
21	0x15	ctrl+U	53	0x35	5	85	0x55	U	117	0x75	u
22	0x16	ctrl+V	54	0x36	6	86	0x56	V	118	0x76	v
23	0x17	ctrl+W	55	0x37	7	87	0x57	W	119	0x77	w
24	0x18	ctrl+X	56	0x38	8	88	0x58	X	120	0x78	x
25	0x19	ctrl+Y	57	0x39	9	89	0x59	Y	121	0x79	y
26	0x1A	ctrl+Z	58	0x3A	:	90	0x5A	Z	122	0x7A	z
27	0x1B	ctrl+[	59	0x3B	;	91	0x5B	[	123	0x7B	{
28	0x1C	ctrl+\	60	0x3C	<	92	0x5C	\	124	0x7C	|
29	0x1D	ctrl+]	61	0x3D	=	93	0x5D	]	125	0x7D	}
30	0x1E	ctrl+^	62	0x3E	>	94	0x5E	^	126	0x7E	~
31	0x1F	ctrl+_	63	0x3F	?	95	0x5F	_	127	0x7F	DELETE

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/

XI-H - Nuancier RGB hexa

R/G/B	Teinte
00 / 00 / 00	
00 / 00 / 40	
00 / 00 / 80	
00 / 00 / C0	
00 / 00 / FF	
00 / 40 / 00	
00 / 40 / 40	
00 / 40 / 80	
00 / 40 / C0	
00 / 40 / FF	
00 / 80 / 00	
00 / 80 / 40	
00 / 80 / 80	
00 / 80 / C0	
00 / 80 / FF	
00 / C0 / 00	
00 / C0 / 40	
00 / C0 / 80	
00 / C0 / C0	
00 / C0 / FF	
00 / FF / 00	
00 / FF / 40	
00 / FF / 80	
00 / FF / C0	
00 / FF / FF	

R/G/B	Teinte
40 / 00 / 00	
40 / 00 / 40	
40 / 00 / 80	
40 / 00 / C0	
40 / 00 / FF	
40 / 40 / 00	
40 / 40 / 40	
40 / 40 / 80	
40 / 40 / C0	
40 / 40 / FF	
40 / 80 / 00	
40 / 80 / 40	
40 / 80 / 80	
40 / 80 / C0	
40 / 80 / FF	
40 / C0 / 00	
40 / C0 / 40	
40 / C0 / 80	
40 / C0 / C0	
40 / C0 / FF	
40 / FF / 00	
40 / FF / 40	
40 / FF / 80	
40 / FF / C0	
40 / FF / FF	

R/G/B	Teinte
80 / 00 / 00	
80 / 00 / 40	
80 / 00 / 80	
80 / 00 / C0	
80 / 00 / FF	
80 / 40 / 00	
80 / 40 / 40	
80 / 40 / 80	
80 / 40 / C0	
80 / 40 / FF	
80 / 80 / 00	
80 / 80 / 40	
80 / 80 / 80	
80 / 80 / C0	
80 / 80 / FF	
80 / C0 / 00	
80 / C0 / 40	
80 / C0 / 80	
80 / C0 / C0	
80 / C0 / FF	
80 / FF / 00	
80 / FF / 40	
80 / FF / 80	
80 / FF / C0	
80 / FF / FF	

R/G/B	Teinte
C0 / 00 / 00	
C0 / 00 / 40	
C0 / 00 / 80	
C0 / 00 / C0	
C0 / 00 / FF	
C0 / 40 / 00	
C0 / 40 / 40	
C0 / 40 / 80	
C0 / 40 / C0	
C0 / 40 / FF	
C0 / 80 / 00	
C0 / 80 / 40	
C0 / 80 / 80	
C0 / 80 / C0	
C0 / 80 / FF	
C0 / C0 / 00	
C0 / C0 / 40	
C0 / C0 / 80	
C0 / C0 / C0	
C0 / C0 / FF	
C0 / FF / 00	
C0 / FF / 40	
C0 / FF / 80	
C0 / FF / C0	
C0 / FF / FF	

R/G/B	Teinte
FF / 00 / 00	
FF / 00 / 40	
FF / 00 / 80	
FF / 00 / C0	
FF / 00 / FF	
FF / 40 / 00	
FF / 40 / 40	
FF / 40 / 80	
FF / 40 / C0	
FF / 40 / FF	
FF / 80 / 00	
FF / 80 / 40	
FF / 80 / 80	
FF / 80 / C0	
FF / 80 / FF	
FF / C0 / 00	
FF / C0 / 40	
FF / C0 / 80	
FF / C0 / C0	
FF / C0 / FF	
FF / FF / 00	
FF / FF / 40	
FF / FF / 80	
FF / FF / C0	
FF / FF / FF	

Les sources présentées sur cette page sont libres de droits et vous pouvez les utiliser à votre convenance. Par contre, la page de présentation constitue une œuvre intellectuelle protégée par les droits d'auteur. Copyright ® 2013 Alexandre Galodé. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts.
http://deusyss.developpez.com/tutoriels/Python/python-en-bref/